

HookRelay — Sıfırdan İnşa Rehberi

Spring Boot 3 · Java 21 · Kafka · Redis · Docker ile adım adım,
gerekçeleriyle

Yasin Akçıl

Ağustos 2026

Güvenilir webhook teslimat servisi

İçindekiler

Bu Rehber Nasıl Kullanılır	6
Rehberin çalışma biçimi	6
Neyi neden yaptığımız	6
Kafka'yı sona bırakıyoruz — bilinçli olarak	6
Tuzaklar	6
Ön koşullar	6
Yol haritası	7
Bölüm 1 — Ne İnşa Ediyoruz	8
1.1 Problem	8
1.2 Neden bu proje öğretici	8
1.3 Mimari — kuşbakışı	9
1.4 Tek kural: bir veriyi yalnız bir servis yazar	9
1.5 Klasör iskeleti	10
Kontrol noktası	11
Bölüm 2 — Maven İskeleti	12
2.1 Kök pom.xml	12
2.2 Modül pom'ları	13
Kontrol noktası	14
Bölüm 3 — Sözleşmeler	15
3.1 Topic kataloğu	15
3.2 Neden gecikme süresi topic adında yok	15
3.3 Başlıklar	16
3.4 Olay kayıtları	16
Kontrol noktası	17
Bölüm 4 — İmzalama ve Redis İlkelleri	18
4.1 Neden Lua — atomiklik problemi	18
4.2 Token bucket	18
4.3 Dağıtık devre kesici	19
4.4 Idempotency deposu	21
4.5 HMAC imzalama	21
4.6 Konfigürasyon replikası	22
4.7 Modülün tamamı	22
Kontrol noktası	39
Bölüm 5 — Transactional Outbox	40
5.1 Bozuk kod	40
5.2 Çözüm	40
5.3 Tablo	40
5.4 `FOR UPDATE SKIP LOCKED`	41
5.5 Poller	41

5.6 Sıra garantisi üzerine dürüst not	42
5.7 `Propagation.MANDATORY` — sessiz hatayı gürültülü yapmak	42
5.8 Modülün tamamı	43
Kontrol noktası	49
Bölüm 6 — admin-api: Kontrol Düzlemi	50
6.1 Kontrol düzlemi ile veri düzlemi ayrımı	50
6.2 Compacted topic ile konfigürasyon replikasyonu	50
6.3 Topic'leri kod oluşturur	51
6.4 API anahtarı	52
6.5 Şema	52
6.6 Servis: değişiklik ve yayın tek transaction'da	53
6.7 CQRS projeksiyonu	54
6.8 Servisin tamamı	55
Kontrol noktası	78
Bölüm 7 — ingest-api: Olay Kabulü	79
7.1 Neden 202, neden 200 değil	79
7.2 API anahtarı doğrulama — replikadan	79
7.3 İki katmanlı idempotency	79
7.4 Fan-out — neden burada	80
7.5 Tuzak: `@Transactional` ve kendi kendine çağrı	81
7.6 Servisin tamamı	82
Kontrol noktası	95
Bölüm 8 — dispatcher: Teslimat Hattı	96
8.1 Akış	96
8.2 Chain of Responsibility	96
8.3 Bulkhead — gürültülü komşu	97
8.4 HTTP gönderim ve sanal iş parçacıkları	97
8.5 Strategy: yeniden deneme politikası	99
8.6 Tüketici ve offset kararı	100
8.7 Mükerrer teslimat koruması — ve buradaki tuzak	101
8.8 Servisin tamamı	102
Kontrol noktası	139
Bölüm 9 — retry-scheduler: Kafka'da Geciktirmeli Mesaj	140
9.1 Problem	140
9.2 Neden `Thread.sleep` felaket	140
9.3 Çözüm: katmanlı topic + `pause`/'`seek`	140
9.4 Neden sadece baştaki kayda bakmak yetiyor	141
9.5 Rebalance'ta durumu temizlemek	142
9.6 `KafkaConsumer` thread-safe değildir	142
9.7 Neden ayrı bir servis	143
9.8 Servisin tamamı	143
Kontrol noktası	150
Bölüm 10 — chaos-target ve gateway	151

10.1 chaos-target — demonun kurbanı	151
10.2 gateway	151
10.3 İki servisin tamamı	153
Bölüm 11 — Docker Compose ve Altyapı	163
11.1 Kafka — KRaft modu	163
11.2 Redis — `noeviction` kararı	163
11.3 Sağlık kontrolleri — "başladı" ile "hazır" farkı	164
11.4 Dockerfile — katman önbellesi	164
11.5 Üç veritabanı, tek sunucu	165
11.6 nginx — CORS'u ortadan kaldırmak	165
11.7 Dosyaların tamamı	165
Kontrol noktası	173
Bölüm 12 — Arayüz	174
12.1 Panelin tamamı	174
Bölüm 13 — Gözlemlenebilirlik	185
13.1 Metrikler	185
13.2 Hangi metrikler önemli	185
13.3 Grafana	185
13.4 Yapılandırma dosyaları	186
Bölüm 14 — Testler	187
14.1 Saf birim testleri	187
14.2 Testcontainers ile entegrasyon	187
14.3 Sürekli entegrasyon	188
14.4 Yük testi	189
14.5 Testlerin tamamı	190
Bölüm 15 — Kırma Senaryoları	203
15.1 Kafka'yı durdurun	203
15.2 Redis'i durdurun	203
15.3 dispatcher'ı öldürün	203
15.4 Ölçekleyin	204
15.5 Yavaş müşteri	204
15.6 Devre kesiciyi açın	204
Bölüm 16 — GitHub'a Hazırlık	205
16.1 README ilk otuz saniyede kazanılır	205
16.2 Kod yorumları	205
16.3 Commit geçmişi	205
16.4 Neyi eklemeyin	206
16.5 Mülakatta ne sorulur	206
Bölüm 17 — Gözden Geçirme: Kodun İkinci Okunuşu	207
17.1 Önce mekanik tarama	207
17.2 Sonra "az geçen alan" taraması	207
17.3 Asıl iş: her `catch` bloğunu sorgulayın	207

17.4 Metriklerin anlamını sorgulayın	208
17.5 "Neden bir eksik?" dedirten satırları arayın	208
17.6 Çalışmayan anotasyonları arayın	208
17.7 Sayfalama ile filtrelemenin sırasını kontrol edin	209
17.8 Bir düzeltmeyi nasıl kanıtlarsınız	209
17.9 Gözden geçirme kontrol listesi	210
Kontrol noktası	210
Ek A — Komut Referansı	212
Çalıştırma	212
Kırma senaryoları	212
Adresler	212
API	212
Kafka	213
Betiklerin tamamı	213
#### baslat.sh `scripts/baslat.sh`	213
#### demo.sh `scripts/demo.sh`	214
#### kir.sh `scripts/kir.sh`	215
#### durdur.sh `scripts/durdur.sh`	216
#### git-hazirla.sh `scripts/git-hazirla.sh`	217
Ek B — Sık Karşılaşılan Hatalar	219
`Topic ... not present in metadata after 60000 ms`	219
Konteynerler `kafka:9092`'ye bağlanamıyor	219
`No existing transaction found for transaction marked with propagation 'mand...`	219
Teslimatlar `DISCARDED` oluyor	219
`CommitFailedException: ... group has already rebalanced`	219
Flyway `checksum mismatch`	219
Grafana panoları boş	219
Port çakışması	219
Kapanış	221
Buradan sonra	221

Bu Rehber Nasıl Kullanılır

Bu rehber, **HookRelay** adlı bir webhook teslimat servisini sıfırdan inşa eder. Sonunda elinizde çalışan bir sistem ve — daha önemlisi — Kafka, Redis ve mikroservis mimarisinin *neden* öyle kurulduğuna dair somut bir kavrayış olacak.

Rehberin çalışma biçimi

Her bölüm çalışan bir sistemle biter. On altıncı bölümü beklemeden, üçüncü bölümün sonunda elinizde derlenen bir şey olur. Bu bilinçli: yarım bırakılmış projelerin en yaygın sebebi, ilk çalışan çıktının çok geç gelmesidir.

Kod blokları eksiksizdir. Kopyalayıp yapıştırdığınızda çalışır. Kısaltma yapıldığında `// ...` ile açıkça belirtilir.

Her bölüm iki katmanlı: önce **kararlar** — hangi problem, hangi alternatifler, neyin bedeli — sonra **dosyaların tamamı**. İlk okumada kararları takip edin; yazarken listelere dönün. Projedeki her Java sınıfı, her `application.yml`, her migration ve her betik bu rehberde tam hâliyle var.

Neyi neden yaptığımız

Bu rehberin asıl konusu kod değil, **karar**. Her önemli adımda üç soru cevaplanır:

1. Hangi problemi çözüyoruz?
2. Hangi alternatifler vardı?
3. Bu seçimle neyi feda ettik?

Üçüncü soru en önemlisi. Bedeli olmayan mimari karar yoktur; bedelini bilmeden verilen karar, karar değil tahmindir.

Kafka'yı sona bırakıyoruz — bilinçli olarak

Dördüncü bölümde Kafka yok. Beşinci bölümde de yok. Kafka'yı sisteme, **ona ihtiyaç duyduğumuz acıyı hissettikten sonra** ekleyeceğiz.

Sebebi şu: "Kafka'yı şuraya koyuyoruz" diye başlayan bir rehber, Kafka'nın ne işe yaradığını öğretmez. Önce senkron bir sistem kurup yavaş bir müşterinin bütün API'nizi kilitlediğini göreceksiniz. Kafka o zaman bir çözüm gibi görünecek — çünkü öyle.

Tuzaklar

Metin boyunca dikkat çekilen tuzaklar, uydurma değil. Hepsi bu projeyi yazarken gerçekten karşılaşılan ya da yakın geçmişte üretim sistemlerini düşürmüş problemler.

Ön koşullar

Araç	Sürüm	Doğrulama
JDK	21+	<code>java -version</code>
Maven	3.8+	<code>mvn -v</code>
Docker	24+	<code>docker --version</code>

Araç	Sürüm	Doğrulama
Docker Compose	v2+	<code>docker compose version</code>
curl, jq	herhangi	<code>curl --version, jq --version</code>

Bilgi olarak: Spring Boot ile REST API yazmış olmak, JPA ve SQL'e aşinalık yeterli. Kafka bilgisi **gerekmiyor** — zaten onu öğreniyoruz.

Yol haritası

Bölüm	Ne inşa edilir	Ne öğrenilir
1-2	İskelet, çok modüllü Maven	Modül sınırları
3	Sözleşmeler	Topic tasarımı
4	İmzalama, Redis ilkeleri	HMAC, Lua atomikliği
5	Transactional Outbox	İki sistemin atomik olamaması
6	Kontrol düzlemi	Veri sahipliği, CQRS
7	Olay kabulü	Idempotency, fan-out
8	Teslimatçı	Devre kesici, bulkhead, yeniden deneme
9	Zamanlayıcı	Kafka'da geciktirmeli mesaj
10-13	Altyapı, arayüz, metrik	Compose, Prometheus, Grafana
14-15	Testler, kırma senaryoları	Testcontainers, kaos
16	GitHub'a hazırlık	Sunum
17	Gözden geçirme turu	Kodun ikinci okunuşu

Sonunda elinizde 6 servis, 3 kütüphane modülü, ~5.800 satır Java, 47 test ve çalışan bir Docker yığını olacak.

Bölüm 1 — Ne İnşa Ediyoruz

1.1 Problem

Bir SaaS ürününüz var. Müşterileriniz, sizde bir şey olduğunda haberdar olmak istiyor: sipariş ödendi, fatura kesildi, kullanıcı silindi. Çözüm belli — webhook. Müşteri bir URL veriyor, siz oraya POST atıyorsunuz.

İlk kod on satır:

```
@EventListener
public void onOrderPaid(OrderPaidEvent event) {
    for (Endpoint endpoint : endpoints.findByCustomerId(event.customerId())) {
        restClient.post()
            .uri(endpoint.url())
            .body(event)
            .retrieve()
            .toBodilessEntity();
    }
}
```

Bu kod üretimde bir hafta yaşar. Sonra sırayla şunlar olur:

Birinci gün. Bir müşterinin sunucusu 500 dönüyor. Olay kayboldu. Müşteri iki gün sonra "siparişlerin yarısı gelmiyor" diye yazıyor. Elinizde hiçbir kayıt yok — ne zaman gönderdiğinizizi, ne cevap aldığınızı bilmiyorsunuz.

Üçüncü gün. Bir müşterinin sunucusu 30 saniyede cevap veriyor. `restClient` senkron. O müşteriye giden her istek bir Tomcat iş parçasığını 30 saniye tutuyor. 200 iş parçasığınız var. Müşterinin 200 olayı birikince **bütün API'niz** cevap vermiyor. Bir müşterinin yavaşlığı, bütün sisteminizin çöküşü oldu.

Birinci hafta. Bir müşteri bakıma girdi, üç saat kapalı kaldı. Üç saatlik olayların hepsi kayıp. Geri gönderme yolu yok, çünkü ne gönderdiğinizizi kaydetmediniz.

İkinci hafta. Yeniden deneme eklediniz. Şimdi müşteri iki kez "sipariş oluştu" bildirimi alıyor ve iki kez kargo çıkıyor.

Bu liste uzar. Her maddesi ayrı bir dağıtık sistem problemi.

Bu problem kimin problemi

Stripe, GitHub, Shopify, Slack — hepsi bu problemi çözmek zorunda kaldı. Svix ve Hookdeck bu problemin üzerine şirket kurdu. Yani uydurma bir domain değil; parası olan, çözümü belgelenmiş, iyi tanımlanmış bir mühendislik problemi.

1.2 Neden bu proje öğretici

Domain **çok küçük**: dört tablo. Uygulama, endpoint, mesaj, deneme. Ekran tasarımıyla, karmaşık iş kurallarıyla, raporlamayla haftalar harcamıyorsunuz.

Ama arkasındaki problemler **çok derin**. Bütün emek dağıtık sistem konularına gidiyor:

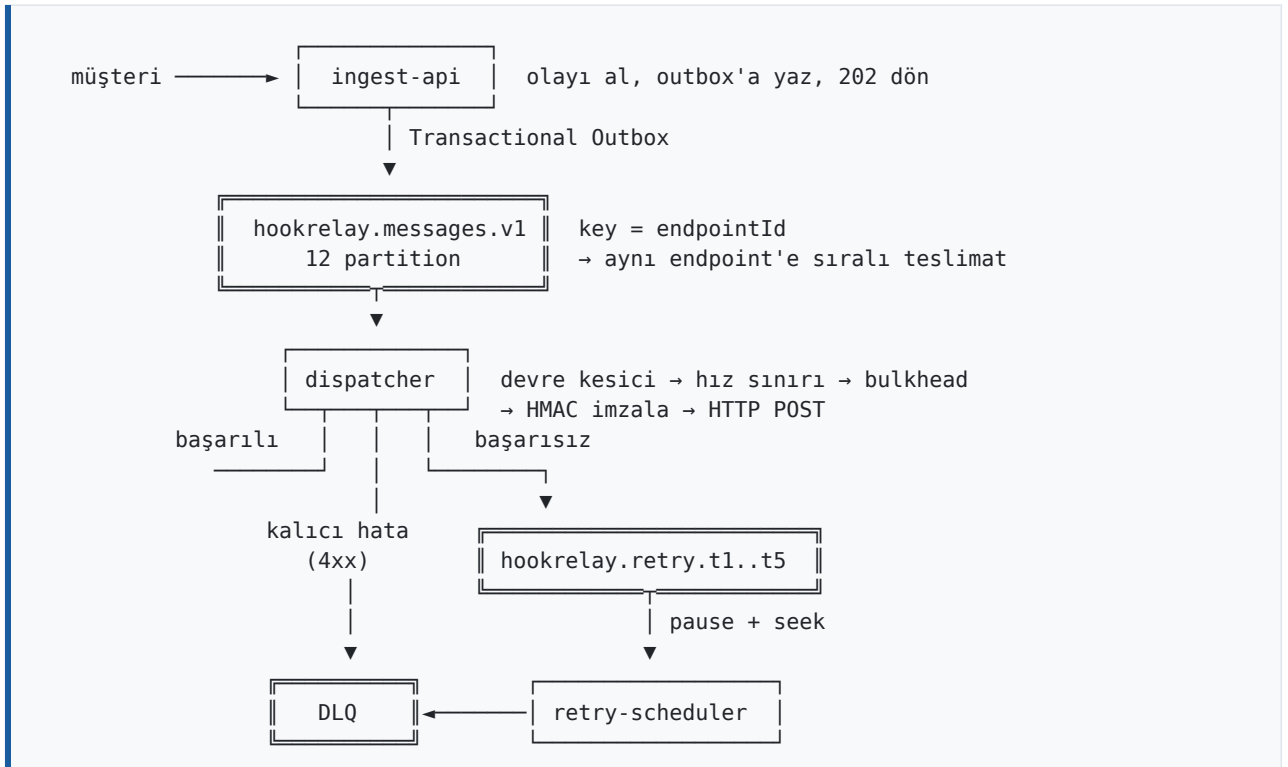
Problem	Öğrettiği
---------	-----------

Problem	Öğrettiği
Aynı endpoint'e sıralı teslimat	Partition ve key tasarımı
Katmanlı yeniden deneme	Kafka'da geciktirmeli mesaj
Çökmüş müşteri	Dağıtık devre kesici
Yavaş müşteri	Bulkhead, gürültülü komşu
Mükerrer olay	Idempotency, iki katmanlı
İmzalı istek	HMAC, replay saldırısı
Veritabanı + Kafka atomikliği	Transactional Outbox
"Webhook gelmedi" şikâyeti	Denetim izi, CQRS projeksiyonu

Hiçbiri süs değil. Her biri, çözülmezse sistemin çalışmadığı bir problem.

1.3 Mimari — kuşbakışı

Altı servis. Sayı önemli değil; **sınırların nereden geçtiği** önemli.



1.4 Tek kural: bir veriyi yalnız bir servis yazar

Mikroservisin en sık atlanan kuralı budur. İki servis aynı tabloya yazıyorsa elinizde mikroservis değil **dağıtık monolit** vardır: bütün maliyeti (ağ, dağıtım karmaşıklığı, hata ayıklama zorluğu), hiçbir faydası (bağımsız değişebilme).

Bizim tablomuz:

Servis	Yazdığı veri	Okuduğu
--------	--------------	---------

Servis	Yazdığı veri	Okuduğu
admin-api	uygulama, endpoint, sağlık projeksiyonu	—
ingest-api	mesaj, outbox	endpoint konfigürasyonu (replika)
dispatcher	teslimat, deneme kaydı	endpoint konfigürasyonu (replika)
retry-scheduler	(hiçbir şey — durumsuz)	—

"Replika" kelimesi kritik. `dispatcher` endpoint tablosunu **okumak için** `admin-api`'ye **HTTP atmaz**. Konfigürasyonu bir Kafka topic'inden Redis'e replike eder ve oradan okur. Altıncı bölümde bunu kuracağız.

Sonucu tek cümlede: `admin-api` **tamamen çökse bile teslimat devam eder**.

1.5 Klasör iskeleti

```
mkdir -p hookrelay && cd hookrelay
mkdir -p libs/{contracts,common,outbox}
mkdir -p services/{gateway,admin-api,ingest-api,dispatcher,retry-scheduler,chaos-target}
mkdir -p ops/{sql,prometheus,grafana,k6} ui docs scripts
```

Yapı:

```
hookrelay/
├── pom.xml                kök – sürüm yönetimi, modül listesi
├── libs/
│   ├── contracts/        topic adları, olay şemaları (bağımlılık: yok)
│   ├── common/           imzalama, Redis ilkelleri, hata sözleşmesi
│   └── outbox/           Transactional Outbox
├── services/
│   ├── gateway/          8080
│   ├── admin-api/        8081 kontrol düzlemi
│   ├── ingest-api/       8082 veri düzlemi girişi
│   ├── dispatcher/       8083 teslimatçı
│   ├── retry-scheduler/  8084 gecikmeli yeniden deneme
│   └── chaos-target/     8085 demo kurbanı
├── ops/                  compose, prometheus, grafana, k6
├── ui/                   tek dosyalık kontrol paneli
└── scripts/              başlat, demo, kır
```

Neden üç ayrı kütüphane modülü

`contracts` hiçbir şeye bağımlı değil — sadece kayıtlar ve sabitler. Bunu ayırmanın sebebi, ileride başka bir dilde istemci yazılacaksa şemanın tek bir yerde durması.

`common` Spring'e bağımlı: imzalama, Redis, hata yönetimi. Her servis kullanıyor.

`outbox` JPA'ya bağımlı. `gateway` ve `chaos-target` veritabanı kullanmıyor; `common`'a JPA koysaydık onlar da JPA taşımak zorunda kalır ve açılıştaki boş bir `DataSource` arayıp çökerlerdi.

Bağımlılık yönü tek yönlüdür: `services` → `libs`. Bir kütüphane bir servisi import ediyorsa, o kütüphane aslında bir servistir.

Kontrol noktası

```
find . -type d -not -path '*/.*' | sort
```

On altı klasör görmelisiniz. Henüz tek satır kod yok — bu normal.

Bölüm 2 — Maven İskeleti

2.1 Kök pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>dev.hookrelay</groupId>
  <artifactId>hookrelay-parent</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
    <relativePath/>
  </parent>

  <modules>
    <module>libs/contracts</module>
    <module>libs/common</module>
    <module>libs/outbox</module>
    <module>services/gateway</module>
    <module>services/admin-api</module>
    <module>services/ingest-api</module>
    <module>services/dispatcher</module>
    <module>services/retry-scheduler</module>
    <module>services/chaos-target</module>
  </modules>

  <properties>
    <java.version>21</java.version>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <spring-cloud.version>2023.0.3</spring-cloud.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>dev.hookrelay</groupId>
        <artifactId>contracts</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>dev.hookrelay</groupId>
        <artifactId>common</artifactId>
        <version>${project.version}</version>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

```

<dependency>
  <groupId>dev.hookrelay</groupId>
  <artifactId>outbox</artifactId>
  <version>${project.version}</version>
</dependency>
</dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>

```

Neden `dependencyManagement`, neden `dependencies` değil

`dependencyManagement` bir **sürüm sözlüğüdür**; kimseye bağımlılık eklemes. Alt modül `contracts`'ı istediğinde sürümü buradan alır. Doğrudan `dependencies`'e koysaydık `gateway` de `outbox`'ı, dolayısıyla JPA'yı taşımak zorunda kalırdı.

İstisna: `spring-boot-starter-test`'i bilinçli olarak `dependencies`'e koyduk. Test bağımlılığını her modülde ayrı ayrı yazmanın hiçbir faydası yok.

Sürüm seçimi

Spring Boot 3.3.4 ve Spring Cloud 2023.0.3 birbiriyle uyumlu. Bu **rastgele bir eşleşme değil** — Spring Cloud'un her sürüm treni belirli bir Boot minör sürümüne bağlıdır. Uyumsuz eşleştirirseniz hata mesajı genellikle alakasız bir `NoSuchMethodError` olur ve saatlerinizi yer.

2.2 Modül pom'ları

`libs/contracts/pom.xml` — en sade olanı:

```

<project>
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../pom.xml</relativePath>
  </parent>
  <artifactId>contracts</artifactId>

  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-annotations</artifactId>
    </dependency>
  </dependencies>
</project>

```

Servis pom'ları `spring-boot-maven-plugin` ekler ve `finalName`'i sabitler:

```
<build>
  <finalName>dispatcher</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

`finalName` neden önemli: Dockerfile `target/dispatcher.jar` diyecek. Varsayılan ad `dispatcher-1.0.0.jar` olurdu ve sürüm her değiştiğinde Dockerfile'ı güncellemek gerekirdi.

Kontrol noktası

```
mvn -q compile
```

Hata vermeden bitmeli. Henüz Java dosyası yok; Maven yalnızca yapıyı doğruluyor.

Bölüm 3 — Sözleşmeler

`contracts` modülü, servislerin **birbirine bakmadan** anlaşabilmesi için var. İçinde iş mantığı yok, yalnızca isimler ve şekiller.

3.1 Topic kataloğu

`libs/contracts/src/main/java/dev/hookrelay/contracts/Topics.java`:

```
public final class Topics {

    private Topics() {}

    /** Ana teslimat kuyruğu. key = endpointId → aynı endpoint sıralı gider. */
    public static final String MESSAGES = "hookrelay.messages.v1";

    public static final List<String> RETRY_TIERS = List.of(
        "hookrelay.retry.t1.v1",
        "hookrelay.retry.t2.v1",
        "hookrelay.retry.t3.v1",
        "hookrelay.retry.t4.v1",
        "hookrelay.retry.t5.v1"
    );

    public static final String DLQ = "hookrelay.dlq.v1";
    public static final String DELIVERY_RESULTS = "hookrelay.delivery-results.v1";
    public static final String ENDPOINT_CONFIG = "hookrelay.endpoint-config.v1";
    public static final String APP_CONFIG = "hookrelay.app-config.v1";

    public static int tierCount() { return RETRY_TIERS.size(); }

    public static String retryTier(int tier) {
        if (tier < 1 || tier > RETRY_TIERS.size()) {
            throw new IllegalArgumentException("Geçersiz katman: " + tier);
        }
        return RETRY_TIERS.get(tier - 1);
    }
}
```

İsimlendirme: `{ürün}.{alan}.{sürüm}`

Sürüm eki (`.v1`) bugün hiçbir işe yaramıyor. Yarın şema kırıcı bir değişiklik gerektiğinde ise tek çıkış yolu o olacak: `.v2` açıp iki tüketiciyi bir süre yan yana çalıştırmak, üreticiyi geçirmek, sonra `.v1`'i kapatmak.

Sürüm eki olmadan bu manevrayı yapamazsınız. Topic adına sonradan sürüm eklemek, mevcut bütün tüketicileri aynı anda değiştirmek demektir.

3.2 Neden gecikme süresi topic adında yok

İlk içgüdü `hookrelay.retry.10s` yazmaktır. Okunaklı görünür. **Yapmayın.**

Gecikme süresi bir *çalışma zamanı* uyarı. Üretimde 10 saniye uygunken demoda 5 saniye, yoğun bir müşteri için 2 dakika olabilir. Topic adına yazarsanız süreyi değiştirmek **topic'i yeniden adlandırmak** demektir — ve o topic'te bekleyen bütün mesajlar kaybolur.

Katman numarası sabittir, süre konfigürasyondan gelir:

```
hookrelay:
  retry:
    delays: 10s,1m,5m,30m,2h      # üretim
    # delays: 5s,15s,30s,60s,120s # demo
```

Kural. Topic adı yapıyı anlatır, davranışı değil. Yapı nadiren değişir; davranış her ortamda farklıdır.

3.3 Başlıklar

```
public final class Headers {

    // --- Kafka kayıt başlıkları ---
    public static final String NOT_BEFORE = "x-hr-not-before";
    public static final String ATTEMPT    = "x-hr-attempt";
    public static final String DEAD_REASON = "x-hr-dead-reason";

    // --- Müşteriye giden HTTP başlıkları ---
    public static final String OUT_ID      = "X-HookRelay-Id";
    public static final String OUT_TIMESTAMP = "X-HookRelay-Timestamp";
    public static final String OUT_SIGNATURE = "X-HookRelay-Signature";
    public static final String OUT_EVENT_TYPE = "X-HookRelay-Event-Type";
    public static final String OUT_ATTEMPT  = "X-HookRelay-Attempt";
}
```

NOT_BEFORE bütün sistemin en kritik başlığı. Dokuzuncu bölümde zamanlayıcı **topic'in adına değil bu başlığa** bakacak. Sayesinde bir kayıt, bulunduğu katmanın süresinden farklı bir zamanda işlenebilecek — sunucu **Retry-After: 300** dediğinde tam olarak buna ihtiyaç duyacağız.

3.4 Olay kayıtları

```
public record DeliveryTask(
    UUID deliveryId,
    UUID messageId,
    UUID applicationId,
    UUID endpointId,
    String eventType,
    String payload,
    int attempt,
    Instant firstQueuedAt
) {
    public DeliveryTask withAttempt(int next) {
        return new DeliveryTask(deliveryId, messageId, applicationId, endpointId,
            eventType, payload, next, firstQueuedAt);
    }
}
```

Bu kaydın **bir endpoint'e yapılacak tek teslimatı** temsil ettiğine dikkat edin — mantıksal olayı değil. Bir olay üç endpoint'e gidiyorsa üç **DeliveryTask** üretilir.

Ayrımı kaybetmek pahalıya patlar: "kaç olay geldi" ile "kaç istek attık" birbirine karışır, faturalandırma ve metrikler yanlış olur.

`firstQueuedAt` alanı yaşam süresi kontrolü için. Bir teslimat sonsuza kadar yeniden denenemez; 24 saat sonra DLQ'ya gider. Bu alan olmadan "ne zamandır uğraşıyoruz" sorusunu cevaplayamazsınız.

Diğer kayıtlar

```
public record DeliveryResult(
    UUID deliveryId, UUID endpointId, UUID applicationId, String eventType,
    Outcome outcome, Integer httpStatus, long latencyMs, int attempt,
    String error, Instant occurredAt
) {
    public enum Outcome { SUCCEEDED, RETRYING, EXHAUSTED, SHORT_CIRCUITED }
}

public record EndpointConfig(
    UUID endpointId, UUID applicationId, String url, String secret,
    Set<String> eventTypes, boolean enabled, boolean deleted,
    int rateLimitPerSecond, int maxAttempts, int timeoutMs, long version
) {
    public boolean subscribesTo(String eventType) {
        return eventTypes.contains("*") || eventTypes.contains(eventType);
    }
    public boolean deliverable() { return enabled && !deleted; }
}
```

`EndpointConfig`, `dispatcher`'ın sıcak yolda ihtiyaç duyduğu **her şeyi** taşır. Amaç tek: bir mesajı gönderirken başka bir servise soru sormak zorunda kalmamak.

`deleted` alanı neden var — Kafka'da silme "tombstone" (boş gövde) ile yapılır ama tombstone alan tüketici hangi uygulamaya ait olduğunu bilemez, çünkü key yalnızca `endpointId`'dir. Açık bir bayrak taşımak hem okunaklı hem hata ayıklanabilir.

Kontrol noktası

```
mvn -q compile -pl libs/contracts
```

Bölüm 4 — İmzalama ve Redis İlkelleri

Bu bölümde dört şey kuruyoruz: HMAC imzalama, hız sınırı, dağıtık devre kesici ve idempotency deposu. Üçü Redis'te ve üçü de **Lua** ile yazılacak. Önce neden.

4.1 Neden Lua — atomiklik problemi

Hız sınırını Java'da yazalım:

```
// BOZUK
int used = redis.get("kova:" + endpointId);
if (used < limit) {
    redis.set("kova:" + endpointId, used + 1);
    return IZIN_VAR;
}
return IZIN_YOK;
```

İki dispatcher örneği aynı anda çalışıyor. Limit 10, mevcut değer 9.

Örnek A	Örnek B
get → 9	get → 9
9 < 10 ✓	9 < 10 ✓
set 10	set 10
İZİN	İZİN

İkisi de izin verdi, sayaç 10'da kaldı, aslında 11 istek gitti. Bu bir **yarış durumu** (race condition) ve yükte kaybolmaz — tam tersine, yük arttıkça sıklaşıyor.

INCR gibi atomik komutlar bazı durumları kurtarır ama token bucket "geçen süreye göre doldur, sonra harca" mantığı istiyor; tek komutla ifade edilemiyor.

Çözüm: Redis, Lua betiklerini **tek iş parçacığında, kesintisiz** çalıştırır. Betik başladığında hiçbir başka komut araya giremez. Yani bir Lua betiği, ne kadar karmaşık olursa olsun, atomiktir.

Bedeli: Redis tek iş parçacıklı olduğu için uzun süren bir Lua betiği **bütün Redis'i bloke eder**. Betikler kısa olmak zorunda. Bizimkiler en fazla on satır.

4.2 Token bucket

libs/common/src/main/resources/lua/rate_limit.lua:

```
-- KEYS[1] = kova anahtarı          ARGV[1] = kapasite (jeton)
-- ARGV[2] = saniyedeki dolun hızı   ARGV[3] = şimdi (ms)
-- ARGV[4] = istenen jeton
-- Döner: {izin(0|1), beklenmesi_gereken_ms}

local key      = KEYS[1]
```

```

local capacity = tonumber(ARGV[1])
local rate     = tonumber(ARGV[2])
local now      = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

if rate <= 0 then return {1, 0} end -- 0 = sınırsız

local data = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(data[1])
local ts     = tonumber(data[2])

if tokens == nil then
    tokens = capacity
    ts = now
end

-- Geçen süreye göre kovayı doldur (tavan: kapasite)
local elapsed = now - ts
if elapsed < 0 then elapsed = 0 end
tokens = math.min(capacity, tokens + (elapsed * rate / 1000.0))

local allowed = 0
local wait = 0
if tokens >= requested then
    tokens = tokens - requested
    allowed = 1
else
    wait = math.ceil(((requested - tokens) / rate) * 1000)
end

redis.call('HSET', key, 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', key, math.ceil((capacity / rate) * 1000) + 10000)

return {allowed, wait}

```

Neden "token bucket", neden sabit pencere değil

Sabit pencere (her saniye en fazla 10) sınır anlarında iki katına izin verir: 12:00:00.999'da 10 istek, 12:00:01.001'de 10 istek daha — 2 milisaniyede 20 istek.

Token bucket zamanı sürekli sayar. Kova sürekli dolar, her istek bir jeton harcar. Kısa patlamalara izin verir (kova doluysa 10 istek arka arkaya geçer) ama ortalamayı korur.

`wait` neden dönüyor

Sadece "hayır" demek yetmez. "Ne kadar sonra?" sorusunun cevabı, çağıranın mesajı doğru katmana koymasını sağlar. Bilmeseydik en kısa katmana koyup tekrar reddedilmeyi göze alırdık.

4.3 Dağıtık devre kesici

Önce **neden hazır kütüphane kullanmadığımız**.

Resilience4j'nin devre kesicisi JVM içinde yaşar. Dört **dispatcher** örneği çalıştırdığınızda dört ayrı devre olur:

- Müşteri çıktığında her örnek ayrı ayrı beş hata biriktirmek zorunda → yirmi boşa istek.
- Bir örnek yeniden başlatıldığında o örneğin devre bilgisi sıfırlanır.
- "Bu endpoint'in devresi açık mı" sorusunun tek bir cevabı yok.

Endpoint bazlı devre kesici **paylaşılan durum** ister. O yüzden Redis'te.

`libs/common/src/main/resources/lua/circuit_breaker.lua` (özet):

```
-- KEYS[1] = devre anahtarı
-- ARGV[1] = işlem: 'allow' | 'success' | 'failure'
-- ARGV[2] = şimdi (ms)      ARGV[3] = hata eşiği
-- ARGV[4] = açık kalma (ms) ARGV[5] = kapanmak için yoklama sayısı
-- ARGV[6] = kayan pencere (ms)
-- Döner: {izin(0|1), durum, pencere_hata_sayısı}

local h = redis.call('HMGET', key, 'state', 'failures', 'openedAt',
                    'inFlight', 'probeOk', 'winStart')
local state = h[1] or 'CLOSED'
-- ... (alanlar okunur)

-- OPEN süresi dolduysa kendiliğinden HALF_OPEN'a geç
if state == 'OPEN' and (now - openedAt) >= openMs then
    state = 'HALF_OPEN'; inFlight = 0; probeOk = 0
end

-- Kayan pencere: son hatadan bu yana windowMs geçtiyse sayaç sıfırlanır
if state == 'CLOSED' and (now - winStart) > windowMs then
    failures = 0; winStart = now
end

if op == 'allow' then
    if state == 'CLOSED' then
        allowed = 1
    elseif state == 'HALF_OPEN' then
        -- TEK yoklama geçirilir. Toparlanmaya çalışan sunucuyu boğmamak için.
        if inFlight < 1 then allowed = 1; inFlight = inFlight + 1 end
    else
        allowed = 0
    end
end

elseif op == 'failure' then
    if state == 'HALF_OPEN' then
        state = 'OPEN'; openedAt = now -- yoklama da patladı, tam süre yeniden
    else
        failures = failures + 1
        if failures >= threshold then state = 'OPEN'; openedAt = now end
    end
end
```

Üç durum, üç anlam

Durum	Ne oluyor	Ne zaman çıkılır
CLOSED	Normal. İstekler geçiyor.	Pencerede eşik kadar hata → OPEN
OPEN	Hiçbir istek gitmiyor.	openMs geçince → HALF_OPEN
HALF_OPEN	Tek bir yoklama geçiyor.	Başarılı → CLOSED, başarısız → OPEN

HALF_OPEN'da neden **tek** yoklama: yeni ayağa kalkmış bir sunucuya biriken 50.000 mesajı bir anda göndermek onu ikinci kez devirir. Önce kapıyı tıklatıyoruz.

Kayan pencere neden gerekli

Eşik "beş hata" ise, sayacı hiç sıfırlamazsanız günde beş hata veren sağlıklı bir endpoint'in devresi bir hafta sonra açılır. Pencere (60 saniye), "beş hata"yı "60 saniyede beş hata"ya çevirir.

4.4 Idempotency deposu

```
local existing = redis.call('GET', KEYS[1])
if existing then
    return {0, existing}
end
redis.call('SET', KEYS[1], ARGV[1], 'PX', tonumber(ARGV[2]))
return {1, ''}
```

Küçük ama kritik: `GET` ve `SETNX`'i ayrı komut yapsaydınız iki istek aynı anda `GET` yapıp ikisi de "yeni" sanabilirdi. Tek betik bunu kapatır.

4.5 HMAC imzalama

```
@Component
public class WebhookSigner {

    private static final String ALGO = "HmacSHA256";
    private static final HexFormat HEX = HexFormat.of();

    public String sign(String secret, String payload, Instant timestamp) {
        long ts = timestamp.getEpochSecond();
        String signed = ts + "." + payload;
        return "t=" + ts + ",v1=" + HEX.formatHex(hmac(secret, signed));
    }

    public boolean verify(String secret, String payload, String headerValue,
        long toleranceSeconds) {
        // t= ve v1= ayrıştırılır
        long age = Math.abs(Instant.now().getEpochSecond() - ts);
        if (age > toleranceSeconds) return false;

        byte[] expected = hmac(secret, ts + "." + payload);
        byte[] actual = HEX.parseHex(v1);

        // Sabit zamanlı karşılaştırma
        return MessageDigest.isEqual(expected, actual);
    }
}
```

Zaman damgası neden imzanın içinde

Yalnız gövde imzalarsaydı, araya giren biri geçerli bir isteği kaydedip yarın tekrar gönderebilirdi — imza hâlâ geçerli olurdu. Buna **replay saldırısı** denir.

Zaman damgası imzanın içinde olduğu için değiştirilemez. Saldırgan `t=` değerini güncellemeye kalkarsa `v1` artık tutmaz. Alıcı da "beş dakikadan eskisini kabul etme" diyerek pencereyi kapatır.

Bunu bir testle kanıtıyoruz:

```

@Test
@DisplayName("Zaman damgası kurcalanırsa imza tutmaz")
void zaman_damgası_kurcalanırsa_reddedilir() {
    // Saldırgan bir saat önce yakaladığı isteği tekrar göndermek istiyor.
    // Eskidiği için tolerans kontrolüne takılacak; o yüzden t= günceller.
    String header = signer.sign(SECRET, PAYLOAD, Instant.now().minusSeconds(3600));
    String forged = header.replaceFirst("t=\\d+", "t=" + Instant.now().getEpochSecond());

    assertThat(signer.verify(SECRET, PAYLOAD, forged, 300)).isFalse();
}

```

`MessageDigest.isEqual` neden `String.equals` değil

`String.equals` ilk farklı baytta çıkar. Saldırgan, imzayı bayt bayt deneyerek cevabın **ne kadar sürdüğüne** bakabilir: doğru tahmin biraz daha uzun sürer. Buna **zamanlama saldırısı** denir.

`MessageDigest.isEqual` bütün baytları her zaman karşılaştırır; süre içeriğe bağlı değildir.

`v1` öneki

Yarın SHA-256'dan vazgeçmek gerekirse `v2` eklenip geçiş süresince iki imza birden gönderilebilir. Müşteriler kırılmaz. Bugün bir satır fazla; yarın tek çıkış yolu.

4.6 Konfigürasyon replikası

`EndpointConfigCache`, endpoint konfigürasyonunu Redis'te tutar:

```

@Component
public class EndpointConfigCache {

    private String epKey(UUID endpointId) { return "hr:ep:" + endpointId; }
    private String appKey(UUID appId) { return "hr:app:" + appId + ":eps"; }

    public void put(EndpointConfig cfg) {
        if (cfg.deleted()) { remove(cfg.applicationId(), cfg.endpointId()); return; }
        redis.opsForValue().set(epKey(cfg.endpointId()), mapper.writeValueAsString(cfg));
        redis.opsForSet().add(appKey(cfg.applicationId()), cfg.endpointId().toString());
    }

    /** Fan-out'un temeli: bir uygulamanın bütün endpoint'leri. */
    public List<EndpointConfig> listByApplication(UUID applicationId) { /* ... */ }
}

```

İki anahtar deseni var: endpoint'in kendisi ve uygulamaya ait endpoint kimlikleri kümesi. İkincisi fan-out için — "bu uygulamanın hangi endpoint'leri var" sorusu bir `SMEMBERS` ile cevaplanıyor.

Bu bir cache değil, replika. Cache'in TTL'i olur ve kaçırdığı güncelleme olabilir. Bu veri compacted bir Kafka topic'inden besleniyor; kaçırılan güncelleme yok, invalidation problemi yok. Altıncı bölümde bunu kuracağız.

4.7 Modülün tamamı

Buraya kadar Lua betiklerini ve imzalamanın mantığını gördük. Şimdi `common` modülünün bütün dosyaları, yazılma sırasıyla. Bu sıra önemli: her dosya yalnızca kendinden öncekilere bağımlı,

dolayısıyla her adımda derlenir.

4.7.1 `libs/common/pom.xml`

`common` Spring'e bağımlı: web, Redis, Kafka, doğrulama. `contracts`'ı içeri alıyor ama hiçbir servise bağımlı değil — bağımlılık yönü tek yönlü.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
    maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../pom.xml</relativePath>
  </parent>
  <artifactId>common</artifactId>
  <name>common</name>
  <description>İmzalama, Redis ilkelleri (hiz siniri, devre kesici, idempotency), hata sozle
    smesi</description>

  <dependencies>
    <dependency>
      <groupId>dev.hookrelay</groupId>
      <artifactId>contracts</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-redis</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.kafka</groupId>
      <artifactId>spring-kafka</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-core</artifactId>
    </dependency>
  </dependencies>
</project>
```

4.7.2 `libs/common/src/main/java/dev/hookrelay/common/crypto/ApiKeys.java`

Anahtar üretimi ve hash'leme. Sınıf `final`, kurucu `private`: bu bir yardımcı, örneği alınmaz. 24 bayt rastgele → 48 karakter hex → `hr_` önekiyle 51 karakter.

```
package dev.hookrelay.common.crypto;
```

```

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.HexFormat;

/**
 * API anahtarı üretimi ve hash'lenmesi.
 *
 * Anahtarın kendisi HICBİR YERDE saklanmaz - ne veritabanında, ne Kafka'da,
 * ne logda. Sadece SHA-256 hash'i saklanır. Kullanıcıya düz metin bir kez,
 * oluşturma cevabında gösterilir. Kaybederse yenisini üretiriz.
 *
 * Neden bcrypt değil SHA-256: bu bir kullanıcı parolası değil, 256 bitlik
 * rastgele bir dize. Sözlük saldırısı yapılamayacağı için yavaslatmaya
 * (bcrypt/argon2) gerek yok; her istekte doğrulanacağı için de hızlı olmalı.
 */
public final class ApiKeys {

    private static final SecureRandom RANDOM = new SecureRandom();
    private static final HexFormat HEX = HexFormat.of();
    private static final String PREFIX = "hr_";

    private ApiKeys() {}

    public static String generate() {
        byte[] raw = new byte[24];
        RANDOM.nextBytes(raw);
        return PREFIX + HEX.formatHex(raw);
    }

    public static String hash(String apiKey) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            return HEX.formatHex(md.digest(apiKey.getBytes(StandardCharsets.UTF_8)));
        } catch (Exception e) {
            throw new IllegalStateException("API anahtarı hash'lenemedi", e);
        }
    }

    /** Arayuzda "hr_3f9a..." diye gösterebilmek için. Tek basına ise yaramaz. */
    public static String preview(String apiKey) {
        return apiKey.length() <= 10 ? apiKey : apiKey.substring(0, 10) + "...";
    }
}

```

4.7.3 `libs/common/src/main/java/dev/hookrelay/common/crypto/WebhookSigner`

İmzalamanın tam hali. `sign` ve `verify` aynı sınıfta duruyor çünkü ikisi de aynı sözleşmeyi biliyor; ayırsaydık biri değişince diğeri sessizce uyumsuz kalırdı.

```

package dev.hookrelay.common.crypto;

import org.springframework.stereotype.Component;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.time.Instant;

```



```

import java.util.HexFormat;

/**
 * Giden webhook'lari imzalar, gelenleri dogrular.
 *
 * Bicim (Stripe'in kullandigi sema):
 * imzalanan metin = "{timestamp}.{govde}"
 * baslik          = "t=1723459200,v1=<hex-hmac-sha256>"
 *
 * Zaman damgasi neden imzaya dahil:
 * Sadece govde imzalarsaydi, araya giren biri gecerli bir istegi kaydedip
 * yarin tekrar gonderebilirdi - imza hala gecerli olurdu. Zaman damgasi
 * imzanin icinde oldugu icin degistirilemez, alici da "5 dakikadan eskisini
 * kabul etme" diyerek replay penceresini kapatir.
 *
 * v1 oneki neden var: yarin SHA-256'dan vazgecmek gerekirse v2 eklenip
 * gecis suresince iki imza birden gonderilebilir. Musteriler kirilmaz.
 */
@Component
public class WebhookSigner {

    private static final String ALGO = "HmacSHA256";
    private static final HexFormat HEX = HexFormat.of();

    public String sign(String secret, String payload, Instant timestamp) {
        long ts = timestamp.getEpochSecond();
        String signed = ts + "." + payload;
        return "t=" + ts + ",v1=" + HEX.formatHex(hmac(secret, signed));
    }

    /**
     * Alici tarafın yapması gereken dogrulama. chaos-target bunu kullaniyor,
     * yani projede imzanin gercekten dogrulandigi calisir bir ornek var.
     */
    public boolean verify(String secret, String payload, String headerValue, long toleranceSeconds) {
        if (headerValue == null || headerValue.isBlank()) return false;

        long ts = -1;
        String v1 = null;
        for (String part : headerValue.split(",")) {
            String[] kv = part.split("=", 2);
            if (kv.length != 2) continue;
            switch (kv[0].trim()) {
                case "t" -> {
                    try { ts = Long.parseLong(kv[1].trim()); } catch (NumberFormatException ignored) { }
                }
                case "v1" -> v1 = kv[1].trim();
            }
        }
        if (ts < 0 || v1 == null) return false;

        long age = Math.abs(Instant.now().getEpochSecond() - ts);
        if (age > toleranceSeconds) return false;

        byte[] expected = hmac(secret, ts + "." + payload);
        byte[] actual;
        try {
            actual = HEX.parseHex(v1);
        } catch (IllegalArgumentException e) {

```

```

        return false;
    }
    // Sabit zamanli karsilastirma: String.equals erken cikar ve
    // baytlarin kacinin tuttugunu zamanlamayla sizdirir.
    return MessageDigest.isEqual(expected, actual);
}

private byte[] hmac(String secret, String data) {
    try {
        Mac mac = Mac.getInstance(ALGO);
        mac.init(new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), ALGO));
        return mac.doFinal(data.getBytes(StandardCharsets.UTF_8));
    } catch (Exception e) {
        throw new IllegalStateException("HMAC uretilemedi", e);
    }
}
}

```

4.7.4 `libs/common/src/main/java/dev/hookrelay/common/redis/RedisScripts.java`

Lua betiklerini bean olarak yükler. `DefaultRedisScript` SHA'yı bir kez hesaplar ve sonraki çağrılarda `EvalSHA` kullanır — betiğin gövdesi her istekte tel üzerinden gitmez.

Üç bean de aynı tipte (`RedisScript<List>`), o yüzden enjeksiyonda `@Qualifier` şart. Bean adı metod adıdır.

```

package dev.hookrelay.common.redis;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.data.redis.core.script.DefaultRedisScript;
import org.springframework.data.redis.core.script.RedisScript;

import java.util.List;

/**
 * Lua betiklerini bean olarak yukler.
 *
 * DefaultRedisScript SHA'yi bir kez hesaplar ve sonraki cagrilarda EVALSHA
 * kullanir; betigin govdesi her istekte tel uzerinden gitmez.
 */
@Configuration
public class RedisScripts {

    @Bean
    @SuppressWarnings("unchecked")
    public RedisScript<List> rateLimitScript() {
        DefaultRedisScript<List> s = new DefaultRedisScript<>();
        s.setLocation(new ClassPathResource("lua/rate_limit.lua"));
        s.setResultType(List.class);
        return s;
    }

    @Bean
    @SuppressWarnings("unchecked")
    public RedisScript<List> circuitBreakerScript() {
        DefaultRedisScript<List> s = new DefaultRedisScript<>();
        s.setLocation(new ClassPathResource("lua/circuit_breaker.lua"));
    }
}

```

```

        s.setResultType(List.class);
        return s;
    }

    @Bean
    @SuppressWarnings("unchecked")
    public RedisScript<List> idempotencyScript() {
        DefaultRedisScript<List> s = new DefaultRedisScript<>();
        s.setLocation(new ClassPathResource("lua/idempotency.lua"));
        s.setResultType(List.class);
        return s;
    }
}

```

4.7.5 `libs/common/src/main/java/dev/hookrelay/common/redis/RedisRateLimiter`

`rate_limit.lua`'nın Java tarafı. Kapasite bilinçli olarak `perSecond`'a eşit: bir saniyelik patlamaya izin verir, ortalamayı korur.

```

package dev.hookrelay.common.redis;

import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.RedisScript;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.UUID;

/** Endpoint bazlı token bucket. Betigin tamamı rate_limit.lua icinde. */
@Component
public class RedisRateLimiter {

    private final StringRedisTemplate redis;
    private final RedisScript<List> script;

    public RedisRateLimiter(StringRedisTemplate redis,
                           @org.springframework.beans.factory.annotation.Qualifier("rateLimitScript")
                           RedisScript<List> script) {
        this.redis = redis;
        this.script = script;
    }

    public record Decision(boolean allowed, long waitMillis) {}

    /**
     * @param perSecond saniyede kac teslimat. 0 = sinirsiz.
     * @return izin ve reddedildiyse ne kadar sonra tekrar denenebilecegi
     */
    @SuppressWarnings("unchecked")
    public Decision tryAcquire(UUID endpointId, int perSecond) {
        if (perSecond <= 0) return new Decision(true, 0);

        // Kapasite = 1 saniyelik hiz, en az 1. Kisa patlamalara izin verir
        // ama ortalama hizi perSecond'da tutar.
        int capacity = Math.max(1, perSecond);

        List<Long> r = redis.execute(script,
                                     List.of("hr:rl:" + endpointId),

```

```

        String.valueOf(capacity),
        String.valueOf(perSecond),
        String.valueOf(System.currentTimeMillis()),
        "1");

    if (r == null || r.size() < 2) return new Decision(true, 0);
    return new Decision(r.get(0) == 1L, r.get(1));
}
}

```

4.7.6 `libs/common/src/main/java/dev/hookrelay/common/redis/CircuitBreakerSet

Ayarlar ayrı bir üst seviye `record`. İç içe kayıt olarak yazılıydı `@ConfigurationProperties` taraması onu bulmakta kırılgan davranırdı.

Compact constructor'da varsayılan atanıyor: servis `application.yml`'inde hiçbir şey tanımlamasa da çalışır.

```

package dev.hookrelay.common.redis;

import org.springframework.boot.context.properties.ConfigurationProperties;

/**
 * Devre kesici ayarlari. Hepsinin makul varsayilani var - servis
 * application.yml'inde hic tanimlamasa da calisir.
 */
@ConfigurationProperties(prefix = "hookrelay.circuit-breaker")
public record CircuitBreakerSettings(
    Integer failureThreshold,
    Long openMillis,
    Integer halfOpenProbes,
    Long windowMillis
) {
    public CircuitBreakerSettings {
        if (failureThreshold == null || failureThreshold <= 0) failureThreshold = 5;
        if (openMillis == null || openMillis <= 0) openMillis = 30_000L;
        if (halfOpenProbes == null || halfOpenProbes <= 0) halfOpenProbes = 2;
        if (windowMillis == null || windowMillis <= 0) windowMillis = 60_000L;
    }
}

```

4.7.7 `libs/common/src/main/java/dev/hookrelay/common/redis/RedisCircuitBreak

`circuit_breaker.lua`'nın Java tarafı. Üç işlem — `allow`, `success`, `failure` — aynı betiğe farklı `op` argümanı ile gidiyor. Ayrı betikler yazsaydık durum okuma/yazma mantığı üç kez tekrarlanırdı.

`state()` metodu betiği çalıştırmadan sadece okuyor; arayüzde göstermek için.

```

package dev.hookrelay.common.redis;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.RedisScript;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.UUID;

```

```

/**
 * Endpoint bazlı, ORNEKLER ARASI PAYLASILAN devre kesici.
 *
 * Neden gerekli: bir musterinin sunucusu tamamen cektuyse ona 50.000 istek
 * daha atmak hem bizim is parcaciklarimizi hem onlari ayaga kalkma sansini
 * yakar. Devre acilınca teslimatlar HIC DENENMEDEN bir sonraki katmana
 * ertelenir - mesaj kaybolmaz, sadece bosa istek atılmaz.
 */
@Component
public class RedisCircuitBreaker {

    public enum State { CLOSED, OPEN, HALF_OPEN }

    public record Verdict(boolean allowed, State state, long failures) {}

    private final StringRedisTemplate redis;
    private final RedisScript<List> script;
    private final CircuitBreakerSettings settings;

    public RedisCircuitBreaker(StringRedisTemplate redis,
                              @Qualifier("circuitBreakerScript") RedisScript<List> script,
                              CircuitBreakerSettings settings) {
        this.redis = redis;
        this.script = script;
        this.settings = settings;
    }

    public Verdict allow(UUID endpointId) { return run(endpointId, "allow"); }
    public Verdict recordSuccess(UUID id) { return run(id, "success"); }
    public Verdict recordFailure(UUID id) { return run(id, "failure"); }

    public State state(UUID endpointId) {
        String s = (String) redis.opsForHash().get(key(endpointId), "state");
        return s == null ? State.CLOSED : State.valueOf(s);
    }

    @SuppressWarnings("unchecked")
    private Verdict run(UUID endpointId, String op) {
        List<Object> r = redis.execute(script,
            List.of(key(endpointId),
                op,
                String.valueOf(System.currentTimeMillis()),
                String.valueOf(settings.failureThreshold()),
                String.valueOf(settings.openMillis()),
                String.valueOf(settings.halfOpenProbes()),
                String.valueOf(settings.windowMillis())));

        if (r == null || r.size() < 3) return new Verdict(true, State.CLOSED, 0);
        return new Verdict(
            ((Number) r.get(0)).intValue() == 1,
            State.valueOf(String.valueOf(r.get(1))),
            ((Number) r.get(2)).longValue());
    }

    private String key(UUID endpointId) { return "hr:cb:" + endpointId; }
}

```

4.7.8 `libs/common/src/main/java/dev/hookrelay/common/redis/RedisIdempotency`

`reserveOrGet` rezervasyon ve okumayı tek adımda yapar. `put` ise iş bittikten sonra gerçek cevabı yerine koyar — arada kalan "PENDING" değeri, aynı anahtarla gelen ikinci isteğe "işlem sürüyor" der.

```
package dev.hookrelay.common.redis;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.data.redis.core.script.RedisScript;
import org.springframework.stereotype.Component;

import java.time.Duration;
import java.util.List;
import java.util.Optional;

/**
 * Idempotency-Key deposu.
 *
 * Kullanım: "bu anahtarı ilk kez mi görüyorum?" Cevap hayırsa daha önce
 * dondurulmuş cevabı aynen tekrar döndür - istemcinin gözünde işlem
 * bir kez olmuş gibi görünür.
 */
@Component
public class RedisIdempotencyStore {

    private final StringRedisTemplate redis;
    private final RedisScript<List> script;

    public RedisIdempotencyStore(StringRedisTemplate redis,
                                @Qualifier("idempotencyScript") RedisScript<List> script) {
        this.redis = redis;
        this.script = script;
    }

    /** Bos Optional = anahtar ilk kez görüldü ve rezerve edildi. */
    @SuppressWarnings("unchecked")
    public Optional<String> reserveOrGet(String scope, String key, String value, Duration ttl) {
        List<Object> r = redis.execute(script,
            List.of("hr:idem:" + scope + ":" + key),
            value,
            String.valueOf(ttl.toMillis()));

        if (r == null || r.size() < 2) return Optional.empty();
        boolean isNew = ((Number) r.get(0)).intValue() == 1;
        return isNew ? Optional.empty() : Optional.of(String.valueOf(r.get(1)));
    }

    /** Rezervasyondan sonra gerçek cevabı yerine koymak için. */
    public void put(String scope, String key, String value, Duration ttl) {
        redis.opsForValue().set("hr:idem:" + scope + ":" + key, value, ttl);
    }
}
```

4.7.9 `libs/common/src/main/java/dev/hookrelay/common/redis/EndpointConfigCa

İki anahtar deseni: endpoint'in kendisi (`hr:ep:{id}`) ve uygulamaya ait kimlikler kümesi (`hr:app:{id}:eps`). İkincisi fan-out için.

`listByApplication` içindeki `MGET` bir performans düzeltmesinin sonucu — ilk hali her endpoint

için ayrı **GET** yapıyordu ve kabul isteğinin sıcak yolunda 7 gidiş-dönüş demekti.

```
package dev.hookrelay.common.redis;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.contracts.EndpointConfig;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.UUID;

/**
 * Endpoint konfigürasyonunun yerel kopyası.
 *
 * Bu sınıfın varlık sebebi tek bir cümle: dispatcher bir mesajı gönderirken
 * admin-api'ye HTTP atmamalı. Atarsa saniyede binlerce istek control plane'e
 * yagar, admin-api çıktığında teslimat da durur ve iki servis birbirine
 * yeniden yapışır.
 *
 * Veriyi kim dolduruyor: EndpointConfigReplicator, compacted Kafka
 * topic'ini dinleyerek. Yani bu bir "cache" değil, bir REPLIKA -
 * kacırlan güncelleme yok, TTL yok, invalidation problemi yok.
 */
@Component
public class EndpointConfigCache {

    private static final Logger log = LoggerFactory.getLogger(EndpointConfigCache.class);

    private final StringRedisTemplate redis;
    private final ObjectMapper mapper;

    public EndpointConfigCache(StringRedisTemplate redis, ObjectMapper mapper) {
        this.redis = redis;
        this.mapper = mapper;
    }

    private String epKey(UUID endpointId) { return "hr:ep:" + endpointId; }
    private String appKey(UUID appId) { return "hr:app:" + appId + ":eps"; }

    public void put(EndpointConfig cfg) {
        try {
            if (cfg.deleted()) {
                remove(cfg.applicationId(), cfg.endpointId());
                return;
            }
            redis.opsForValue().set(epKey(cfg.endpointId()), mapper.writeValueAsString(cfg));
            ;
            redis.opsForSet().add(appKey(cfg.applicationId()), cfg.endpointId().toString());
        } catch (Exception e) {
            log.error("Endpoint konfigürasyonu yazılamadı: {}", cfg.endpointId(), e);
        }
    }

    public void remove(UUID applicationId, UUID endpointId) {
        redis.delete(epKey(endpointId));
    }
}
```

```

        redis.opsForSet().remove(appKey(applicationId), endpointId.toString());
    }

    public Optional<EndpointConfig> get(UUID endpointId) {
        String json = redis.opsForValue().get(epKey(endpointId));
        if (json == null) return Optional.empty();
        try {
            return Optional.of(mapper.readValue(json, EndpointConfig.class));
        } catch (Exception e) {
            log.error("Endpoint konfigurasyonu okunamadi: {}", endpointId, e);
            return Optional.empty();
        }
    }

    /**
     * Fan-out'un temeli: bir uygulamanin butun endpoint'leri.
     *
     * TEK MGET, N AYRI GET DEGIL
     * İlk yazilisi kume uyelerini alip her biri icin ayri GET yapiyordu.
     * 6 endpoint = 7 Redis gidis-donusu, ve bu KABUL ISTEGININ SICAK
     * YOLUNDA. Her gidis-donus ~0,2 ms olsa bile 100 olay/sn'de saniyede
     * 700 gidis-donus demek.
     *
     * MGET hepsini tek komutta getirir: 2 gidis-donus (SMEMBERS + MGET),
     * endpoint sayisindan bagimsiz.
     *
     * Bu, N+1 probleminin Redis'teki hali. Veritabaninda herkesin bildigi
     * bu tuzak, onbellekte cogu zaman gozden kaciyor.
     */
    public List<EndpointConfig> listByApplication(UUID applicationId) {
        Set<String> ids = redis.opsForSet().members(appKey(applicationId));
        if (ids == null || ids.isEmpty()) return List.of();

        List<String> keys = ids.stream().map(id -> epKey(UUID.fromString(id))).toList();
        List<String> payloads = redis.opsForValue().multiGet(keys);
        if (payloads == null) return List.of();

        List<EndpointConfig> out = new ArrayList<>(payloads.size());
        for (String json : payloads) {
            if (json == null) continue; // kume ile deger arasinda yaris - atla
            try {
                out.add(mapper.readValue(json, EndpointConfig.class));
            } catch (Exception e) {
                log.error("Endpoint konfigurasyonu ayristirilamadi", e);
            }
        }
        return out;
    }
}

```

4.7.10 `libs/common/src/main/java/dev/hookrelay/common/redis/AppConfigCache

Aynı desen, uygulamalar için. Ek olarak bir **ters indeks** var: `hr:apikey:{hash}` → `applicationId`. API anahtarı doğrulaması bu sayede tek `GET` ile oluyor.

```

package dev.hookrelay.common.redis;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.contracts.ApplicationConfig;

```



```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Component;

import java.util.Optional;
import java.util.UUID;

/** Uygulama (musteri) konfigürasyonunun yerel replikası. Bkz. EndpointConfigCache. */
@Component
public class AppConfigCache {

    private static final Logger log = LoggerFactory.getLogger(AppConfigCache.class);

    private final StringRedisTemplate redis;
    private final ObjectMapper mapper;

    public AppConfigCache(StringRedisTemplate redis, ObjectMapper mapper) {
        this.redis = redis;
        this.mapper = mapper;
    }

    private String key(UUID id) { return "hr:app:" + id + ":cfg"; }
    private String byKeyHash(String hash) { return "hr:apikey:" + hash; }

    public void put(ApplicationConfig cfg) {
        try {
            if (cfg.deleted()) {
                remove(cfg);
                return;
            }
            redis.opsForValue().set(key(cfg.applicationId()), mapper.writeValueAsString(cfg));
            // Ters indeks: API anahtarından uygulamaya O(1). ingest'in sıcak yolu bu.
            redis.opsForValue().set(byKeyHash(cfg.apiKeyHash()), cfg.applicationId().toString());
        } catch (Exception e) {
            log.error("Uygulama konfigürasyonu yazılamadı: {}", cfg.applicationId(), e);
        }
    }

    public void remove(ApplicationConfig cfg) {
        redis.delete(key(cfg.applicationId()));
        if (cfg.apiKeyHash() != null) redis.delete(byKeyHash(cfg.apiKeyHash()));
    }

    public Optional<ApplicationConfig> get(UUID id) {
        String json = redis.opsForValue().get(key(id));
        if (json == null) return Optional.empty();
        try {
            return Optional.of(mapper.readValue(json, ApplicationConfig.class));
        } catch (Exception e) {
            return Optional.empty();
        }
    }

    /** API anahtarının SHA-256 hash'inden uygulamayı bul. */
    public Optional<ApplicationConfig> findByApiKeyHash(String hash) {
        String id = redis.opsForValue().get(byKeyHash(hash));
        if (id == null) return Optional.empty();
        return get(UUID.fromString(id));
    }
}

```

```

    }
}

```

4.7.11 `libs/common/src/main/java/dev/hookrelay/common/kafka/KafkaHeaderCodec

Kafka başlıkları ham `byte[]`. Her yerde aynı dönüşümü elle yazmamak için küçük bir yardımcı. `longValue` hatalı veriyi yutup varsayılan dönüyor — bozuk bir başlık yüzünden tüketici çökmemeli.

```

package dev.hookrelay.common.kafka;

import org.apache.kafka.common.header.Header;
import org.apache.kafka.common.header.Headers;

import java.nio.charset.StandardCharsets;

/**
 * Kafka basliklari ham byte[]. Her yerde ayni donusumu elle yazmamak icin.
 */
public final class KafkaHeaderCodec {

    private KafkaHeaderCodec() {}

    public static byte[] of(long value) {
        return String.valueOf(value).getBytes(StandardCharsets.UTF_8);
    }

    public static byte[] of(String value) {
        return value == null ? new byte[0] : value.getBytes(StandardCharsets.UTF_8);
    }

    public static String string(Headers headers, String key, String fallback) {
        Header h = headers.lastHeader(key);
        if (h == null || h.value() == null) return fallback;
        return new String(h.value(), StandardCharsets.UTF_8);
    }

    public static long longValue(Headers headers, String key, long fallback) {
        String s = string(headers, key, null);
        if (s == null) return fallback;
        try {
            return Long.parseLong(s.trim());
        } catch (NumberFormatException e) {
            return fallback;
        }
    }

    public static int intValue(Headers headers, String key, int fallback) {
        return (int) longValue(headers, key, fallback);
    }
}

```

4.7.12 `libs/common/src/main/java/dev/hookrelay/common/kafka/EndpointConfig

Bölüm 6.2'de tasarımını tartışmıştık; tam hali burada. `@ConditionalOnProperty` ile kapalı geliyor — yalnızca ihtiyacı olan servisler (`ingest`, `dispatcher`) `replicate: true` diyerek açıyor.

```

package dev.hookrelay.common.kafka;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.redis.EndpointConfigCache;
import dev.hookrelay.contracts.EndpointConfig;
import dev.hookrelay.contracts.Topics;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.common.TopicPartition;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.listener.ConsumerSeekAware;
import org.springframework.stereotype.Component;

import java.util.Map;

/**
 * Compacted konfigürasyon topic'ini Redis replikasına akitir.
 *
 * İki alisilmadık tarafı var, ikisi de kasıtlı:
 *
 * groupId'de rastgele UUID -- her örnek kendi grubunda. Bu bir iş kuyruğu
 * değil durum tablosu; her dispatcher örneğinin bütün endpoint'leri
 * bilmesi gerekiyor. Ortak grup olsaydı partition'lar paylaştırılır ve
 * her örnek endpoint'lerin uçte birini görürdü.
 *
 * seekToBeginning -- her acilista bastan okunur. Topic compacted olduğu
 * için bu, geçmişi değil güncel tabloyu okumak demek. Offset saklamaya
 * gerek yok: durumun kaynağı topic'in kendisi.
 *
 * @TopicPartition ile elle atama da mümkundu ama partition numaralarını
 * anotasyona yazmayı gerektiriyor; sayı değişince sessizce eksik okunur.
 */
@Component
@ConditionalOnProperty(name = "hookrelay.endpoint-config.replicate", havingValue = "true")
public class EndpointConfigReplicator implements ConsumerSeekAware {

    private static final Logger log = LoggerFactory.getLogger(EndpointConfigReplicator.class);

    private final EndpointConfigCache cache;
    private final ObjectMapper mapper;

    public EndpointConfigReplicator(EndpointConfigCache cache, ObjectMapper mapper) {
        this.cache = cache;
        this.mapper = mapper;
    }

    @KafkaListener(
        topics = Topics.ENDPOINT_CONFIG,
        groupId = "endpoint-config-#{T(java.util.UUID).randomUUID().toString()}"
    )
    public void onConfig(ConsumerRecord<String, String> record) {
        try {
            // Mezar taşı (tombstone): compacted topic'te null gövde silme demek.
            // Biz silmeyi deleted=true ile yapıyoruz ama başka bir araç
            // tombstone basarsa da çokmeyelim.
            if (record.value() == null || record.value().isBlank()) {
                log.debug("Mezar taşı alındı: {}", record.key());
                return;
            }
        }
    }

```

```

    }
    EndpointConfig cfg = mapper.readValue(record.value(), EndpointConfig.class);
    cache.put(cfg);
    log.debug("Endpoint replikasi guncellendi: {} v{}", cfg.endpointId(), cfg.version());
} catch (Exception e) {
    // Tek bir bozuk kayıt bütün replikasyonu durdurmamalı.
    log.error("Konfigurasyon kaydi islenemedi: key={}", record.key(), e);
}
}

@Override
public void onPartitionsAssigned(Map<TopicPartition, Long> assignments,
    ConsumerSeekCallback callback) {
    if (assignments.isEmpty()) return;
    callback.seekToBeginning(assignments.keySet());
    log.info("Endpoint konfigurasyonu bastan okunuyor: {} partition", assignments.size());
}
}

```

4.7.13 `libs/common/src/main/java/dev/hookrelay/common/kafka/AppConfigReplicator`

Aynı kalıp, uygulamalar için. İki sınıfı tek bir jenerik sınıfa indirmek mümkündür ama `@KafkaListener` anotasyonundaki topic adı sabit olmak zorunda — jenerikleştirme, anotasyonu kaybettirip elle tüketici kurmayı gerektirirdi. Kopya, buradaki karmaşadan ucuz.

```

package dev.hookrelay.common.kafka;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.redis.AppConfigCache;
import dev.hookrelay.contracts.ApplicationConfig;
import dev.hookrelay.contracts.Topics;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.common.TopicPartition;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.listener.ConsumerSeekAware;
import org.springframework.stereotype.Component;

import java.util.Map;

/**
 * app-config compacted topic'ini Redis replikasına akitir.
 * Tasarım gerekçeleri için bkz. {@link EndpointConfigReplicator}.
 */
@Component
@ConditionalOnProperty(name = "hookrelay.app-config.replicate", havingValue = "true")
public class AppConfigReplicator implements ConsumerSeekAware {

    private static final Logger log = LoggerFactory.getLogger(AppConfigReplicator.class);

    private final AppConfigCache cache;
    private final ObjectMapper mapper;

    public AppConfigReplicator(AppConfigCache cache, ObjectMapper mapper) {
        this.cache = cache;
    }

```

```

        this.mapper = mapper;
    }

    @KafkaListener(
        topics = Topics.APP_CONFIG,
        groupId = "app-config-#{T(java.util.UUID).randomUUID().toString()}")
    public void onConfig(ConsumerRecord<String, String> record) {
        try {
            if (record.value() == null || record.value().isBlank()) return;
            ApplicationConfig cfg = mapper.readValue(record.value(), ApplicationConfig.class);
            cache.put(cfg);
            log.debug("Uygulama replikasi guncellendi: {} v{}", cfg.applicationId(), cfg.version());
        } catch (Exception e) {
            log.error("Uygulama konfigurasyon kaydi islenemedi: key={}", record.key(), e);
        }
    }

    @Override
    public void onPartitionsAssigned(Map<TopicPartition, Long> assignments,
                                     ConsumerSeekCallback callback) {
        if (assignments.isEmpty()) return;
        callback.seekToBeginning(assignments.keySet());
        log.info("Uygulama konfigurasyonu bastan okunuyor: {} partition", assignments.size());
    }
}

```

4.7.14 `libs/common/src/main/java/dev/hookrelay/common/error/ApiException.java`

Kontrollü hata. Statik fabrikalar (`notFound`, `badRequest`, ...) çağrı yerinde HTTP kodunu düşünmeyi gereksiz kılıyor.

```

package dev.hookrelay.common.error;

import org.springframework.http.HttpStatus;

/** Kontrollü hata. GlobalExceptionHandler bunu düz bir cevaba çevirir. */
public class ApiException extends RuntimeException {

    private final HttpStatus status;
    private final String code;

    public ApiException(HttpStatus status, String code, String message) {
        super(message);
        this.status = status;
        this.code = code;
    }

    public static ApiException notFound(String what, Object id) {
        return new ApiException(HttpStatus.NOT_FOUND, "NOT_FOUND", what + " bulunamadi: " + id);
    }

    public static ApiException badRequest(String message) {
        return new ApiException(HttpStatus.BAD_REQUEST, "BAD_REQUEST", message);
    }
}

```

```

    public static ApiException unauthorized(String message) {
        return new ApiException(HttpStatus.UNAUTHORIZED, "UNAUTHORIZED", message);
    }

    public static ApiException conflict(String message) {
        return new ApiException(HttpStatus.CONFLICT, "CONFLICT", message);
    }

    public HttpStatus status() { return status; }
    public String code() { return code; }
}

```

4.7.15 `libs/common/src/main/java/dev/hookrelay/common/error/ErrorResponse.java`

Tek hata sözleşmesi. Bütün servisler aynı gövdeyi döner, istemci tarafında tek bir ayrıştırıcı yeter.

```

package dev.hookrelay.common.error;

import java.time.Instant;
import java.util.List;

/**
 * Tek hata sozlesmesi. Butun servisler ayni govdeyi doner, istemci
 * tarafinda tek bir hata ayristirici yeter.
 */
public record ErrorResponse(
    String code,
    String message,
    List<String> details,
    String path,
    Instant timestamp
) {
    public static ErrorResponse of(String code, String message, String path) {
        return new ErrorResponse(code, message, List.of(), path, Instant.now());
    }

    public static ErrorResponse of(String code, String message, List<String> details, String
        path) {
        return new ErrorResponse(code, message, details, path, Instant.now());
    }
}

```

4.7.16 `libs/common/src/main/java/dev/hookrelay/common/error/GlobalExceptionHandler.java`

Üç seviye: kontrollü hata, doğrulama hatası, beklenmeyen.

Sonuncusunda istisnanın detayı **istemciye gitmez**, loga gider. Yığın izini cevap gövdesinde döndürmek, saldırgana sınıf adlarını ve kütüphane sürümlerini hediye etmektir.

```

package dev.hookrelay.common.error;

import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

```

```

import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.List;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(ApiException.class)
    public ResponseEntity<ErrorResponse> handleApi(ApiException e, HttpServletRequest req) {
        return ResponseEntity.status(e.status())
            .body(ErrorResponse.of(e.code(), e.getMessage(), req.getRequestURI()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException e,
        HttpServletRequest req) {
        List<String> details = e.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .toList();
        return ResponseEntity.badRequest()
            .body(ErrorResponse.of("VALIDATION_FAILED", "Istek dogrulanamadi", details,
                req.getRequestURI()));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleUnexpected(Exception e, HttpServletRequest req) {
        // Beklenmeyen hatanın detayı İSTEMCIYE gitmez, loga gider.
        log.error("Beklenmeyen hata: {} {}", req.getMethod(), req.getRequestURI(), e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ErrorResponse.of("INTERNAL_ERROR", "Beklenmeyen bir hata olustu",
                req.getRequestURI()));
    }
}

```

Kontrol noktası

```
mvn -q test -pl libs/common
```

Sekiz imzalama testi geçmeli.

Bölüm 5 — Transactional Outbox

Bu bölüm, projedeki en önemli tek desenin bölümü.

5.1 Bozuk kod

Bu kod, görünüşe rağmen bozuk:

```
@Transactional
void kaydet(Mesaj m) {
    repo.save(m);           // (1)
    kafka.send("konu", m);  // (2)
}
```

Üç senaryo:

(1) ile (2) arasında süreç ölürse. Transaction geri sarılır. Veritabanında mesaj yok, Kafka'da da yok. Sorun yok — bu tek iyi senaryo.

(2) çalıştı, commit'ten önce süreç öldü. Kafka'da mesaj **var**, veritabanında **yok**. Tüketici, var olmayan bir kaydı işlemeye çalışır. Hayalet teslimat.

(2) broker'a ulaşamadı ama istisna atmadı (asenكرون gönderim). Veritabanında mesaj var, Kafka'da yok. Mesaj sonsuza kadar kayıp — ve hiç kimse fark etmez.

Kök sebep basit: **iki ayrı sistem tek bir transaction'a giremez**. XA/2PC diye bir mekanizma var ama Kafka'da pratikte kullanılmaz — yavaştır, koordinatör tek hata noktasıdır, kilitlenir.

5.2 Çözüm

Kafka'ya gidecek kaydı, **aynı veritabanı transaction'ında** bir tabloya yazın. Tek transaction, tek sistem → atomik. Ayrı bir süreç tabloyu tarayıp Kafka'ya basar.

<u>İş transaction'ı</u>		<u>Ayrı süreç</u>
BEGIN		
INSERT message		her 200 ms:
INSERT outbox_event	→	SELECT ... FOR UPDATE SKIP LOCKED
COMMIT		kafka.send(...)
(atomik)		UPDATE published_at

Bedeli: mesaj **en az bir kez** gider. Basma ile işaretleme arasında ölürsek tekrar basarız. Bu yüzden tüketici idempotent olmak **zorunda**.

Dağıtık sistemde "exactly-once" diye bir şey yoktur. Olan şey: en-az-bir-kez teslimat + idempotent tüketici. Bu ikisi birlikte, dışarıdan bakınca exactly-once gibi *görünür*.

5.3 Tablo

```
CREATE TABLE outbox_event (
    id          UUID PRIMARY KEY,
    aggregate_type VARCHAR(64) NOT NULL,
```



```

    aggregate_id    VARCHAR(64) NOT NULL,
    topic           VARCHAR(128) NOT NULL,
    msg_key         VARCHAR(128),
    payload         TEXT NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL,
    published_at    TIMESTAMPTZ,
    attempts        INT NOT NULL DEFAULT 0,
    last_error      VARCHAR(1000)
);

CREATE INDEX idx_outbox_pending
ON outbox_event (created_at, id)
WHERE published_at IS NULL;

```

Kısmi indeks neden

Tablo zamanla çoğunlukla *yayınlanmış* kayıtlardan oluşur. Poller'ın tek sorgusu ise sadece yayınlanmamışlara bakar. Bütün tabloyu indekslemek hem yer harcar hem her `INSERT`'te gereksiz indeks yazması yapar.

`WHERE published_at IS NULL` ile indeks yalnızca bekleyen kayıtları içerir — genellikle birkaç yüz satır. Sorgu her zaman hızlı kalır, tablo ne kadar büyürse büyüsün.

5.4 `FOR UPDATE SKIP LOCKED`

```

@Query(value = """
    SELECT * FROM outbox_event
    WHERE published_at IS NULL
    ORDER BY created_at, id
    LIMIT :limit
    FOR UPDATE SKIP LOCKED
    """, nativeQuery = true)
List<OutboxEvent> lockUnpublished(@Param("limit") int limit);

```

`FOR UPDATE` satırları kilitler. `SKIP LOCKED` ise "başkası kilitlemişse **bekleme**, atla" der.

Bu ikisi olmadan iki poller örneği aynı satırlarda birbirini bekler ve iki örnek, tek örnek hızında çalışır. `SKIP LOCKED` ile her örnek farklı satırlar alır ve gerçekten paralelleşir.

Postgres'e özgü değil: MySQL 8+ ve Oracle da destekler. SQL standardında yok.

5.5 Poller

```

@Scheduled(fixedDelayString = "${hookrelay.outbox.poll-interval-ms:200}")
@Transactional
public void drain() {
    List<OutboxEvent> batch = repository.lockUnpublished(batchSize);
    if (batch.isEmpty()) return;

    for (OutboxEvent event : batch) {
        try {
            kafka.send(new ProducerRecord<>(event.getTopic(), event.getMsgKey(),
                event.getPayload()))
                .get(sendTimeout.toMillis(), TimeUnit.MILLISECONDS);
        }
    }
}

```

```

        event.markPublished();
    } catch (Exception e) {
        event.markFailed(e.getMessage());
        log.error("Outbox kaydı yayınlanamadı: id={}", event.getId(), e);
        break; // sırayı bozmamak için dur
    }
}
repository.saveAll(batch);
}

```

`.get()` neden — asenkron göndermek yetmez mi

Yetmez. `send()` asenkrondur ve hemen döner. Hemen `markPublished()` deseydik, broker kaydı reddettiğinde bunu asla öğrenemez ve kaydı "yayınlandı" diye işaretlemiş olurduk. Kayıp mesaj. `.get(timeout)` broker'ın kabul ettiğini teyit eder. Yavaşlatır — ama outbox'ın var olma sebebi hız değil, **kayıpsızlık**.

`.fixedDelay` neden, `.fixedRate` değil

`fixedRate` sabit aralıklarla tetikler; bir tur 300 ms sürerse ve aralık 200 ms ise turlar üst üste biner. `fixedDelay` bir tur bittikten sonra sayar.

Hata durumunda `.break`, `.throw` değil

İstisna atsaydık transaction geri sarılır ve o turda başarıyla gönderilmiş kayıtlar da "yayınlanmadı" olarak kalırdı — sonraki turda tekrar gönderilirlerdi. `break` ile önceki başarılar işaretli kalır, sorunlu kayıt sonraki turda tekrar denenir.

5.6 Sıra garantisi üzerine dürüst not

Tek poller örneği çalışırken sıra korunur: `ORDER BY created_at` ile okuyup aynı sırayla basarız, idempotent üretici partition içinde de bu sırayı korur.

İki örnek aynı anda çalışırsa `SKIP LOCKED` ikisine farklı satırlar verir ve A örneğindeki eski kayıt, B örneğindeki yeni kayıttan **sonra** basılabilir. Yani "aynı endpoint'e sıralı teslimat" garantisi kırılır.

Çözümler:

Yaklaşım	Nasıl	Ne zaman
Tek poller	<code>ingest</code> yatay ölçeklenir, poller ölçeklenmez	Bizim seçimimiz
Bölümleme	Outbox'ı key hash'ine göre böl, her örneğe bir dilim	Poller darboğaz olunca
Debezium	WAL'dan doğrudan oku	Çok yüksek hacim

Ölçülen kapasite: tek poller ~5.000 kayıt/sn. Hedeflerimizin çok üstünde.

Bunu yazmamızın sebebi: bir kısıtın *bilinmesi*, *seçilmiş* olması demektir. Yazılmayan kısıt, unutulmuş kısıttır ve altı ay sonra kimse neden orada olduğunu bilemez.

5.7 `Propagation.MANDATORY` — sessiz hatayı gürültülü yapmak

```
@Transactional(propagation = Propagation.MANDATORY)
public void record(String aggregateType, String aggregateId,
                  String topic, String msgKey, Object payload) {
    repository.save(new OutboxEvent(aggregateType, aggregateId, topic, msgKey, json));
}
```

MANDATORY: bu metot **çağırının transaction'ı içinde** çalışmak zorunda. Transaction yoksa istisna atar.

Neden: outbox'ın tek amacı, kaydın iş verisiyle **aynı** transaction'da yazılması. Transaction dışında çağılırsa desen anlamını tamamen kaybeder — ve bu, hiçbir hata vermeden olur.

MANDATORY sessiz hatayı gürültülü hataya çevirir.

5.8 Modülün tamamı

5.8.1 `libs/outbox/pom.xml`

JPA ve Kafka'ya bağımlı. Bu yüzden ayrı bir modül: **gateway** ve **chaos-target** veritabanı kullanmıyor, **common**'a JPA koysaydık onlar da taşımak zorunda kalır ve açılışta boş bir **DataSource** arayıp çökerlerdi.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../../pom.xml</relativePath>
  </parent>
  <artifactId>outbox</artifactId>
  <name>outbox</name>
  <description>Transactional Outbox – veritabanı commit'i ile Kafka yayını atomik yapar</description>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.kafka</groupId>
      <artifactId>spring-kafka</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-core</artifactId>
    </dependency>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
    </dependency>
  </dependencies>
</project>
```

5.8.2 `libs/outbox/src/main/java/dev/hookrelay/outbox/OutboxEvent.java`

Kimlik uygulama tarafında üretiliyor (`UUID.randomUUID()`), veritabanı sırasından değil. Sebebi: kaydı oluşturan kod, id'yi commit'ten önce bilmek zorunda — loglayabilmek ve gerekirse ilişkilendirebilmek için.

`markFailed` hata mesajını 1000 karaktere kırpıyor. Kırpmasaydı, uzun bir yığın izi sütunu taşırır ve **outbox yazımı** patlardı — yani hata kaydetmeye çalışırken yeni bir hata üretirdik.

```
package dev.hookrelay.outbox;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.UUID;

/**
 * Yayınlanmayı bekleyen Kafka kaydının veritabanındaki hali.
 *
 * "repo.save(m); kafka.send(m);" ikilisi atomik degildir: arada surec
 * olurse ya mesaj kaybolur ya hayalet teslimat olur. Iki ayri sistem tek
 * transaction'a giremez (XA/2PC pratikte Kafka ile kullanilmaz).
 *
 * Cozum: Kafka'ya gidecek kaydi ayni transaction'da bu tabloya yaz. Ayri
 * bir surec tarayip basar ve yayinlandi diye isaretler.
 *
 * Bedeli: mesaj en az bir kez gider, dolayisiyla tuketici idempotent
 * olmak zorunda.
 */
@Entity
@Table(name = "outbox_event")
public class OutboxEvent {

    @Id
    private UUID id;

    @Column(name = "aggregate_type", nullable = false, length = 64)
    private String aggregateType;

    @Column(name = "aggregate_id", nullable = false, length = 64)
    private String aggregateId;

    @Column(name = "topic", nullable = false, length = 128)
    private String topic;

    /** Kafka partition anahtari. null olabilir ama bizde hep dolu -- sira garantisi buna ba
        gli. */
    @Column(name = "msg_key", length = 128)
    private String msgKey;

    @Column(name = "payload", nullable = false, columnDefinition = "text")
    private String payload;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    /** null = henuz yayinlanmadi. Poller bu sutuna bakiyor. */
    @Column(name = "published_at")
    private Instant publishedAt;
}
```

```

@Column(name = "attempts", nullable = false)
private int attempts;

@Column(name = "last_error", length = 1000)
private String lastError;

protected OutboxEvent() {}

public OutboxEvent(String aggregateType, String aggregateId, String topic,
                    String msgKey, String payload) {
    this.id = UUID.randomUUID();
    this.aggregateType = aggregateType;
    this.aggregateId = aggregateId;
    this.topic = topic;
    this.msgKey = msgKey;
    this.payload = payload;
    this.createdAt = Instant.now();
    this.attempts = 0;
}

public void markPublished() {
    this.publishedAt = Instant.now();
    this.lastError = null;
}

public void markFailed(String error) {
    this.attempts++;
    this.lastError = error == null ? null
        : error.substring(0, Math.min(error.length(), 1000));
}

public UUID getId() { return id; }
public String getTopic() { return topic; }
public String getMsgKey() { return msgKey; }
public String getPayload() { return payload; }
public Instant getCreatedAt() { return createdAt; }
public Instant getPublishedAt() { return publishedAt; }
public int getAttempts() { return attempts; }
public String getAggregateId() { return aggregateId; }
}

```

5.8.3 `libs/outbox/src/main/java/dev/hookrelay/outbox/OutboxRepository.java`

Üç sorgu: kilitleyerek okuma, bekleyen sayımı, temizlik.

`lockUnpublished` native çünkü `FOR UPDATE SKIP LOCKED` JPQL'de yok. `deletePublishedBefore` ise JPQL — orada özel bir şey gerekmiyor.

```

package dev.hookrelay.outbox;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.time.Instant;
import java.util.List;
import java.util.UUID;

public interface OutboxRepository extends JpaRepository<OutboxEvent, UUID> {

```

```

/**
 * Yayinlanmamis kayitlari SIRAYLA kilitleyerek al.
 *
 * FOR UPDATE SKIP LOCKED: baska bir poller ornegi ayni satirlari
 * kilitlemisse bekleme, atla ve sonrakini al. Bu olmadan iki ornek
 * ayni satirda birbirini bekler ve tek ornek hizinda calisirsiniz.
 *
 * ORDER BY created_at, id: ayni endpoint'e giden olaylari sirasi
 * korunsun diye. (Tek poller ornegi icin yeterli. Cok ornekte sıra
 * ornekler ARASINDA bozulabilir - bkz. OutboxPublisher javadoc.)
 */
@Query(value = """
    SELECT * FROM outbox_event
    WHERE published_at IS NULL
    ORDER BY created_at, id
    LIMIT :limit
    FOR UPDATE SKIP LOCKED
    """, nativeQuery = true)
List<OutboxEvent> lockUnpublished(@Param("limit") int limit);

long countByPublishedAtIsNull();

@Query("DELETE FROM OutboxEvent o WHERE o.publishedAt IS NOT NULL AND o.publishedAt < :before")
@org.springframework.data.jpa.repository.Modifying
int deletePublishedBefore(@Param("before") Instant before);
}

```

5.8.4 `libs/outbox/src/main/java/dev/hookrelay/outbox/OutboxRecorder.java`

İş mantığının çağırdığı tek yer. `Propagation.MANDATORY` bilinçli: transaction yoksa patlar. 5.7'de anlattığımız erken uyarı mekanizması.

```

package dev.hookrelay.outbox;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

/**
 * İş mantığının çağırdığı tek yer.
 *
 * MANDATORY yayılımı bilinçli: bu metod CAGIRANIN transaction'i içinde
 * çalışmak ZORUNDA. Transaction dışında çağırırsa hata fırlatır -
 * çünkü transaction yoksa outbox'ın tüm anlamı kaybolur.
 */
@Component
public class OutboxRecorder {

    private final OutboxRepository repository;
    private final ObjectMapper mapper;

    public OutboxRecorder(OutboxRepository repository, ObjectMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }
}

```

```

@Transactional(propagation = Propagation.MANDATORY)
public void record(String aggregateType, String aggregateId,
                  String topic, String msgKey, Object payload) {
    try {
        String json = payload instanceof String s ? s : mapper.writeValueAsString(payload);
        repository.save(new OutboxEvent(aggregateType, aggregateId, topic, msgKey, json));
    } catch (Exception e) {
        throw new IllegalStateException("Outbox kaydi olusturulamadi", e);
    }
}
}

```

5.8.5 `libs/outbox/src/main/java/dev/hookrelay/outbox/OutboxPublisher.java`

Tabloyu Kafka'ya akıtan zamanlanmış görev.

`drain()` içindeki iki döngüye dikkat: önce **hepsi** gönderiliyor, sonra teyitler toplanıyor. İlk hali her kayıta `send().get()` çağırıyordu ve poller tek iş parçacıklı olduğu için bu doğrudan sistemin tavanyıdı — ölçüldüğünde giriş 67 olay/sn'de tıkanıyordu.

`cleanup()` yayınlanmış kayıtları altı saat sonra siliyor. Silmeseydik tablo büyür, kısmi indeks yine hızlı kalırdı ama disk şişerdi.

```

package dev.hookrelay.outbox;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;

/**
 * Outbox tablosunu Kafka'ya akitir.
 *
 * Sıra garantisi tek poller ornegiyle korunur. Iki ornek ayni anda
 * calisirsas SKIP LOCKED ikisine farkli satirlar verir ve eski kayit
 * yeniden sonra basilabilir. Bu yuzden ingest yatay olcekleniyor,
 * poller olceklenmiyor. Alternatifler: outbox'i key hash'ine gore
 * bolmek, ya da Debezium ile WAL'dan okumak.
 */
@Component
@ConditionalOnProperty(name = "hookrelay.outbox.enabled", havingValue = "true", matchIfMissing = true)
public class OutboxPublisher {

```

```

private static final Logger log = LoggerFactory.getLogger(OutboxPublisher.class);

private final OutboxRepository repository;
private final KafkaTemplate<String, String> kafka;
private final int batchSize;
private final Duration sendTimeout;
private final Counter published;
private final Counter failed;

public OutboxPublisher(OutboxRepository repository,
    KafkaTemplate<String, String> kafka,
    MeterRegistry meters,
    @org.springframework.beans.factory.annotation.Value("${hookrelay.
        outbox.batch-size:200}") int batchSize,
    @org.springframework.beans.factory.annotation.Value("${hookrelay.
        outbox.send-timeout-ms:5000}") long sendTimeoutMs) {
    this.repository = repository;
    this.kafka = kafka;
    this.batchSize = batchSize;
    this.sendTimeout = Duration.ofMillis(sendTimeoutMs);
    this.published = Counter.builder("hookrelay.outbox.published").register(meters);
    this.failed = Counter.builder("hookrelay.outbox.failed").register(meters);
}

/**
 * fixedDelay (fixedRate degil): bir tur bitmeden yenisi baslmasin.
 * fixedRate olsaydi yavas bir tur sirasinda turlar ust uste binerdi.
 */
@Scheduled(fixedDelayString = "${hookrelay.outbox.poll-interval-ms:200}")
@Transactional
public void drain() {
    List<OutboxEvent> batch = repository.lockUnpublished(batchSize);
    if (batch.isEmpty()) return;

    // ONCE HEPSINI GONDER, SONRA TEYITLERI TOPLA
    //
    // İlk yazilisi her kayitta send().get() cagiriyordu - yani her
    // kayıt için bir gidis-donus bekleniyordu. 500 kayitlik bir paket,
    // 1 ms'lik gidis-donus ile 500 ms suruyordu ve poller tek is
    // parcacikli oldugu icin bu dogrudan sistemin tavani oluyordu.
    // Olculdu: giris ~67 olay/sn'de tikaniyor, p95 5,7 saniye.
    //
    // Simdi butun paket once uretici tamponuna birakiliyor (send
    // asenkrondur, aninda doner), sonra teyitler toplaniyor. Kafka
    // istemcisi kayitlari paketleyip tek istekte gonderiyor.
    //
    // SIRA GARANTISI KORUNUYOR: send cagrilari created_at sirasiyla
    // YAPILIYOR ve idempotent uretici, ayni partition'a yapilan
    // cagrilarin sirasini koruyor. Degisen tek sey, teyidi ne zaman
    // bekledigimiz.
    //
    // KAYIPSIZLIK KORUNUYOR: hala hicbir kaydi teyit almadan
    // "yayinlandi" diye isaretlemiyoruz.
    List<CompletableFuture<SendResult<String, String>>> futures = new ArrayList<>(batch.
        size());
    for (OutboxEvent event : batch) {
        futures.add(kafka.send(new ProducerRecord<>(
            event.getTopic(), event.getMsgKey(), event.getPayload())));
    }
}

```



```

    for (int i = 0; i < batch.size(); i++) {
        OutboxEvent event = batch.get(i);
        try {
            futures.get(i).get(sendTimeout.toMillis(), TimeUnit.MILLISECONDS);
            event.markPublished();
            published.increment();
        } catch (Exception e) {
            // Kismi basarisizlik artik sorun degil: basarili olanlar
            // isaretlenir, olmayanlar bir sonraki turda tekrar denenir.
            // (Onceki surumde ilk hatada break ediliyordu; asenkron
            // gönderimde "sonrakiler zaten gitmis olabilir" durumu
            // dogdugu icin her kaydın kendi teyidine bakıyoruz.)
            event.markFailed(e.getMessage());
            failed.increment();
            log.error("Outbox kaydı yayınlanamadı: id={} topic={} deneme={}",
                event.getId(), event.getTopic(), event.getAttempts(), e);
        }
    }
    repository.saveAll(batch);
}

/** Yayınlanmış kayıtları sonsuza kadar tutmak tabloyu sisirir. */
@Scheduled(cron = "${hookrelay.outbox.cleanup-cron:0 */10 * * * *}")
@Transactional
public void cleanup() {
    int removed = repository.deletePublishedBefore(Instant.now().minus(Duration.ofHours(
        6)));
    if (removed > 0) log.info("Outbox temizliği: {} kayıt silindi", removed);
}
}

```

Kontrol noktası

```
mvn -q compile -pl libs/outbox
```

Bölüm 6 — admin-api: Kontrol Düzlemi

6.1 Kontrol düzlemi ile veri düzlemi ayrımı

Dağıtık sistemlerin en faydalı kavramlarından biri:

- **Kontrol düzlemi** (control plane): yapılandırma. Kim var, nereye gönderiyoruz, hangi limitler. Trafiği düşük, tutarlılık önemli.
- **Veri düzlemi** (data plane): asıl iş. Olay kabulü, teslimat. Trafiği yüksek, gecikme kritik.

İkisini ayırmanın somut kazancı tek cümlede: **kontrol düzlemi çöktüğünde veri düzlemi çalışmaya devam eder.**

Bu bedava gelmez. `dispatcher`'ın endpoint URL'ine ihtiyacı var ve o veri `admin-api`'de. Naif çözüm HTTP çağrısı — ve tam da bu, iki servisi birbirine yeniden yapıştırır.

6.2 Compacted topic ile konfigürasyon replikasyonu

Çözüm: `admin-api` her değişikliği bir Kafka topic'ine basar, `dispatcher` ve `ingest` bunu Redis'e replike eder.

Kritik detay: topic **compacted**.

```
@Bean
public NewTopic endpointConfigTopic() {
    return TopicBuilder.name(Topics.ENDPOINT_CONFIG)
        .partitions(3).replicas(1)
        .config(TopicConfig.CLEANUP_POLICY_CONFIG, TopicConfig.CLEANUP_POLICY_COMPACT)
        .build();
}
```

`cleanup.policy=compact` ile Kafka eski kayıtları silmez — **aynı key'in eski sürümlerini** siler. Sonuçta topic, "her key için son değer" tutan bir **tabloya** dönüşür.

```
Yazılan sırayla:
ep-1 → {url: "a.com", v1}
ep-2 → {url: "b.com", v1}
ep-1 → {url: "a.com/yeni", v2}
ep-1 → {url: "a.com/yeni", v3, enabled:false}

Sıkıştırılmadan sonra topic'te kalan:
ep-2 → {url: "b.com", v1}
ep-1 → {url: "a.com/yeni", v3, enabled:false}
```

Yeni bir `dispatcher` ayağa kalktığında offset 0'dan okur ve saniyeler içinde bütün endpoint tablosunu öğrenir — altı aylık geçmişi değil, **güncel tabloyu**.

Her örnek kendi tüketici grubunda — bilinçli

```
@KafkaListener(
    topics = Topics.ENDPOINT_CONFIG,
    groupId = "endpoint-config-#{T(java.util.UUID).randomUUID().toString()}")
public void onConfig(ConsumerRecord<String, String> record) {
```

```

    if (record.value() == null || record.value().isBlank()) return; // tombstone
    cache.put(mapper.readValue(record.value(), EndpointConfig.class));
}

@Override
public void onPartitionsAssigned(Map<TopicPartition, Long> assignments,
    ConsumerSeekCallback callback) {
    callback.seekToBeginning(assignments.keySet());
}

```

`groupId` içinde rastgele bir UUID var. Normalde bu bir hatadır — tüketici grubu tam olarak "iş bölüşelim" demek içindir.

Ama burada iş bölüşmek **istemiyoruz**. Bu bir iş kuyruğu değil, bir **durum tablosu**. Her `dispatcher` örneğinin bütün endpoint'leri bilmesi gerekiyor. Ortak bir grup kullansaydık Kafka partition'ları örnekler arasında paylaşırdı ve her örnek endpoint'lerin yalnızca üçte birini görürdü — kalanlar için "endpoint bulunamadı" deyip teslimatları düşürürdü.

Benzersiz grup ile her örnek **bütün** partition'ları alır.

`seekToBeginning` ise her açılışta baştan okumayı sağlar. Topic compacted olduğu için "baştan okumak" altı aylık geçmişini okumak değil, **güncel tabloyu** okumak demek. Bu yüzden offset saklamaya da gerek yok: durumun kaynağı topic'in kendisi.

Tuzak. Aynı işi `@TopicPartition(partitions = {"0","1","2"}, partitionOffsets = @PartitionOffset(partition = "*", initialOffset = "0"))` ile de yapabilirsiniz. Ama partition numaralarını anotasyona **yazmak zorundasınız** — `partitions` boş bırakılırsa Spring Kafka "At least one partition required" diye açılışta patlar. Topic'in partition sayısı ileride değişirse bu sınıf sessizce eksik okumaya başlar. `ConsumerSeekAware` yolu dinamiktir.

Kural. İş bölüşmek istiyorsanız ortak tüketici grubu. Herkesin her şeyi bilmesi gerekiyorsa örnek başına ayrı grup (veya elle atama).

6.3 Topic'leri kod oluşturur

```

@Bean
public NewTopic messagesTopic() {
    return TopicBuilder.name(Topics.MESSAGES)
        .partitions(12).replicas(1)
        .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(7L*24*3600*1000))
        .build();
}

```

Ve compose'da otomatik oluşturma **kapalı**:

```
KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
```

Açık olsaydı, adını yanlış yazdığınız bir topic sessizce oluşur ve mesajlarınız kimsenin dinlemediği bir yere akardı. Bulması saatler süren bir hata. Kapalı olunca yanlış isim anında patlar.

Partition sayısı = paralellik tavanı

12 partition varsa 13. tüketici **boş oturur**. Kafka bir partition'ı aynı grupta yalnız bir tüketiciye verir.

Yukarı çıkarmak kolay, aşağı inmek **imkânsız** (Kafka partition silmeye izin vermez). Bu yüzden başlangıçta ihtiyaçtan bir tık fazlası seçilir.

Ayrıca: partition sayısını artırmak, mevcut key'lerin dağılımını değiştirir. `hash(key) % partitionSayısı` formülü nedeniyle bir endpoint'in kayıtları artıştan sonra başka bir partition'a düşer — ve **sıra garantisi geçici olarak kırılır**. Üretimde partition artırmak bakım penceresi ister.

6.4 API anahtarı

```
public final class ApiKeys {

    public static String generate() {
        byte[] raw = new byte[24];
        RANDOM.nextBytes(raw);
        return "hr_" + HexFormat.of().formatHex(raw);
    }

    public static String hash(String apiKey) {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        return HexFormat.of().formatHex(md.digest(apiKey.getBytes(UTF_8)));
    }
}
```

Anahtarın kendisi **hiçbir yerde** saklanmaz: ne veritabanında, ne Kafka'da, ne logda. Yalnız SHA-256 hash'i. Kullanıcıya düz metin bir kez, oluşturma cevabında gösterilir.

Neden bcrypt değil SHA-256

Bu bir kullanıcı parolası değil, 192 bitlik rastgele bir dizge. Sözlük saldırısı yapılamaz, dolayısıyla yavaşlatmaya (bcrypt, argon2) gerek yok. Üstelik her istekte doğrulanacak — bcrypt kullansaydık her olay kabulü 100 ms sürerdi.

Parolalar için bcrypt, rastgele tokenlar için SHA-256. Fark, girdinin tahmin edilebilirliğinde.

6.5 Şema

```
CREATE TABLE application (
    id                UUID PRIMARY KEY,
    name              VARCHAR(120) NOT NULL UNIQUE,
    api_key_hash      VARCHAR(128) NOT NULL,
    api_key_preview   VARCHAR(32)  NOT NULL,
    enabled           BOOLEAN      NOT NULL DEFAULT TRUE,
    version           BIGINT       NOT NULL DEFAULT 1,
    created_at        TIMESTAMPTZ  NOT NULL
);

CREATE TABLE endpoint (
    id                UUID PRIMARY KEY,
    application_id     UUID        NOT NULL REFERENCES application (id) ON DELETE CASCADE,
    DE,
```

```

url          VARCHAR(2000) NOT NULL,
secret       VARCHAR(128)  NOT NULL,
event_types  VARCHAR(2000) NOT NULL DEFAULT '*',
enabled      BOOLEAN       NOT NULL DEFAULT TRUE,
rate_limit_per_second INT   NOT NULL DEFAULT 0,
max_attempts INT           NOT NULL DEFAULT 6,
timeout_ms   INT           NOT NULL DEFAULT 10000,
version      BIGINT        NOT NULL DEFAULT 1,
created_at   TIMESTAMPTZ   NOT NULL,
updated_at   TIMESTAMPTZ   NOT NULL
);

```

`version` alanı her değişiklikte artar. Compacted topic'te "hangi kayıt daha yeni" sorusunu cevaplar ve hata ayıklamada ("replika v3'te kalmış, veritabanı v5") çok işe yarar.

`event_types` neden ayrı tablo değil: bu alan her zaman endpoint'le birlikte okunuyor, üzerinde ilişkisel sorgu yapmıyoruz ve compacted topic'e bütün hâlinde gidiyor. Ayrı tablo yalnızca bir JOIN maliyeti eklerdi.

6.6 Servis: değişiklik ve yayın tek transaction'da

```

@Transactional
public Created create(String name) {
    if (repository.existsByName(name)) throw ApiException.conflict("...");

    String key = ApiKeys.generate();
    WebhookApplication app = new WebhookApplication(name, ApiKeys.hash(key),
                                                    ApiKeys.preview(key));

    repository.save(app);
    publish(app); // ← AYNİ transaction. Outbox'ın bütün anlamı bu.
    return new Created(app, key);
}

private void publish(WebhookApplication app) {
    outbox.record("application", app.getId().toString(),
                 Topics.APP_CONFIG, app.getId().toString(),
                 new ApplicationConfig(app.getId(), app.getName(), app.getApiKeyHash(),
                                       app.isEnabled(), false, app.getVersion()));
}

```

`ApplicationConfig` içinde `apiKeyHash` gidiyor, anahtarın kendisi **asla**. Kafka topic'i uzun ömürlü bir kayıttır; içine düz metin sır yazılmaz.

Silme: tombstone değil, açık bayrak

```

@Transactional
public void delete(UUID id) {
    Endpoint endpoint = get(id);
    publish(endpoint, /* deleted */ true);
    repository.delete(endpoint);
}

```

Kafka'nın kendi silme mekanizması "tombstone"dur: aynı key'e `null` gövde yazarsınız, compaction o key'i tamamen kaldırır. Kullanmıyoruz çünkü tombstone alan tüketici hangi **uygulamaya** ait olduğunu bilemez — key yalnızca `endpointId`'dir. Replikadan çıkarmak için

uygulama kimliği gerekiyor.

6.7 CQRS projeksiyonu

```
@KafkaListener(topics = Topics.DELIVERY_RESULTS, groupId = "admin-health-projection")
@Transactional
public void onResult(String payload) {
    DeliveryResult r = mapper.readValue(payload, DeliveryResult.class);

    EndpointHealth h = repository.findById(r.endpointId())
        .orElseGet(() -> new EndpointHealth(r.endpointId(), r.applicationId()));

    if (r.outcome() == DeliveryResult.Outcome.SUCCEEDED) {
        h.recordSuccess(r.httpStatus(), r.latencyMs(), r.occurredAt());
    } else {
        h.recordFailure(r.httpStatus(), r.latencyMs(), r.error(), r.occurredAt());
    }
    repository.save(h);
}
```

Bu sınıfın en önemli özelliği: **dispatcher'da tek satır değiştirmeden eklendi.**

Olay tabanlı mimarinin somut kazancı bu. Yeni bir okuyucu eklemek, yazanı ilgilendirmiyor. Yarın bir bildirim servisi aynı topic'i dinleyip "endpoint'iniz 10 kez üst üste hata verdi" maili atabilir — yine **dispatcher'a** dokunmadan.

endpoint_health tablosu bir **projeksiyon**: hiçbir kullanıcı yazmaz, tamamı olaylardan türetilir. Silinip topic baştan okunsa aynen geri gelir.

Tuzak: "SUCCEEDED değilse hatadır" yanlış

İlk yazılışı böyleydi ve ölçtüğümüzde yalan söylediği ortaya çıktı:

```
if (r.outcome() == Outcome.SUCCEEDED) h.recordSuccess(...);
else h.recordFailure(...); // ← yanlış
```

Silinmiş bir endpoint'in 25.657 düşürülmüş teslimatı, o endpoint'in "25.657 kez hata verdiği" gibi görünüyordu — oysa ona **tek bir istek bile atılmamıştı**. Devresi açık bir endpoint de hiç istek almadığı hâlde saniyede yüzlerce "hata" biriktiriyordu.

Ayırım, sonucun kendisine ait bir soru:

```
public enum Outcome {
    SUCCEEDED, RETRYING, EXHAUSTED, // gerçek HTTP denemesi yapıldı
    SHORT_CIRCUITED, DISCARDED;    // istek hiç atılmadı

    public boolean isRealAttempt() {
        return this == SUCCEEDED || this == RETRYING || this == EXHAUSTED;
    }
}
```

```
if (!r.outcome().isRealAttempt()) {
    if (r.outcome() == Outcome.SHORT_CIRCUITED) h.recordShortCircuit(r.occurredAt());
    return; // DISCARDED: sağlık kaydını kirletme
}
```

Kısa devreler ayrı bir `short_circuited` sütununda birikiyor — arayüzde "340 atlandı" diye görünüyor ve başarı oranını bozmuyor.

Yanlış bir metrik, olmayan bir metriktir kötüdür. Olmayana bakmazsınız; yanlış olana güvenirsiniz.

Kazancı somut: "bu endpoint sağlıklı mı" sorusu tek satırlık bir `SELECT`. Aynı soruyu `delivery_attempt` üzerinde sormak milyonlarca satır taramaktır.

6.8 Servisin tamamı

Sıra önemli: pom → giriş noktası → şema → entity → repository → servis → API → projeksiyon → yapılandırma. Her adımda derlenir.

6.8.1 `services/admin-api/pom.xml`

`common` ve `outbox`'ı içeri alıyor, JPA ve Flyway ekliyor. `finalName` sabitlenmiş: Dockerfile `target/admin-api.jar` diyecek, sürüm değişince Dockerfile'ı güncellemek gerekmesin.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../../pom.xml</relativePath>
  </parent>
  <artifactId>admin-api</artifactId>
  <name>admin-api</name>
  <description>Kontrol düzlemi: uygulama ve endpoint yönetimi, sağlık projeksiyonu</description>

  <dependencies>
    <dependency><groupId>dev.hookrelay</groupId><artifactId>outbox</artifactId></dependency>
    <dependency><groupId>dev.hookrelay</groupId><artifactId>common</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-actuator</artifactId></dependency>
    <dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
    <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></dependency>
    <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></dependency>
    <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
  </dependencies>

  <build>
    <finalName>admin-api</finalName>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
</plugins>
</build>
</project>

```

6.8.2 `services/admin-api/src/main/java/dev/hookrelay/adminapi/AdminApiApplication`

Üç anotasyona dikkat:

`scanBasePackages = "dev.hookrelay"` — `common` içindeki bean'ler de taransın.

`@EnableJpaRepositories` ve `@EntityScan` **elle** verilmiş ve iki paket listeliyor. Varsayılan davranış yalnızca ana sınıfın paketinin altına bakar; `outbox` modülü başka bir pakette olduğu için elle eklenmeseydi `OutboxRepository` hiç bulunmaz ve uygulama "no bean of type" diye açılışta patlardı.

`@EnableScheduling` outbox poller'ı için.

```

package dev.hookrelay.adminapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
import org.springframework.scheduling.annotation.EnableScheduling;

/**
 * KONTROL DUZLEMI (control plane).
 *
 * Uygulama ve endpoint tanımlarının TEK sahibi. Bu servis cuktugunde
 * yeni endpoint eklenemez ama TESLIMAT DURMAZ - cunku dispatcher
 * konfigurasyonu Redis replikasından okuyor, buraya HTTP atmıyor.
 * Kontrol duzlemi ile veri duzlemini ayirmanin butun anlami bu cumlede.
 */
@SpringBootApplication(scanBasePackages = "dev.hookrelay")
@ConfigurationPropertiesScan("dev.hookrelay")
@EnableScheduling
@org.springframework.data.jpa.repository.config.EnableJpaRepositories(
    basePackages = {"dev.hookrelay.adminapi.repo", "dev.hookrelay.outbox"})
@org.springframework.boot.autoconfigure.domain.EntityScan(
    basePackages = {"dev.hookrelay.adminapi.domain", "dev.hookrelay.outbox"})
public class AdminApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(AdminApiApplication.class, args);
    }
}

```

6.8.3 `services/admin-api/src/main/resources/db/migration/V1\_\_outbox.sql`

Outbox kullanan **her** servisin kendi veritabanında ayrı ayrı bulunur. Paylaşılmaz: her servis kendi verisinin sahibi.

```

-- Transactional Outbox tablosu.
-- Bu sema, outbox kullanan HER servisin kendi veritabanında ayrı ayrı bulunur.
-- Paylaşılmaz: her servis kendi verisinin sahibi.

CREATE TABLE outbox_event (

```



```

    id                UUID PRIMARY KEY,
    aggregate_type    VARCHAR(64) NOT NULL,
    aggregate_id      VARCHAR(64) NOT NULL,
    topic             VARCHAR(128) NOT NULL,
    msg_key           VARCHAR(128),
    payload           TEXT NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL,
    published_at      TIMESTAMPTZ,
    attempts          INT NOT NULL DEFAULT 0,
    last_error        VARCHAR(1000)
);

-- Poller'in tek sorgusu bu indeksi kullanir:
-- WHERE published_at IS NULL ORDER BY created_at
-- Kismi indeks (WHERE published_at IS NULL) secildi cunku tablo cogunlukla
-- yayinlanmis kayitlardan olusur; onlari indekslemek bosuna yer ve yazma maliyeti.
CREATE INDEX idx_outbox_pending
    ON outbox_event (created_at, id)
    WHERE published_at IS NULL;

CREATE INDEX idx_outbox_published_at
    ON outbox_event (published_at)
    WHERE published_at IS NOT NULL;

```

6.8.4 `services/admin-api/src/main/resources/db/migration/V2__control_schema.s

`endpoint.application_id` üzerinde `ON DELETE CASCADE`: uygulama silinince endpoint'leri de gider. Yetim endpoint bırakmak, "hangi uygulamaya aitti" sorusunu cevapsız bırakır.

`endpoint_health` bir **projeksiyon** — hiçbir kullanıcı yazmaz.

```

-- Kontrol duzlemi semasi: kim var, nereye gonderiyoruz.

CREATE TABLE application (
    id                UUID PRIMARY KEY,
    name              VARCHAR(120) NOT NULL UNIQUE,
    api_key_hash      VARCHAR(128) NOT NULL,
    api_key_preview   VARCHAR(32) NOT NULL,
    enabled            BOOLEAN NOT NULL DEFAULT TRUE,
    version            BIGINT NOT NULL DEFAULT 1,
    created_at        TIMESTAMPTZ NOT NULL
);

CREATE UNIQUE INDEX idx_application_key_hash ON application (api_key_hash);

CREATE TABLE endpoint (
    id                UUID PRIMARY KEY,
    application_id     UUID NOT NULL REFERENCES application (id) ON DELETE CASCA
        DE,
    url                VARCHAR(2000) NOT NULL,
    secret              VARCHAR(128) NOT NULL,
    description         VARCHAR(255),
    event_types         VARCHAR(2000) NOT NULL DEFAULT '*',
    enabled              BOOLEAN NOT NULL DEFAULT TRUE,
    rate_limit_per_second INT NOT NULL DEFAULT 0,
    max_attempts        INT NOT NULL DEFAULT 6,
    timeout_ms          INT NOT NULL DEFAULT 10000,
    version              BIGINT NOT NULL DEFAULT 1,
    created_at          TIMESTAMPTZ NOT NULL,

```

```

        updated_at          TIMESTAMPTZ    NOT NULL
    );

    CREATE INDEX idx_endpoint_application ON endpoint (application_id, created_at DESC);

    -- PROJEKSIYON. Kaynak veri degil; delivery-results topic'inden turetilir.
    -- Silinip topic bastan okunsa aynen geri gelir.
    CREATE TABLE endpoint_health (
        endpoint_id          UUID PRIMARY KEY,
        application_id        UUID          NOT NULL,
        succeeded             BIGINT        NOT NULL DEFAULT 0,
        failed                BIGINT        NOT NULL DEFAULT 0,
        consecutive_failures  INT           NOT NULL DEFAULT 0,
        last_status           INT,
        last_error            VARCHAR(500),
        last_latency_ms       BIGINT,
        last_delivery_at      TIMESTAMPTZ,
        updated_at           TIMESTAMPTZ    NOT NULL
    );

    CREATE INDEX idx_endpoint_health_app ON endpoint_health (application_id);

```

6.8.5 `services/admin-api/src/main/resources/db/migration/V3\_\_endpoint\_health\_`

Sonradan eklendi. Uygulanmış bir migration'ı **değiştirmek yerine** yenisini eklemek kuraldır: Flyway sağlama toplamı tutmayınca bütün ortamları kilitler.

```

-- Devre kesici / hiz siniri yuzunden HIC ATILMAYAN istekler.
--
-- failed sutunundan ayri: bunlar musterinin hatasi degil, bizim kararimiz.
-- Ayni kovada tutmak "basari oranı" metrigini anlamsiz yapıyordu – devresi
-- acik bir endpoint hicbir istek almadigi halde saniyede yuzlerce "hata"
-- biriktiriyordu.
ALTER TABLE endpoint_health
    ADD COLUMN short_circuited BIGINT NOT NULL DEFAULT 0;

```

6.8.6 `services/admin-api/src/main/java/dev/hookrelay/adminapi/domain/Webhook`

Sınıf adı `Application` değil — Spring'in kendi kavramıyla karışıyor ve import kazalarına yol açıyor.

`version` her değişiklikte artıyor. Compacted topic'te "hangi kayıt daha yeni" sorusunu cevaplar; hata ayıklamada ("replika v3'te kalmış, veritabanı v5") çok işe yarar.

Kurucu `protected` — JPA proxy üretebilsin ama uygulama kodu yanlışlıkla boş nesne oluşturamasın.

```

package dev.hookrelay.adminapi.domain;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.UUID;

/**
 * Bir musteri / kiraci. Olaylari bu kimlikle gonderir.
 * Sinif adi "Application" degil cunku Spring'in kendi Application

```

```

* kavramıyla karışıyor ve import kazalarına yol açıyor.
*/
@Entity
@Table(name = "application")
public class WebhookApplication {

    @Id
    private UUID id;

    @Column(nullable = false, unique = true, length = 120)
    private String name;

    /** Anahtarın kendisi değil, SHA-256 hash'i. Bkz. ApiKeys. */
    @Column(name = "api_key_hash", nullable = false, length = 128)
    private String apiKeyHash;

    /** Arayüzde gösterilecek "hr_3f9a..." onizlemesi. Tek basına ise yaramaz. */
    @Column(name = "api_key_preview", nullable = false, length = 32)
    private String apiKeyPreview;

    @Column(nullable = false)
    private boolean enabled = true;

    /** Her değişiklikte artar; compacted topic'te eski/yeni ayrımı için. */
    @Column(nullable = false)
    private long version = 1;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    protected WebhookApplication() {}

    public WebhookApplication(String name, String apiKeyHash, String apiKeyPreview) {
        this.id = UUID.randomUUID();
        this.name = name;
        this.apiKeyHash = apiKeyHash;
        this.apiKeyPreview = apiKeyPreview;
        this.enabled = true;
        this.version = 1;
        this.createdAt = Instant.now();
    }

    public void rotateKey(String newHash, String newPreview) {
        this.apiKeyHash = newHash;
        this.apiKeyPreview = newPreview;
        this.version++;
    }

    public void setEnabled(boolean enabled) {
        this.enabled = enabled;
        this.version++;
    }

    public UUID getId() { return id; }
    public String getName() { return name; }
    public String getApiKeyHash() { return apiKeyHash; }
    public String getApiKeyPreview() { return apiKeyPreview; }
    public boolean isEnabled() { return enabled; }
    public long getVersion() { return version; }
    public Instant getCreatedAt() { return createdAt; }
}

```

6.8.7 `services/admin-api/src/main/java/dev/hookrelay/adminapi/domain/Endpoint`

`eventTypes` virgülle ayrılmış tek sütun. Ayrı tablo değil çünkü bu alan her zaman endpoint'le birlikte okunuyor, üzerinde ilişkisel sorgu yapılmıyor ve compacted topic'e bütün hâlinde gidiyor. Ayrı tablo yalnızca bir `JOIN` maliyeti eklerdi.

`update` metodunda her alan `null` kontrolünden geçiyor: PATCH semantiği. Gönderilmeyen alan değişmez.

```
package dev.hookrelay.adminapi.domain;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.UUID;

/** Bir musterinin webhook alacagi URL. */
@Entity
@Table(name = "endpoint")
public class Endpoint {

    @Id
    private UUID id;

    @Column(name = "application_id", nullable = false)
    private UUID applicationId;

    @Column(nullable = false, length = 2000)
    private String url;

    /** Bu endpoint'e giden isteklerin HMAC anahtari. Her endpoint'in kendine ait. */
    @Column(nullable = false, length = 128)
    private String secret;

    @Column(length = 255)
    private String description;

    /**
     * Abone olunan olay tipleri, virgulle ayrılmis. "*" = hepsi.
     *
     * Neden ayri tablo degil: burada iliskisel sorgu yapmiyoruz, bu alan
     * her zaman endpoint'le birlikte okunuyor ve compacted topic'e
     * butun halinde gidiyor. Ayri tablo sadece bir JOIN maliyeti eklerdi.
     */
    @Column(name = "event_types", nullable = false, length = 2000)
    private String eventTypes;

    @Column(nullable = false)
    private boolean enabled = true;

    /** 0 = sinirsiz. Yavas musteriye bogmamak icin. */
    @Column(name = "rate_limit_per_second", nullable = false)
    private int rateLimitPerSecond;

    /** Kac denemeden sonra DLQ. Katman sayisi + 1'i gecemez. */
    @Column(name = "max_attempts", nullable = false)
    private int maxAttempts = 6;
}
```

```

@Column(name = "timeout_ms", nullable = false)
private int timeoutMs = 10_000;

@Column(nullable = false)
private long version = 1;

@Column(name = "created_at", nullable = false)
private Instant createdAt;

@Column(name = "updated_at", nullable = false)
private Instant updatedAt;

protected Endpoint() {}

public Endpoint(UUID applicationId, String url, String secret, String description,
                Set<String> eventTypes, int rateLimitPerSecond,
                int maxAttempts, int timeoutMs) {
    this.id = UUID.randomUUID();
    this.applicationId = applicationId;
    this.url = url;
    this.secret = secret;
    this.description = description;
    this.eventTypes = String.join(",", eventTypes);
    this.enabled = true;
    this.rateLimitPerSecond = rateLimitPerSecond;
    this.maxAttempts = maxAttempts;
    this.timeoutMs = timeoutMs;
    this.version = 1;
    this.createdAt = Instant.now();
    this.updatedAt = this.createdAt;
}

public void update(String url, String description, Set<String> eventTypes,
                  Boolean enabled, Integer rateLimitPerSecond,
                  Integer maxAttempts, Integer timeoutMs) {
    if (url != null) this.url = url;
    if (description != null) this.description = description;
    if (eventTypes != null && !eventTypes.isEmpty()) this.eventTypes = String.join(",",
        eventTypes);
    if (enabled != null) this.enabled = enabled;
    if (rateLimitPerSecond != null) this.rateLimitPerSecond = rateLimitPerSecond;
    if (maxAttempts != null) this.maxAttempts = maxAttempts;
    if (timeoutMs != null) this.timeoutMs = timeoutMs;
    this.version++;
    this.updatedAt = Instant.now();
}

public Set<String> eventTypeSet() {
    Set<String> out = new LinkedHashSet<>();
    for (String s : eventTypes.split(",")) {
        String t = s.trim();
        if (!t.isEmpty()) out.add(t);
    }
    return out;
}

public UUID getId() { return id; }
public UUID getApplicationId() { return applicationId; }
public String getUrl() { return url; }
public String getSecret() { return secret; }
public String getDescription() { return description; }

```

```

    public String getEventTypes()      { return eventTypes; }
    public boolean isEnabled()         { return enabled; }
    public int getRateLimitPerSecond() { return rateLimitPerSecond; }
    public int getMaxAttempts()        { return maxAttempts; }
    public int getTimeoutMs()          { return timeoutMs; }
    public long getVersion()           { return version; }
    public Instant getCreatedAt()       { return createdAt; }
    public Instant getUpdatedAt()       { return updatedAt; }
}

```

6.8.8 `services/admin-api/src/main/java/dev/hookrelay/adminapi/domain/EndpointHealth`

`succeeded` / `failed` / `shortCircuited` üç ayrı sayaç. Üçüncüsü sonradan eklendi ve sebebi 6.7'de yazılı.

`successRate()` sıfıra bölmeye karşı korumalı: hiç teslimat yoksa 1.0 döner. Yeni bir endpoint "%0 başarılı" diye kırmızı görünmemeli.

```

package dev.hookrelay.adminapi.domain;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.UUID;

/**
 * PROJEKSİYON - bu tabloyu hiçbir kullanıcı yazmaz.
 *
 * Tamami hookrelay.delivery-results.v1 topic'i dinlenerek türetilir.
 * Silinip topic bastan okunsa aynen yeniden oluşur. CQRS'in okuma tarafı
 * tam olarak budur: yazma modeli (dispatcher'daki delivery_attempt)
 * ile okuma modeli (bu tablo) ayrı şekilde, ayrı servislerde, ayrı
 * amaçlar için tutulur.
 *
 * Kazancı somut: arayüzde "bu endpoint sağlıklı mı" sorusu 1 satırlık
 * bir SELECT. Aynı soruyu delivery_attempt üzerinde sormak milyonlarca
 * satırı taramak demektir.
 */
@Entity
@Table(name = "endpoint_health")
public class EndpointHealth {

    @Id
    @Column(name = "endpoint_id")
    private UUID endpointId;

    @Column(name = "application_id", nullable = false)
    private UUID applicationId;

    @Column(nullable = false) private long succeeded;
    @Column(nullable = false) private long failed;

    /**
     * Devre kesici / hız sınırı yüzünden HIC ATILMAYAN istekler.
     *
     * failed'dan ayrı tutuluyor çünkü bunlar müşterinin hatası değil,
     * bizim kararımız. Aynı kovaya koymak "başarı oranı" metrigini
     * anlamsız yapardı: devresi açık bir endpoint, hiçbir istek
     * almadığı halde saniyede yüzlerce "hata" biriktirirdi.
     */
}

```

```

    */
    @Column(name = "short_circuited", nullable = false) private long shortCircuited;
    @Column(name = "consecutive_failures", nullable = false) private int consecutiveFailures
    ;

    @Column(name = "last_status") private Integer lastStatus;
    @Column(name = "last_error", length = 500) private String lastError;
    @Column(name = "last_latency_ms") private Long lastLatencyMs;
    @Column(name = "last_delivery_at") private Instant lastDeliveryAt;
    @Column(name = "updated_at", nullable = false) private Instant updatedAt;

    protected EndpointHealth() {}

    public EndpointHealth(UUID endpointId, UUID applicationId) {
        this.endpointId = endpointId;
        this.applicationId = applicationId;
        this.updatedAt = Instant.now();
    }

    public void recordSuccess(Integer status, long latencyMs, Instant at) {
        this.succeeded++;
        this.consecutiveFailures = 0;
        this.lastStatus = status;
        this.lastError = null;
        this.lastLatencyMs = latencyMs;
        this.lastDeliveryAt = at;
        this.updatedAt = Instant.now();
    }

    public void recordFailure(Integer status, long latencyMs, String error, Instant at) {
        this.failed++;
        this.consecutiveFailures++;
        this.lastStatus = status;
        this.lastError = error == null ? null : error.substring(0, Math.min(error.length(),
            500));
        this.lastLatencyMs = latencyMs;
        this.lastDeliveryAt = at;
        this.updatedAt = Instant.now();
    }

    /** Istek atilmadi; sadece atlanan sayisini artir. */
    public void recordShortCircuit(Instant at) {
        this.shortCircuited++;
        this.lastDeliveryAt = at;
        this.updatedAt = Instant.now();
    }

    public double successRate() {
        long total = succeeded + failed;
        return total == 0 ? 1.0 : (double) succeeded / total;
    }

    public UUID getEndpointId() { return endpointId; }
    public UUID getApplicationId() { return applicationId; }
    public long getSucceeded() { return succeeded; }
    public long getFailed() { return failed; }
    public long getShortCircuited() { return shortCircuited; }
    public int getConsecutiveFailures() { return consecutiveFailures; }
    public Integer getLastStatus() { return lastStatus; }
    public String getLastError() { return lastError; }
    public Long getLastLatencyMs() { return lastLatencyMs; }

```

```

    public Instant getLastDeliveryAt() { return lastDeliveryAt; }
    public Instant getUpdatedAt()      { return updatedAt; }
}

```

6.8.9 `services/admin-api/src/main/java/dev/hookrelay/adminapi/repo/Application

Spring Data'nın metot adından sorgu türetmesi yeterli; özel sorgu yok.

```

package dev.hookrelay.adminapi.repo;

import dev.hookrelay.adminapi.domain.WebhookApplication;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;
import java.util.UUID;

public interface ApplicationRepository extends JpaRepository<WebhookApplication, UUID> {
    Optional<WebhookApplication> findByName(String name);
    boolean existsByName(String name);
}

```

6.8.10 `services/admin-api/src/main/java/dev/hookrelay/adminapi/repo/EndpointR

```

package dev.hookrelay.adminapi.repo;

import dev.hookrelay.adminapi.domain.Endpoint;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.UUID;

public interface EndpointRepository extends JpaRepository<Endpoint, UUID> {
    List<Endpoint> findByIdOrderByIdDesc(UUID applicationId);
}

```

6.8.11 `services/admin-api/src/main/java/dev/hookrelay/adminapi/repo/EndpointH

```

package dev.hookrelay.adminapi.repo;

import dev.hookrelay.adminapi.domain.EndpointHealth;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.UUID;

public interface EndpointHealthRepository extends JpaRepository<EndpointHealth, UUID> {
    List<EndpointHealth> findById(UUID applicationId);
}

```

6.8.12 `services/admin-api/src/main/java/dev/hookrelay/adminapi/service/Applica

`create` içinde `repository.save` ve `publish` aynı transaction'da. Outbox'ın bütün anlamı bu satırda.

`republishAll` bir kurtarma aracı: Kafka'yı sıfırdan kurduğunuzda ya da replika bozulduğunda

bütün konfigürasyonu yeniden basar. Kaynak doğru (veritabanı), türetilmiş olan yeniden üretilebilir — sağlıklı bir mimarinin işareti tam olarak budur.

Düz metin API anahtarı yalnızca `Created` kaydında, yalnızca bir kez dönüyor.

```
package dev.hookrelay.adminapi.service;

import dev.hookrelay.adminapi.domain.WebhookApplication;
import dev.hookrelay.adminapi.repo.ApplicationRepository;
import dev.hookrelay.common.crypto.ApiKeys;
import dev.hookrelay.common.error.ApiException;
import dev.hookrelay.contracts.ApplicationConfig;
import dev.hookrelay.contracts.Topics;
import dev.hookrelay.outbox.OutboxRecorder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.UUID;

@Service
public class ApplicationService {

    private final ApplicationRepository repository;
    private final OutboxRecorder outbox;

    public ApplicationService(ApplicationRepository repository, OutboxRecorder outbox) {
        this.repository = repository;
        this.outbox = outbox;
    }

    /** Düz metin API anahtarı SADECE burada, SADECE bir kez doner. */
    public record Created(WebhookApplication application, String plaintextApiKey) {}

    @Transactional
    public Created create(String name) {
        if (repository.existsByName(name)) {
            throw ApiException.conflict("Bu isimde bir uygulama zaten var: " + name);
        }
        String key = ApiKeys.generate();
        WebhookApplication app = new WebhookApplication(name, ApiKeys.hash(key), ApiKeys.preview(key));
        repository.save(app);
        publish(app); // AYNI transaction - outbox'in tüm anlamı bu
        return new Created(app, key);
    }

    @Transactional
    public String rotateKey(UUID id) {
        WebhookApplication app = get(id);
        String key = ApiKeys.generate();
        app.rotateKey(ApiKeys.hash(key), ApiKeys.preview(key));
        publish(app);
        return key;
    }

    @Transactional
    public WebhookApplication setEnabled(UUID id, boolean enabled) {
        WebhookApplication app = get(id);
        app.setEnabled(enabled);
        publish(app);
    }
}
```

```

        return app;
    }

    /**
     * Butun uygulamalari compacted topic'e yeniden basar.
     *
     * Ne zaman lazim: Kafka'yi sifirdan kurdunuz, ya da replika bozuldu.
     * Kaynak dogru (veritabani), turetilmis olan yeniden uretilebilir -
     * saglikli bir mimarinin isareti tam olarak budur.
     */
    @Transactional
    public int republishAll() {
        List<WebhookApplication> all = repository.findAll();
        all.forEach(this::publish);
        return all.size();
    }

    @Transactional(readOnly = true)
    public List<WebhookApplication> list() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public WebhookApplication get(UUID id) {
        return repository.findById(id)
            .orElseThrow(() -> ApiException.notFound("Uygulama", id));
    }

    private void publish(WebhookApplication app) {
        outbox.record("application", app.getId().toString(),
            Topics.APP_CONFIG, app.getId().toString(),
            new ApplicationConfig(app.getId(), app.getName(), app.getApiKeyHash(),
                app.isEnabled(), false, app.getVersion()));
    }
}

```

6.8.13 `services/admin-api/src/main/java/dev/hookrelay/adminapi/service/Endpoint`

`clampAttempts` sessizce tavana kırıyor: 5 katman varsa 6. deneme yapacak bir topic yok. Kullanıcı 99 yazsa bile sistem tutarlı kalır.

`delete` önce `publish(deleted=true)` sonra `repository.delete` — sırası önemli. Ters olsaydı `publish` için gereken alanlar (uygulama kimliği) elimizde olmazdı.

```

package dev.hookrelay.adminapi.service;

import dev.hookrelay.adminapi.domain.Endpoint;
import dev.hookrelay.adminapi.repo.EndpointRepository;
import dev.hookrelay.common.error.ApiException;
import dev.hookrelay.contracts.EndpointConfig;
import dev.hookrelay.contracts.Topics;
import dev.hookrelay.outbox.OutboxRecorder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.security.SecureRandom;
import java.util.HexFormat;
import java.util.List;
import java.util.Set;

```

```

import java.util.UUID;

@Service
public class EndpointService {

    private static final SecureRandom RANDOM = new SecureRandom();

    private final EndpointRepository repository;
    private final ApplicationService applications;
    private final OutboxRecorder outbox;
    private final int tierCount;

    public EndpointService(EndpointRepository repository, ApplicationService applications,
        OutboxRecorder outbox) {
        this.repository = repository;
        this.applications = applications;
        this.outbox = outbox;
        this.tierCount = Topics.tierCount();
    }

    @Transactional
    public Endpoint create(UUID applicationId, String url, String description,
        Set<String> eventTypes, Integer rateLimitPerSecond,
        Integer maxAttempts, Integer timeoutMs, String secret) {
        applications.get(applicationId); // uygulama yoksa 404
        validateUrl(url);

        int attempts = clampAttempts(maxAttempts == null ? tierCount + 1 : maxAttempts);
        Endpoint endpoint = new Endpoint(
            applicationId, url,
            secret == null || secret.isBlank() ? generateSecret() : secret,
            description,
            eventTypes == null || eventTypes.isEmpty() ? Set.of("*") : eventTypes,
            rateLimitPerSecond == null ? 0 : rateLimitPerSecond,
            attempts,
            timeoutMs == null ? 10_000 : timeoutMs);

        repository.save(endpoint);
        publish(endpoint, false);
        return endpoint;
    }

    @Transactional
    public Endpoint update(UUID id, String url, String description, Set<String> eventTypes,
        Boolean enabled, Integer rateLimitPerSecond,
        Integer maxAttempts, Integer timeoutMs) {
        Endpoint endpoint = get(id);
        if (url != null) validateUrl(url);
        endpoint.update(url, description, eventTypes, enabled, rateLimitPerSecond,
            maxAttempts == null ? null : clampAttempts(maxAttempts), timeoutMs);
        publish(endpoint, false);
        return endpoint;
    }

    /**
     * Silme = compacted topic'e deleted=true basmak.
     *
     * Neden tombstone (null govde) değil: tombstone'u alan tüketici
     * "silindi mi yoksa sema mi bozuk" ayrimini yapamaz ve hangi
     * uygulamaya ait oldugunu bilemez (key sadece endpointId). Acik
     * bir deleted bayragi tasimak hem okunakli hem hata ayiklanabilir.

```

```

    */
    @Transactional
    public void delete(UUID id) {
        Endpoint endpoint = get(id);
        publish(endpoint, true);
        repository.delete(endpoint);
    }

    @Transactional
    public int republishAll() {
        List<Endpoint> all = repository.findAll();
        all.forEach(e -> publish(e, false));
        return all.size();
    }

    @Transactional(readOnly = true)
    public List<Endpoint> listByApplication(UUID applicationId) {
        return repository.findByApplicationIdOrderByCreatedAtDesc(applicationId);
    }

    @Transactional(readOnly = true)
    public List<Endpoint> listAll() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public Endpoint get(UUID id) {
        return repository.findById(id).orElseThrow(() -> ApiException.notFound("Endpoint", id));
    }

    private void publish(Endpoint e, boolean deleted) {
        outbox.record("endpoint", e.getId().toString(),
            Topics.ENDPOINT_CONFIG, e.getId().toString(),
            new EndpointConfig(e.getId(), e.getApplicationId(), e.getUrl(), e.getSecret(),
                e.eventTypeSet(), e.isEnabled(), deleted, e.getRateLimitPerSecond(),
                e.getMaxAttempts(), e.getTimeoutMs(), e.getVersion()));
    }

    /**
     * maxAttempts katman sayisini asamaz: 5 katman varsa 6. deneme
     * yapılacak bir topic yok. Sessizce tavana kirpiyoruz - kullanıcı
     * 99 yazsa bile sistem tutarlı kalır.
     */
    private int clampAttempts(int requested) {
        return Math.max(1, Math.min(requested, tierCount + 1));
    }

    private void validateUrl(String url) {
        if (url == null || url.isBlank()) throw ApiException.badRequest("url boş olamaz");
        if (!url.startsWith("http://") && !url.startsWith("https://")) {
            throw ApiException.badRequest("url http:// veya https:// ile başlamalı");
        }
    }

    private String generateSecret() {
        byte[] raw = new byte[32];
        RANDOM.nextBytes(raw);
        return "whsec_" + HexFormat.of().formatHex(raw);
    }

```

```
}
```

6.8.14 `services/admin-api/src/main/java/dev/hookrelay/adminapi/api/Dtos.java`

Bütün istek/cevap kayıtları tek dosyada. Hepsi küçük ve birlikte okunuyor; 15 ayrı dosyaya bölmek gezinmeyi zorlaştırırdı.

`EndpointResponse secret` alanını **döndürüyor** — kasıtlı, çünkü müşteri imzayı doğrulamak için sırra ihtiyaç duyar ve bu uç yalnızca kontrol düzlemine açık. Uca kimlik doğrulama eklenirse burası gözden geçirilmeli.

```
package dev.hookrelay.adminapi.api;

import dev.hookrelay.adminapi.domain.Endpoint;
import dev.hookrelay.adminapi.domain.EndpointHealth;
import dev.hookrelay.adminapi.domain.WebhookApplication;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

import java.time.Instant;
import java.util.Set;
import java.util.UUID;

/** Istek/cevap kayıtları tek dosyada - hepsi küçük ve birlikte okunuyor. */
public final class Dtos {

    private Dtos() {}

    public record CreateApplicationRequest(
        @NotBlank @Size(max = 120) String name) {}

    public record ApplicationResponse(
        UUID id, String name, String apiKeyPreview, boolean enabled,
        long version, Instant createdAt) {

        public static ApplicationResponse from(WebhookApplication a) {
            return new ApplicationResponse(a.getId(), a.getName(), a.getApiKeyPreview(),
                a.isEnabled(), a.getVersion(), a.getCreatedAt());
        }
    }

    /** apiKey alanı SADECE oluşturma ve yenileme cevabında dolu gelir. */
    public record ApplicationCreatedResponse(
        ApplicationResponse application, String apiKey, String warning) {}

    public record CreateEndpointRequest(
        @NotBlank String url,
        String description,
        Set<String> eventTypes,
        Integer rateLimitPerSecond,
        Integer maxAttempts,
        Integer timeoutMs,
        String secret) {}

    public record UpdateEndpointRequest(
        String url,
        String description,
        Set<String> eventTypes,
        Boolean enabled,
```

```

        Integer rateLimitPerSecond,
        Integer maxAttempts,
        Integer timeoutMs) {}

    public record EndpointResponse(
        UUID id, UUID applicationId, String url, String description,
        Set<String> eventTypes, boolean enabled, int rateLimitPerSecond,
        int maxAttempts, int timeoutMs, String secret, long version,
        Instant createdAt, Instant updatedAt) {

        public static EndpointResponse from(Endpoint e) {
            return new EndpointResponse(e.getId(), e.getApplicationId(), e.getUrl(),
                e.getDescription(), e.getEventTypeSet(), e.isEnabled(),
                e.getRateLimitPerSecond(), e.getMaxAttempts(), e.getTimeoutMs(),
                e.getSecret(), e.getVersion(), e.getCreatedAt(), e.getUpdatedAt());
        }
    }

    public record EndpointHealthResponse(
        UUID endpointId, long succeeded, long failed, long shortCircuited,
        double successRate, int consecutiveFailures, Integer lastStatus, String lastError,
        Long lastLatencyMs, Instant lastDeliveryAt, String circuitState) {

        public static EndpointHealthResponse from(EndpointHealth h, String circuitState) {
            return new EndpointHealthResponse(h.getEndpointId(), h.getSucceeded(), h.getFailed(),
                h.getShortCircuited(), h.getSuccessRate(), h.getConsecutiveFailures(),
                h.getLastStatus(), h.getLastError(), h.getLastLatencyMs(),
                h.getLastDeliveryAt(), circuitState);
        }

        public static EndpointHealthResponse empty(UUID endpointId, String circuitState) {
            return new EndpointHealthResponse(endpointId, 0, 0, 0, 1.0, 0,
                null, null, null, null, circuitState);
        }
    }
}

```

6.8.15 `services/admin-api/src/main/java/dev/hookrelay/adminapi/api/Application

KEY_WARNING sabiti cevapta dönüyor: "bu anahtar bir daha gösterilmeyecek". API'nin kendisi kullanıcıyı uyarmalı; dokümana güvenmek yetmez.

```

package dev.hookrelay.adminapi.api;

import dev.hookrelay.adminapi.service.ApplicationService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.UUID;

@RestController
@RequestMapping("/api/admin/applications")
public class ApplicationController {

```

```

private static final String KEY_WARNING =
    "Bu anahtar bir daha gosterilmeyecek. Simdi kaydedin.";

private final ApplicationService service;

public ApplicationController(ApplicationService service) {
    this.service = service;
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Dtos.ApplicationCreatedResponse create(
    @Valid @RequestBody Dtos.CreateApplicationRequest request) {
    var created = service.create(request.name());
    return new Dtos.ApplicationCreatedResponse(
        Dtos.ApplicationResponse.from(created.application()),
        created.plaintextApiKey(), KEY_WARNING);
}

@GetMapping
public List<Dtos.ApplicationResponse> list() {
    return service.list().stream().map(Dtos.ApplicationResponse::from).toList();
}

@GetMapping("/{id}")
public Dtos.ApplicationResponse get(@PathVariable UUID id) {
    return Dtos.ApplicationResponse.from(service.get(id));
}

@PostMapping("/{id}/rotate-key")
public Map<String, String> rotateKey(@PathVariable UUID id) {
    return Map.of("apiKey", service.rotateKey(id), "warning", KEY_WARNING);
}

@PostMapping("/{id}/enabled")
public Dtos.ApplicationResponse setEnabled(@PathVariable UUID id,
    @RequestParam boolean value) {
    return Dtos.ApplicationResponse.from(service.setEnabled(id, value));
}

/** Replikalari yeniden kurmak icin. Bkz. ApplicationService.republishAll. */
@PostMapping("/republish")
public ResponseEntity<Map<String, Integer>> republish() {
    return ResponseEntity.ok(Map.of("published", service.republishAll()));
}
}

```

6.8.16 `services/admin-api/src/main/java/dev/hookrelay/adminapi/api/EndpointCo

`health` ucu iki kaynağı birleştiriyor: projeksiyon tablosu (olaylardan türetilmiş geçmiş) ve Redis'teki canlı devre durumu (anlık). İkisi farklı şeyler ve farklı yerlerde durmaları doğru.

404 yerine 403 dönmüyoruz — başka kiracının kaynağının **varlığını** bile sızdırmamak için.

```

package dev.hookrelay.adminapi.api;

import dev.hookrelay.adminapi.domain.EndpointHealth;
import dev.hookrelay.adminapi.repo.EndpointHealthRepository;

```

```

import dev.hookrelay.adminapi.service.EndpointService;
import dev.hookrelay.common.redis.RedisCircuitBreaker;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;
import java.util.UUID;

@RestController
@RequestMapping("/api/admin")
public class EndpointController {

    private final EndpointService service;
    private final EndpointHealthRepository health;
    private final RedisCircuitBreaker breaker;

    public EndpointController(EndpointService service, EndpointHealthRepository health,
                             RedisCircuitBreaker breaker) {
        this.service = service;
        this.health = health;
        this.breaker = breaker;
    }

    @PostMapping("/applications/{applicationId}/endpoints")
    @ResponseStatus(HttpStatus.CREATED)
    public Dtos.EndpointResponse create(@PathVariable UUID applicationId,
                                         @Valid @RequestBody Dtos.CreateEndpointRequest r) {
        return Dtos.EndpointResponse.from(service.create(applicationId, r.url(), r.description(),
                                                         r.eventTypes(), r.rateLimitPerSecond(), r.maxAttempts(), r.timeoutMs(), r.secret()));
    }

    @GetMapping("/applications/{applicationId}/endpoints")
    public List<Dtos.EndpointResponse> listByApp(@PathVariable UUID applicationId) {
        return service.listByApplication(applicationId).stream()
            .map(Dtos.EndpointResponse::from).toList();
    }

    @GetMapping("/endpoints")
    public List<Dtos.EndpointResponse> listAll() {
        return service.listAll().stream().map(Dtos.EndpointResponse::from).toList();
    }

    @GetMapping("/endpoints/{id}")
    public Dtos.EndpointResponse get(@PathVariable UUID id) {
        return Dtos.EndpointResponse.from(service.get(id));
    }

    @PatchMapping("/endpoints/{id}")
    public Dtos.EndpointResponse update(@PathVariable UUID id,
                                         @RequestBody Dtos.UpdateEndpointRequest r) {
        return Dtos.EndpointResponse.from(service.update(id, r.url(), r.description(),
                                                         r.eventTypes(), r.enabled(), r.rateLimitPerSecond(), r.maxAttempts(), r.timeoutMs()));
    }

    @DeleteMapping("/endpoints/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)

```



```

public void delete(@PathVariable UUID id) {
    service.delete(id);
}

/**
 * Saglik = projeksiyon tablosu + Redis'teki canli devre durumu.
 * Iki farkli kaynak: biri olaylardan turetilmis gecmis, digeri anlik durum.
 */
@GetMapping("/endpoints/{id}/health")
public Dtos.EndpointHealthResponse health(@PathVariable UUID id) {
    String state = breaker.state(id).name();
    return health.findById(id)
        .map(h -> Dtos.EndpointHealthResponse.from(h, state))
        .orElseGet(() -> Dtos.EndpointHealthResponse.empty(id, state));
}

@GetMapping("/applications/{applicationId}/health")
public List<Dtos.EndpointHealthResponse> healthByApp(@PathVariable UUID applicationId) {
    List<EndpointHealth> rows = health.findByApplicationId(applicationId);
    return rows.stream()
        .map(h -> Dtos.EndpointHealthResponse.from(h,
            breaker.state(h.getEndpointId()).name()))
        .toList();
}

@PostMapping("/endpoints/republish")
public Map<String, Integer> republish() {
    return Map.of("published", service.republishAll());
}
}

```

6.8.17 `services/admin-api/src/main/java/dev/hookrelay/adminapi/projection/DeliveryResultsTopicProjection.java`

Bu sınıf `dispatcher`'da **tek satır değiştirmeden** eklendi. Olay tabanlı mimarinin somut kazancı bu.

`catch (Exception)` geniş ve bilinçli: projeksiyon kırılırsa teslimat durmaz. Bu tüketici bağımsız.

```

package dev.hookrelay.adminapi.projection;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.adminapi.domain.EndpointHealth;
import dev.hookrelay.adminapi.repo.EndpointHealthRepository;
import dev.hookrelay.contracts.DeliveryResult;
import dev.hookrelay.contracts.Topics;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

/**
 * delivery-results topic'inden endpoint_health projeksiyonunu kurar.
 *
 * Bu sınıf, dispatcher'da TEK SATIR degistirmeden eklendi. Olay tabanlı
 * mimarinin somut kazanci bu: yeni bir okuyucu eklemek, yazani
 * ilgilendirmiyor. Yarın bir bildirim servisi ayni topic'i dinleyip
 * "endpoint'iniz 10 kez ust uste hata verdi" maili atabilir -
 * yine dispatcher'a dokunmadan.
 */

```

```

*/
@Component
public class DeliveryResultProjector {

    private static final Logger log = LoggerFactory.getLogger(DeliveryResultProjector.class)
        ;

    private final EndpointHealthRepository repository;
    private final ObjectMapper mapper;

    public DeliveryResultProjector(EndpointHealthRepository repository, ObjectMapper mapper)
    {
        this.repository = repository;
        this.mapper = mapper;
    }

    @KafkaListener(topics = Topics.DELIVERY_RESULTS, groupId = "admin-health-projection")
    @Transactional
    public void onResult(String payload) {
        try {
            DeliveryResult r = mapper.readValue(payload, DeliveryResult.class);

            EndpointHealth h = repository.findById(r.endpointId())
                .orElseGet(() -> new EndpointHealth(r.endpointId(), r.applicationId()));

            // GERCEK DENEME AYRIMI
            //
            // İlk surum "SUCCEEDED degilse hatadir" diyordu. Sonuc: silinmis
            // bir endpoint'in 25 bin dusurulmus teslimati, o endpoint'in
            // "25 bin kez hata verdigi" gibi gorunuyordu - oysa ona tek bir
            // istek bile atilmamisti. Basari orani da anlamsizlasiyordu.
            if (!r.outcome().isRealAttempt()) {
                if (r.outcome() == DeliveryResult.Outcome.SHORT_CIRCUITED) {
                    h.recordShortCircuit(r.occurredAt());
                    repository.save(h);
                }
                // DISCARDED: endpoint zaten yok, saglik kaydini kirletmeye gerek yok.
                return;
            }

            if (r.outcome() == DeliveryResult.Outcome.SUCCEEDED) {
                h.recordSuccess(r.httpStatus(), r.latencyMs(), r.occurredAt());
            } else {
                h.recordFailure(r.httpStatus(), r.latencyMs(), r.error(), r.occurredAt());
            }
            repository.save(h);
        } catch (Exception e) {
            // Projeksiyon kirilirse TESLIMAT DURMAZ. Bu tuketici bagimsiz.
            log.error("Teslimat sonucu islenemedi: {}", payload, e);
        }
    }
}

```

6.8.18 `services/admin-api/src/main/java/dev/hookrelay/adminapi/config/KafkaTo

Topic'leri kod olusturur. `retryTopics()` dönüş tipine dikkat: `KafkaAdmin.NewTopics`, `List<NewTopic>` değil. İlk hali `List` döndürüyordu ve **sessizce çalışmıyordu** — `KafkaAdmin` o tipi tanımıyor, bean oluştuyor, kimse bakmıyor, beş retry topic'i hiç açılmıyordu.

```

package dev.hookrelay.adminapi.config;

import dev.hookrelay.contracts.Topics;
import org.apache.kafka.clients.admin.NewTopic;
import org.apache.kafka.common.config.TopicConfig;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;
import org.springframework.kafka.core.KafkaAdmin;

/**
 * Topic'leri kod olusturur, elle degil.
 *
 * Neden onemli: partition sayisi ve cleanup.policy calisma zamaninda
 * degistirilmesi zor kararlar. Burada durunca hem versiyonlanir hem de
 * "docker compose up" diyen herkeste ayni sistem kurulur.
 *
 * KafkaAdmin bu bean'leri gorup topic'leri olusturur; VAR OLAN topic'i
 * degistirmez (partition sayisini kod dusurmez).
 */
@Configuration
public class KafkaTopicsConfig {

    /**
     * Ana teslimat topic'i. 12 partition.
     *
     * PARTITION SAYISI = PARALELLIK TAVANI. 12 partition varsa 13. tuketici
     * bos oturur. Yukari cikarmak kolay, asagi inmek imkansiz - bu yuzden
     * baslangicta ihtiyactan bir tik fazlasi secilir.
     */
    @Bean
    public NewTopic messagesTopic() {
        return TopicBuilder.name(Topics.MESSAGES)
            .partitions(12).replicas(1)
            .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(7L * 24 * 3600 * 100
                0))
            .build();
    }

    /**
     * Katmanli yeniden deneme topic'leri. Daha az trafik → daha az partition.
     *
     * TUZAK - DONUS TIPI KafkaAdmin.NewTopics OLMAK ZORUNDA
     * Ilk yazilisi soyleydi ve SESSIZCE calismadi:
     *
     * @Bean
     * public List<NewTopic> retryTopics() { ... }
     *
     * KafkaAdmin, uygulama baglaminda NewTopic ve KafkaAdmin.NewTopics
     * tipindeki bean'leri arar. List<NewTopic> bunlardan hicbiri degil -
     * Spring bean'i olusturur, KafkaAdmin gormezden gelir, hata cikmaz.
     *
     * Sonuc: bes retry topic'i hic olusmaz. Ilk yeniden deneme gerekene
     * kadar da fark edilmez; sonra "Topic ... not present in metadata"
     * hatasi gelir ve sebebi bu bean'de aranmaz.
     *
     * Birden fazla topic'i tek bean'de tanimlamanin dogru yolu budur.
     */
    @Bean

```

```

public KafkaAdmin.NewTopics retryTopics() {
    NewTopic[] topics = Topics.RETRY_TIERS.stream()
        .map(name -> TopicBuilder.name(name).partitions(6).replicas(1)
            .config(TopicConfig.RETENTION_MS_CONFIG,
                String.valueOf(3L * 24 * 3600 * 1000))
            .build())
        .toArray(NewTopic[]::new);
    return new KafkaAdmin.NewTopics(topics);
}

/** DLQ 30 gun tutulur: elle kurtarma icin insan zamani gerekir. */
@Bean
public NewTopic dlqTopic() {
    return TopicBuilder.name(Topics.DLQ)
        .partitions(3).replicas(1)
        .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(30L * 24 * 3600 * 1000))
        .build();
}

@Bean
public NewTopic deliveryResultsTopic() {
    return TopicBuilder.name(Topics.DELIVERY_RESULTS)
        .partitions(12).replicas(1)
        .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(2L * 24 * 3600 * 1000))
        .build();
}

/**
 * COMPACTED topic'ler.
 *
 * cleanup.policy=compact → Kafka eski kayitlari SILMEZ, ayni key'in
 * eski surumlerini siler. Sonucta topic "her key icin son deger"
 * tutan bir tabloya donusur. Yeni bir dispatcher offset 0'dan okur
 * ve 3 saniyede butun endpoint tablosunu ogrenir.
 *
 * segment.ms + min.cleanable.dirty.ratio dusuk: demo sirasinda
 * sikistirmanin gorulebilmesi icin. Uretimde varsayilanlar birakilir.
 */
@Bean
public NewTopic endpointConfigTopic() {
    return TopicBuilder.name(Topics.ENDPOINT_CONFIG)
        .partitions(3).replicas(1)
        .config(TopicConfig.CLEANUP_POLICY_CONFIG, TopicConfig.CLEANUP_POLICY_COMPACT)
        .config(TopicConfig.SEGMENT_MS_CONFIG, "60000")
        .config(TopicConfig.MIN_CLEANABLE_DIRTY_RATIO_CONFIG, "0.1")
        .build();
}

@Bean
public NewTopic appConfigTopic() {
    return TopicBuilder.name(Topics.APP_CONFIG)
        .partitions(3).replicas(1)
        .config(TopicConfig.CLEANUP_POLICY_CONFIG, TopicConfig.CLEANUP_POLICY_COMPACT)
        .config(TopicConfig.SEGMENT_MS_CONFIG, "60000")
        .config(TopicConfig.MIN_CLEANABLE_DIRTY_RATIO_CONFIG, "0.1")
        .build();
}

```

```
}
```

6.8.19 `services/admin-api/src/main/resources/application.yml`

Dikkat edilecek üç satır:

`ddl-auto: none` — şemayı Flyway yönetir. Başka bir değer, üretimde veri kaybettiren tek satırdır.

`open-in-view: false` — varsayılan `true` ve sessizce her isteğin süresince bir veritabanı bağlantısı tutar. Kapatmazsanız yavaş bir görüntüleme katmanı bağlantı havuzunu tüketir.

`enable.idempotence: true` — Kafka 3.x'te zaten varsayılan, ama açıkça yazmak birinin `acks`'i düşürüp bunu farkında olmadan kapatmasını önler.

```
server:
  port: 8081

spring:
  application:
    name: admin-api

datasource:
  url: ${DB_URL:jdbc:postgresql://localhost:5432/hookrelay_control}
  username: ${DB_USER:hookrelay}
  password: ${DB_PASSWORD:hookrelay}
  hikari:
    maximum-pool-size: 10
    pool-name: admin-pool

jpa:
  # Semayı Flyway yönetir. ddl-auto: none dışında bir değer, üretimde
  # veri kaybettiren tek satırdır.
  hibernate:
    ddl-auto: none
  open-in-view: false
  properties:
    hibernate.jdbc.time_zone: UTC

flyway:
  enabled: true
  locations: classpath:db/migration
  baseline-on-migrate: true

data:
  redis:
    host: ${REDIS_HOST:localhost}
    port: ${REDIS_PORT:6379}

kafka:
  bootstrap-servers: ${KAFKA_BOOTSTRAP:localhost:9092}
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.apache.kafka.common.serialization.StringSerializer
    # acks=all: lider + bütün senkron replikalar yazdı demeden başarılı sayma.
    acks: all
    # Idempotent üretici: ağlar tekrarında aynı kaydın iki kez yazılmasını
    # broker tarafında engeller. Kafka 3.x'te varsayılan açık ama açıkça
    # yazmak, birinin acks'i düşürüp bunu farkında olmadan kapatmasını önler.
```

```

    properties:
      enable.idempotence: true
      max.in.flight.requests.per.connection: 5
      compression.type: lz4
    consumer:
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      auto-offset-reset: earliest
      enable-auto-commit: false
      properties:
        max.poll.records: 200
    listener:
      ack-mode: batch
      concurrency: 3

hookrelay:
  outbox:
    enabled: true
    poll-interval-ms: 200
    batch-size: 200
  # admin-api replika TUTMAZ: konfigurasyonun kaynagi zaten kendi veritabani.
  endpoint-config:
    replicate: false
  app-config:
    replicate: false
  circuit-breaker:
    failure-threshold: 5
    open-millis: 30000
    half-open-probes: 2
    window-millis: 60000

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      service: admin-api

logging:
  level:
    dev.hookrelay: INFO

```

Kontrol noktası

```
mvn -q compile -pl services/admin-api -am
```

Bölüm 7 — ingest-api: Olay Kabulü

7.1 Neden 202, neden 200 değil

```
@PostMapping("/events")
@ResponseStatus(HttpStatus.ACCEPTED) // 202
public EventResponse ingest(...) { }
```

200 dönmek "işiniz bitti" demektir ve **yalan** olurdu — teslimat henüz olmadı. 202 tam olarak "aldım, sırada, sonucu ayrı yerden takip et" anlamına gelir.

Bu bir HTTP kodu detayı değil, bir **sözleşme kararı**. 200 dönseydiniz müşteri "webhook gitti" varsayardı; şikâyet geldiğinde tartışma "ama siz 200 dönmüştünüz"de düğümlenirdi.

7.2 API anahtarı doğrulama — replikadan

```
@Component
public class ApiKeyFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain chain) {
        String header = request.getHeader(AUTHORIZATION);
        if (header == null || !header.startsWith("Bearer ")) { reject(...); return; }

        String apiKey = header.substring(7).trim();
        Optional<ApplicationConfig> app = apps.findByApiKeyHash(ApiKeys.hash(apiKey));

        if (app.isEmpty() || !app.get().enabled()) { reject(...); return; }

        request.setAttribute(ATTR_APPLICATION_ID, app.get().applicationId());
        chain.doFilter(request, response);
    }
}
```

Doğrulama Redis replikasıdan. Sonuç:

- Sıcak yolda ağ çağırısı yok, gecikme ~0.2 ms.
- **admin-api** tamamen çöktüğünde bile olay kabulü **çalışmaya devam eder**.

Bedeli: anahtar iptal edildiğinde replikaya ulaşması birkaç yüz milisaniye sürer. Bilinçli bir takas — kabul edilen olay zaten müşterinin kendi olayı; saniyelik bir gecikme güvenlik açığı değil.

Neden Spring Security değil

Burada rol yok, oturum yok, CSRF yok, form login yok. Tek bir bearer token karşılaştırması için Spring Security kurmak, projeye anlamadığınız 200 satırlık bir filtre zinciri sokmaktır. Yirmi satırlık bir filtre hem daha okunaklı hem daha hızlı.

(Kontrol düzlemine ileride gerçek kullanıcı girişi eklenirse, Spring Security **orada** kurulur.)

7.3 İki katmanlı idempotency

```

public Accepted ingest(UUID applicationId, String eventType, Object payload,
                        String idempotencyKey) {

    if (idempotencyKey != null && !idempotencyKey.isBlank()) {
        Optional<String> cached = idempotency.reserveOrGet(
            applicationId.toString(), idempotencyKey, "PENDING", TTL);

        if (cached.isPresent() && !"PENDING".equals(cached.get())) {
            return readCached(cached.get()); // 1. KATMAN: Redis
        }
    }

    Accepted result;
    try {
        var written = writer.store(applicationId, eventType, payloadJson, idempotencyKey);
        result = new Accepted(written.message().getId(), eventType, written.deliveryIds(), false);
    } catch (DataIntegrityViolationException e) {
        // 2. KATMAN: Postgres UNIQUE kısıtı.
        // Bu bir HATA DEĞİL — idempotency'nin çalıştığının kanıtı.
        Message existing = messages
            .findByApplicationIdAndIdempotencyKey(applicationId, idempotencyKey)
            .orElseThrow(...);
        result = new Accepted(existing.getId(), existing.getEventType(), List.of(), true);
    }
    // ...
}

```

Ve şema:

```

CREATE UNIQUE INDEX uk_message_idempotency
ON message (application_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;

```

Neden ikisi birden

Redis tek başına yetmez. Redis bir cache'tir: bellekten taşabilir, restart'ta boşalabilir, TTL dolabilir. Tek başına Redis'e güvenen bir idempotency, "çoğu zaman çalışan" bir idempotency'dir — ve para ile ilgili bir sistemde bu yeterli değildir.

Veritabanı tek başına yetmez. Her istekte disk I/O demek. Saniyede binlerce olay geldiğinde gereksiz yük.

Hız Redis'ten, doğruluk veritabanından.

`WHERE idempotency\_key IS NOT NULL` neden

Postgres'te **NULL**'lar birbirine eşit sayılmaz, dolayısıyla **UNIQUE** kısıtı anahtar vermeyen istekleri zaten engellemez. Kısmi indeks bunu açıkça ifade eder ve indeksin boyutunu küçültür: anahtar veren korunur, vermeyen korunmaz.

7.4 Fan-out — neden burada

```

List<EndpointConfig> targets = endpoints.listByApplication(applicationId).stream()
    .filter(EndpointConfig::deliverable)
    .filter(c -> c.subscribeTo(eventType))

```



```

        .toList();

    for (EndpointConfig target : targets) {
        DeliveryTask task = new DeliveryTask(UUID.randomUUID(), message.getId(),
            applicationId, target.endpointId(), eventType, payloadJson, 1, now);

        outbox.record("delivery", task.deliveryId().toString(),
            Topics.MESSAGES, target.endpointId().toString(), task);
        //                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ Kafka key
    }

```

Fan-out **dispatcher**'da da yapılabilirdi. Yapılmadı çünkü **Kafka key'i endpointId olmak zorunda**.

dispatcher'da yapsaydık, ana topic'e yazarken key olarak **messageId** kullanmak zorunda kalırdık. O zaman aynı endpoint'in iki olayı farklı partition'lara düşer ve sıra garantisi kaybolurdu.

Bedeli: yazma amplifikasyonu. 100 endpoint'e abone bir uygulama, tek olayda 100 outbox satırı yazar. Kabul edilebilir — çünkü alternatifi bir iş kuralını kaybetmek.

7.5 Tuzak: `@Transactional` ve kendi kendine çağrı

Bu kod **çalışmaz**:

```

@Service
public class IngestService {

    public Accepted ingest(...) {
        return store(...);           // ← self-invocation
    }

    @Transactional
    protected Accepted store(...) { // ← anotasyon HİÇBİR ŞEY yapmıyor
        repo.save(...);
        outbox.record(...);
    }
}

```

@Transactional Spring'de **proxy** ile çalışır. Çağrı önce proxy'ye gelir, proxy transaction'ı açar, sonra gerçek nesneye geçer. Aynı sınıf içinden yapılan **this.store(...)** çağrısı proxy'ye **uğramaz** — doğrudan metoda gider ve transaction hiç açılmaz.

Anotasyon orada durur ve hiçbir şey yapmaz. Test ortamında fark etmezsiniz; üretimde bir gün yarım yazılmış veri bulursunuz.

Çözüm — transaction sınırını ayrı bir bean'e taşıyın:

```

@Component
public class MessageWriter {

    @Transactional
    public Written store(UUID applicationId, String eventType,
        String payloadJson, String idempotencyKey) {

        // ...
    }
}

```

```
@Service
public class IngestService {
    private final MessageWriter writer;    // ← enjekte edilen PROXY

    public Accepted ingest(...) {
        var written = writer.store(...);    // ← proxy üzerinden geçiyor
    }
}
```

Bizim durumumuzda hata sessiz de kalmazdı: `OutboxRecorder Propagation.MANDATORY` kullanıyor ve transaction yoksa pathiyor. Beşinci bölümde konuştuğu anda bunun bir "erken uyarı mekanizması" olduğunu söylemiştik — işte tam olarak bu senaryo için.

7.6 Servisin tamamı

7.6.1 `services/ingest-api/pom.xml`

`admin-api` ile aynı bağımlılıklar, artı Testcontainers (test kapsamında). Entegrasyon testi bu serviste çünkü giriş akışı en çok parçayı bir arada kullanan yer: Redis replikası, veritabanı kısıtı, outbox.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>dev.hookrelay</groupId>
        <artifactId>hookrelay-parent</artifactId>
        <version>1.0.0</version>
        <relativePath>../../pom.xml</relativePath>
    </parent>
    <artifactId>ingest-api</artifactId>
    <name>ingest-api</name>
    <description>Veri düzlemi girişi: olay kabulü, idempotency, outbox, fan-out</description>

    <dependencies>
        <dependency><groupId>dev.hookrelay</groupId><artifactId>outbox</artifactId></dependency>
        <dependency><groupId>dev.hookrelay</groupId><artifactId>common</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-actuator</artifactId></dependency>
        <dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></dependency>
        <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
        <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></dependency>
        <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></dependency>
        <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-testcontainers</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

```

</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>kafka</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.awaitility</groupId>
  <artifactId>awaitility</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <finalName>ingest-api</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

7.6.2 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/IngestApiApplication

`admin-api` ile aynı kalıp. Fark, `hookrelay.*-config.replicate` ayarlarının `true` olması — bu servis hem endpoint hem uygulama replikası tutuyor.

```

package dev.hookrelay.ingestapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
import org.springframework.scheduling.annotation.EnableScheduling;

/**
 * VERİ DÜZLEMİNİN GİRİŞİ.
 *
 * Tek işi var ve hızlı yapması gerekiyor: olayı al, dayanıklı şekilde yaz,
 * 202 dön. Teslimatla ilgilenmez, HTTP atmaz, müşteriye bakmaz.
 *
 * Neden 202 (Accepted) ve 200 değil: teslimat HENÜZ OLMADI. 200 dönmek
 * "işiniz bitti" demek olurdu ve yalan olurdu. 202 tam olarak
 * "aldım, sırada, sonucu ayrı yerden takip et" anlamına gelir.
 */
@SpringBootApplication(scanBasePackages = "dev.hookrelay")
@ConfigurationPropertiesScan("dev.hookrelay")
@EnableScheduling

```

```
@org.springframework.data.jpa.repository.config.EnableJpaRepositories(
    basePackages = {"dev.hookrelay.ingestapi.repo", "dev.hookrelay.outbox"})
@org.springframework.boot.autoconfigure.domain.EntityScan(
    basePackages = {"dev.hookrelay.ingestapi.domain", "dev.hookrelay.outbox"})
public class IngestApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(IngestApiApplication.class, args);
    }
}
```

7.6.3 `services/ingest-api/src/main/resources/db/migration/V2\_\_message.sql`

Kısmi **UNIQUE** indekse dikkat: **WHERE idempotency_key IS NOT NULL**.

Postgres'te **NULL**'lar birbirine eşit sayılmaz, dolayısıyla kısıt zaten anahtar vermeyen istekleri engellemezdi. Kısmi indeks bunu **açıkça** ifade ediyor ve indeksin boyutunu küçültüyor.

```
-- Kabul edilen mantıksal olaylar.

CREATE TABLE message (
    id                UUID PRIMARY KEY,
    application_id    UUID      NOT NULL,
    event_type        VARCHAR(120) NOT NULL,
    payload            TEXT      NOT NULL,
    idempotency_key    VARCHAR(200),
    delivery_count     INT        NOT NULL DEFAULT 0,
    created_at        TIMESTAMPTZ NOT NULL
);

-- Idempotency'nin ASIL garantisi burada. Redis hızlı katman; doğruluk bu satırda.
--
-- Postgres'te NULL'lar birbirine eşit sayılmadığı için bu kısıt,
-- Idempotency-Key göndermeyen istekleri engellemez. Tam istediğimiz davranış:
-- anahtar veren korunur, vermeyen korunmaz.
CREATE UNIQUE INDEX uk_message_idempotency
    ON message (application_id, idempotency_key)
    WHERE idempotency_key IS NOT NULL;

CREATE INDEX idx_message_app_created ON message (application_id, created_at DESC);
```

7.6.4 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/domain/Message`

deliveryCount fan-out sonucunu saklıyor. Türetilbilir bir değer (outbox satırlarını sayarak) ama outbox temizlendiğinde kaybolurdu. Gözlem için burada duruyor.

```
package dev.hookrelay.ingestapi.domain;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.UUID;

/**
 * Musterinin gönderdiği MANTIKSAL olay.
 *
 * Dikkat: bu bir teslimat değil. Bir olay 3 endpoint'e gidiyorsa
 * 1 Message + 3 DeliveryTask olur. Ayrımı kaybetmek, sonradan
```

```

* "kac olay geldi" ile "kac istek attik" sorularini birbirine karistirir.
*/
@Entity
@Table(name = "message",
    uniqueConstraints = @UniqueConstraint(
        name = "uk_message_idempotency",
        columnNames = {"application_id", "idempotency_key"}))
public class Message {

    @Id
    private UUID id;

    @Column(name = "application_id", nullable = false)
    private UUID applicationId;

    @Column(name = "event_type", nullable = false, length = 120)
    private String eventType;

    @Column(nullable = false, columnDefinition = "text")
    private String payload;

    /**
     * Musterinin verdigi Idempotency-Key. NULL olabilir (istege bagli).
     *
     * Postgres'te NULL'lar birbirine esit sayilmaz, yani UNIQUE kisiti
     * anahtar vermeyen istekleri engellemez. Tam istedigimiz davranis.
     */
    @Column(name = "idempotency_key", length = 200)
    private String idempotencyKey;

    /** Fan-out sonucu kac teslimat uretildigi. Gozlem icin. */
    @Column(name = "delivery_count", nullable = false)
    private int deliveryCount;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    protected Message() {}

    public Message(UUID applicationId, String eventType, String payload, String idempotencyKey) {
        this.id = UUID.randomUUID();
        this.applicationId = applicationId;
        this.eventType = eventType;
        this.payload = payload;
        this.idempotencyKey = idempotencyKey;
        this.deliveryCount = 0;
        this.createdAt = Instant.now();
    }

    public void setDeliveryCount(int deliveryCount) { this.deliveryCount = deliveryCount; }

    public UUID getId() { return id; }
    public UUID getApplicationId() { return applicationId; }
    public String getEventType() { return eventType; }
    public String getPayload() { return payload; }
    public String getIdempotencyKey() { return idempotencyKey; }
    public int getDeliveryCount() { return deliveryCount; }
    public Instant getCreatedAt() { return createdAt; }
}

```

7.6.5 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/repo/MessageRe

`findTop50By...` — sayfalama yerine sabit tavan. Bu uç bir hata ayıklama aracı, tam sayfalama gerektirmiyor; sınırsız bırakmak ise bir gün 2 milyon satır çekmek demek.

```
package dev.hookrelay.ingestapi.repo;

import dev.hookrelay.ingestapi.domain.Message;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;
import java.util.UUID;

public interface MessageRepository extends JpaRepository<Message, UUID> {

    Optional<Message> findByIdAndIdempotencyKey(UUID applicationId, String key);

    List<Message> findTop50ByApplicationIdOrderByCreatedAtDesc(UUID applicationId);
}
```

7.6.6 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/security/ApiKey

`shouldNotFilter` ile yalnızca `/v1/**` korunuyor — `/actuator/health` kimlik istemiyor, yoksa Docker sağlık kontrolü çalışmazdı.

Reddetme cevabı `ErrorResponse` ile aynı biçimde dönüyor. Filtre katmanını `@RestControllerAdvice` 'tan önce çalıştığı için hata sözleşmesini burada elle kurmak gerekiyor; aksi halde 401'ler diğer hatalardan farklı görünürdü.

```
package dev.hookrelay.ingestapi.security;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.crypto.ApiKeys;
import dev.hookrelay.common.error.ErrorResponse;
import dev.hookrelay.common.redis.AppConfigCache;
import dev.hookrelay.contracts.ApplicationConfig;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Optional;

/**
 * API anahtari dogrulaması.
 *
 * TASARIMIN CAN ALICI NOKTASI: dogrulama Redis replikasinan yapiliyor,
 * admin-api'ye HTTP atilmiyor. Sonuc:
 * - Sicak yolda ag cagrisi yok, gecikme ~0.2 ms.
 * - admin-api tamamen coktugunde bile olay kabulü CALISMAYA DEVAM EDER.
 *
 * Bedeli: anahtar iptal edildiginde replikaya ulasmasi birkac yuz
```

```

* milisaniye surer. Bilincli bir takas - kabul edilen olay zaten
* musterinin kendi olayi, saniyelik gecikme guvenlik acigi degil.
*
* Neden Spring Security degil: burada rol, oturum, CSRF, form login yok.
* Tek bir bearer token karsilastirmasi icin Spring Security kurmak,
* projeye anlamadigin 200 satirlik bir filtre zinciri sokmak demek.
* (admin-api ileride gercek kullanıcı girişi isterse orada kurulur.)
*/
@Component
public class ApiKeyFilter extends OncePerRequestFilter {

    public static final String ATTR_APPLICATION_ID = "hookrelay.applicationId";

    private final AppConfigCache apps;
    private final ObjectMapper mapper;

    public ApiKeyFilter(AppConfigCache apps, ObjectMapper mapper) {
        this.apps = apps;
        this.mapper = mapper;
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        return !path.startsWith("/v1/");
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response
        ,
        FilterChain chain) throws ServletException, IOException
    {
        String header = request.getHeader(HttpHeaders.AUTHORIZATION);
        if (header == null || !header.startsWith("Bearer ")) {
            reject(request, response, "Authorization: Bearer <api-key> basligi gerekli");
            return;
        }

        String apiKey = header.substring("Bearer ".length()).trim();
        Optional<ApplicationConfig> app = apps.findByApiKeyHash(ApiKeys.hash(apiKey));

        if (app.isEmpty()) {
            reject(request, response, "API anahtarı geçersiz");
            return;
        }
        if (!app.get().enabled()) {
            reject(request, response, "Uygulama devre dışı");
            return;
        }

        request.setAttribute(ATTR_APPLICATION_ID, app.get().applicationId());
        chain.doFilter(request, response);
    }

    private void reject(HttpServletRequest request, HttpServletResponse response, String message)
        throws IOException {
        response.setStatus(HttpStatus.UNAUTHORIZED);
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.setCharacterEncoding("UTF-8");
    }
}

```

```

        mapper.writeValue(response.getWriter(),
            ErrorResponse.of("UNAUTHORIZED", message, request.getRequestURI()));
    }
}

```

7.6.7 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/service/Message

Transaction sınırı. Ayrı bean olmasının sebebi 7.5'te: `@Transactional` proxy ile çalışır, aynı sınıftan yapılan çağrı proxy'ye uğramaz.

```

package dev.hookrelay.ingestapi.service;

import dev.hookrelay.common.redis.EndpointConfigCache;
import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.contracts.EndpointConfig;
import dev.hookrelay.contracts.Topics;
import dev.hookrelay.ingestapi.domain.Message;
import dev.hookrelay.ingestapi.repo.MessageRepository;
import dev.hookrelay.outbox.OutboxRecorder;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

/**
 * Yazma islemi neden AYRI BIR BEAN'de?
 *
 * * @Transactional Spring'de proxy ile calisir: cagri once proxy'ye gelir,
 * * proxy transaction'i acar, sonra gercek nesneye gecer. AYNI SINIF ICINDEN
 * * yapilan cagri (this.store(...)) proxy'ye ugramaz - dogrudan metoda gider
 * * ve transaction HIC ACILMAZ. Anotasyon orada durur, hicbir sey yapmaz.
 *
 * * Bu, Spring'in en sik dusulen tuzagi. Burada acikca kaciniyoruz:
 * * transaction sinirini ayri bir bean'e tasiyoruz ki cagri gercekten
 * * proxy uzerinden geccsin.
 *
 * * (Bizim durumumuzda hata sessiz de kalmazdi: OutboxRecorder
 * * Propagation.MANDATORY kullaniyor ve transaction yoksa patliyor.
 * * Bilincli bir erken uyari mekanizmasi.)
 */
@Component
public class MessageWriter {

    private static final Logger log = LoggerFactory.getLogger(MessageWriter.class);

    private final MessageRepository messages;
    private final EndpointConfigCache endpoints;
    private final OutboxRecorder outbox;
    private final Counter fannedOut;

    public MessageWriter(MessageRepository messages, EndpointConfigCache endpoints,
        OutboxRecorder outbox, MeterRegistry meters) {

```



```

        this.messages = messages;
        this.endpoints = endpoints;
        this.outbox = outbox;
        this.fannedOut = Counter.builder("hookrelay.ingest.deliveries.created").register(meters);
    }

    public record Written(Message message, List<UUID> deliveryIds) {}

    /**
     * TEK TRANSACTION: mesaj satiri + N adet outbox satiri.
     * Ya hepsi ya hicbiri. "Mesaj yazildi ama 3 teslimattan 2'si kuyruğa
     * girdi" diye bir ara durum yok.
     */
    @Transactional
    public Written store(UUID applicationId, String eventType,
                        String payloadJson, String idempotencyKey) {

        Message message = new Message(applicationId, eventType, payloadJson, idempotencyKey)
            ;

        // FAN-OUT BURADA - dispatcher'da değil.
        //
        // Neden: Kafka kaydinin key'i endpointId olmalı ki "ayni endpoint'e
        // sirali teslimat" garantisi partition'dan bedava gelsin. Fan-out'u
        // dispatcher'a birakirsak key olarak messageId kullanmak zorunda
        // kalirdik ve o garanti kaybolurdu.
        List<EndpointConfig> targets = endpoints.listByApplication(applicationId).stream()
            .filter(EndpointConfig::deliverable)
            .filter(c -> c.subscribesTo(eventType))
            .toList();

        List<UUID> deliveryIds = new ArrayList<>(targets.size());
        Instant now = Instant.now();

        for (EndpointConfig target : targets) {
            DeliveryTask task = new DeliveryTask(
                UUID.randomUUID(), message.getId(), applicationId, target.endpointId(),
                eventType, payloadJson, 1, now);

            outbox.record("delivery", task.deliveryId().toString(),
                Topics.MESSAGES, target.endpointId().toString(), task);
            deliveryIds.add(task.deliveryId());
        }

        message.setDeliveryCount(deliveryIds.size());
        messages.save(message);
        fannedOut.increment(deliveryIds.size());

        if (targets.isEmpty()) {
            log.debug("Olay kabul edildi ama abone endpoint yok: app={} type={}",
                applicationId, eventType);
        }
        return new Written(message, deliveryIds);
    }
}

```

7.6.8 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/service/IngestSe

İki katmanlı idempotency'nin tam hali.

`catch` bloğundaki `if (!hasKey) throw e;` satırına dikkat: anahtar yoksa bu bizim beklediğimiz çatışma değildir, başka bir kısıt patlamıştır. Yutmak, gerçek bir şema hatasını "idempotency çatışması" diye maskelerdi.

`accepted` sayacı mükerrer yolda **artmıyor** — iki yolun aynı şeyi ölçmesi için.

```
package dev.hookrelay.ingestapi.service;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.error.ApiException;
import dev.hookrelay.common.redis.RedisIdempotencyStore;
import dev.hookrelay.ingestapi.domain.Message;
import dev.hookrelay.ingestapi.repo.MessageRepository;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Duration;
import java.util.List;
import java.util.Optional;
import java.util.UUID;

@Service
public class IngestService {

    private static final Logger log = LoggerFactory.getLogger(IngestService.class);
    private static final Duration IDEMPOTENCY_TTL = Duration.ofHours(24);

    private final MessageRepository messages;
    private final MessageWriter writer;
    private final RedisIdempotencyStore idempotency;
    private final ObjectMapper mapper;
    private final Counter accepted;
    private final Counter duplicates;

    public IngestService(MessageRepository messages, MessageWriter writer,
                        RedisIdempotencyStore idempotency,
                        ObjectMapper mapper, MeterRegistry meters) {
        this.messages = messages;
        this.writer = writer;
        this.idempotency = idempotency;
        this.mapper = mapper;
        this.accepted = Counter.builder("hookrelay.ingest.accepted").register(meters);
        this.duplicates = Counter.builder("hookrelay.ingest.duplicates").register(meters);
    }

    public record Accepted(UUID messageId, String eventType, List<UUID> deliveryIds,
                        boolean duplicate) {}

    /**
     * IDEMPOTENCY: İKİ KATMANLI, BİLİNCİ OLARAK
     * 1. Redis (hızlı): aynı anahtarı saniyeler içinde tekrar goren istegi
     *    veritabanına hic gitmeden geri çevirir. Sıcak yolun korumasi.
     * 2. Postgres UNIQUE kisiti (dogru): Redis restart olsa, anahtar TTL'i
     *    dolsa, Redis tamamen kaybolrsa bile ikinci kayıt ACILAMAZ.
     *
     * Neden ikisi birden: Redis bir CACHE'tir, dogruluk garantisi vermez -
    */
}
```

```

* bellekten tasabilir, restart'ta bosalabilir. Tek basına Redis'e
* guvenen bir idempotency, "cogu zaman calisan" bir idempotency'dir.
* Tek basına veritabanı ise her istekte disk I/O demek.
* Hiz Redis'ten, dogruluk veritabanından.
*/
public Accepted ingest(UUID applicationId, String eventType, Object payload,
    String idempotencyKey) {

    String payloadJson = toJson(payload);

    boolean hasKey = idempotencyKey != null && !idempotencyKey.isBlank();

    if (hasKey) {
        Optional<String> cached = idempotency.reserveOrGet(
            applicationId.toString(), idempotencyKey, "PENDING", IDEMPOTENCY_TTL);

        if (cached.isPresent() && !"PENDING".equals(cached.get())) {
            duplicates.increment();
            return readCached(cached.get());
        }
        // "PENDING" gorduysek: ayni anahtarli baska bir istek SU AN
        // isleniyor demektir. Veritabanı kisiti nasil olsa yakalayacak,
        // devam ediyoruz.
    }

    try {
        var written = writer.store(applicationId, eventType, payloadJson, idempotencyKey);
        Accepted result = new Accepted(
            written.message().getId(), eventType, written.deliveryIds(), false);

        if (hasKey) {
            idempotency.put(applicationId.toString(), idempotencyKey,
                toJson(result), IDEMPOTENCY_TTL);
        }
        accepted.increment();
        return result;
    } catch (DataIntegrityViolationException e) {
        // Anahtar YOKSA bu bizim bekledigimiz catisma degildir -
        // baska bir kisit patlamistir. Yutmak, gercek bir semayi
        // "idempotency catismasi" diye maskeler. Oldugu gibi yukari
        // birakiyoruz ki GlobalExceptionHandler 500 donsun ve loga
        // gercek sebep dusun.
        if (!hasKey) throw e;

        // Yaris kaybedildi: ayni anahtarla baska bir istek once yazdi.
        // Bu bir HATA DEGIL, idempotency'nin calistiginin kanitidir.
        duplicates.increment();
        Message existing = messages
            .findByApplicationIdAndIdempotencyKey(applicationId, idempotencyKey)
            .orElseThrow(() -> ApiException.conflict(
                "Idempotency catismasi cozulemedi: " + idempotencyKey));

        // accepted sayaci ARTMIYOR: bu yeni bir olay degil.
        // (Onceki surumde artiyordu ve "kabul edilen olay" metrigi
        // mukerrerleri de sayiyordu - Redis'ten donen mukerrerler
        // ise saymiyordu. Iki yol iki farkli sey olcuyordu.)
        return new Accepted(existing.getId(), existing.getEventType(), List.of(), true);
    }
}

```

```

@Transactional(readOnly = true)
public List<Message> recent(UUID applicationId) {
    return messages.findTop50ByApplicationIdOrderByCreatedAtDesc(applicationId);
}

@Transactional(readOnly = true)
public Message get(UUID applicationId, UUID messageId) {
    Message m = messages.findById(messageId)
        .orElseThrow(() -> ApiException.notFound("Mesaj", messageId));
    if (!m.getApplicationId().equals(applicationId)) {
        // Baska kiracinin mesaji. 403 degil 404: kaynagin VARLIGINI bile sizdirmayiz.
        throw ApiException.notFound("Mesaj", messageId);
    }
    return m;
}

private String toJson(Object o) {
    try {
        return o instanceof String s ? s : mapper.writeValueAsString(o);
    } catch (Exception e) {
        throw ApiException.badRequest("Govde JSON'a cevrilememi: " + e.getMessage());
    }
}

private Accepted readCached(String json) {
    try {
        Accepted a = mapper.readValue(json, Accepted.class);
        return new Accepted(a.messageId(), a.eventType(), a.deliveryIds(), true);
    } catch (Exception e) {
        // Saklanan cevap okunamiyor. Sessizce yeni bir mesaj
        // olusturmak YANLIS olurdu - idempotency sozunu bozardi.
        // 409 donup istemcinin tekrar denemesini istiyoruz.
        log.warn("Saklanan idempotency cevabi ayristirilamadi: {}", json, e);
        throw ApiException.conflict("Ayni Idempotency-Key ile islem devam ediyor");
    }
}
}

```

7.6.9 `services/ingest-api/src/main/java/dev/hookrelay/ingestapi/api/IngestControl

`@RequestAttribute` ile filtreden gelen uygulama kimliği alınıyor. Controller kimlik doğrulamayı bilmiyor; yalnızca sonucunu kullanıyor.

202 `Accepted` kararı 7.1'de.

```

package dev.hookrelay.ingestapi.api;

import dev.hookrelay.ingestapi.domain.Message;
import dev.hookrelay.ingestapi.security.ApiKeyFilter;
import dev.hookrelay.ingestapi.service.IngestService;
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```

import java.time.Instant;
import java.util.List;
import java.util.UUID;

@RestController
@RequestMapping("/v1")
public class IngestController {

    private final IngestService service;

    public IngestController(IngestService service) {
        this.service = service;
    }

    public record EventRequest(
        @NotBlank @Size(max = 120) String eventType,
        @NotNull Object payload) {}

    public record EventResponse(
        UUID messageId, String eventType, List<UUID> deliveryIds,
        int deliveryCount, boolean duplicate, String status) {}

    public record MessageResponse(
        UUID id, String eventType, String payload, int deliveryCount, Instant createdAt)
    {

        static MessageResponse from(Message m) {
            return new MessageResponse(m.getId(), m.getEventType(), m.getPayload(),
                m.getDeliveryCount(), m.getCreatedAt());
        }
    }

    /**
     * 202 Accepted - ve bunun bir tercih oldugunu bilerek.
     *
     * Bu uc, teslimatin BASARILI oldugunu soylemez. Sadece "olayi
     * dayanikli sekilde kaydettim, teslimattan ben sorumluyum" der.
     * Musteri sonucu delivery uclarindan veya kendi webhook'undan ogrenir.
     */
    @PostMapping("/events")
    @ResponseStatus(HttpStatus.ACCEPTED)
    public EventResponse ingest(
        @RequestAttribute(ApiKeyFilter.ATTR_APPLICATION_ID) UUID applicationId,
        @RequestHeader(value = "Idempotency-Key", required = false) String idempotencyKey,
        @Valid @RequestBody EventRequest request) {

        var accepted = service.ingest(applicationId, request.eventType(),
            request.payload(), idempotencyKey);

        return new EventResponse(accepted.messageId(), accepted.eventType(),
            accepted.deliveryIds(), accepted.deliveryIds().size(), accepted.duplicate(),
            accepted.duplicate() ? "zaten_kabul_edilmis" : "kuyruga_alindi");
    }

    @GetMapping("/messages")
    public List<MessageResponse> recent(
        @RequestAttribute(ApiKeyFilter.ATTR_APPLICATION_ID) UUID applicationId) {
        return service.recent(applicationId).stream().map(MessageResponse::from).toList();
    }
}

```

```

@GetMapping("/messages/{id}")
public ResponseEntity<MessageResponse> get(
    @RequestAttribute(ApiKeyFilter.ATTR_APPLICATION_ID) UUID applicationId,
    @PathVariable UUID id) {
    return ResponseEntity.ok(MessageResponse.from(service.get(applicationId, id)));
}
}

```

7.6.10 `services/ingest-api/src/main/resources/application.yml`

`admin-api`'den farklı üç yer:

`poll-interval-ms: 100` — giriş sıcak yol, gecikme düşük tutuluyor.

`hibernate.jdbc.batch_size: 50` + `order_inserts: true` — fan-out'ta tek transaction'da N outbox satırı yazılıyor; toplu insert bunu tek gidiş-dönüşe indiriyor.

`replicate: true` (her ikisi) — bu servis iki replikayı da tutuyor.

```

server:
  port: 8082
  tomcat:
    threads:
      max: 200

spring:
  application:
    name: ingest-api

  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/hookrelay_ingest}
    username: ${DB_USER:hookrelay}
    password: ${DB_PASSWORD:hookrelay}
    hikari:
      maximum-pool-size: 20
      pool-name: ingest-pool

  jpa:
    hibernate:
      ddl-auto: none
    open-in-view: false
    properties:
      hibernate.jdbc.time_zone: UTC
      hibernate.jdbc.batch_size: 50
      hibernate.order_inserts: true

  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true

  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: ${REDIS_PORT:6379}
      lettuce:
        pool:
          max-active: 32

  kafka:

```

```

bootstrap-servers: ${KAFKA_BOOTSTRAP:localhost:9092}
producer:
  key-serializer: org.apache.kafka.common.serialization.StringSerializer
  value-serializer: org.apache.kafka.common.serialization.StringSerializer
  acks: all
  properties:
    enable.idempotence: true
    max.in.flight.requests.per.connection: 5
    linger.ms: 5
    compression.type: lz4
consumer:
  key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
  value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
  auto-offset-reset: earliest
  enable-auto-commit: false

hookrelay:
  outbox:
    enabled: true
    poll-interval-ms: 100      # giris sicak yol: gecikme dusuk tutulur
    batch-size: 500
  # ingest IKI replikayi da tutar:
  # endpoint-config → fan-out hedeflerini bulmak icin
  # app-config      → API anahtarini dogrulamak icin
  endpoint-config:
    replicate: true
  app-config:
    replicate: true

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      service: ingest-api

logging:
  level:
    dev.hookrelay: INFO

```

Kontrol noktası

```
mvn -q compile -pl services/ingest-api -am
```

Bölüm 8 — dispatcher: Teslimat Hattı

Sistemin kalbi. Kafka'dan okur, kontrollerden geçirir, imzalar, gönderir, sonuca göre karar verir.

8.1 Akış

1. Bu teslimat zaten başarılı mı? → atla (mükerrer)
2. Endpoint hâlâ var mı, açık mı? → yoksa düşür
3. Yaşam süresi doldu mu? → DLQ
4. Ön kontroller (devre kesici, hız sınırı) → engellendiyse erteleye
5. Bulkhead izni → yoksa erteleye
6. Gönder
7. Sonuca göre: bitti / yeniden dene / DLQ

`DeliveryProcessor` sınıfı **hiçbir iş yapmaz** — HTTP atmaz, veritabanına yazmaz, Kafka'ya basmaz. Yalnızca sırayı ve kararı yönetir. Her adım ayrı bir bean'de ve tek başına test edilebilir.

8.2 Chain of Responsibility

```
public interface PreflightCheck {
    record Block(DeliveryResult.Outcome outcome, String reason,
                Duration minDelay, boolean consumesAttempt) {}

    Optional<Block> check(DeliveryContext context);

    int order();
}
```

Bugün iki uygulaması var: devre kesici (order 10) ve hız sınırı (order 20).

Neden zincir, neden tek bir `if` bloğu değil: yarın "müşteri IP'si kara listede mi", "bu olay tipi geçici duraklatıldı mı", "bakım penceresi mi" eklenecek. Her biri `DeliveryProcessor`'a bir `if` daha eklemek yerine bir sınıf olarak gelir ve processor'a **hiç dokunulmaz**.

Sıra `order()` ile, Spring'in bean sırasıyla değil

```
this.checks = checks.stream()
    .sorted(Comparator.comparingInt(PreflightCheck::order))
    .toList();
```

Spring bir `List<PreflightCheck>` enjekte ederken sırayı garanti etmez — paket adına, sınıf adına, hatta derleyici çıktısının sırasına göre değişebilir. Üzerine sistem kurulacak bir şey değil.

Sıra ucuzdan pahalıya dizilmiş: devre kesici önce (bir Redis çağrısı, çoğu zaman kapalı → hemen geç), hız sınırı sonra (jeton harcıyor — devre açıkken boşuna jeton harcamayalım).

`consumesAttempt` — adalet meselesi

Devre kesici engellemesi bir **deneme hakkı yakmaz**:


```
int attempt = block.consumesAttempt() ? task.attempt() + 1 : task.attempt();
```

Müşteri beş dakika bakımda diye teslimatın bütün haklarını tüketmesi, bakım bitince mesajların çoktan DLQ'ya düşmüş olması demektir. Suç bizde değil, onlarda da değil — sadece zamanlama. Sonsuz döngüyü ne engelliyor: yaşam süresi kontrolü (varsayılan 24 saat). O dolunca DLQ.

8.3 Bulkhead — gürültülü komşu

Problem. Tek bir müşterinin sunucusu 30 saniyede cevap veriyor. `dispatcher`'da 12 tüketici iş parçacığı var. O müşteriye 12 mesaj gelirse 12 iş parçacığının hepsi 30 saniye bloke olur ve **diğer bütün müşterilerin** teslimatı durur.

Çözüm. Her endpoint'e ayrı kota:

```
@Component
public class EndpointBulkhead {

    private final Map<UUID, Semaphore> permits = new ConcurrentHashMap<>();

    public boolean tryAcquire(UUID endpointId) {
        return semaphore(endpointId).tryAcquire(); // ← beklemeden
    }

    private Semaphore semaphore(UUID endpointId) {
        return permits.computeIfAbsent(endpointId,
            id -> new Semaphore(maxConcurrentPerEndpoint));
    }
}
```

Kota doluysa mesaj **beklemez** — kuyruğa geri konur, iş parçacığı serbest kalır ve başka müşterilere hizmete devam eder.

Neden `tryAcquire()` ve `tryAcquire(5, SECONDS)` değil: beklemek, bloke olmanın başka adıdır. Bekleyen iş parçacığı da çalışmıyor demektir.

Gemi metaforu buradan: gövdeyi bölmelere ayırırsınız, bir bölme su alınca gemi batmaz.

`finally` şart

```
if (!bulkhead.tryAcquire(endpoint.endpointId())) { /* ertele */ return; }
try {
    SendResult result = sender.send(ctx);
    handleResult(ctx, result);
} finally {
    bulkhead.release(endpoint.endpointId());
}
```

`send()` istisna atarsa izin sonsuza kadar kilitli kalır ve o endpoint bir daha **hiç** teslimat almaz. Bulması çok zor bir hata: sistem çalışıyor görünür, sadece bir müşteri sessizce hizmet almaz.

8.4 HTTP gönderim ve sanal iş parçacıkları

```
@Bean
public HttpClient webhookHttpClient(@Value("${...connect-timeout-ms:5000}") long timeout) {
    return HttpClient.newBuilder()
        .connectTimeout(Duration.ofMillis(timeout))
        .followRedirects(HttpClient.Redirect.NEVER)
        .executor(Executors.newVirtualThreadPerTaskExecutor())
        .build();
}
```

Sanal iş parçacıkları (Java 21)

Bu servisin işi neredeyse tamamen "istek at, cevabı bekle". Klasik platform iş parçacığı bu beklemede 1 MB yığın tutar ve bir işletim sistemi iş parçacığıdır — birkaç bin tanesini açamazsınız.

Sanal iş parçacığı beklerken serbest bırakılır; on binlercesi açılabilir.

Yavaş bir müşteri artık "bir iş parçacığını işgal eden" değil, "bir sanal iş parçacığını bekleten" bir şey. Bu bulkhead'in işini kolaylaştırır ama **yerini almaz**: sanal iş parçacığı bedava olsa da uzaktaki sunucunun bağlantı kotası bedava değil.

`followRedirects(NEVER)` — güvenlik

Webhook'ta yönlendirme takip etmek bir açıktır. Müşteri URL'i bir yere yönlendirirse **imzalı isteğimiz** bilmediğimiz bir sunucuya gider. Buna SSRF (Server-Side Request Forgery) denir.

Hata sınıflandırması — ve neden ayrı bir sınıf

İlk yazılışta bu mantık `SendResult` kaydının içindeydi:

```
public record SendResult(Integer httpStatus, ...) {
    public boolean permanentFailure() { ... } // ← yanlış yer
}
```

Yanlış yer olmasının sebebi: **sınıflandırma bir politikadır, veri değildir**. `SendResult` bir HTTP denemesinde *ne olduğunu* taşır. "Bunu yeniden denemeye değer mi" sorusu ise bir karardır ve konfigürasyona bağlıdır. Kayıt içine gömüldüğünde ne ayarlanabilir ne de ayrı test edilebilir.

```
@Component
public class FailureClassifier {

    private final boolean retryOn4xx;

    public FailureClassifier(
        @Value("${hookrelay.delivery.retry-on-4xx:false}") boolean retryOn4xx) {
        this.retryOn4xx = retryOn4xx;
    }

    public boolean permanent(SendResult result) {
        Integer status = result.httpStatus();

        if (status == null) return false; // ağ hatası → geçici
        if (status >= 200 && status < 300) return false;
        if (status == 410) return true; // ayardan BAĞIMSIZ
        if (status == 408 || status == 429) return false;
        if (status >= 400 && status < 500) return !retryOn4xx;
    }
}
```

```

        return false; // 5xx geçici
    }
}

```

Durum	Karar	Neden
2xx	başarı	—
ağ hatası (kod yok)	geçici	Sunucu yarın ayakta olabilir
408, 429	geçici	"Şimdi olmaz" demek, "asla" değil
410 Gone	her zaman kalıcı	"Bu kaynak kalıcı olarak yok"un başka ifadesi yok
diğer 4xx	varsayılan kalıcı	Aynı isteği 5 kez daha göndermek aynı cevabı alır
5xx	geçici	—

retry-on-4xx ayarı neden var: bazı müşteriler token yenilerken geçici 403 döner. Onlar için 4xx'i yeniden denemek doğru olabilir. Kararı biz değil, sistemi çalıştıran versin.

410 neden ayardan muaf: 410 Gone, HTTP'de "bu kaynak vardı, kalıcı olarak kaldırıldı" demenin özel yoludur. 404'ten farkı tam olarak bu kesinlik. Israr etmek kabalık.

Genel kural. Bir kayıt (record) ne olduğunu taşır; ne yapılacağını bir servis söyler. Bu ayırım kaybolduğunda konfigürasyon veri sınıflarına sızmaya başlar ve test etmek imkânsızlaşır.

`Retry-After` iki biçimde gelir

```

private Optional<Duration> parseRetryAfter(HttpResponse<String> response) {
    return response.headers().firstValue("Retry-After").flatMap(raw -> {
        try {
            return Optional.of(Duration.ofSeconds(Long.parseLong(raw.trim())));
        } catch (NumberFormatException ignored) { }
        // sayı değilse HTTP tarihi olabilir
        Instant when = Instant.from(DateTimeFormatter.RFC_1123_DATE_TIME.parse(raw.trim()));
        return Optional.of(Duration.between(Instant.now(), when));
    });
}

```

RFC 9110 iki biçime izin verir: saniye (**120**) veya HTTP tarihi (**Wed, 21 Oct 2026 07:28:00 GMT**). Çoğu istemci ikincisini atlar ve 429'ları yanlış zamanlar.

8.5 Strategy: yeniden deneme politikası

```

public interface RetryPolicy {
    record Reschedule(int tier, Instant notBefore, Duration delay) {}

    Optional<Reschedule> afterFailure(int attempt, int maxAttempts, Duration serverRetryAfter);
    Reschedule withoutConsumingAttempt(Duration minDelay);
    boolean expired(Instant firstQueuedAt);
}

```

```
}
```

Bugün tek uygulaması var. Arayüz olarak durmasının gerçek sebebi: müşteri bazlı politika. Bir müşteri "beni 3 kez dene sonra bırak" derken bir diğeri "24 saat boyunca dene" diyebilir. O gün geldiğinde `DeliveryProcessor`'a dokunulmaz, yeni bir `RetryPolicy` yazılır.

Arayüzü bugün kullanmayacak olsaydık yazmazdık. **Kullanılmayan soyutlama, desen değil süslemedir.**

Uygulama:

```
@Override
public Optional<Reschedule> afterFailure(int attempt, int maxAttempts, Duration serverRetryAfter) {
    if (attempt >= maxAttempts || attempt > delays.size()) return Optional.empty();

    int tier = attempt; // 1. deneme bitti → t1'e koy
    Duration delay = delays.get(tier - 1);

    // Sunucu "Retry-After: 120" dediyse ona SAYGI GÖSTER.
    if (serverRetryAfter != null && serverRetryAfter.compareTo(delay) > 0) {
        delay = serverRetryAfter;
    }
    return Optional.of(new Reschedule(tier, Instant.now().plus(delay), delay));
}
```

Dikkat: `Retry-After` uzun olduğunda **katman değişmiyor**, sadece `not-before` ileri alınıyor. Tasarım bunu bedavaya destekliyor çünkü zamanlayıcı topic adına değil kaydın üzerindeki `not-before` başlığına bakıyor. Üçüncü bölümde bu başlığı koyarken söylediğimiz şeyin karşılığı burada.

8.6 Tüketici ve offset kararı

```
@KafkaListener(topics = Topics.MESSAGES, groupId = "${hookrelay.consumer.group:dispatcher}",
    concurrency = "${hookrelay.consumer.concurrency:6}")
public void onMessage(String payload, Acknowledgment ack) {
    try {
        DeliveryTask task = mapper.readValue(payload, DeliveryTask.class);
        processor.process(task);
    } catch (Exception e) {
        log.error("Teslimat görevi işlenemedi, atlanıyor: {}", payload, e);
    } finally {
        ack.acknowledge();
    }
}
```

Offset ne zaman commit edilir

İki seçenek:

	Ne zaman commit	Sonuç	Risk
(a)	İşlemeden önce	En fazla bir kez	Süreç çökerse mesaj kaybolur

	Ne zaman commit	Sonuç	Risk
(b)	İşlemeden sonra	En az bir kez	Mesaj iki kez gidebilir

(b) seçildi. Webhook dünyasında "iki kez geldi" çözülebilir bir problemdir — alıcı **X-HookRelay-Id** ile ayıklar. "Hiç gelmedi" çözülemez.

Mükerrer teslimatı **DeliveryProcessor**'daki "zaten başarılı mı" kontrolü büyük ölçüde engeller — ama tamamen değil. Gönderim ile veritabanı yazımı arasındaki mikrosaniyelerde çökersek tekrar göndeririz. Bu pencereyi kapatmanın yolu yok.

Dağıtık sistemlerde exactly-once bir pazarlama terimidir. Doğru cevap: alıcıyı idempotent yapmak — ki biz her isteğe benzersiz bir kimlik göndererek bunu **mümkün kılıyoruz**.

Neden her durumda ack ediyoruz

İşlem başarısız olsa bile offset ilerletiliyor. Kulağa yanlış geliyor ama doğrusu bu: başarısız teslimat **kaybolmuyor**, bir yeniden deneme topic'ine taşınıyor.

Ack etmeseydik aynı kayıt aynı partition'da sonsuza kadar tekrar okunur ve **arkasındaki bütün mesajları bloklardı**. Buna *head-of-line blocking* denir. Kafka'da "bu mesajı atla, diğerine geç" diye bir şey yok — ilerlemek zorundasınız.

`max.poll.records: 50`

```
spring.kafka.consumer.properties:
  max.poll.records: 50
  max.poll.interval.ms: 300000
```

Yüksek tutmak cazip ama her kayıt bir HTTP isteği demek. 500 kayıt alıp 500 istek atmaya çalışırsak **max.poll.interval.ms**'i aşar ve gruptan atılırız. Küçük paket = sık heartbeat = sağlıklı tüketici.

8.7 Mükerrer teslimat koruması — ve buradaki tuzak

Kısıt basit:

```
CREATE UNIQUE INDEX uk_attempt_delivery_no ON delivery_attempt (delivery_id, attempt);
```

Uygulama kodundaki kontrol unutulabilir; kısıt unutulmaz. Ama **kısıtı nasıl karşıladığınız** kritik.

Bu kod bozuk — ve çok yaygın

```
try {
  attempts.save(new DeliveryAttempt(...));
} catch (DataIntegrityViolationException e) {
  log.debug("Deneme zaten kayıtlı, sorun değil");
}
deliveries.save(delivery); // ← ARTIK ÇALIŞMAZ
```

Kulağa gayet makul geliyor: kısıt patlıyor, yutuyoruz, devam ediyoruz. **JPA'da böyle çalışmaz**.

Hibernate bir kısıt ihlali gördüğünde persistence context'i **tutarsız** kabul eder ve transaction'ı "yalnızca geri sarılabilir" (**rollback-only**) olarak işaretler. İstisnayı yakalayıp devam etmeniz bunu değiştirmez:

- Sonraki yazmalar çalışır **görünür**.
- Commit anında **UnexpectedRollbackException** gelir.
- **Tüm** transaction geri sarılır.

Sonuç: mükerrer teslimat geldiğinde — ki bu Kafka'da **normal** bir durumdur — teslimatın durum güncellemesi de kaybolur. Ve hata mesajı suçlunun yerini göstermez; **deliveries.save()** satırını suçlarsınız.

Doğrusu: istisnayı hiç doğurmamak

```
@Modifying
@Query(value = """
    INSERT INTO delivery_attempt
      (id, delivery_id, endpoint_id, attempt, http_status,
       latency_ms, error, response_snippet, occurred_at)
    VALUES (:id, :deliveryId, :endpointId, :attempt, :httpStatus,
            :latencyMs, :error, :snippet, :occurredAt)
    ON CONFLICT (delivery_id, attempt) DO NOTHING
    """, nativeQuery = true)
int insertIgnoringDuplicate(...);
```

Postgres çatışmayı veritabanının içinde, atomik olarak, sessizce yutar. Hibernate'in haberi bile olmaz. Dönüş değeri (0 = atlandı) isterseniz loglanabilir.

Bedeli: native sorgu ve Postgres bağımlılığı (**ON CONFLICT** standart SQL değil; MySQL'de **INSERT IGNORE**).

Genel kural. Beklediğiniz bir çatışmayı istisna olarak karşılamayın, **önleyin**. İstisna gerçekten beklenmeyen durumlar içindir; beklenen bir durumu istisnaya yönetmek hem pahalıdır hem — JPA örneğinde olduğu gibi — sizi tanımsız bir duruma sokabilir.

8.8 Servisin tamamı

Projenin en büyük servisi. Sıra: pom → giriş → şema → entity → repository → gönderim parçaları → politika → hat → tüketici → API → yapılandırma.

8.8.1 `services/dispatcher/pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
    maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../../pom.xml</relativePath>
  </parent>
```

```

<artifactId>dispatcher</artifactId>
<name>dispatcher</name>
<description>Teslimatci: Kafka'dan okur, imzalar, HTTP atar, sonucu yazar</description>

<dependencies>
  <dependency><groupId>dev.hookrelay</groupId><artifactId>common</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-actuator</artifactId></dependency>
  <dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></dependency>
  <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-database-postgresql</artifactId></dependency>
  <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
</dependencies>

<build>
  <finalName>dispatcher</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

8.8.2 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/DispatcherApp

Outbox kullanmıyor, o yüzden `@EnableScheduling` ve `outbox` paket taraması yok. Yatay ölçeklenen tek servis budur.

```

package dev.hookrelay.dispatcher;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

/**
 * TESLIMATCI - sistemin kalbi.
 *
 * Kafka'dan okur, endpoint konfigurasyonunu Redis replikasından alır,
 * istegi imzalar, HTTP atar, sonucu yazar ve gerekiyorsa bir sonraki
 * yeniden deneme katmanına bırakır.
 *
 * Yatay ölçeklenen tek servis budur: 3 örnek çalıştırınca Kafka
 * partition'ları aralarında paylaştırır. Tavan, topic'in partition
 * sayısidir (12) - 13. örnek boş oturur.
 */
@SpringBootApplication(scanBasePackages = "dev.hookrelay")
@ConfigurationPropertiesScan("dev.hookrelay")
@org.springframework.data.jpa.repository.config.EnableJpaRepositories(
    basePackages = "dev.hookrelay.dispatcher.repo")
@org.springframework.boot.autoconfigure.domain.EntityScan(
    basePackages = "dev.hookrelay.dispatcher.domain")

```

```
public class DispatcherApplication {
    public static void main(String[] args) {
        SpringApplication.run(DispatcherApplication.class, args);
    }
}
```

8.8.3 `services/dispatcher/src/main/resources/db/migration/V1\_\_delivery\_schema`

`delivery_attempt` yalnızca **eklenir**; güncellenmez, silinmez. Bir denetim izi.

`uk_attempt_delivery_no` tekil indeksi mükerrer teslimat korumasının veritabanı seviyesindeki hâli. Uygulama kodundaki kontrol unutulabilir; kısıt unutulmaz.

```
-- Teslimat durumu ve denetim izi. Bu veritabanının sahibi: dispatcher.

CREATE TABLE delivery (
    id                UUID PRIMARY KEY,
    message_id        UUID          NOT NULL,
    application_id     UUID          NOT NULL,
    endpoint_id        UUID          NOT NULL,
    event_type         VARCHAR(120) NOT NULL,
    payload            TEXT,
    status             VARCHAR(20)  NOT NULL,
    attempts           INT          NOT NULL DEFAULT 0,
    last_status        INT,
    last_error         VARCHAR(1000),
    next_attempt_at    TIMESTAMPTZ,
    created_at         TIMESTAMPTZ  NOT NULL,
    updated_at         TIMESTAMPTZ  NOT NULL
);

CREATE INDEX idx_delivery_endpoint ON delivery (endpoint_id, created_at DESC);
CREATE INDEX idx_delivery_app      ON delivery (application_id, created_at DESC);
CREATE INDEX idx_delivery_status   ON delivery (status, updated_at DESC);
CREATE INDEX idx_delivery_message  ON delivery (message_id);

-- Sadece EKLENİR. Guncellenmez, silinmez.
CREATE TABLE delivery_attempt (
    id                UUID PRIMARY KEY,
    delivery_id        UUID          NOT NULL,
    endpoint_id        UUID          NOT NULL,
    attempt            INT          NOT NULL,
    http_status        INT,
    latency_ms         BIGINT        NOT NULL,
    error              VARCHAR(1000),
    response_snippet   VARCHAR(2000),
    occurred_at        TIMESTAMPTZ  NOT NULL
);

-- MUKERRER TESLIMAT KORUMASI.
--
-- Kafka "en az bir kez" teslim ettigi icin ayni (teslimat, deneme)
-- ikilisi iki kez islenebilir. Bu kisit, ikinci kaydın acilmasını
-- veritabanı seviyesinde imkansız kılar. Uygulama kodundaki kontrol
-- unutulabilir; kisit unutulmaz.
CREATE UNIQUE INDEX uk_attempt_delivery_no ON delivery_attempt (delivery_id, attempt);

CREATE INDEX idx_attempt_endpoint ON delivery_attempt (endpoint_id, occurred_at DESC);
```


8.8.4 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/domain/Delivery`

Durum makinesi metotlarla ifade ediliyor: `succeeded`, `retrying`, `exhausted`, `blocked`, `discarded`. Alanları dışarıdan set eden bir arayüz yok — geçersiz bir duruma girmek mümkün değil.

`payload` alanı bilinçli bir denormalizasyon: gövdenin asıl sahibi `ingest-api`, buraya da yazılmasının tek sebebi yeniden gönderim.

`blocked()` metodu `attempts`'e dokunmuyor. İlk hali bunu `retrying(task.attempt() - 1, ...)` diye ifade ediyordu — çalışıyordu ama okuyana "neden bir eksik?" diye sorduruyordu.

```
package dev.hookrelay.dispatcher.domain;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.UUID;

/** Bir endpoint'e yapılacak tek teslimatin guncel durumu. */
@Entity
@Table(name = "delivery")
public class Delivery {

    public enum Status {
        /** Kuyrukta veya deneniyor. */
        PENDING,
        /** 2xx alındı. Bitti. */
        SUCCEEDED,
        /** Butun denemeler tukendi, DLQ'da. */
        EXHAUSTED,
        /** Endpoint silinmis/kapali - hic denenmedi. */
        DISCARDED
    }

    @Id
    private UUID id;

    @Column(name = "message_id", nullable = false)    private UUID messageId;
    @Column(name = "application_id", nullable = false) private UUID applicationId;
    @Column(name = "endpoint_id", nullable = false)   private UUID endpointId;
    @Column(name = "event_type", nullable = false, length = 120) private String eventType;

    /**
     * Govdenin kopyasi.
     *
     * Bu bir DENORMALIZASYON ve bilincli. Govdenin "asil" sahibi
     * ingest-api'nin message tablosu. Buraya da yazmamizin tek sebebi
     * YENIDEN GONDERIM: kullanıcı arayuzden "tekrar dene" dediginde
     * dispatcher'in baska bir servise HTTP atmasi gerekmesin.
     *
     * Bedeli: disk. Kazanci: dispatcher'in kendi kendine yetmesi.
     * Bir mikroservisin baska bir servise sormadan isini bitirebilmesi,
     * odenmeye deger bir disk maliyetidir.
     */
    @Column(columnDefinition = "text")                private String payload;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Status status = Status.PENDING;
}
```

```

@Column(nullable = false) private int attempts;

@Column(name = "last_status")           private Integer lastStatus;
@Column(name = "last_error", length = 1000) private String lastError;
@Column(name = "next_attempt_at")       private Instant nextAttemptAt;
@Column(name = "created_at", nullable = false) private Instant createdAt;
@Column(name = "updated_at", nullable = false) private Instant updatedAt;

protected Delivery() {}

public Delivery(UUID id, UUID messageId, UUID applicationId, UUID endpointId,
                String eventType, String payload) {
    this.id = id;
    this.messageId = messageId;
    this.applicationId = applicationId;
    this.endpointId = endpointId;
    this.eventType = eventType;
    this.payload = payload;
    this.status = Status.PENDING;
    this.createdAt = Instant.now();
    this.updatedAt = this.createdAt;
}

public void succeeded(int attempt, Integer httpStatus) {
    this.status = Status.SUCCEEDED;
    this.attempts = attempt;
    this.lastStatus = httpStatus;
    this.lastError = null;
    this.nextAttemptAt = null;
    this.updatedAt = Instant.now();
}

public void retrying(int attempt, Integer httpStatus, String error, Instant nextAt) {
    this.status = Status.PENDING;
    this.attempts = attempt;
    this.lastStatus = httpStatus;
    this.lastError = truncate(error);
    this.nextAttemptAt = nextAt;
    this.updatedAt = Instant.now();
}

/**
 * Hic istek atilmeden ertelendi (devre kesici / hiz siniri / bulkhead).
 *
 * attempts'e DOKUNMAZ. Onceki surumde bu, recordBlocked icinde
 * "retrying(task.attempt() - 1, ...)" diye ifade ediliyordu -
 * calisiyordu ama okuyana "neden bir eksik?" diye sorduruyordu.
 * Niyeti kodun kendisi soylemeli.
 */
public void blocked(String reason, Instant nextAt) {
    this.status = Status.PENDING;
    this.lastStatus = null;
    this.lastError = truncate(reason);
    this.nextAttemptAt = nextAt;
    this.updatedAt = Instant.now();
}

public void exhausted(int attempt, Integer httpStatus, String error) {
    this.status = Status.EXHAUSTED;
    this.attempts = attempt;
    this.lastStatus = httpStatus;

```

```

        this.lastError = truncate(error);
        this.nextAttemptAt = null;
        this.updatedAt = Instant.now();
    }

    public void discarded(String reason) {
        this.status = Status.DISCARDED;
        this.lastError = truncate(reason);
        this.nextAttemptAt = null;
        this.updatedAt = Instant.now();
    }

    /** Yeniden gönderim için durumu sıfırla. */
    public void resetForReplay() {
        this.status = Status.PENDING;
        this.attempts = 0;
        this.lastError = null;
        this.nextAttemptAt = null;
        this.updatedAt = Instant.now();
    }

    private static String truncate(String s) {
        return s == null ? null : s.substring(0, Math.min(s.length(), 1000));
    }

    public UUID getId() { return id; }
    public UUID getMessageId() { return messageId; }
    public UUID getApplicationId() { return applicationId; }
    public UUID getEndpointId() { return endpointId; }
    public String getEventType() { return eventType; }
    public String getPayload() { return payload; }
    public Status getStatus() { return status; }
    public int getAttempts() { return attempts; }
    public Integer getLastStatus() { return lastStatus; }
    public String getLastError() { return lastError; }
    public Instant getNextAttemptAt() { return nextAttemptAt; }
    public Instant getCreatedAt() { return createdAt; }
    public Instant getUpdatedAt() { return updatedAt; }
}

```

8.8.5 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/domain/Deliverable`

`responseSnippet` ilk 2000 karakter. Hata ayıklamanın en çok işe yarayan alanı: müşteri "webhook gelmedi" dediğinde onların sunucusunun ne cevap verdiğini görürsünüz.

```

package dev.hookrelay.dispatcher.domain;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.UUID;

/**
 * Tek bir HTTP denemesinin kaydı. Silinmez, güncellenmez - sadece eklenir.
 *
 * Bu tablo bir DENETİM İZİ. Müsteri "webhook gelmedi" dediğinde açılan
 * ilk yer burası olacak: ne zaman, hangi URL'e, kaç ms'de, hangi kodu
 * donduk, cevabının ilk 2000 karakteri neydi.
 */

```

```

@Entity
@Table(name = "delivery_attempt",
        uniqueConstraints = @UniqueConstraint(
            name = "uk_attempt_delivery_no",
            columnNames = {"delivery_id", "attempt"}))
public class DeliveryAttempt {

    @Id
    private UUID id;

    @Column(name = "delivery_id", nullable = false) private UUID deliveryId;
    @Column(name = "endpoint_id", nullable = false) private UUID endpointId;
    @Column(nullable = false) private int attempt;

    @Column(name = "http_status") private Integer httpStatus;
    @Column(name = "latency_ms", nullable = false) private long latencyMs;
    @Column(length = 1000) private String error;

    /** Cevabin ilk 2000 karakteri. Hata ayıklamanin en cok ise yarayan alanı. */
    @Column(name = "response_snippet", length = 2000) private String responseSnippet;

    @Column(name = "occurred_at", nullable = false) private Instant occurredAt;

    protected DeliveryAttempt() {}

    public DeliveryAttempt(UUID deliveryId, UUID endpointId, int attempt, Integer httpStatus
        ,
            long latencyMs, String error, String responseSnippet) {
        this.id = UUID.randomUUID();
        this.deliveryId = deliveryId;
        this.endpointId = endpointId;
        this.attempt = attempt;
        this.httpStatus = httpStatus;
        this.latencyMs = latencyMs;
        this.error = cut(error, 1000);
        this.responseSnippet = cut(responseSnippet, 2000);
        this.occurredAt = Instant.now();
    }

    private static String cut(String s, int max) {
        return s == null ? null : s.substring(0, Math.min(s.length(), max));
    }

    public UUID getId() { return id; }
    public UUID getDeliveryId() { return deliveryId; }
    public UUID getEndpointId() { return endpointId; }
    public int getAttempt() { return attempt; }
    public Integer getHttpStatus() { return httpStatus; }
    public long getLatencyMs() { return latencyMs; }
    public String getError() { return error; }
    public String getResponseSnippet(){ return responseSnippet; }
    public Instant getOccurredAt() { return occurredAt; }
}

```

8.8.6 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/repo/DeliveryF

`findStatusById` yalnızca durum sütununu çekiyor. Önceki sürüm `isAlreadySucceeded` diye bir boolean döndürüyordu; durumun kendisini döndürmek aynı maliyette ama çağırana daha çok şey söylüyor.

`findByEndpointIdAndStatus...` sonradan eklendi: toplu yeniden gönderim önce 500 kayıt çekip Java'da filtreliyordu ve `limit` yalan söylüyordu.

```
package dev.hookrelay.dispatcher.repo;

import dev.hookrelay.dispatcher.domain.Delivery;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;
import java.util.Optional;
import java.util.UUID;

public interface DeliveryRepository extends JpaRepository<Delivery, UUID> {

    Page<Delivery> findByEndpointIdOrderByCreatedAtDesc(UUID endpointId, Pageable pageable);

    Page<Delivery> findByApplicationIdOrderByCreatedAtDesc(UUID applicationId, Pageable pageable);

    Page<Delivery> findByStatusOrderByUpdatedAtDesc(Delivery.Status status, Pageable pageable);

    /**
     * Belirli bir endpoint'in belirli durumdaki teslimatları.
     *
     * Neden ayrı metod: toplu yeniden gönderim önce 500 kayıt çekip
     * sonra Java'da endpoint'e göre filtreliyordu. Sonuç: "en fazla
     * 500 gönder" dediginizde, o 500'un içinde o endpoint'ten sadece
     * 12 tane varsa 12 tane gönderiliyordu - sessizce eksik iş.
     * Filtreleme veritabanında yapılmalı ki limit ne ditiyse o olsun.
     */
    List<Delivery> findByEndpointIdAndStatusOrderByUpdatedAtDesc(
        UUID endpointId, Delivery.Status status, Pageable pageable);

    List<Delivery> findById(UUID messageId);

    long countByStatus(Delivery.Status status);

    long countByEndpointIdAndStatus(UUID endpointId, Delivery.Status status);

    @Query("SELECT d.status, COUNT(d) FROM Delivery d GROUP BY d.status")
    List<Object[]> countGroupedByStatus();

    /**
     * Sadece durum sütunu. Bos Optional = teslimat henüz yok.
     *
     * Önceki sürüm "isAlreadySucceeded" adında bir boolean donduruyordu.
     * Durumun kendisini dondurmek aynı maliyette ama çağırana daha çok
     * şey söylüyor - ve loglara "zaten DISCARDED" gibi anlamlı bir
     * mesaj yazılabiliyor.
     */
    @Query("SELECT d.status FROM Delivery d WHERE d.id = :id")
    Optional<Delivery.Status> findById(@Param("id") UUID id);
}
```

8.8.7 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/repo/DeliveryAttemptRepository.java`

ON CONFLICT DO NOTHING. Neden `try/catch` olmadığı uzun uzun javadoc'ta; özeti: JPA'da kısıt ihlalini yakalayıp devam etmek transaction'ı sessizce öldürür.

```
package dev.hookrelay.dispatcher.repo;

import dev.hookrelay.dispatcher.domain.DeliveryAttempt;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.time.Instant;
import java.util.List;
import java.util.UUID;

public interface DeliveryAttemptRepository extends JpaRepository<DeliveryAttempt, UUID> {

    List<DeliveryAttempt> findByDeliveryIdOrderByAttemptAsc(UUID deliveryId);

    List<DeliveryAttempt> findTop100ByEndpointIdOrderByOccurredAtDesc(UUID endpointId);

    /**
     * MUKERRER DENEMEYI ISTISNA ATMADAN YUTAR.
     *
     * BURADA CİDDİ BİR HATA VARDI - VE ÇOK YAYGIN BİR HATADIR
     *
     * İlk yazilisi soyleydi:
     *
     * try {
     *     attempts.save(new DeliveryAttempt(...));
     * } catch (DataIntegrityViolationException e) {
     *     log.debug("zaten kayitli, sorun degil");
     * }
     * deliveries.save(delivery);          // ← ARTIK CALISMAZ
     *
     * Kulaga makul geliyor: kisit patliyor, yutuyoruz, devam ediyoruz.
     * Ama JPA'da BOYLE CALISMAZ.
     *
     * Hibernate bir kisit ihlali gordugunde persistence context'i
     * TUTARSIZ kabul eder ve transaction'i "yalnizca geri sarilabilir"
     * (rollback-only) olarak isaretler. Istisnayi yakalayip devam
     * etmeniz bir sey degistirmez: sonraki yazmalar calisir gibi
     * gorunur, sonra commit aninda UnexpectedRollbackException gelir
     * ve TUM transaction geri sarilir.
     *
     * Sonuc: mukerrer teslimat geldiginde - ki bu Kafka'da NORMAL bir
     * durumdur - teslimatin durum guncellemesi de kaybolur. Hata
     * mesaji da suclunun yerini gostermez.
     *
     * COZUM: istisnayi hic dogurmamak. Postgres'in ON CONFLICT'i
     * catismayi veritabani icinde, atomik olarak, sessizce yutar.
     * Hibernate'in haberi bile olmaz.
     *
     * @return eklendiyse 1, catisma nedeniyle atlandiyse 0
     */
    @Modifying
    @Query(value = "INSERT INTO delivery_attempt
```

```

        (id, delivery_id, endpoint_id, attempt, http_status,
         latency_ms, error, response_snippet, occurred_at)
VALUES (:id, :deliveryId, :endpointId, :attempt, :httpStatus,
        :latencyMs, :error, :snippet, :occurredAt)
ON CONFLICT (delivery_id, attempt) DO NOTHING
""", nativeQuery = true)
int insertIgnoringDuplicate(@Param("id") UUID id,
                           @Param("deliveryId") UUID deliveryId,
                           @Param("endpointId") UUID endpointId,
                           @Param("attempt") int attempt,
                           @Param("httpStatus") Integer httpStatus,
                           @Param("latencyMs") long latencyMs,
                           @Param("error") String error,
                           @Param("snippet") String snippet,
                           @Param("occurredAt") Instant occurredAt);
}

```

8.8.8 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/config/HttpClient

Sanal iş parçacıkları ve `followRedirects(NEVER)`. İkincisi bir güvenlik kararı: yönlendirme takip etmek, imzalı isteğimizi bilmediğimiz bir sunucuya göndermek demek (SSRF).

```

package dev.hookrelay.dispatcher.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.net.http.HttpClient;
import java.time.Duration;
import java.util.concurrent.Executors;

@Configuration
public class HttpClientConfig {

    /**
     * Giden webhook'lar için tek, paylaşılan HttpClient.
     *
     * SANAL İŞ PARÇACIKLARI (Java 21)
     * executor olarak newVirtualThreadPerTaskExecutor verildi. Sebep:
     * bu servisin yaptığı iş neredeyse tamamen "istek at, cevabi bekle".
     * Klasik platform iş parçacığı bu beklemede 1 MB yığın tutar ve
     * işletim sistemi iş parçacığıdır - birkaç bin tanesini acamazsınız.
     * Sanal iş parçacığı beklerken serbest bırakılır, on binlercesi
     * kullanılabilir.
     *
     * Yavaş bir müşteri artık "bir iş parçacığını isgal eden" değil,
     * "bir sanal iş parçacığını bekleten" bir şey. Bu, bulkhead'in
     * işini kolaylaştırır ama YERİNİ ALMAZ: sanal iş parçacığı bedava
     * olsa da uzaktaki sunucunun bağlantı kotası bedava değil.
     *
     * followRedirects NEVER: webhook'ta yönlendirme takip etmek
     * güvenlik acıdır. Müşteri URL'i bir yere yönlendirirse imzalı
     * isteğimiz bilmediğimiz bir sunucuya gider (SSRF).
     */
    @Bean
    public HttpClient webhookHttpClient(
        @Value("${hookrelay.http.connect-timeout-ms:5000}") long connectTimeoutMs) {
        return HttpClient.newBuilder()
    }

```

```

        .connectTimeout(Duration.ofMillis(connectTimeoutMs))
        .followRedirects(HttpClient.Redirect.NEVER)
        .version(HttpClient.Version.HTTP_1_1)
        .executor(Executors.newVirtualThreadPerTaskExecutor())
        .build();
    }
}

```

8.8.9 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/DeliveryTask`

Bir denemenin bağlamı. `record` çünkü değişmez ve yalnızca veri taşıyor.

```

package dev.hookrelay.dispatcher.delivery;

import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.contracts.EndpointConfig;

/** Bir teslimat denemesi boyunca tasınan veri. */
public record DeliveryContext(DeliveryTask task, EndpointConfig endpoint) {

    public int attempt() { return task.attempt(); }
}

```

8.8.10 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/SendResult`

Yalnızca **ne olduğunu** taşır. "Bu hata kalıcı mı" sorusu burada değil — o bir politika kararı ve konfigürasyona bağlı.

```

package dev.hookrelay.dispatcher.delivery;

import java.time.Duration;

/**
 * Tek bir HTTP denemesinin ham sonucu.
 *
 * @param retryAfter sunucunun Retry-After başlığıyla söylediği bekleme süresi
 */
public record SendResult(
    Integer httpStatus,
    long latencyMs,
    String responseSnippet,
    String error,
    Duration retryAfter
) {
    public boolean success() {
        return httpStatus != null && httpStatus >= 200 && httpStatus < 300;
    }

    // Not: "bu hata kalıcı mı" sorusu bilincli olarak burada DEĞİL.
    // 0 bir politika kararı ve konfigürasyona bağlı -- bkz. FailureClassifier.
    // Bu kayıt sadece ne olduğunu taşır, ne yapılacağını değil.
}

```

8.8.11 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/FailureClassifier`

Politika. `retry-on-4xx` ile ayarlanabiliyor, 410 Gone ayardan muaf.


```

package dev.hookrelay.dispatcher.delivery;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

/**
 * "Bu hatayı yeniden denemeye değer mi?"
 *
 * Sınıflandırma bir POLİTİKADIR, veri değildir. O yüzden {@link SendResult}
 * kaydının içinde değil, ayrı bir bean'de: müşteri bazlı veya ortam bazlı
 * değisebilmesi gerekiyor ve test edilebilir olması lazım.
 *
 * Kural:
 * 2xx → başarılı
 * 4xx hatası (kod yok) → gecici. Sunucu yarın ayakta olabilir.
 * 408, 429 → gecici. İkisi de "şimdi olmaz" demek, "asla" değil.
 * 410 Gone → HER ZAMAN kalıcı. "Bu kaynak kalıcı olarak yok"
 * demenin başka bir yolu yok; ısrar etmek kabalık.
 * diğer 4xx → varsayılan kalıcı, ayarla acilabilir
 * 5xx → gecici
 *
 * Neden diğer 4xx varsayılan olarak kalıcı: "istegin kendisi yanlış" demek.
 * Aynı isteği beş kez daha göndermek aynı cevabı beş kez daha almaktır --
 * bosa kaynak, bosa gecikme, bosa DLQ gecikmesi.
 *
 * Neden yine de acilabilir: bazı müşteriler token yenilerken gecici 403
 * doner. Onlar için 4xx'i yeniden denemek doğru olabilir. Kararı biz değil,
 * sistemi çalıştıran versin.
 */
@Component
public class FailureClassifier {

    private final boolean retryOn4xx;

    public FailureClassifier(
        @Value("${hookrelay.delivery.retry-on-4xx:false}") boolean retryOn4xx) {
        this.retryOn4xx = retryOn4xx;
    }

    public boolean permanent(SendResult result) {
        Integer status = result.httpStatus();

        if (status == null) return false; // 4xx hatası → gecici
        if (status >= 200 && status < 300) return false;
        if (status == 410) return true; // ayardan bağımsız
        if (status == 408 || status == 429) return false;
        if (status >= 400 && status < 500) return !retryOn4xx;

        return false; // 5xx ve diğerleri gecici
    }

    public boolean retryOn4xx() { return retryOn4xx; }
}

```

8.8.12 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/Web

Tek işi imzalı POST atmak ve ne olduğunu düzgünce raporlamak. **Karar vermez** — yeniden denenecek mi, DLQ'ya mı gidecek, o `DeliveryProcessor`'ın işi. Bu ayrım sınıfı tek başına test edilebilir kılıyor.

Dört ayrı `catch`: zaman aşımı, `IOException`, kesilme, beklenmeyen. Hepsi aynı `SendResult` tipine dönüşüyor; çağırana istisna yakalamak zorunda kalmıyor.

```
package dev.hookrelay.dispatcher.delivery;

import dev.hookrelay.common.crypto.WebhookSigner;
import dev.hookrelay.contracts.Headers;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.time.Duration;
import java.time.Instant;
import java.time.format.DateTimeFormatter;
import java.util.Optional;

/**
 * Tek isi: imzalı HTTP POST atmak ve ne olduğunu düzgünce raporlamak.
 *
 * Karar VERMEZ (yeniden denenecek mi, DLQ'ya mı gidecek) - o
 * DeliveryProcessor'in isi. Bu ayırım, sinifi tek basına test
 * edilebilir kılıyor.
 */
@Component
public class WebhookSender {

    private static final Logger log = LoggerFactory.getLogger(WebhookSender.class);
    private static final int SNIPPET_MAX = 2000;

    private final HttpClient client;
    private final WebhookSigner signer;
    private final Timer timer;

    public WebhookSender(HttpClient client, WebhookSigner signer, MeterRegistry meters) {
        this.client = client;
        this.signer = signer;
        this.timer = Timer.builder("hookrelay.dispatch.http")
            .publishPercentiles(0.5, 0.95, 0.99)
            .register(meters);
    }

    public SendResult send(DeliveryContext ctx) {
        var endpoint = ctx.endpoint();
        var task = ctx.task();
        Instant now = Instant.now();
        String signature = signer.sign(endpoint.secret(), task.payload(), now);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(endpoint.url()))
            .timeout(Duration.ofMillis(endpoint.timeoutMs()))
            .header("Content-Type", "application/json; charset=utf-8")
            .header("User-Agent", "HookRelay/1.0")
            // Alicinin idempotency anahtarı BUDUR. "En az bir kez"
```

```

        // teslimat yaptigimiz icin ayni id ile ikinci kez
        // gelebiliriz; alicinin bunu ayirt etmesi lazim.
        .header(Headers.OUT_ID, task.deliveryId().toString())
        .header(Headers.OUT_TIMESTAMP, String.valueOf(now.getEpochSecond()))
        .header(Headers.OUT_SIGNATURE, signature)
        .header(Headers.OUT_EVENT_TYPE, task.eventType())
        .header(Headers.OUT_ATTEMPT, String.valueOf(task.attempt()))
        .POST(HttpRequest.BodyPublishers.ofString(task.payload()))
        .build();

    long start = System.nanoTime();
    try {
        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.o
            fString());
        long latency = elapsedMs(start);
        timer.record(Duration.ofMillis(latency));

        return new SendResult(
            response.statusCode(),
            latency,
            snippet(response.body()),
            null,
            parseRetryAfter(response).orElse(null));
    } catch (HttpTimeoutException e) {
        long latency = elapsedMs(start);
        timer.record(Duration.ofMillis(latency));
        return new SendResult(null, latency, null,
            "Zaman asimi (" + endpoint.timeoutMs() + " ms)", null);
    } catch (IOException e) {
        long latency = elapsedMs(start);
        timer.record(Duration.ofMillis(latency));
        // DNS coz-ulemedi, baglanti reddedildi, TLS el sikismasi basarisiz...
        return new SendResult(null, latency, null,
            e.getClass().getSimpleName() + ": " + e.getMessage(), null);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        return new SendResult(null, elapsedMs(start), null, "Kesildi", null);
    } catch (Exception e) {
        log.error("Beklenmeyen gonderim hatasi: endpoint={}", endpoint.endpointId(), e);
        return new SendResult(null, elapsedMs(start), null,
            e.getClass().getSimpleName() + ": " + e.getMessage(), null);
    }
}

private long elapsedMs(long startNanos) {
    return (System.nanoTime() - startNanos) / 1_000_000;
}

private String snippet(String body) {
    if (body == null) return null;
    return body.length() <= SNIPPET_MAX ? body : body.substring(0, SNIPPET_MAX);
}

/**
 * Retry-After iki bicimde gelebilir (RFC 9110):
 *   Retry-After: 120                                → saniye
 *   Retry-After: Wed, 21 Oct 2026 07:28:00 GMT       → HTTP tarihi

```

```

    * Ikisini de destekliyoruz; cogu istemci ikincisini atlar ve
    * 429'lari yanlis zamanlar.
    */
    private Optional<Duration> parseRetryAfter(HttpResponse<String> response) {
        return response.headers().firstValue("Retry-After").flatMap(raw -> {
            String value = raw.trim();
            try {
                long seconds = Long.parseLong(value);
                return seconds >= 0 ? Optional.of(Duration.ofSeconds(seconds)) : Optional.empty();
            } catch (NumberFormatException ignored) {
                // sayi degilse tarih olabilir
            }
            try {
                Instant when = Instant.from(DateTimeFormatter.RFC_1123_DATE_TIME.parse(value));
                Duration d = Duration.between(Instant.now(), when);
                return d.isNegative() ? Optional.empty() : Optional.of(d);
            } catch (Exception e) {
                return Optional.empty();
            }
        });
    }
}

```

8.8.13 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/RetryProperties`

`delays` listesi katman topic'leriyle birebir eşleşir. Compact constructor varsayılan atıyor.

```

package dev.hookrelay.dispatcher.delivery;

import org.springframework.boot.context.properties.ConfigurationProperties;

import java.time.Duration;
import java.util.List;

/**
 * Yeniden deneme ayarlari.
 *
 * delays listesi katman topic'leriyle BIREBIR eslesir:
 * delays[0] -> hookrelay.retry.t1.v1
 * delays[1] -> hookrelay.retry.t2.v1 ...
 *
 * Uretimde: 10s, 1m, 5m, 30m, 2h (toplam ~2.5 saatlik kurtarma penceresi)
 * Demo'da: 5s, 15s, 30s, 60s, 120s (docker-compose'da override ediliyor,
 * yoksa demoyu izlemek 2.5 saat surer)
 */
@ConfigurationProperties(prefix = "hookrelay.retry")
public record RetryProperties(
    List<Duration> delays,
    Duration maxLifetime
) {
    public RetryProperties {
        if (delays == null || delays.isEmpty()) {
            delays = List.of(
                Duration.ofSeconds(10), Duration.ofMinutes(1), Duration.ofMinutes(5),
                Duration.ofMinutes(30), Duration.ofHours(2));
        }
        if (maxLifetime == null || maxLifetime.isZero()) {

```

```

        maxLifetime = Duration.ofHours(24);
    }
}

```

8.8.14 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/RetryPolicy`

Strategy arayüzü. Bugün tek uygulaması var; arayüz olarak durmasının sebebi müşteri bazlı politika ihtiyacı. Kullanılmayan soyutlama desen değil süslemedir — ama bunun somut bir kullanım senaryosu var.

```

package dev.hookrelay.dispatcher.delivery;

import java.time.Duration;
import java.time.Instant;
import java.util.Optional;

/**
 * STRATEGY deseni.
 *
 * Yeniden deneme "ne zaman ve nereye" sorusunun cevabi. Bugun tek bir
 * uygulaması var (katmanlı topic'ler) ama arayüz olarak durmasının
 * gerçek bir sebebi var: müşteri bazlı politika. Bir müşteri "beni
 * 3 kez dene, sonra bırak" derken bir diğeri "24 saat boyunca dene"
 * diyebilir. 0 gün geldiğinde DeliveryProcessor'a dokunulmaz,
 * yeni bir RetryPolicy yazılır.
 *
 * (Arayuzu bugün kullanmayacak olsaydık yazmazdık. Kullanılmayan
 * soyutlama, desen değil süslemedir.)
 */
public interface RetryPolicy {

    /**
     * @param tier      1 tabanlı katman numarası
     * @param notBefore bu andan önce işlenmemeli
     */
    record Reschedule(int tier, Instant notBefore, Duration delay) {}

    /** Basarisiz denemeden sonra. Bos donerse: artik deneme yok, DLQ. */
    Optional<Reschedule> afterFailure(int attempt, int maxAttempts, Duration serverRetryAfter);

    /**
     * Deneme SAYILMADAN erteleme. Devre kesici acikken veya hiz siniri
     * doldugunda kullanilir: musterinin sucu degil, bizim kararimiz.
     */
    Reschedule withoutConsumingAttempt(Duration minDelay);

    /** Teslimat, dogum tarihinden bu yana yasam suresini asti mi? */
    boolean expired(Instant firstQueuedAt);
}

```

8.8.15 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/TieredRetryPolicy`

`withoutConsumingAttempt` istenen gecikmeyi karşılayan **ilk** katmanı seçiyor. İstenen süre en büyük katmandan da uzunsa son katmana konuyor ama `notBefore` gerçek ana ayarlanıyor — kayıt birkaç tur dönebilir, asla erken işlenmez.

```

package dev.hookrelay.dispatcher.delivery;

import org.springframework.stereotype.Component;

import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.Optional;

/**
 * Katmanlı (tiered) yeniden deneme.
 *
 * * KAFKA'DA GECIKMELI MESAJ PROBLEMI
 * * RabbitMQ'da "bu mesajı 30 dakika sonra teslim et" diyebilirsiniz.
 * * Kafka'da BOYLE BIR SEY YOK. Kafka bir kuyruk degil, bir GUNLUK:
 * * kayitlar yazildiklari sirada durur, tuketici bastan sona okur.
 * * Tek bir kaydi "sonraya birakmak" diye bir islem yoktur.
 *
 * * Naif cozum: tuketici kaydi okur, Thread.sleep(30 dakika) yapar.
 * * Felaket: max.poll.interval.ms (varsayilan 5 dk) asilir, broker
 * * tuketiciyi olmus sayar, rebalance tetiklenir, mesaj baskasina gider,
 * * o da uyur... Butun grup kilitlenir.
 *
 * * COZUM: HER GECIKME ICIN AYRI TOPIC
 * * t1=10sn, t2=1dk, t3=5dk, t4=30dk, t5=2sa.
 *
 * * Bir topic'teki BUTUN kayitlar AYNI gecikmeye sahip oldugu icin,
 * * partition icinde "islenme zamani" sirasi = "yazilma" sirasi.
 * * Yani tuketicinin sadece BASTAKI kayda bakmasi yeterli:
 * * o hazir degilse arkasindakiler de hazir degildir.
 *
 * * Bu tek gozlem butun cozumu mumkun kiliyor. Karisik gecikmeler ayni
 * * topic'te olsaydi, arkadaki hazir bir kaydi gormek icin ondekini
 * * atlamak gerekirdi - Kafka'da mumkun degil (offset geriye/ileriye
 * * atlanir ama "atla ve sonra don" diye bir sey yok).
 */
@Component
public class TieredRetryPolicy implements RetryPolicy {

    private final List<Duration> delays;
    private final Duration maxLifetime;

    public TieredRetryPolicy(RetryProperties properties) {
        this.delays = properties.delays();
        this.maxLifetime = properties.maxLifetime();
    }

    @Override
    public Optional<Reschedule> afterFailure(int attempt, int maxAttempts, Duration serverRe
tryAfter) {
        // attempt = simdi biten deneme. Sonraki katman indeksi = attempt (0 tabanlı).
        if (attempt >= maxAttempts || attempt > delays.size()) {
            return Optional.empty();
        }
        int tier = attempt; // 1. deneme bitti → t1'e koy
        Duration delay = delays.get(tier - 1);

        // Sunucu "Retry-After: 120" dediyse ona SAYGI GOSTER.
        //
        // Bunu yapmak sadece kibarlik degil: 429 donen bir sunucuya

```

```

        // erken donmek, ikinci bir 429 ve bir deneme daha harcamak demek.
        // Katman topic'i degismiyor, sadece not-before ileri aliniyor -
        // tasarim bunu bedavaya destekliyor cunku zamanlayici topic adina
        // degil, kaydın üzerindeki not-before basligina bakiyor.
        if (serverRetryAfter != null && serverRetryAfter.compareTo(delay) > 0) {
            delay = serverRetryAfter;
        }
        return Optional.of(new Reschedule(tier, Instant.now().plus(delay), delay));
    }

    @Override
    public Reschedule withoutConsumingAttempt(Duration minDelay) {
        Duration wanted = minDelay == null ? delays.get(0) : minDelay;
        for (int i = 0; i < delays.size(); i++) {
            if (delays.get(i).compareTo(wanted) >= 0) {
                return new Reschedule(i + 1, Instant.now().plus(delays.get(i)), delays.get(i));
            }
        }
        // Istenen gecikme en büyük katmandan da uzun: en buyugune koy,
        // not-before'u istenen ana ayarla. Kayıt birkaç tur donebilir
        // ama asla erken islenmez.
        int last = delays.size();
        return new Reschedule(last, Instant.now().plus(wanted), wanted);
    }

    @Override
    public boolean expired(Instant firstQueuedAt) {
        if (firstQueuedAt == null) return false;
        return Instant.now().isAfter(firstQueuedAt.plus(maxLifetime));
    }
}

```

8.8.16 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/Endp

JVM içi semafor. Redis'te tutulmuyor çünkü bu bir **süreç yerel** kaynağı koruyor: bu örneğin iş parçacıkları. Dağıtık olması gerekmiyor.

```

package dev.hookrelay.dispatcher.delivery;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import java.util.Map;
import java.util.UUID;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.Semaphore;

/**
 * BULKHEAD (gemi bolmesi) deseni - "gurultulu komşu" problemi.
 *
 * PROBLEM
 * Tek bir musterinin sunucusu 30 saniyede cevap veriyor. Dispatcher'da
 * 12 tüketici iş parçacığı var. 0 müşteriye 12 mesaj gelirse 12 iş
 * parçacığının hepsi 30 saniye bloke olur ve DİĞER BUTUN musterilerin
 * teslimatı durur. Bir musterinin yavaşlığı, sistemin tamamının
 * yavaşlığına dönüşür.
 *
 * COZUM

```

```

* -----
* Her endpoint'e ayrı bir kota: aynı anda en fazla N istek. Kota
* dolduysa mesaj beklemes - kuyruğa geri konur, iş parçası serbest
* kalır ve başka müşterilere hizmet etmeye devam eder.
*
* Gemi metaforu buradan: gövdeyi bölmelere ayırırsınız, bir bölme su
* alınca gemi batmaz.
*
* Neden tryAcquire(0) ve tryAcquire(5 saniye) değil: beklemek, bloke
* olmanın başka adı. Bekleyen iş parçası da çalışmıyor demektir.
*/
@Component
public class EndpointBulkhead {

    private final Map<UUID, Semaphore> permits = new ConcurrentHashMap<>();
    private final int maxConcurrentPerEndpoint;

    public EndpointBulkhead(
        @Value("${hookrelay.bulkhead.max-concurrent-per-endpoint:4}") int max) {
        this.maxConcurrentPerEndpoint = max;
    }

    /** @return true ise izin alındı ve release() çağrılmak ZORUNDA. */
    public boolean tryAcquire(UUID endpointId) {
        return semaphore(endpointId).tryAcquire();
    }

    public void release(UUID endpointId) {
        semaphore(endpointId).release();
    }

    public int available(UUID endpointId) {
        return semaphore(endpointId).availablePermits();
    }

    private Semaphore semaphore(UUID endpointId) {
        return permits.computeIfAbsent(endpointId,
            id -> new Semaphore(maxConcurrentPerEndpoint));
    }
}

```

8.8.17 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/check

Chain of Responsibility arayüzü. `order()` sıralamayı kodun kendisine yazıyor; Spring'in bean sırasına güvenmek üzerine sistem kurulacak bir şey değil.

```

package dev.hookrelay.dispatcher.delivery.checks;

import dev.hookrelay.contracts.DeliveryResult;
import dev.hookrelay.dispatcher.delivery.DeliveryContext;

import java.time.Duration;
import java.util.Optional;

/**
 * CHAIN OF RESPONSIBILITY.
 *
 * İstek gönderilmeden önce geçmesi gereken kontroller. Her biri
 * bağımsız, her biri tek bir şey biliyor, sırası order() ile belli.

```



```

*
* Neden zincir, neden tek bir if bloğu değil: bugün 2 kontrol var
* (devre kesici, hız sınırı). Yarın "müsteri IP'si kara listede mi",
* "bu olay tipi gecici olarak duraklatıldı mı", "bakım penceresi mi"
* eklenecek. Her biri DeliveryProcessor'a bir if daha eklemek yerine
* bir sınıf olarak gelir ve processor'a hiç dokunulmaz.
*
* Zincirin sırası ÖNEMLİ ve ucuzdan pahalıya dizilmiş:
* 10 devre kesici (1 Redis çağırışı, çoğu zaman kapalı → hemen geç)
* 20 hız sınırı (1 Redis çağırışı, jeton harcıyor -- devre açıkken
*                bosuna jeton harcamamak için sonra)
*/
public interface PreflightCheck {

    /**
     * @param outcome        sonuca ne yazılacak
     * @param reason          insan okuyacak açıklama
     * @param minDelay        en erken ne kadar sonra tekrar denenebilir
     * @param consumesAttempt bu engelleme bir deneme hakkı yaşıyor mu
     */
    record Block(DeliveryResult.Outcome outcome, String reason,
                Duration minDelay, boolean consumesAttempt) {}

    /** Bos Optional = geçti, sonraki kontrole devam. */
    Optional<Block> check(DeliveryContext context);

    int order();
}

```

8.8.18 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/check

```

package dev.hookrelay.dispatcher.delivery.checks;

import dev.hookrelay.common.redis.RedisCircuitBreaker;
import dev.hookrelay.contracts.DeliveryResult;
import dev.hookrelay.dispatcher.delivery.DeliveryContext;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.stereotype.Component;

import java.time.Duration;
import java.util.Optional;

/**
 * Müsteri çıktıysa ona 50.000 istek daha atmayalım.
 *
 * ÖNEMLİ: engelleme bir DENEME HAKKI YAKMAZ (consumesAttempt = false).
 * Sebep adil olmak: müşteri 5 dakika bakımda diye teslimatın bütün
 * haklarını tüketmesi, bakım bitince mesajların çoktan DLQ'ya düşmüş
 * olması demektir. Şu bizde değil, onlarda da değil - sadece zamanlama.
 *
 * Sonsuz döngüyü ne engelliyor: RetryPolicy.expired() - teslimatın
 * toplam yaşam süresi (varsayılan 24 saat). 0 dolunca DLQ.
 */
@Component
public class CircuitBreakerCheck implements PreflightCheck {

    private final RedisCircuitBreaker breaker;
    private final Counter blocked;

```

```

public CircuitBreakerCheck(RedisCircuitBreaker breaker, MeterRegistry meters) {
    this.breaker = breaker;
    this.blocked = Counter.builder("hookrelay.dispatch.short_circuited").register(meters);
}

@Override
public Optional<Block> check(DeliveryContext ctx) {
    var verdict = breaker.allow(ctx.endpoint().endpointId());
    if (verdict.allowed()) return Optional.empty();

    blocked.increment();
    return Optional.of(new Block(
        DeliveryResult.Outcome.SHORT_CIRCUITED,
        "Devre kesici acik (durum=" + verdict.state() + ", hata=" + verdict.failures
            () + ")",
        Duration.ofSeconds(30),
        false));
}

@Override
public int order() { return 10; }
}

```

8.8.19 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/check`

İkisi de `consumesAttempt = false` döndürüyor: müşterinin hatası olmayan bir engelleme, onun deneme hakkını yakmamalı.

```

package dev.hookrelay.dispatcher.delivery.checks;

import dev.hookrelay.common.redis.RedisRateLimiter;
import dev.hookrelay.contracts.DeliveryResult;
import dev.hookrelay.dispatcher.delivery.DeliveryContext;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.springframework.stereotype.Component;

import java.time.Duration;
import java.util.Optional;

/**
 * Endpoint'in kaldirabilecegi hizdan fazlasini gondermeyelim.
 *
 * Bu, bizim degil MUSTERININ korunmasi. Kucuk bir Rails sunucusu
 * saniyede 10 istek kaldiriyorsa, elimizde 5.000 mesaj birikti diye
 * hepsini bir anda atmak onlari devirir - sonra da 5.000 mesajin
 * hepsini yeniden denemek zorunda kaliriz. Yavas gondermek herkes
 * icin daha hizli bitirir.
 *
 * Deneme hakki yakmaz: hiz siniri musterinin hatasi degil.
 */
@Component
public class RateLimitCheck implements PreflightCheck {

    private final RedisRateLimiter limiter;
    private final Counter throttled;
}

```

```

public RateLimitCheck(RedisRateLimiter limiter, MeterRegistry meters) {
    this.limiter = limiter;
    this.throttled = Counter.builder("hookrelay.dispatch.rate_limited").register(meters);
}

@Override
public Optional<Block> check(DeliveryContext ctx) {
    int perSecond = ctx.endpoint().rateLimitPerSecond();
    if (perSecond <= 0) return Optional.empty();

    var decision = limiter.tryAcquire(ctx.endpoint().endpointId(), perSecond);
    if (decision.allowed()) return Optional.empty();

    throttled.increment();
    return Optional.of(new Block(
        DeliveryResult.Outcome.RETRYING,
        "Hiz siniri: " + perSecond + "/sn, " + decision.waitMillis() + " ms sonra",
        Duration.ofMillis(Math.max(decision.waitMillis(), 1000)),
        false));
}

@Override
public int order() { return 20; }
}

```

8.8.20 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/Deli

Kafka'ya çıkan her şey buradan geçer. Yeniden zamanlamada **key değişmiyor** — hâlâ `endpointId`, yani katman topic'i içinde de sıra korunuyor.

`deadLetter` DLQ'ya basıyor ve DLQ'nun tüketicisi **yok**. Otomatik tüketen bir DLQ, DLQ değil bir döngüdür.

```

package dev.hookrelay.dispatcher.delivery;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.kafka.KafkaHeaderCodec;
import dev.hookrelay.contracts.DeliveryResult;
import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.contracts.Headers;
import dev.hookrelay.contracts.Topics;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

import java.time.Instant;

/** Kafka'ya cikan her sey buradan gecer: yeniden deneme, DLQ, sonuc. */
@Component
public class DeliveryPublisher {

    private static final Logger log = LoggerFactory.getLogger(DeliveryPublisher.class);

    private final KafkaTemplate<String, String> kafka;

```

```

private final ObjectMapper mapper;
private final Counter rescheduled;
private final Counter deadLettered;

public DeliveryPublisher(KafkaTemplate<String, String> kafka, ObjectMapper mapper,
    MeterRegistry meters) {
    this.kafka = kafka;
    this.mapper = mapper;
    this.rescheduled = Counter.builder("hookrelay.dispatch.rescheduled").register(meters);
    this.deadLettered = Counter.builder("hookrelay.dispatch.dead_lettered").register(meters);
}

/**
 * Teslimati bir yeniden deneme katmanına bırakır.
 *
 * KEY DEGİSMİYOR: hala endpointId. Katman topic'i içinde de aynı
 * endpoint'in kayıtları aynı partition'a duser, sıra korunur.
 *
 * NOT_BEFORE başlığı kritik: zamanlayıcı, topic'in adına değil
 * BU BASLIGA bakar. Sayesinde Retry-After gibi topic gecikmesinden
 * farklı bir bekleme süresi de temsil edilebiliyor.
 */
public void reschedule(DeliveryTask task, int tier, Instant notBefore, int nextAttempt)
{
    String topic = Topics.retryTier(tier);
    DeliveryTask next = task.withAttempt(nextAttempt);

    ProducerRecord<String, String> record = new ProducerRecord<>(
        topic, task.endpointId().toString(), toJson(next));
    record.headers()
        .add(Headers.NOT_BEFORE, KafkaHeaderCodec.of(notBefore.toEpochMilli()))
        .add(Headers.ATTEMPT, KafkaHeaderCodec.of(nextAttempt));

    kafka.send(record);
    rescheduled.increment();
    log.debug("Yeniden zamanlandı: delivery={} katman=t{} not-before={}",
        task.deliveryId(), tier, notBefore);
}

/**
 * DLQ'ya bırakır.
 *
 * DLQ'nun tüketicisi YOK ve olmamalı. Buraya düşen mesaj bir İNSAN
 * bakana kadar bekler. Otomatik tüketen bir DLQ, DLQ değil bir
 * dongudur - mesaj olur, geri gelir, yine olur.
 */
public void deadLetter(DeliveryTask task, String reason) {
    ProducerRecord<String, String> record = new ProducerRecord<>(
        Topics.DLQ, task.endpointId().toString(), toJson(task));
    record.headers()
        .add(Headers.DEAD_REASON, KafkaHeaderCodec.of(reason))
        .add(Headers.ATTEMPT, KafkaHeaderCodec.of(task.attempt()));

    kafka.send(record);
    deadLettered.increment();
    log.warn("DLQ: delivery={} endpoint={} sebep={}",
        task.deliveryId(), task.endpointId(), reason);
}

```

```

/** Sonuc yayini. Kimin dinledigi dispatcher'i ilgilendirmez. */
public void publishResult(DeliveryResult result) {
    kafka.send(Topics.DELIVERY_RESULTS, result.endpointId().toString(), toJson(result));
}

private String toJson(Object o) {
    try {
        return mapper.writeValueAsString(o);
    } catch (Exception e) {
        throw new IllegalStateException("Kafka govdesi serilestirilemedi", e);
    }
}
}

```

8.8.21 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/Deli

İki ayrı kısa transaction: gönderimden önce durum okuması, gönderimden sonra sonuç yazımı. Tek transaction içine alsaydık yavaş bir müşteri 30 saniye boyunca bir veritabanı bağlantısını tutardı.

`ensure private` ve `@Transactional yok` — ikisi de bilinçli, sebebi javadoc'ta.

```

package dev.hookrelay.dispatcher.delivery;

import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.dispatcher.domain.Delivery;
import dev.hookrelay.dispatcher.repo.DeliveryAttemptRepository;
import dev.hookrelay.dispatcher.repo.DeliveryRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.Optional;
import java.util.UUID;

/**
 * Teslimat durumunun veritabanına yazilmasi.
 *
 * Ayri bean olmasinin sebebi MessageWriter ile ayni: @Transactional
 * proxy uzerinden calisir, ayni siniftan yapilan cagri proxy'ye ugramaz.
 *
 * DIKKAT - TRANSACTION SINIRLARI: HTTP istegi ile veritabanı transaction'i
 * ASLA ic ice olmamali. 0 yuzden burada iki ayri kısa transaction var:
 * gonderimden once durum okumasi, gonderimden sonra sonuc yazimi.
 * Tek bir transaction icine alsaydik, yavas bir musteri 30 saniye
 * boyunca bir veritabanı baglantisini tutardi ve havuz tukenirdi.
 */
@Component
public class DeliveryStore {

    private static final Logger log = LoggerFactory.getLogger(DeliveryStore.class);

    private final DeliveryRepository deliveries;
    private final DeliveryAttemptRepository attempts;

    public DeliveryStore(DeliveryRepository deliveries, DeliveryAttemptRepository attempts)
    {

```

```

        this.deliveries = deliveries;
        this.attempts = attempts;
    }

    /** Teslimatin guncel durumu. Bos = bu teslimat ilk kez goruluyor. */
    @Transactional(readOnly = true)
    public Optional<Delivery.Status> currentStatus(UUID deliveryId) {
        return deliveries.findStatusById(deliveryId);
    }

    /**
     * Teslimat satirini bulur, yoksa olusturur.
     *
     * private VE @Transactional YOK - ikisi de bilincli.
     *
     * Bu metot yalnızca bu sinifin icinden, hepsi zaten @Transactional
     * olan metotlardan cagriliyor. Uzerine @Transactional yazsaydik
     * anotasyon HICBIR SEY YAPMAZDI: Spring'de transaction proxy ile
     * calisir ve ayni siniftan yapilan cagri proxy'ye ugramaz.
     *
     * Calismayan bir anotasyon, olmayan bir anotasyondan kotudur:
     * okuyan kisi "burasi kendi transaction'ini aciyor" sanir ve
     * yanlis bir zihin modeli kurar. Bkz. MessageWriter - ayni tuzagin
     * gercekten zarar verdigi yer.
     */
    private Delivery ensure(DeliveryTask task) {
        return deliveries.findById(task.deliveryId()).orElseGet(() ->
            deliveries.save(new Delivery(task.deliveryId(), task.messageId(),
                task.applicationId(), task.endpointId(), task.eventType(), task.payload()));
    }

    /**
     * Gercek bir HTTP denemesini ve teslimatin yeni durumunu tek
     * transaction'da yazar.
     *
     * Deneme kaydi ON CONFLICT DO NOTHING ile ekleniyor - mukerrer
     * teslimatta istisna DOGMUYOR. Sebebi DeliveryAttemptRepository'de
     * uzun uzun yazili; ozeti: JPA'da kisit ihlalini yakalayip devam
     * etmek transaction'i sessizce oldurur.
     */
    @Transactional
    public void recordAttempt(DeliveryTask task, SendResult result, Delivery.Status newStatus,
        Instant nextAttemptAt) {

        int inserted = attempts.insertIgnoringDuplicate(
            UUID.randomUUID(), task.deliveryId(), task.endpointId(), task.attempt(),
            result.httpStatus(), result.latencyMs(),
            cut(result.error(), 1000), cut(result.responseSnippet(), 2000), Instant.now());

        if (inserted == 0) {
            log.debug("Deneme zaten kayitli (mukerrer teslimat): delivery={} attempt={}",
                task.deliveryId(), task.attempt());
        }

        Delivery delivery = ensure(task);
        String error = result.error() != null ? result.error() : "HTTP " + result.httpStatus();
    }

```

```

        switch (newStatus) {
            case SUCCEEDED -> delivery.succeeded(task.attempt(), result.httpStatus());
            case EXHAUSTED -> delivery.exhausted(task.attempt(), result.httpStatus(), error)
                                ;
            case PENDING    -> delivery.retrying(task.attempt(), result.httpStatus(), error,
                                                nextAttemptAt);
            case DISCARDED -> delivery.discarded(error);
        }
        deliveries.save(delivery);
    }

    /**
     * Hic istek atılmadan ertelenen teslimat.
     *
     * delivery_attempt'e YAZMAZ - cunku o tablo "atilan HTTP istekleri"
     * demek. Atılmayan bir istegi oraya yazmak, denetim izini yalanci
     * yapar: musterisi "bize 40 istek atmissiniz" der, oysa 5 tanesi
     * gercek, 35'i devre kesici kaydidir.
     */
    @Transactional
    public void recordBlocked(DeliveryTask task, String reason, Instant nextAttemptAt) {
        Delivery delivery = ensure(task);
        delivery.blocked(reason, nextAttemptAt);
        deliveries.save(delivery);
    }

    /** Butun denemeler tukendi ama ortada yeni bir HTTP denemesi yok. */
    @Transactional
    public void markExhausted(DeliveryTask task, String reason) {
        Delivery delivery = ensure(task);
        delivery.exhausted(task.attempt(), null, reason);
        deliveries.save(delivery);
    }

    @Transactional
    public void discard(DeliveryTask task, String reason) {
        Delivery delivery = ensure(task);
        delivery.discarded(reason);
        deliveries.save(delivery);
    }

    private static String cut(String s, int max) {
        return s == null ? null : s.substring(0, Math.min(s.length(), max));
    }
}

```

8.8.22 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/delivery/DeliveryTask`

Beyin. Kendisi hiçbir iş yapmaz; sırayı ve kararı yönetir.

`bulkhead.release` neden `finally` içinde: `send()` istisna atarsa izin sonsuza kadar kilitli kalır ve o endpoint bir daha hiç teslimat almaz. Bulması çok zor bir hata — sistem çalışıyor görünür, sadece bir müşteri sessizce hizmet almaz.

İki `outcome` fabrikası: biri gerçek HTTP cevabı olan durumlar, diğeri istek atılmadan verilen kararlar için. Üç ayrı yerde elle kurulması hem tekrardı hem alan sırasını karıştırmaya davetiyeydi — hepsi UUID ve String.

```

package dev.hookrelay.dispatcher.delivery;

import dev.hookrelay.common.redis.EndpointConfigCache;
import dev.hookrelay.common.redis.RedisCircuitBreaker;
import dev.hookrelay.contracts.DeliveryResult;
import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.contracts.EndpointConfig;
import dev.hookrelay.dispatcher.delivery.checks.PreflightCheck;
import dev.hookrelay.dispatcher.domain.Delivery;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import java.time.Duration;
import java.time.Instant;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

/**
 * TESLIMATIN BEYNI.
 *
 * Bu sınıf kendisi HICBIR IS YAPMAZ - HTTP atmaz, veritabanına yazmaz,
 * Kafka'ya basmaz. Sadece SIRAYI ve KARARI yönetir. Her adım ayrı bir
 * bean'de ve tek basına test edilebilir.
 *
 * Akis:
 * 1. Bu teslimat zaten basarili mi?           → atla (mukerrer teslimat)
 * 2. Endpoint hala var mi, acik mi?           → yoksa dusur
 * 3. Yasam suresi doldu mu?                   → DLQ
 * 4. On kontroller (devre kesici, hiz siniri) → engellendiyse ertele
 * 5. Bulkhead izni                             → yoksa ertele
 * 6. Gonder
 * 7. Sonuca gore: bitti / yeniden dene / DLQ
 */
@Component
public class DeliveryProcessor {

    private static final Logger log = LoggerFactory.getLogger(DeliveryProcessor.class);

    private final EndpointConfigCache configs;
    private final List<PreflightCheck> checks;
    private final EndpointBulkhead bulkhead;
    private final WebhookSender sender;
    private final DeliveryStore store;
    private final DeliveryPublisher publisher;
    private final RetryPolicy retryPolicy;
    private final RedisCircuitBreaker breaker;
    private final FailureClassifier classifier;

    private final Counter succeeded;
    private final Counter failed;
    private final Counter skipped;

    public DeliveryProcessor(EndpointConfigCache configs, List<PreflightCheck> checks,
                           EndpointBulkhead bulkhead, WebhookSender sender,
                           DeliveryStore store, DeliveryPublisher publisher,
                           RetryPolicy retryPolicy, RedisCircuitBreaker breaker,

```



```

        FailureClassifier classifier, MeterRegistry meters) {
    this.configs = configs;
    // Zincirin sirasi order() ile belirlenir, Spring'in bean sirasiyla degil.
    // Bean sirasi paket adina, sinif adina, hatta derleyiciye gore degisir -
    // uzerine sistem kurulacak bir sey degil.
    this.checks = checks.stream()
        .sorted(Comparator.comparingInt(PreflightCheck::order))
        .toList();
    this.bulkhead = bulkhead;
    this.sender = sender;
    this.store = store;
    this.publisher = publisher;
    this.retryPolicy = retryPolicy;
    this.breaker = breaker;
    this.classifier = classifier;
    this.succeeded = Counter.builder("hookrelay.dispatch.succeeded").register(meters);
    this.failed = Counter.builder("hookrelay.dispatch.failed").register(meters);
    this.skipped = Counter.builder("hookrelay.dispatch.skipped_duplicate").register(meters);
}

public void process(DeliveryTask task) {

    // 1. MUKERRER TESLIMAT KORUMASI
    //
    // Kafka "en az bir kez" teslim eder. Offset commit edilmeden
    // once surec olurse ayni kayit tekrar gelir. Zaten basarili
    // olmus bir teslimati ikinci kez gondermek, musterinin
    // sistemine ikinci kez "odeme alindi" demek olabilir.
    Optional<Delivery.Status> current = store.currentStatus(task.deliveryId());
    if (current.filter(Delivery.Status.SUCCEEDED::equals).isPresent()) {
        skipped.increment();
        log.debug("Zaten teslim edilmiş, atlanıyor: {}", task.deliveryId());
        return;
    }

    // 2. ENDPOINT HALA GECERLI MI
    Optional<EndpointConfig> found = configs.get(task.endpointId());
    if (found.isEmpty()) {
        drop(task, "Endpoint konfigurasyonu bulunamadi (silinmiş olabilir)");
        return;
    }
    EndpointConfig endpoint = found.get();
    if (!endpoint.deliverable()) {
        drop(task, "Endpoint devre dışı veya silinmiş");
        return;
    }

    // 3. YASAM SURESI
    if (retryPolicy.expired(task.firstQueuedAt())) {
        toDeadLetter(task, "Yasam suresi doldu");
        return;
    }

    DeliveryContext ctx = new DeliveryContext(task, endpoint);

    // 4. ON KONTROLLER
    for (PreflightCheck check : checks) {
        Optional<PreflightCheck.Block> block = check.check(ctx);
        if (block.isPresent()) {
            handleBlock(ctx, block.get());
        }
    }
}

```

```

        return;
    }
}

// 5. BULKHEAD - izin yoksa BEKLEME, geri koy.
if (!bulkhead.tryAcquire(endpoint.endpointId())) {
    handleBlock(ctx, new PreflightCheck.Block(
        DeliveryResult.Outcome.RETRYING,
        "Endpoint es zamanlilik kotasi dolu",
        Duration.ofSeconds(5), false));
    return;
}

try {
    // 6. GONDER
    SendResult result = sender.send(ctx);
    handleResult(ctx, result);
} finally {
    // finally SART: send() istisna atarsa izin sonsuza kadar
    // kilitli kalir ve o endpoint bir daha hic teslimat almaz.
    bulkhead.release(endpoint.endpointId());
}
}

// Sonuc isleme

private void handleResult(DeliveryContext ctx, SendResult result) {
    DeliveryTask task = ctx.task();
    EndpointConfig endpoint = ctx.endpoint();

    if (result.success()) {
        breaker.recordSuccess(endpoint.endpointId());
        store.recordAttempt(task, result, Delivery.Status.SUCCEEDED, null);
        publisher.publishResult(outcome(ctx, result, DeliveryResult.Outcome.SUCCEEDED));
        succeeded.increment();
        log.debug("Teslim edildi: delivery={} status={} {}ms",
            task.deliveryId(), result.httpStatus(), result.latencyMs());
        return;
    }

    breaker.recordFailure(endpoint.endpointId());
    failed.increment();

    // Kalici hata: yeniden denemek ayni cevabi almaktır.
    if (classifier.permanent(result)) {
        store.recordAttempt(task, result, Delivery.Status.EXHAUSTED, null);
        publisher.deadLetter(task, "Kalici hata: HTTP " + result.httpStatus());
        publisher.publishResult(outcome(ctx, result, DeliveryResult.Outcome.EXHAUSTED));
        log.info("Kalici hata, DLQ: delivery={} status={}",
            task.deliveryId(), result.httpStatus());
        return;
    }

    Optional<RetryPolicy.Reschedule> next = retryPolicy.afterFailure(
        task.attempt(), endpoint.maxAttempts(), result.retryAfter());

    if (next.isEmpty()) {
        store.recordAttempt(task, result, Delivery.Status.EXHAUSTED, null);
        publisher.deadLetter(task, "Butun denemeler tukendi (" + task.attempt() + ")");
        publisher.publishResult(outcome(ctx, result, DeliveryResult.Outcome.EXHAUSTED));
        return;
    }
}

```

```

    }

    RetryPolicy.Reschedule r = next.get();
    store.recordAttempt(task, result, Delivery.Status.PENDING, r.notBefore());
    publisher.reschedule(task, r.tier(), r.notBefore(), task.attempt() + 1);
    publisher.publishResult(outcome(ctx, result, DeliveryResult.Outcome.RETRYING));
    log.debug("Yeniden denenecek: delivery={} katman=t{} {} sonra",
        task.deliveryId(), r.tier(), r.delay());
}

private void handleBlock(DeliveryContext ctx, PreflightCheck.Block block) {
    DeliveryTask task = ctx.task();

    if (retryPolicy.expired(task.firstQueuedAt())) {
        toDeadLetter(task, "Yasam suresi doldu (engelliyken): " + block.reason());
        return;
    }

    RetryPolicy.Reschedule r = retryPolicy.withoutConsumingAttempt(block.minDelay());
    store.recordBlocked(task, block.reason(), r.notBefore());

    // DENEME SAYISI ARTMIYOR: ayni attempt ile geri konuyor.
    // Musterinin hatasi olmayan bir engelleme, onun deneme
    // hakkini yakmamali.
    int attempt = block.consumesAttempt() ? task.attempt() + 1 : task.attempt();
    publisher.reschedule(task, r.tier(), r.notBefore(), attempt);

    publisher.publishResult(outcome(task, block.outcome(), block.reason()));

    log.debug("Engellendi: delivery={} sebep={} → t{}",
        task.deliveryId(), block.reason(), r.tier());
}

/** Endpoint artık yok/kapali. Hic istek atilmadi. */
private void drop(DeliveryTask task, String reason) {
    store.discard(task, reason);
    publisher.publishResult(
        outcome(task, DeliveryResult.Outcome.DISCARDED, reason));
    log.info("Teslimat dusuruldu: {} - {}", task.deliveryId(), reason);
}

/**
 * Yasam suresi doldu. DLQ'ya gider ama YENI BIR DENEME YOK -
 * o yuzden recordAttempt degil markExhausted.
 */
private void toDeadLetter(DeliveryTask task, String reason) {
    store.markExhausted(task, reason);
    publisher.deadLetter(task, reason);
    publisher.publishResult(
        outcome(task, DeliveryResult.Outcome.EXHAUSTED, reason));
}

// DeliveryResult fabrikalari
//
// Ikisi de ayni kaydi kuruyor; ayrim, ortada gercek bir HTTP cevabi
// olup olmamasi. Uc ayri yerde elle kurulmasi hem tekrardi hem de
// alan sirasini karistirmaya davetiyeydi (hepsi UUID ve String).

/** Gercek bir HTTP denemesinin sonucu. */
private DeliveryResult outcome(DeliveryContext ctx, SendResult r, DeliveryResult.Outcome
    o) {

```

```

        DeliveryTask task = ctx.task();
        return new DeliveryResult(task.deliveryId(), task.endpointId(), task.applicationId()
            ,
            task.eventType(), o, r.httpStatus(), r.latencyMs(), task.attempt(),
            r.error(), Instant.now());
    }

    /** İstek atılmadan verilen karar (engelleme, dusurme, sure asimi). */
    private DeliveryResult outcome(DeliveryTask task, DeliveryResult.Outcome o, String reason) {
        return new DeliveryResult(task.deliveryId(), task.endpointId(), task.applicationId()
            ,
            task.eventType(), o, null, 0, task.attempt(), reason, Instant.now());
    }
}

```

8.8.23 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/consumer/DeliveryConsumer.java`

`finally { ack.acknowledge(); }` — her durumda ack. Başarısız teslimat kaybolmuyor, bir retry topic'ine taşınıyor.

```

package dev.hookrelay.dispatcher.consumer;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.contracts.Topics;
import dev.hookrelay.dispatcher.delivery.DeliveryProcessor;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.stereotype.Component;

/**
 * Ana teslimat tuketicisi.
 *
 * * Offset islemden SONRA commit ediliyor, yani "en az bir kez" teslimat.
 * * Webhook'ta "iki kez geldi" cozulebilir bir problem (alici
 * * X-HookRelay-Id ile ayiklar), "hic gelmedi" cozulemez.
 *
 * * Islem basarisiz olsa bile ack ediyoruz: basarisiz teslimat kaybolmuyor,
 * * bir retry topic'ine tasiniyor. Ack etmeseydik ayni kayit partition'i
 * * bloklardi (head-of-line blocking).
 */
@Component
public class DeliveryConsumer {

    private static final Logger log = LoggerFactory.getLogger(DeliveryConsumer.class);

    private final DeliveryProcessor processor;
    private final ObjectMapper mapper;

    public DeliveryConsumer(DeliveryProcessor processor, ObjectMapper mapper) {
        this.processor = processor;
        this.mapper = mapper;
    }

    @KafkaListener(
        topics = Topics.MESSAGES,

```

```

        groupId = "${hookrelay.consumer.group:dispatcher}",
        concurrency = "${hookrelay.consumer.concurrency:6}")
    public void onMessage(String payload, Acknowledgment ack) {
        try {
            DeliveryTask task = mapper.readValue(payload, DeliveryTask.class);
            processor.process(task);
        } catch (Exception e) {
            // Buraya dusmek beklenmeyen bir durum: govde ayristirilamadi
            // ya da altyapi coktu. Mesaji bloklamiyoruz, logluyoruz.
            log.error("Teslimat gorevi islenemedi, atlaniyor: {}", payload, e);
        } finally {
            ack.acknowledge();
        }
    }
}

```

8.8.24 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/api/DeliveryQueryController`

"Webhook gelmedi" diyen müşteriye bakılacak ilk yer. Bu uçlar olmadan sistem bir kara kutudur.

`size` 200'e kırılıyor: `?size=1000000` diyen bir istek belleği yakmasın.

```

package dev.hookrelay.dispatcher.api;

import dev.hookrelay.common.error.ApiException;
import dev.hookrelay.dispatcher.domain.Delivery;
import dev.hookrelay.dispatcher.domain.DeliveryAttempt;
import dev.hookrelay.dispatcher.repo.DeliveryAttemptRepository;
import dev.hookrelay.dispatcher.repo.DeliveryRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

/**
 * Teslimat gunlugu.
 *
 * "Webhook gelmedi" diyen musteriye bakilacak ilk yer. Her denemenin
 * ne zaman yapildigi, hangi kodun dondugu, cevabin ilk 2000 karakteri.
 * Bu uclar olmadan sistem bir kara kutudur.
 */
@RestController
@RequestMapping("/api/deliveries")
public class DeliveryQueryController {

    private final DeliveryRepository deliveries;
    private final DeliveryAttemptRepository attempts;

    public DeliveryQueryController(DeliveryRepository deliveries,
                                   DeliveryAttemptRepository attempts) {
        this.deliveries = deliveries;
        this.attempts = attempts;
    }
}

```

```

}

public record DeliveryView(
    UUID id, UUID messageId, UUID endpointId, String eventType, String status,
    int attempts, Integer lastStatus, String lastError,
    Instant nextAttemptAt, Instant createdAt, Instant updatedAt) {

    static DeliveryView from(Delivery d) {
        return new DeliveryView(d.getId(), d.getMessageId(), d.getEndpointId(),
            d.getEventType(), d.getStatus().name(), d.getAttempts(),
            d.getLastStatus(), d.getLastError(), d.getNextAttemptAt(),
            d.getCreatedAt(), d.getUpdatedAt());
    }
}

public record AttemptView(
    int attempt, Integer httpStatus, long latencyMs, String error,
    String responseSnippet, Instant occurredAt) {

    static AttemptView from(DeliveryAttempt a) {
        return new AttemptView(a.getAttempt(), a.getHttpStatus(), a.getLatencyMs(),
            a.getError(), a.getResponseSnippet(), a.getOccurredAt());
    }
}

public record DeliveryDetail(DeliveryView delivery, String payload, List<AttemptView> attempts) {}

@GetMapping
public Page<DeliveryView> list(
    @RequestParam(required = false) UUID endpointId,
    @RequestParam(required = false) UUID applicationId,
    @RequestParam(required = false) Delivery.Status status,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "25") int size) {

    Pageable pageable = PageRequest.of(page, Math.min(size, 200));

    if (endpointId != null) {
        return deliveries.findByEndpointIdOrderByCreatedAtDesc(endpointId, pageable)
            .map(DeliveryView::from);
    }
    if (applicationId != null) {
        return deliveries.findByApplicationIdOrderByCreatedAtDesc(applicationId, pageable)
            .map(DeliveryView::from);
    }
    if (status != null) {
        return deliveries.findByStatusOrderByUpdatedAtDesc(status, pageable)
            .map(DeliveryView::from);
    }
    return deliveries.findAll(pageable).map(DeliveryView::from);
}

@GetMapping("/{id}")
public DeliveryDetail get(@PathVariable UUID id) {
    Delivery d = deliveries.findById(id)
        .orElseThrow(() -> ApiException.notFound("Teslimat", id));
    List<AttemptView> list = attempts.findByDeliveryIdOrderByAttemptAsc(id)
        .stream().map(AttemptView::from).toList();
    return new DeliveryDetail(DeliveryView.from(d), d.getPayload(), list);
}

```

```

    }

    @GetMapping("/stats")
    public Map<String, Long> stats() {
        Map<String, Long> out = new LinkedHashMap<>();
        for (Delivery.Status s : Delivery.Status.values()) out.put(s.name(), 0L);
        for (Object[] row : deliveries.countGroupedByStatus()) {
            out.put(((Delivery.Status) row[0]).name(), (Long) row[1]);
        }
        return out;
    }
}

```

8.8.25 `services/dispatcher/src/main/java/dev/hookrelay/dispatcher/api/ReplayCo

Yeniden gönderimde `attempt` 1'e ve `firstQueuedAt` şu ana sıfırlanıyor: teslimat taze bir teslimat gibi yeniden doğuyor. Aksi hâlde eski yaşam süresi kontrolüne takılıp anında DLQ'ya geri düşerdi.

```

package dev.hookrelay.dispatcher.api;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.error.ApiException;
import dev.hookrelay.contracts.DeliveryTask;
import dev.hookrelay.contracts.Topics;
import dev.hookrelay.dispatcher.domain.Delivery;
import dev.hookrelay.dispatcher.repo.DeliveryRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.data.domain.PageRequest;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.UUID;

/**
 * YENIDEN GONDERIM (replay).
 *
 * DLQ'nun otomatik tüketicisi yok - olmamalı da. Ama insanın
 * mudahale edebileceği bir kapi OLMAK ZORUNDA, yoksa DLQ bir cop
 * kutusuna donusur.
 *
 * Tipik senaryo: musterinin sunucusu 3 saat cokmus, 8.000 teslimat
 * tukenip DLQ'ya dusmus. Sunucu geri geldi. Bir dugmeye basip hepsini
 * yeniden kuyruğa koyabilmek gerekiyor.
 *
 * Yeniden gonderimde attempt 1'e ve firstQueuedAt su ana sifirlanir:
 * teslimat, taze bir teslimat gibi yeniden dogar. Aksi halde eski
 * yasam suresi kontrolune takilip aninda DLQ'ya geri duserdi.
 */
@RestController
@RequestMapping("/api/deliveries")
public class ReplayController {

    private static final Logger log = LoggerFactory.getLogger(ReplayController.class);

```

```

private final DeliveryRepository deliveries;
private final KafkaTemplate<String, String> kafka;
private final ObjectMapper mapper;

public ReplayController(DeliveryRepository deliveries, KafkaTemplate<String, String> kaf
    ka,
                        ObjectMapper mapper) {
    this.deliveries = deliveries;
    this.kafka = kafka;
    this.mapper = mapper;
}

@PostMapping("/{id}/replay")
@Transactional
public Map<String, Object> replay(@PathVariable UUID id) {
    Delivery d = deliveries.findById(id)
        .orElseThrow(() -> ApiException.notFound("Teslimat", id));
    requeue(d);
    return Map.of("deliveryId", id, "status", "yeniden_kuyruga_alindi");
}

/** Bir endpoint'in butun tukenmis teslimatlarini yeniden kuyruga koyar. */
@PostMapping("/replay-exhausted")
@Transactional
public Map<String, Object> replayExhausted(
    @RequestParam(required = false) UUID endpointId,
    @RequestParam(defaultValue = "500") int limit) {

    PageRequest page = PageRequest.of(0, Math.min(limit, 5000));

    // Filtreleme VERITABANINDA. Once sayfalayip sonra Java'da
    // filtrelemek, limit'i yalan soyleyen bir sayiya cevirir.
    List<Delivery> targets = endpointId == null
        ? deliveries.findByStatusOrderByUpdatedAtDesc(Delivery.Status.EXHAUSTED, pag
            e)
        : deliveries.findByEndpointIdAndStatusOrderByUpdatedAtDesc(
            endpointId, Delivery.Status.EXHAUSTED, page);

    targets.forEach(this::requeue);
    log.info("Toplu yeniden gonderim: {} teslimat", targets.size());
    return Map.of("requeued", targets.size());
}

private void requeue(Delivery d) {
    if (d.getPayload() == null) {
        throw ApiException.badRequest(
            "Bu teslimatin govdesi saklanmamis, yeniden gonderilemez: " + d.getId())
        ;
    }
    DeliveryTask task = new DeliveryTask(
        d.getId(), d.getMessageId(), d.getApplicationId(), d.getEndpointId(),
        d.getEventType(), d.getPayload(), 1, Instant.now());

    d.resetForReplay();
    deliveries.save(d);

    try {
        kafka.send(Topics.MESSAGES, d.getEndpointId().toString(),
            mapper.writeValueAsString(task));
    }
}

```



```

        } catch (Exception e) {
            throw new IllegalStateException("Yeniden gönderim yayınlanamadi", e);
        }
    }
}

```

8.8.26 `services/dispatcher/src/main/resources/application.yml`

Dört kritik satır:

`spring.threads.virtual.enabled: true` — bu servisin işi neredeyse tamamen "istek at, bekle".

`max.poll.records: 50` — her kayıt bir HTTP isteği. 500 alıp 500 istek atmaya çalışmak

`max.poll.interval.ms`'i aşar ve gruptan atılmak demektir.

`ack-mode: manual_immediate` — offset'i biz commit ediyoruz.

`concurrency: 6` — iki örnekle 12, yani partition sayısı. Fazlası boş oturur.

```

server:
  port: 8083

spring:
  application:
    name: dispatcher

# Java 21 sanal iş parçacıkları. Bu servisin işi neredeyse tamamen
# "istek at, bekle" olduğu için en çok fayda eden yer burası.
threads:
  virtual:
    enabled: true

datasource:
  url: ${DB_URL:jdbc:postgresql://localhost:5432/hookrelay_delivery}
  username: ${DB_USER:hookrelay}
  password: ${DB_PASSWORD:hookrelay}
  hikari:
    maximum-pool-size: 20
    pool-name: dispatcher-pool

jpa:
  hibernate:
    ddl-auto: none
    open-in-view: false
  properties:
    hibernate.jdbc.time_zone: UTC

flyway:
  enabled: true
  locations: classpath:db/migration
  baseline-on-migrate: true

data:
  redis:
    host: ${REDIS_HOST:localhost}
    port: ${REDIS_PORT:6379}
  lettuce:
    pool:
      max-active: 64

```

```

kafka:
  bootstrap-servers: ${KAFKA_BOOTSTRAP:localhost:9092}
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.apache.kafka.common.serialization.StringSerializer
    acks: all
    properties:
      enable.idempotence: true
      max.in.flight.requests.per.connection: 5
      compression.type: lz4
  consumer:
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    auto-offset-reset: earliest
    enable-auto-commit: false
    properties:
      # Bir poll'da en fazla 50 kayıt. Yüksek tutmak cazip ama
      # her kayıt bir HTTP istegi demek: 500 kayıt alıp 500 istek
      # atmaya çalışırsak max.poll.interval.ms'i asar ve gruptan
      # atılırız. Küçük paket = sık heartbeat = sağlıklı tüketici.
      max.poll.records: 50
      max.poll.interval.ms: 300000
  listener:
    # MANUAL_IMMEDIATE: offset'i biz commit ediyoruz, her kayıttan sonra.
    # Bkz. DeliveryConsumer javadoc – "en az bir kez" kararı.
    ack-mode: manual_immediate

hookrelay:
  consumer:
    group: dispatcher
    # Örnek başına tüketici iş parçası. 6 x 2 örnek = 12 = partition sayısı.
    concurrency: 6
  outbox:
    enabled: false          # dispatcher outbox kullanmıyor
  endpoint-config:
    replicate: true         # sıcak yol için ZORUNLU
  app-config:
    replicate: false
  delivery:
    # 4xx varsayılan olarak KALICI hata sayılır ve tek denemede DLQ'ya gider.
    # true yapmak, gecici 403 dönen müşteriler için yeniden denemeyi acar.
    # 410 Gone bu ayardan etkilenmez – her zaman kalıcıdır.
    retry-on-4xx: false
  http:
    connect-timeout-ms: 5000
  bulkhead:
    max-concurrent-per-endpoint: 4
  circuit-breaker:
    failure-threshold: 5
    open-millis: 30000
    half-open-probes: 2
    window-millis: 60000
  retry:
    # Üretim değerleri. docker-compose demo profilinde kısaltılıyor.
    delays: 10s,1m,5m,30m,2h
    max-lifetime: 24h

management:
  endpoints:
    web:
      exposure:

```

```
    include: health,info,prometheus,metrics
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      service: dispatcher
  logging:
    level:
      dev.hookrelay: INFO
```

Kontrol noktası

```
mvn -q test -pl services/dispatcher -am
```

Yeniden deneme politikası ve hata sınıflandırma testleri geçmeli.

Bölüm 9 — retry-scheduler: Kafka'da Geciktirmeli Mesaj

Projedeki en ilginç bölüm. Kafka'nın **desteklemediği** bir şeyi, Kafka'nın kendi ilkelleriyle kuracağız.

9.1 Problem

RabbitMQ'da "bu mesajı 30 dakika sonra teslim et" diyebilirsiniz. Kafka'da **böyle bir şey yok**.

Sebebi mimari: Kafka bir kuyruk değil, bir **günlüktür**. Kayıtlar yazıldıkları sırada durur, tüketici baştan sona okur. Tek bir kaydı "sonraya bırakmak" diye bir işlem yoktur — çünkü kayıtların bir "sırası" vardır ve onu değiştiremezsiniz.

9.2 Neden `Thread.sleep` felaket

İlk akla gelen:

```
@KafkaListener(topics = "retry")
public void onRetry(String payload) {
    long notBefore = ...;
    Thread.sleep(notBefore - System.currentTimeMillis()); // FELAKET
    process(payload);
}
```

Ne oluyor:

1. Tüketici 30 dakika uyur.
2. `max.poll.interval.ms` (varsayılan 5 dakika) aşılr.
3. Broker tüketiciyi ölmüş sayar ve gruptan atar.
4. Rebalance tetiklenir; partition başka bir örneğe gider.
5. O örnek de aynı kaydı alır ve uyur.
6. Adım 2'ye dön.

Sonuç: bütün tüketici grubu kilitlenir. Hiçbir mesaj işlenmez. Loglarda sürekli rebalance görürsünüz ve sebebini bulmanız günler alır.

9.3 Çözüm: katmanlı topic + `pause`/`seek`

Birinci parça: her gecikme için ayrı topic.

```
t1 → 10 saniye
t2 → 1 dakika
t3 → 5 dakika
t4 → 30 dakika
t5 → 2 saat
```

İkinci parça: `pause` + `seek`.

```

for (TopicPartition partition : records.partitions()) {
    long lastProcessed = -1;

    for (ConsumerRecord<String, String> record : records.records(partition)) {
        long notBefore = KafkaHeaderCodec.longValue(
            record.headers(), Headers.NOT_BEFORE, record.timestamp());

        if (now < notBefore) {
            // BAŞTAKİ KAYIT HENÜZ HAZIR DEĞİL.
            consumer.pause(List.of(partition)); // bu partition'dan kayıt gelmesin
            consumer.seek(partition, record.offset()); // offset'i geri al, kayıt kaybolması
            n
            resumeAt.put(partition, notBefore);
            break; // bu partition'da devam etme
        }

        forward(record);
        lastProcessed = record.offset();
    }

    if (lastProcessed >= 0) {
        // Sadece GERÇEKTEN işlenenlere kadar commit.
        commits.put(partition, new OffsetAndMetadata(lastProcessed + 1));
    }
}

```

Ve döngünün başında:

```

private void resumeDuePartitions() {
    long now = System.currentTimeMillis();
    List<TopicPartition> due = resumeAt.entrySet().stream()
        .filter(e -> now >= e.getValue())
        .map(Map.Entry::getKey)
        .toList();

    Set<TopicPartition> assigned = consumer.assignment();
    List<TopicPartition> resumable = due.stream().filter(assigned::contains).toList();

    if (!resumable.isEmpty()) consumer.resume(resumable);
    due.forEach(resumeAt::remove);
}

```

Neden bu çalışıyor

`poll()` çağrılmaya **devam ediyor**. Duraklatılmış partition'dan kayıt gelmez ama `poll()` heartbeat gönderir → broker bizi canlı sayar → rebalance olmaz.

`seek()` ise offset'i geri alır. Duraklatma kalktığında aynı kayıt yeniden okunur; hiçbir şey kaybolmaz.

9.4 Neden sadece baştaki kayda bakmak yetiyor

Bu, çözümün kilit taşı.

Bir katman topic'indeki **bütün kayıtların gecikmesi aynıdır** (t3'te hepsi 5 dakika). Kayıtlar yazılma sırasına göre durur. Dolayısıyla:

```
not-before sırası = yazılma sırası = offset sırası
```

Baştaki kaydın vakti gelmediyse, arkasındakilerin de gelmemiştir. Tek kontrol yeter.

Karışık gecikmeler tek topic'te olsaydı bu çalışmazdı:

```
offset 100: not-before = 12:30    (30 dakikalık)
offset 101: not-before = 12:05    (5 dakikalık)   ← şu an hazır
offset 102: not-before = 12:31
```

Saat 12:05'te offset 101'i işlemek istiyorsunuz ama 100'ün üzerinden atlamanız gerekiyor. Kafka'da bunu yapamazsınız — offset ileri alınabilir ama "atla ve sonra geri dön" diye bir mekanizma yoktur. 101'i işleyip offset'i 102'ye commit ederseniz 100'ü **sonsuz kadar kaybedersiniz**.

Katmanlı topic tasarımının asıl sebebi budur. Okunaklı olsun diye değil — **başka türlü mümkün olmadığı için**.

9.5 Rebalance'ta durumu temizlemek

```
private class ClearPausedStateOnRebalance implements ConsumerRebalanceListener {
    @Override
    public void onPartitionsRevoked(Collection<TopicPartition> partitions) {
        partitions.forEach(resumeAt::remove);
    }

    @Override
    public void onPartitionsAssigned(Collection<TopicPartition> partitions) {
        partitions.forEach(resumeAt::remove);
    }
}
```

Elimizden alınan bir partition için "şu saatte devam ettir" notu tutmaya devam edersek, o partition başka bir örnekteyken `resume` etmeye çalışırsınız ve `IllegalStateException` alırsınız.

9.6 `KafkaConsumer` thread-safe değildir

```
@PreDestroy
public void stop() {
    if (!running.compareAndSet(true, false)) return;
    if (consumer != null) consumer.wakeup(); // ← tek güvenli dışarıdan müdahale
    thread.join(10_000);
}
```

`KafkaConsumer` **thread-safe değildir**. Başka bir iş parçacığından `close()` çağırırsanız tanımsız davranış alırsınız.

`wakeup()` tek istisnadır: `poll()` içinde bloke olan tüketiciyi `WakeupException` ile uyandırır. Kapatma her zaman şöyle yapılır: bayrağı indir, `wakeup()` çağır, döngünün kendi kendini bitirmesini bekle.

9.7 Neden ayrı bir servis

Tüketim semantiği `dispatcher`'dan tamamen farklı:

	<code>dispatcher</code>	<code>retry-scheduler</code>
Ne yapar	Hızlı tüketir, HTTP atar	Çoğu zaman duraklatılmış oturur
Ayarlar	Küçük paket, sık poll	Uzun poll, <code>pause/seek</code>
Ölçekleme	Yatay, partition sayısına kadar	Nadiren gerekir
Durum	Veritabanı	Yok — durumsuz

İkisini aynı sürece koymak, birinin ayarlarının diğerini bozması demektir.

9.8 Servisin tamamı

9.8.1 `services/retry-scheduler/pom.xml`

En yalın servis: veritabanı yok, JPA yok. Yalnızca `common` ve actuator.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/
    maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../../pom.xml</relativePath>
  </parent>
  <artifactId>retry-scheduler</artifactId>
  <name>retry-scheduler</name>
  <description>Katmanli gecikmeli yeniden deneme: duraklatılan partition'lar</description>

  <dependencies>
    <dependency><groupId>dev.hookrelay</groupId><artifactId>common</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-a
      ctuator</artifactId></dependency>
    <dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</
      artifactId></dependency>
  </dependencies>

  <build>
    <finalName>retry-scheduler</finalName>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

9.8.2 `services/retry-scheduler/src/main/java/dev/hookrelay/retryscheduler/Retry

```

package dev.hookrelay.retryscheduler;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

/**
 * GECIKMELI YENIDEN DENEME ZAMANLAYICISI.
 *
 * Projedeki en ilginç servis. Kafka'nin desteklemediği bir şeyi -
 * "bu mesajı 30 dakika sonra işle" - Kafka'nin kendi ilkelleriyle kurar.
 *
 * Ayri bir servis olmasının sebebi: tüketim semantigi dispatcher'dan
 * tamamen farklı. Dispatcher hızlı tüketir ve HTTP atar; bu servis
 * cüğü zaman duraklatılmış (paused) partition'larla oturur ve saate bakar.
 * İkisini aynı surece koymak, birinin ayarlarının diğerini bozması demektir.
 */
@SpringBootApplication(scanBasePackages = "dev.hookrelay")
@ConfigurationPropertiesScan("dev.hookrelay")
public class RetrySchedulerApplication {
    public static void main(String[] args) {
        SpringApplication.run(RetrySchedulerApplication.class, args);
    }
}

```

9.8.3 `services/retry-scheduler/src/main/java/dev/hookrelay/retryscheduler/runner

Ham `KafkaConsumer` ile yazıldı, `@KafkaListener` ile değil. Sebebi: `pause/seek/resume` üçlüsünü ve `rebalance` dinleyicisini elle yönetmek gerekiyor; Spring soyutlaması bu kontrolü gizliyor.

Üç noktaya dikkat:

`@EventListener(ApplicationReadyEvent.class)` ile başlıyor, kurucu içinde değil. Kurucuda başlatsaydık, henüz hazır olmayan bean'lere erişme riski olurdu.

`@PreDestroy` içinde `wakeup()` — `KafkaConsumer` **thread-safe değildir**, başka bir iş parçacığından `close()` çağırmak tanımsız davranıştır. `wakeup()` tek istisnadır.

`resumeDuePartitions` yalnızca **hâlâ bize atanmış** partition'ları devam ettiriyor. Rebalance sırasında elimizden alınmış bir partition'ı `resume` etmek `IllegalStateException` verir.

```

package dev.hookrelay.retryscheduler.runner;

import dev.hookrelay.common.kafka.KafkaHeaderCodec;
import dev.hookrelay.contracts.Headers;
import dev.hookrelay.contracts.Topics;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.Gauge;
import io.micrometer.core.instrument.MeterRegistry;
import jakarta.annotation.PreDestroy;
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.errors.WakeupException;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.context.event.ApplicationReadyEvent;

```



```

import org.springframework.context.event.EventListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

import java.time.Duration;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicBoolean;

/**
 * Kafka'da geciktirmeli mesaj: her gecikme için ayrı topic + pause/seek.
 *
 * Kafka bir kuyruk değil gunluktur; "bunu 30 dakika sonra ver" diye bir
 * işlem yok. Thread.sleep ise max.poll.interval.ms'i asar, broker
 * tüketiciyi olmuş sayar ve grup sonsuz rebalance'a girer.
 *
 * Bunun yerine: vakti gelmemiş kaydı görünce partition'i duraklat,
 * offset'i o kayda geri al, poll() çağırma devam et (heartbeat gitsin).
 * Vakit gelince resume.
 *
 * Bir katman topic'indeki bütün kayıtların gecikmesi aynı olduğu için
 * not-before sırası = offset sırası; bastaki kaydı kontrol etmek yeter.
 * Katmanlı topic tasarımının asıl sebebi bu.
 *
 * Ayrıntılı gerekçe: docs/ içindeki rehber, Bölüm 9.
 */
@Component
public class RetrySchedulerRunner implements Runnable {

    private static final Logger log = LoggerFactory.getLogger(RetrySchedulerRunner.class);

    private final KafkaTemplate<String, String> producer;
    private final MeterRegistry meters;
    private final String bootstrapServers;
    private final String groupId;
    private final long pollMillis;

    private final AtomicBoolean running = new AtomicBoolean(false);
    private final Map<TopicPartition, Long> resumeAt = new ConcurrentHashMap<>();
    private KafkaConsumer<String, String> consumer;
    private Thread thread;

    private Counter forwarded;
    private Counter earlyPaused;

    public RetrySchedulerRunner(KafkaTemplate<String, String> producer,
                               MeterRegistry meters,
                               @Value("${spring.kafka.bootstrap-servers}") String bootstrapServers,
                               @Value("${hookrelay.scheduler.group:retry-scheduler}") String groupId,
                               @Value("${hookrelay.scheduler.poll-ms:500}") long pollMillis) {
        this.producer = producer;
        this.meters = meters;
        this.bootstrapServers = bootstrapServers;
        this.groupId = groupId;
        this.pollMillis = pollMillis;
    }

    @EventListener(ApplicationReadyEvent.class)

```

```

public void start() {
    if (!running.compareAndSet(false, true)) return;

    this.forwarded = Counter.builder("hookrelay.scheduler.forwarded").register(meters);
    this.earlyPaused = Counter.builder("hookrelay.scheduler.paused").register(meters);
    Gauge.builder("hookrelay.scheduler.paused_partitions", resumeAt, Map::size)
        .register(meters);

    this.consumer = new KafkaConsumer<>(consumerProperties());
    this.thread = new Thread(this, "retry-scheduler");
    this.thread.setDaemon(false);
    this.thread.start();
    log.info("Yeniden deneme zamanlayicisi basladi. Katmanlar: {}", Topics.RETRY_TIERS);
}

@PreDestroy
public void stop() {
    if (!running.compareAndSet(true, false)) return;
    // wakeup(): poll() icinde bloke olan tuketiciyi WakeupException ile
    // uyandırır. Tek güvenli dışarıdan durdurma yöntemi budur;
    // KafkaConsumer thread-safe DEĞİLDİR, baska metodunu baska
    // is paracacigindan cagiramazsiniz.
    if (consumer != null) consumer.wakeup();
    try {
        if (thread != null) thread.join(10_000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

@Override
public void run() {
    try {
        consumer.subscribe(Topics.RETRY_TIERS, new ClearPausedStateOnRebalance());

        while (running.get()) {
            resumeDuePartitions();

            ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(pollInterval));
            if (records.isEmpty()) {
                idleBackoff();
                continue;
            }

            Map<TopicPartition, OffsetAndMetadata> commits = new HashMap<>();
            long now = System.currentTimeMillis();

            for (TopicPartition partition : records.partitions()) {
                long lastProcessed = -1;

                for (ConsumerRecord<String, String> record : records.records(partition)) {
                    long notBefore = KafkaHeaderCodec.longValue(
                        record.headers(), Headers.NOT_BEFORE, record.timestamp());

                    if (now < notBefore) {
                        // BASTAKI KAYIT HENUZ HAZIR DEĞİL.
                        // Partition'i duraklat, offset'i bu kayda geri al.
                        consumer.pause(List.of(partition));
                        consumer.seek(partition, record.offset());
                    }
                }
            }
        }
    }
}

```

```

        resumeAt.put(partition, notBefore);
        earlyPaused.increment();

        log.debug("Duraklatildi: {} → {} ms sonra",
            partition, notBefore - now);
        break; // bu partition'da devam etme
    }

    forward(record);
    lastProcessed = record.offset();
}

if (lastProcessed >= 0) {
    // Sadece GERCEKTEN islenenlere kadar commit.
    // Duraklatılan kaydın offset'i commit EDİLMEZ.
    commits.put(partition, new OffsetAndMetadata(lastProcessed + 1));
}

if (!commits.isEmpty()) {
    consumer.commitSync(commits);
}

} catch (WakeupException e) {
    if (running.get()) log.error("Beklenmeyen wakeup", e);
} catch (Exception e) {
    log.error("Zamanlayıcı dongusu coku", e);
} finally {
    try {
        consumer.close(Duration.ofSeconds(5));
    } catch (Exception ignored) {}
    log.info("Yeniden deneme zamanlayicisi durdu");
}

}

/** Vakti gelen partition'lari yeniden aktif et. */
private void resumeDuePartitions() {
    if (resumeAt.isEmpty()) return;
    long now = System.currentTimeMillis();

    List<TopicPartition> due = resumeAt.entrySet().stream()
        .filter(e -> now >= e.getValue())
        .map(Map.Entry::getKey)
        .toList();

    if (due.isEmpty()) return;
    // Sadece BIZE HALA ATANMIS olanlari devam ettir; rebalance
    // sirasinda elimizden alinmis bir partition'i resume etmek hata verir.
    Set<TopicPartition> assigned = consumer.assignment();
    List<TopicPartition> resumable = due.stream().filter(assigned::contains).toList();

    if (!resumable.isEmpty()) {
        consumer.resume(resumable);
        log.debug("Devam ettirildi: {}", resumable);
    }
    due.forEach(resumeAt::remove);
}

private void forward(ConsumerRecord<String, String> record) {
    // Key DEĞİSMİYOR (endpointId): ana topic'te de aynı partition'a
    // duser, aynı endpoint için sıra korunur.

```

```

        producer.send(Topics.MESSAGES, record.key(), record.value());
        forwarded.increment();
    }

    /**
     * Butun partition'lar duraklatilmissa poll() aninda bos doner ve
     * dongu CPU yakar. Kucuk bir nefes.
     */
    private void idleBackoff() {
        if (resumeAt.isEmpty()) return;
        try {
            Thread.sleep(Math.min(pollMillis, 250));
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private Properties consumerProperties() {
        Properties p = new Properties();
        p.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        p.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);
        p.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        p.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
            "org.apache.kafka.common.serialization.StringDeserializer");
        p.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false); // offset'i biz yönetiyoruz
        p.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        p.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 200);
        return p;
    }

    /**
     * Rebalance'ta duraklatma durumu SIFIRLANMALI.
     *
     * Elimizden alınan bir partition için "su saatte devam ettir" notu
     * tutmaya devam edersek, o partition baska bir ornekteyken resume
     * etmeye calisiriz ve IllegalStateException aliriz. Yeni atanan
     * partition'lar zaten duraklatilmamis gelir.
     */
    private class ClearPausedStateOnRebalance implements ConsumerRebalanceListener {
        @Override
        public void onPartitionsRevoked(Collection<TopicPartition> partitions) {
            partitions.forEach(resumeAt::remove);
            log.debug("Partition'lar geri alindi: {}", partitions);
        }

        @Override
        public void onPartitionsAssigned(Collection<TopicPartition> partitions) {
            partitions.forEach(resumeAt::remove);
            log.info("Partition'lar atandi: {}", partitions);
        }
    }

    /** Arayuzde gosterme icin: hangi partition ne zaman devam edecek. */
    public Map<String, Long> pausedPartitions() {
        Map<String, Long> out = new LinkedHashMap<>();
        long now = System.currentTimeMillis();
        resumeAt.forEach((tp, at) -> out.put(tp.toString(), Math.max(0, at - now)));
        return out;
    }

```

```
}
```

9.8.4 `services/retry-scheduler/src/main/java/dev/hookrelay/retryscheduler/api/S

Zamanlayıcının iç durumunu dışarı açıyor. Bu bilgi olmadan sistem sihirli bir kara kutu gibi görünür; arayüzde "t3 · 18 sn" yazması, tasarımın en öğretici parçasını gözle görülür kılıyor.

```
package dev.hookrelay.retryscheduler.api;

import dev.hookrelay.contracts.Topics;
import dev.hookrelay.retryscheduler.runner.RetrySchedulerRunner;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Zamanlayıcının iç durumunu dışarı açar.
 *
 * "Hangi partition duraklatılmış, ne kadar sonra devam edecek" -
 * bu bilgi olmadan sistem sihirli bir kara kutu gibi görünür.
 * Arayüzde canlı göstermek, tasarımın en öğretici parçasını
 * gözle görülür kılıyor.
 */
@RestController
@RequestMapping("/api/scheduler")
public class SchedulerController {

    private final RetrySchedulerRunner runner;

    public SchedulerController(RetrySchedulerRunner runner) {
        this.runner = runner;
    }

    @GetMapping("/state")
    public Map<String, Object> state() {
        Map<String, Object> out = new LinkedHashMap<>();
        out.put("tiers", Topics.RETRY_TIERS);
        out.put("pausedPartitions", runner.pausedPartitions());
        return out;
    }
}
```

9.8.5 `services/retry-scheduler/src/main/resources/application.yml`

`spring.autoconfigure.exclude` ile `DataSource` ve `JPA` kapatılmış. Bu serviste veritabanı yok; kapatmasaydık Spring boş bir `DataSource` arayıp açılışa patlardı.

```
server:
  port: 8084

spring:
  application:
    name: retry-scheduler
```

```

kafka:
  bootstrap-servers: ${KAFKA_BOOTSTRAP:localhost:9092}
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.apache.kafka.common.serialization.StringSerializer
    acks: all
    properties:
      enable.idempotence: true

data:
  redis:
    host: ${REDIS_HOST:localhost}
    port: ${REDIS_PORT:6379}

autoconfigure:
  # Bu serviste veritabani YOK. Kendi durumu yok, sadece mesaj tasiyor.
  exclude:
    - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
    - org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration

hookrelay:
  scheduler:
    group: retry-scheduler
    poll-ms: 500
  endpoint-config:
    replicate: false
  app-config:
    replicate: false

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      service: retry-scheduler

logging:
  level:
    dev.hookrelay: INFO

```

Kontrol noktası

```
mvn -q compile -pl services/retry-scheduler -am
```

Bölüm 10 — chaos-target ve gateway

10.1 chaos-target — demonun kurbanı

Bu servis projenin en değerli parçalarından biri, çünkü onsuz yeniden deneme, devre kesici ve hız sınırı özelliklerini **gösteremezsiniz**.

Mimari dokümanda "devre kesici var" yazmak kolay. Ekranda açılıp kapandığını izletmek başka bir şey.

Uç	Davranış	Ne kanıtlar
/sink/ok	Her zaman 200	Mutlu yol
/sink/flaky?rate=50	%50 ihtimalle 500	Katmanlı yeniden deneme, üstel azalma
/sink/slow?ms=30000	Yavaş cevap	Zaman aşımı, bulkhead
/sink/ratelimited	429 + Retry-After	Retry-After 'a saygı
/sink/dead	Her zaman 500	Devre kesici açılması
/sink/gone	410 Gone	Kalıcı hata, anında DLQ
/sink/strict?secret=...	İmzayı doğrular	HMAC gerçekten çalışıyor

/sink/strict özellikle önemli:

```
@PostMapping("/strict")
public ResponseEntity<String> strict(@RequestParam String secret,
                                     @RequestBody String body,
                                     HttpServletRequest request) {
    String header = request.getHeader(Headers.OUT_SIGNATURE);
    boolean valid = signer.verify(secret, body, header, 300);

    if (!valid) return ResponseEntity.status(401).body("{\"error\":\"imza geçersiz\"}");
    return ResponseEntity.ok("{\"ok\":true,\"signature\":\"doğrulandı\"}");
}
```

Bu, projenin güvenlik iddiasının **çalışan kanıtı**. Doküman "HMAC ile imzalıyoruz" der; burası imzayı gerçekten doğrular ve yanlışsa 401 döner. Mülakatta "imzaladığınızı nasıl biliyorsunuz" sorusuna verilecek cevap bu uçtur.

10.2 gateway

```
spring.cloud.gateway.routes:
- id: admin
  uri: ${ADMIN_URL:http://localhost:8081}
  predicates: [Path=/api/admin/**]
- id: deliveries
  uri: ${DISPATCHER_URL:http://localhost:8083}
  predicates: [Path=/api/deliveries/**]
- id: ingest
  uri: ${INGEST_URL:http://localhost:8082}
```

```
predicates: [Path=/v1/**]
```

Rotaların sırası önemli: Spring ilk eşleşen rotayı kullanır. Özelden genele dizilir.

Neden Eureka yok

Eureka'nın çözdüğü problem, dinamik adresler ve DNS yokluğudur. Docker Compose'da servis adları zaten DNS ile çözülüyor (<http://dispatcher:8083>). Kubernetes'e geçilse `Service` kaynağı aynı işi görür.

Çözmediği bir problem için beşinci bir süreç çalıştırmak, projeye anlaşılabilir bir parça eklemektir. Mülakatta "neden Eureka yok" diye sorulursa cevap hazır — ve bu cevap, Eureka koymuş olmaktan daha iyi bir cevap.

Tuzak: konteyner yeniden oluşunca gateway kör kalıyor

Bu, projeyi ayağa kaldırırken **gerçekten karşılaşılan** bir hata ve konteyner ortamındaki en sinsi gateway problemi.

Belirti. Bir arka servisi yeniden oluşturuyorsunuz (`docker compose up --build dispatcher`). Servis sağlıklı, `curl localhost:8083` çalışıyor, gateway'in içinden `wget http://dispatcher:8083` bile çalışıyor. Ama gateway o servise giden **her isteği 500** ile cevaplıyor:

```
Connection refused: dispatcher/172.23.0.10:8083
```

Oysa konteynerin yeni adresi `172.23.0.11`.

Sebep. Spring Cloud Gateway, Reactor Netty üzerinde çalışır. Reactor Netty varsayılan olarak **kendi** DNS çözümleyicisini kullanır ve sonuçları DNS kaydının TTL'ine göre ön belleğe alır. Docker'ın gömülü DNS sunucusu konteyner adları için **TTL = 600 saniye** döndürür.

Yani gateway, yeniden oluşturulan bir servisi **on dakika** boyunca eski IP'sinde aramaya devam eder. İşletim sistemi çözümleyicisi (`getent`, `wget`) etkilenmez — o yüzden "elle deniyorum çalışıyor, uygulamadan çalışmıyor" gibi baş ağrıtan bir tabloyla karşılaşabilirsiniz.

Çözüm. Reactor Netty'ye JVM'in kendi çözümleyicisini kullanmasını:

```
@Bean
public HttpClientCustomizer jvmDnsResolverCustomizer() {
    return httpClient -> httpClient.resolver(DefaultAddressResolverGroup.INSTANCE);
}
```

JVM, `networkaddress.cache.ttl`'e uyar. Dockerfile'da bunu 10 saniyeye çekiyoruz:

```
ENV JAVA_OPTS="... -Dnetworkaddress.cache.ttl=10 -Dnetworkaddress.cache.negative.ttl=5 ..."
```

Ve bağlantı havuzunda ölü bağlantılar beklemesin:

```
spring.cloud.gateway.httpclient.pool:
  type: elastic
  max-idle-time: 15s
  max-life-time: 60s
  eviction-interval: 30s
```


Bedeli: JVM çözümleyicisi bloke edicidir (Netty'ninki asenkron). Pratikte önemsiz — işletim sistemi çözümü mikrosaniyeler sürer. On dakika kör kalmanın yanında hiçbir şey.

Bu problem **Kubernetes'te de aynen vardır**. Pod yeniden planlandığında IP değişir ve aynı tabloyla karşılaşabilirsiniz. Servis keşfi kütüphanesi kullanan projeler bunu fark etmez; statik URI kullananlar duvara toslar.

`RequestIdFilter` — gateway'in işini hak etmesi

```
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String requestId = exchange.getRequest().getHeaders().getFirst("X-Request-Id");
        if (requestId == null || requestId.isBlank()) requestId = UUID.randomUUID().toString();

        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header("X-Request-Id", requestId).build();
        exchange.getResponse().getHeaders().set("X-Request-Id", requestId);

        return chain.filter(exchange.mutate().request(mutated).build());
    }

    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

Dağıtık sistemde hata ayıklamanın ilk kuralı: bir isteğin dört servisteki izini sürebilmek. Müşteri "saat 14:32'de hata aldım" dediğinde elinizde kimlik yoksa dört servisin loglarını zaman damgasına göre karşılaştırmaya çalışırsınız.

10.3 İki servisin tamamı

10.3.1 `services/chaos-target/pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>dev.hookrelay</groupId>
        <artifactId>hookrelay-parent</artifactId>
        <version>1.0.0</version>
        <relativePath>../../pom.xml</relativePath>
    </parent>
    <artifactId>chaos-target</artifactId>
    <name>chaos-target</name>
    <description>Demo kurbanı: bozuk davranan müşteri sunucusu</description>

    <dependencies>
        <dependency><groupId>dev.hookrelay</groupId><artifactId>common</artifactId></dependency>
```

```

<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-actuator</artifactId></dependency>
<dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</artifactId></dependency>
</dependencies>

<build>
  <finalName>chaos-target</finalName>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

10.3.2 `services/chaos-target/src/main/java/dev/hookrelay/chaostarget/ChaosTargetApplication.java`

`scanBasePackages` dar tutulmuş: yalnızca kendi paketi ve `dev.hookrelay.common.crypto.common`'ın Redis ve Kafka bean'leri taranmasın diye — bu servisin ikisine de ihtiyacı yok.

```

package dev.hookrelay.chaostarget;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * DEMONUN KURBANI - kotu davranan müşteri sunucusu.
 *
 * Bu servis projenin en değerli parçalarından biri, çünkü onsuz
 * yeniden deneme / devre kesici / hız sınırı gibi özellikleri
 * GOSTEREMEZSİNİZ. Mimari dokümanda "devre kesici var" yazmak kolay;
 * ekranda açılıp kapandığını izletmek başka bir şey.
 *
 * Davranışlar:
 * /sink/ok her zaman 200
 * /sink/flaky yüzde N ihtimalle patlar
 * /sink/slow yavaş cevap verir (zaman asımı testi)
 * /sink/ratelimited 429 + Retry-After
 * /sink/dead her zaman 500
 * /sink/gone 410 - kalıcı hata, DLQ'ya anında gitmeli
 * /sink/strict imzayı DOĞRULAR, geçersizse 401
 */
@SpringBootApplication(scanBasePackages = {"dev.hookrelay.chaostarget", "dev.hookrelay.common.crypto"})
public class ChaosTargetApplication {
    public static void main(String[] args) {
        SpringApplication.run(ChaosTargetApplication.class, args);
    }
}

```

10.3.3 `services/chaos-target/src/main/java/dev/hookrelay/chaostarget/ChaosController.java`

Yedi davranış. `/sink/strict` en önemlisi: imzayı **gerçekten** doğruluyor ve geçersizse 401 dönüyor. Projenin güvenlik iddiasının çalışan kanıtı.

`received` bir `ConcurrentLinkedDeque` ve 500 kayıta sabitleniyor. Sınırsız bıraksaydık yük testinde

bellek dolardı.

```
package dev.hookrelay.chaostarget;

import dev.hookrelay.common.crypto.WebhookSigner;
import dev.hookrelay.contracts.Headers;
import jakarta.servlet.http.HttpServletRequest;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.*;
import java.util.concurrent.ConcurrentLinkedDeque;
import java.util.concurrent.ThreadLocalRandom;
import java.util.concurrent.atomic.AtomicLong;

@RestController
@RequestMapping("/sink")
public class ChaosController {

    private static final Logger log = LoggerFactory.getLogger(ChaosController.class);
    private static final int MEMORY = 500;

    private final WebhookSigner signer;

    /** Son N istegin kaydi. Demoda "gercekten geldi mi" sorusunun cevabi. */
    private final Deque<Received> received = new ConcurrentLinkedDeque<>();
    private final Map<String, AtomicLong> counters = new java.util.concurrent.ConcurrentHashMap<>();

    public ChaosController(WebhookSigner signer) {
        this.signer = signer;
    }

    public record Received(String behavior, String deliveryId, String eventType,
        int attempt, boolean signatureValid, int respondedWith,
        Instant at) {}

    @PostMapping("/ok")
    public ResponseEntity<String> ok(@RequestBody(required = false) String body,
        HttpServletRequest request) {
        record("ok", request, 200, null);
        return ResponseEntity.ok("{\"ok\":true}");
    }

    /**
     * Yuzde rate ihtimalle 500 doner.
     *
     * Demonun kalbi: %50 ile 1000 mesaj gonderin, yarisi ilk denemede
     * gecer, yarisi t1'e duser, onun yarisi t2'ye... Gozunuzle
     * ustel azalmayi izlersiniz.
     */
    @PostMapping("/flaky")
    public ResponseEntity<String> flaky(@RequestParam(defaultValue = "50") int rate,
        @RequestBody(required = false) String body,
        HttpServletRequest request) {
        boolean fail = ThreadLocalRandom.current().nextInt(100) < rate;
        int status = fail ? 500 : 200;
    }
}
```

```

        record("flaky", request, status, null);
        return ResponseEntity.status(status)
            .body(fail ? "{\"error\":\"rastgele hata\"}" : "{\"ok\":true}");
    }

    /** Zaman asimi testi. endpoint.timeoutMs'ten büyük verin. */
    @PostMapping("/slow")
    public ResponseEntity<String> slow(@RequestParam(defaultValue = "30000") long ms,
                                      @RequestBody(required = false) String body,
                                      HttpServletRequest request) throws InterruptedException {
        Thread.sleep(Math.min(ms, 120_000));
        record("slow", request, 200, null);
        return ResponseEntity.ok("{\"ok\":true,\"slow\":true}");
    }

    /** 429 + Retry-After. Dispatcher'in bu basliga saygi gosterdigini kanitlar. */
    @PostMapping("/ratelimited")
    public ResponseEntity<String> rateLimited(@RequestParam(defaultValue = "30") int retryAfter,
                                              @RequestBody(required = false) String body,
                                              HttpServletRequest request) {
        record("ratelimited", request, 429, null);
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .header("Retry-After", String.valueOf(retryAfter))
            .body("{\"error\":\"cok hizli\"}");
    }

    /** Tamamen olmus sunucu. Devre kesiciyi acan senaryo. */
    @PostMapping("/dead")
    public ResponseEntity<String> dead(@RequestBody(required = false) String body,
                                       HttpServletRequest request) {
        record("dead", request, 500, null);
        return ResponseEntity.status(500).body("{\"error\":\"sunucu olu\"}");
    }

    /** 410 Gone - KALICI hata. Yeniden denenmeden DLQ'ya gitmeli. */
    @PostMapping("/gone")
    public ResponseEntity<String> gone(@RequestBody(required = false) String body,
                                       HttpServletRequest request) {
        record("gone", request, 410, null);
        return ResponseEntity.status(HttpStatus.GONE).body("{\"error\":\"bu endpoint kaldiri ldi\"}");
    }

    /**
     * IMZA DOGRULAYAN uc.
     *
     * Bu, projenin guvenlik iddiasinin CALISAN kaniti. Dokuman
     * "HMAC ile imzaliyoruz" der; burasi imzayi gercekten dogrular ve
     * yanlissa 401 doner. secret query parametresinden gelir cunku
     * bu bir demo sunucusu, gercek bir musterisi degil.
     */
    @PostMapping("/strict")
    public ResponseEntity<String> strict(@RequestParam String secret,
                                         @RequestBody(required = false) String body,
                                         HttpServletRequest request) {
        String header = request.getHeader(Headers.OUT_SIGNATURE);
        boolean valid = signer.verify(secret, body == null ? "" : body, header, 300);
        record("strict", request, valid ? 200 : 401, valid);
    }

```

```

        if (!valid) {
            log.warn("İmza doğrulanamadi: {}", header);
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                .body("{\"error\":\"imza gecersiz\"}");
        }
        return ResponseEntity.ok("{\"ok\":true,\"signature\":\"dogrulandi\"}");
    }

    @GetMapping("/received")
    public List<Received> received(@RequestParam(defaultValue = "50") int limit) {
        return received.stream().limit(Math.min(limit, MEMORY)).toList();
    }

    @GetMapping("/counters")
    public Map<String, Long> counters() {
        Map<String, Long> out = new LinkedHashMap<>();
        counters.forEach((k, v) -> out.put(k, v.get()));
        return out;
    }

    @DeleteMapping("/received")
    public Map<String, String> clear() {
        received.clear();
        counters.clear();
        return Map.of("status", "temizlendi");
    }

    private void record(String behavior, HttpServletRequest request, int status, Boolean sigValid) {
        String deliveryId = request.getHeader(Headers.OUT_ID);
        String eventType = request.getHeader(Headers.OUT_EVENT_TYPE);
        int attempt = parseInt(request.getHeader(Headers.OUT_ATTEMPT));

        received.addFirst(new Received(behavior, deliveryId, eventType, attempt,
            sigValid != null && sigValid, status, Instant.now()));
        while (received.size() > MEMORY) received.pollLast();

        counters.computeIfAbsent(behavior + ":" + status, k -> new AtomicLong()).incrementAndGet();
        counters.computeIfAbsent("toplam", k -> new AtomicLong()).incrementAndGet();
    }

    private int parseInt(String s) {
        try {
            return s == null ? 0 : Integer.parseInt(s.trim());
        } catch (NumberFormatException e) {
            return 0;
        }
    }
}

```

10.3.4 `services/chaos-target/src/main/resources/application.yml`

Dört autoconfiguration kapalı: DataSource, JPA, Redis, Kafka. Bu servis `common`'ı bağımlılık olarak taşıyor ama hiçbirini kullanmıyor.

```

server:
  port: 8085

```

```

spring:
  application:
    name: chaos-target
  threads:
    virtual:
      enabled: true    # /sink/slow ile yuzlerce es zamanli yavas istek tutabilmek icin
  autoconfigure:
    exclude:
      - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
      - org.springframework.boot.autoconfigure.orm.jpa.HibernateJpaAutoConfiguration
      - org.springframework.boot.autoconfigure.data.redis.RedisAutoConfiguration
      - org.springframework.boot.autoconfigure.kafka.KafkaAutoConfiguration

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus
  metrics:
    tags:
      service: chaos-target

logging:
  level:
    dev.hookrelay: INFO

```

10.3.5 `services/gateway/pom.xml`

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/
    maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.hookrelay</groupId>
    <artifactId>hookrelay-parent</artifactId>
    <version>1.0.0</version>
    <relativePath>../../pom.xml</relativePath>
  </parent>
  <artifactId>gateway</artifactId>
  <name>gateway</name>
  <description>Tek giris kapişi</description>

  <dependencies>
    <dependency><groupId>org.springframework.cloud</groupId><artifactId>spring-cloud-starter
      -gateway</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-a
      ctuator</artifactId></dependency>
    <dependency><groupId>io.micrometer</groupId><artifactId>micrometer-registry-prometheus</
      artifactId></dependency>
  </dependencies>

  <build>
    <finalName>gateway</finalName>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>

```

```

    </plugin>
  </plugins>
</build>
</project>

```

10.3.6 `services/gateway/src/main/java/dev/hookrelay/gateway/GatewayApplication.java`

```

package dev.hookrelay.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * TEK GIRIS KAPISI.
 *
 * Disaridan gelen her istek buradan gecer, arkadaki 5 servise dagitilir.
 * Kazanci: istemci (arayuz, musteri, k6) tek bir adres bilir; servisler
 * bolunse, tasinsa, portu degisse istemci etkilenmez.
 *
 * Neden Eureka yok: Docker Compose'da servis adlari zaten DNS ile
 * cozuluyor (http://dispatcher:8083). Servis kesfi icin ayri bir
 * bilesen calistirmak, cozdugu problemden buyuk bir problem eklemek olur.
 * Kubernetes'e gecilirse Service kaynagi ayni isi gorur - yine Eureka
 * gerekmez. Eureka'nin yeri, dinamik olceklenen ve DNS'i olmayan
 * ortamlardir.
 */
@SpringBootApplication
public class GatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(GatewayApplication.class, args);
    }
}

```

10.3.7 `services/gateway/src/main/java/dev/hookrelay/gateway/RequestIdFilter.java`

Ordered.HIGHEST_PRECEDENCE — kimlik en başta takılmalı ki sonraki bütün filtreler ve arka servisler görsün.

```

package dev.hookrelay.gateway;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.core.Ordered;
import org.springframework.http.server.reactive.ServerHttpRequest;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

import java.util.UUID;

/**
 * Her isteğe bir kimlik takar ve arkaya iletir.
 *
 * Dagitik sistemde hata ayıklamanin ilk kurali: bir istegin 4 servisteki
 * izini surebilmek. Musteri "saat 14:32'de bir hata aldım" dediginde

```

```

* elinizde bir kimlik yoksa 4 servisin loglarini zaman damgasina gore
* karsilastirmaya calisirsiniz - ki bu, saatler suren bir istir.
*
* (Tam cozum OpenTelemetry ile dagitik izleme. Bu filtre onun basit
* hali: tek baslik, sifir bagimlilik, %80 fayda.)
*/
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {

    private static final Logger log = LoggerFactory.getLogger(RequestIdFilter.class);
    private static final String HEADER = "X-Request-Id";

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String requestId = exchange.getRequest().getHeaders().getFirst(HEADER);
        if (requestId == null || requestId.isBlank()) {
            requestId = UUID.randomUUID().toString();
        }

        ServerHttpRequest mutated = exchange.getRequest().mutate()
            .header(HEADER, requestId)
            .build();

        exchange.getResponse().getHeaders().set(HEADER, requestId);

        String finalId = requestId;
        return chain.filter(exchange.mutate().request(mutated).build())
            .doOnSuccess(v -> log.debug("{} {} → {}",
                mutated.getMethod(), mutated.getURI().getPath(), finalId));
    }

    @Override
    public int getOrder() {
        return Ordered.HIGHEST_PRECEDENCE;
    }
}

```

10.3.8 `services/gateway/src/main/java/dev/hookrelay/gateway/HttpClientDnsCon

Konteyner ortamındaki DNS tuzağının çözümü. Sekiz satır kod, kırk satır gerekçe — çünkü bu sınıf silinirse hata altı ay sonra geri gelir ve kimse sebebini hatırlamaz.

```

package dev.hookrelay.gateway;

import io.netty.resolver.DefaultAddressResolverGroup;
import org.springframework.cloud.gateway.config.HttpClientCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * Reactor Netty'yi JVM'in DNS cozumleyicisine yonlendirir.
 *
 * Yasanan hata: bir arka servis yeniden olusturulunca gateway ona giden
 * her istegi "Connection refused: dispatcher/172.23.0.10" ile reddediyordu
 * -- oysa konteynerin yeni adresi .11 idi ve gateway'in icinden wget
 * calisiyordu.
 *
 * Sebep: Reactor Netty kendi DNS onbellegini kullanir ve Docker'in gomulu
 * DNS'i konteyner adlari icin TTL 600 doner. Yani on dakika eski IP.

```



```

* JVM cozumleyicisi networkaddress.cache.ttl'e uyar; Dockerfile'da 10 sn.
*
* Ayni problem Kubernetes'te de var: pod yeniden planlaninca IP degisir.
*/
@Configuration
public class HttpClientDnsConfig {

    @Bean
    public HttpClientCustomizer jvmDnsResolverCustomizer() {
        return httpClient -> httpClient.resolver(DefaultAddressResolverGroup.INSTANCE);
    }
}

```

10.3.9 `services/gateway/src/main/resources/application.yml`

Rotaların **sırası önemli**: Spring ilk eşleşen rotayı kullanır, özelden genele dizilir.

`httpClient.pool` ayarları DNS düzeltmesinin tamamlayıcısı: ölü bağlantılar havuzda beklemesin.

```

server:
  port: 8080

spring:
  application:
    name: gateway

cloud:
  gateway:
    httpClient:
      connect-timeout: 3000
      response-timeout: 30s
    pool:
      # Bosta duran baglantilari tahliye et. DNS duzeltmesiyle birlikte,
      # yeniden olusturulan bir servise takilip kalmayi tamamen bitirir:
      # eski IP'ye acilmis olu baglantilar havuzda beklemesin.
      type: elastic
      max-idle-time: 15s
      max-life-time: 60s
      eviction-interval: 30s
      # Tarayici arayuzu nginx'ten geliyor ama gelistirirken
      # dogrudan gateway'e de vurulabilsin.
    globalcors:
      cors-configurations:
        '[/*]':
          allowedOriginPatterns: "*"
          allowedMethods: "*"
          allowedHeaders: "*"

      # Yollarin sirasi ONEMLI: Spring ilk eslesen rotayi kullanir.
      # Ozelden genele dizilir.
    routes:
      - id: admin
        uri: ${ADMIN_URL:http://localhost:8081}
        predicates:
          - Path=/api/admin/**

      - id: deliveries
        uri: ${DISPATCHER_URL:http://localhost:8083}
        predicates:

```

```
- Path=/api/deliveries/**

- id: scheduler
  uri: ${SCHEDULER_URL:http://localhost:8084}
  predicates:
    - Path=/api/scheduler/**

- id: ingest
  uri: ${INGEST_URL:http://localhost:8082}
  predicates:
    - Path=/v1/**

- id: chaos
  uri: ${CHAOS_URL:http://localhost:8085}
  predicates:
    - Path=/sink/**

management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,gateway
  endpoint:
    health:
      show-details: always
  metrics:
    tags:
      service: gateway

logging:
  level:
    dev.hookrelay: INFO
```

Bölüm 11 — Docker Compose ve Altyapı

11.1 Kafka — KRaft modu

```
kafka:
  image: apache/kafka:3.8.1
  environment:
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    CLUSTER_ID: hookrelay-local-cluster
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://:29092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,EXTERNAL://localhost:29092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
  ports: ["29092:29092"]
```

KRaft: ZooKeeper yok. Kafka 3.3'ten beri üretime hazır, 4.0'da ZooKeeper tamamen kaldırıldı. İnternette bulacağınız ZooKeeper'lı örnekleri görmezden gelin.

Üç dinleyici, üç farklı amaç:

Dinleyici	Kim kullanır	Adres
PLAINTEXT	Konteynerler arası	kafka:9092
EXTERNAL	Makineniz (IDE, k6)	localhost:29092
CONTROLLER	KRaft iç trafiği	kafka:9093

Tek dinleyici kullanıp `localhost:9092` verseydiniz konteynerler bağlanamazdı (onların `localhost` 'u kendileri). `kafka:9092` verseydiniz makinenizden bağlanamazdınız (o isim host'ta çözülmez). Bu, Kafka'yı Docker'da çalıştıranların en sık takıldığı yer.

11.2 Redis — `noeviction` kararı

```
redis:
  command: >
  redis-server --appendonly yes --maxmemory 512mb --maxmemory-policy noeviction
```

Varsayılan `allkeys-lru` olsaydı Redis bellek dolunca **en az kullanılan anahtarları silerdi**. Bizim durumumuzda o anahtarlar endpoint konfigürasyon replikası olabilirdi. `dispatcher` "endpoint bulunamadı" deyip teslimatları düşürürdü — ve hiçbir hata görmezsiniz.

`noeviction` ile Redis yazma reddeder. Gürültülü bir hata, sessiz veri kaybından iyidir.

Bu, "Redis = cache" varsayımının neden tehlikeli olduğunun somut örneği. Redis'i cache dışında bir şey için kullanıyorsanız, eviction politikasını bilinçli seçmek zorundasınız.

11.3 Sağlık kontrolleri — "başladı" ile "hazır" farkı

```
postgres:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U hookrelay -d postgres"]
    interval: 5s
    retries: 20

admin-api:
  depends_on:
    postgres: {condition: service_healthy}
```

`depends_on` tek başına yalnızca **başlatma sırasını** belirler. Konteyner "başladı" der ama Postgres henüz bağlantı kabul etmiyordur ve servis çöker.

`condition: service_healthy` ile Compose sağlık kontrolünün geçmesini bekler. `pg_isready` olmadan `depends_on` işe yaramaz.

11.4 Dockerfile — katman önbelleği

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build

# ÖNCE sadece pom dosyaları
COPY pom.xml .
COPY libs/contracts/pom.xml libs/contracts/pom.xml
COPY libs/common/pom.xml libs/common/pom.xml
# ... diğer pom'lar
RUN --mount=type=cache,target=/root/.m2 mvn -B -q dependency:go-offline || true

# SONRA kaynak kod
COPY libs libs
COPY services services
RUN --mount=type=cache,target=/root/.m2 mvn -B -q package -DskipTests
```

Docker katmanları sırayla önbelleklenir. Kaynak kod pom'dan önce kopyalansaydı, tek satırlık bir kod değişikliği bütün bağımlılıkları yeniden indirtirdi. Bu sırayla: pom değişmedikçe indirme katmanı önbellekten gelir ve derleme 3 dakikadan 20 saniyeye düşer.

`--mount=type=cache` (BuildKit) ise `~/ .m2`'yi katmanlar arasında da korur.

Çok aşamalı yapı ve root olmayan kullanıcı

```
FROM eclipse-temurin:21-jre-jammy
ARG SERVICE
RUN useradd --system --uid 10001 --create-home hookrelay
USER hookrelay
COPY --from=build --chown=hookrelay:hookrelay \
  /build/services/${SERVICE}/target/${SERVICE}.jar /app/app.jar
```

```
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseG1GC"
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
```

Derleme için Maven imajı (~700 MB), çalıştırma için sadece JRE (~250 MB). Üretim imajında ne Maven var, ne kaynak kod, ne `~/m2`.

`MaxRAMPercentage` neden sabit `-Xmx` değil: JVM'e "konteyner limitinin %75'ini kullan" der. Sabit `-Xmx512m` yazsaydınız ve compose'da limiti 2 GB'a çıkarsaydınız JVM'in haberi olmazdı — ya belleği boşa harcardınız ya `OOMKilled` alırdınız.

`exec` neden: `sh -c "java ..."` yazarsanız PID 1 shell olur ve `SIGTERM` Java'ya ulaşmaz; konteyner 10 saniye sonra zorla öldürülür ve düzgün kapanma (graceful shutdown) çalışmaz. `exec` shell'i Java ile **değiştirir**.

11.5 Üç veritabanı, tek sunucu

```
CREATE DATABASE hookrelay_control;
CREATE DATABASE hookrelay_ingest;
CREATE DATABASE hookrelay_delivery;
```

İzolasyon **şema düzeyinde gerçek**: hiçbir servis diğerinin tablosunu göremez. Üç ayrı konteyner ise yerel geliştirmede 600 MB fazladan RAM.

Üretimde ayrı sunuculara taşımak, sadece bir bağlantı dizgisi değişikliği. Feda edilen: kaynak izolasyonu — bir servisin ağır sorgusu diğerlerini yavaşlatabilir.

11.6 nginx — CORS'u ortadan kaldırmak

```
location /api/ { proxy_pass http://gateway:8080; }
location /v1/ { proxy_pass http://gateway:8080; }
location /     { try_files $uri $uri/ /index.html; }
```

Arayüz ve API **aynı kökenden** servis ediliyor. Tarayıcı açısından her şey `http://localhost:3000` — bu yüzden CORS diye bir problem **yok**. Preflight isteği yok, `Access-Control-*` başlıkları yok, "neden çalışmıyor" yok.

Ayrı portlardan servis edip CORS'la uğraşmaktansa tek kökende toplamak her zaman daha basit.

11.7 Dosyaların tamamı

11.7.1 `Dockerfile`

Tek dosya, altı servis. `ARG SERVICE` ile hangi jar'ın kopyalanacağı seçiliyor.

```
# syntax=docker/dockerfile:1.7
# -----
# Tek Dockerfile, altı servis.
#
# Neden tek dosya: bütün servisler aynı Maven reaktorundan cikiyor ve
# aynı JRE'de çalışıyor. Altı ayrı Dockerfile tutmak, altı kez aynı
# şeyi güncellemek demektir.
#
```

```

# Çok asamali (multi-stage) yapı: derleme için Maven imajı (yaklaşık
# 700 MB), çalıştırma için sadece JRE (yaklaşık 250 MB). Üretim imajında
# ne Maven var ne kaynak kod ne de ~/.m2.
# -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build

# ONCE sadece pom dosyaları, sonra bağımlılıklar.
#
# Docker katmanları sırayla onbelleklenir. Kaynak kod pom'dan önce
# kopyalansaydı, tek satırlık bir kod değişikliği bütün bağımlılıkları
# yeniden indirtirdi. Bu sırayla: pom değişmedikçe indirme katmanı
# onbelleden gelir ve derleme 3 dakikadan 20 saniyeye düşer.
COPY pom.xml .
COPY libs/contracts/pom.xml      libs/contracts/pom.xml
COPY libs/common/pom.xml        libs/common/pom.xml
COPY libs/outbox/pom.xml        libs/outbox/pom.xml
COPY services/gateway/pom.xml    services/gateway/pom.xml
COPY services/admin-api/pom.xml  services/admin-api/pom.xml
COPY services/ingest-api/pom.xml services/ingest-api/pom.xml
COPY services/dispatcher/pom.xml services/dispatcher/pom.xml
COPY services/retry-scheduler/pom.xml services/retry-scheduler/pom.xml
COPY services/chaos-target/pom.xml services/chaos-target/pom.xml
RUN --mount=type=cache,target=/root/.m2 \
    mvn -B -q dependency:go-offline -DskipTests || true

COPY libs libs
COPY services services
# BuildKit onbellek bağlantısı: ~/.m2 katmanları arasında korunur.
# İlk derleme 3 dakika, sonrakiler 30 saniye.
RUN --mount=type=cache,target=/root/.m2 \
    mvn -B -q package -DskipTests

# -----
FROM eclipse-temurin:21-jre-jammy
ARG SERVICE
WORKDIR /app

# root olarak çalıştırmıyoruz. Konteynerden kaçış açığı bulunursa
# saldırgan root değil, hiçbir şeye yetkisi olmayan bir kullanıcı olur.
RUN useradd --system --uid 10001 --create-home hookrelay
USER hookrelay

COPY --from=build --chown=hookrelay:hookrelay /build/services/${SERVICE}/target/${SERVICE}.jar /app/app.jar

# MaxRAMPercentage: JVM'e "konteyner limitinin %75'ini kullan" der.
# Sabit -Xmx yazmak, compose'da limiti degistirdiginizde JVM'in
# haberi olmaması demek – ve OOMKilled.
# networkaddress.cache.ttl: JVM'in DNS onbellegi. Varsayılan 30 saniye,
# ama konteyner ortamında bir servis yeniden oluşunca 30 saniye kor
# kalmak bile fazla. 10 saniyeye çekiyoruz.
# (Bkz. gateway/HttpClientDnsConfig – Reactor Netty'yi JVM çözümleyicisine
# yönlendirmeden bu ayarın gateway'e hiçbir etkisi olmaz.)
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseG1GC \
    -Dnetworkaddress.cache.ttl=10 -Dnetworkaddress.cache.negative.ttl=5 \
    -Djava.security.egd=file:/dev/./urandom"

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]

```

11.7.2 `docker-compose.yml`

`x-service-base` YAML çapası ile ortak yapılandırma bir kez yazılıyor. Servisler `<<: *service-base` ile devralıp yalnızca farklarını belirtiyor.

`depends_on: {condition: service_healthy}` — `admin-api` topic'leri oluşturduğu için diğerleri onu bekliyor.

```
# =====
# HookRelay — tam yigin
#
#   docker compose up -d --build
#   → http://localhost:3000   arayuz
#   → http://localhost:8090   Kafka UI
#   → http://localhost:3001   Grafana (admin / admin)
# =====

name: hookrelay

x-service-base: &service-base
  build:
    context: .
    dockerfile: Dockerfile
  restart: unless-stopped
  networks: [hookrelay]
  environment: &common-env
    DB_USER: hookrelay
    DB_PASSWORD: hookrelay
    REDIS_HOST: redis
    REDIS_PORT: 6379
    KAFKA_BOOTSTRAP: kafka:9092
    TZ: Europe/Istanbul

services:

# -----
# Altyapi
# -----
postgres:
  image: postgres:16-alpine
  container_name: hr-postgres
  environment:
    POSTGRES_USER: hookrelay
    POSTGRES_PASSWORD: hookrelay
    POSTGRES_DB: postgres
  ports: ["5433:5432"]
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./ops/sql/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  healthcheck:
    # pg_isready olmadan depends_on ise yaramaz: konteyner "basladi"
    # der ama Postgres henuz baglanti kabul etmiyordur ve servisler
    # coker. Saglik kontrolu, "basladi" ile "hazir" arasindaki farktir.
    test: ["CMD-SHELL", "pg_isready -U hookrelay -d postgres"]
    interval: 5s
    timeout: 3s
    retries: 20
    networks: [hookrelay]

redis:
```

```

image: redis:7-alpine
container_name: hr-redis
command: >
  redis-server
  --appendonly yes
  --maxmemory 512mb
  --maxmemory-policy noeviction
ports: ["6380:6379"]
volumes: [redisdata:/data]
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 5s
  timeout: 3s
  retries: 20
networks: [hookrelay]

```

```

# maxmemory-policy NEDEN noeviction:
# Redis burada sadece onbellek degil – endpoint konfigurasyon REPLIKASI
# ve idempotency deposu. allkeys-lru olsaydi Redis bellek dolunca
# endpoint konfigurasyonlarini SESSIZCE silerdi ve dispatcher
# "endpoint bulunamadi" deyip teslimatlari dusururdu. noeviction ile
# Redis yazma reddeder – gurultulu bir hata, sessiz veri kaybindan iyidir.

```

```

kafka:
  image: apache/kafka:3.8.1
  container_name: hr-kafka
  ports: ["29092:29092"]
  environment:
    # KRaft modu: ZooKeeper YOK. Kafka 3.3'ten beri uretime hazir,
    # 4.0'da ZooKeeper tamamen kaldırıldı. Yeni proje kuruyorsanız
    # ZooKeeper'li örnekleri gormezden gelin.
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    CLUSTER_ID: hookrelay-local-cluster
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    # Uc dinleyici:
    # PLAINTEXT → konteynerler arasi (kafka:9092)
    # EXTERNAL → makinenizden (localhost:29092), IDE'den baglanmak icin
    # CONTROLLER → KRaft ic trafiği
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://:29092
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,EXTERNAL://localhost:29092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
    # Tek dugumde rebalance gecikmesini beklemenin anlami yok.
    KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
    KAFKA_LOG_RETENTION_HOURS: 168
  volumes: [kafkadata:/var/lib/kafka/data]
  healthcheck:
    test: ["CMD-SHELL", "/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092 >/dev/null 2>&1"]
    interval: 10s
    timeout: 10s
    retries: 20
    start_period: 30s
  networks: [hookrelay]

```



```

# AUTO_CREATE_TOPICS_ENABLE=false NEDEN ONEMLI:
# Acik olsaydi, adini yanlis yazdiginiz bir topic sessizce olur ve
# mesajlariniz kimsenin dinlemedigi bir yere akardi. Kapali olunca
# yanlis isim aninda hata verir. Topic'ler admin-api'deki
# KafkaTopicsConfig tarafından bilincli olarak olusturulur.

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: hr-kafka-ui
  ports: ["8090:8080"]
  environment:
    KAFKA_CLUSTERS_0_NAME: hookrelay
    KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092
    DYNAMIC_CONFIG_ENABLED: "true"
  depends_on:
    kafka: {condition: service_healthy}
  networks: [hookrelay]

# -----
# Uygulama servisleri
# -----

admin-api:
  <<: *service-base
  build: {context: ., dockerfile: Dockerfile, args: {SERVICE: admin-api}}
  container_name: hr-admin-api
  ports: ["8081:8081"]
  environment:
    <<: *common-env
    DB_URL: jdbc:postgresql://postgres:5432/hookrelay_control
  depends_on:
    postgres: {condition: service_healthy}
    redis: {condition: service_healthy}
    kafka: {condition: service_healthy}
  healthcheck:
    test: ["CMD-SHELL", "wget -q0- http://localhost:8081/actuator/health | grep -q UP"]
    interval: 10s
    timeout: 5s
    retries: 12
    start_period: 40s

ingest-api:
  <<: *service-base
  build: {context: ., dockerfile: Dockerfile, args: {SERVICE: ingest-api}}
  container_name: hr-ingest-api
  ports: ["8082:8082"]
  environment:
    <<: *common-env
    DB_URL: jdbc:postgresql://postgres:5432/hookrelay_ingest
  depends_on:
    postgres: {condition: service_healthy}
    redis: {condition: service_healthy}
    kafka: {condition: service_healthy}
  # admin-api ONCE ayaga kalkmali: topic'leri o olusturuyor.
  admin-api: {condition: service_healthy}

dispatcher:
  <<: *service-base
  build: {context: ., dockerfile: Dockerfile, args: {SERVICE: dispatcher}}
  container_name: hr-dispatcher
  ports: ["8083:8083"]

```

```

environment:
  <<: *common-env
  DB_URL: jdbc:postgresql://postgres:5432/hookrelay_delivery
  # DEMO GECIKMELERI. Uretim degerleri 10s,1m,5m,30m,2h idi -
  # demoyu izlemek 2.5 saat surerdi. Katman SAYISI ayni, sadece
  # sureler kisaltildi; topic adlari degismedi (tasarim geregi).
  HOOKRELAY_RETRY_DELAYS: 5s,15s,30s,60s,120s
  HOOKRELAY_RETRY_MAX_LIFETIME: 30m
  HOOKRELAY_CIRCUIT_BREAKER_OPEN_MILLIS: 20000
  HOOKRELAY_CIRCUIT_BREAKER_FAILURE_THRESHOLD: 5
depends_on:
  postgres: {condition: service_healthy}
  redis: {condition: service_healthy}
  kafka: {condition: service_healthy}
  admin-api: {condition: service_healthy}

retry-scheduler:
  <<: *service-base
  build: {context: ., dockerfile: Dockerfile, args: {SERVICE: retry-scheduler}}
  container_name: hr-retry-scheduler
  ports: ["8084:8084"]
  depends_on:
    redis: {condition: service_healthy}
    kafka: {condition: service_healthy}
    admin-api: {condition: service_healthy}

chaos-target:
  <<: *service-base
  build: {context: ., dockerfile: Dockerfile, args: {SERVICE: chaos-target}}
  container_name: hr-chaos-target
  ports: ["8085:8085"]

gateway:
  <<: *service-base
  build: {context: ., dockerfile: Dockerfile, args: {SERVICE: gateway}}
  container_name: hr-gateway
  ports: ["8080:8080"]
  environment:
    <<: *common-env
    ADMIN_URL: http://admin-api:8081
    INGEST_URL: http://ingest-api:8082
    DISPATCHER_URL: http://dispatcher:8083
    SCHEDULER_URL: http://retry-scheduler:8084
    CHAOS_URL: http://chaos-target:8085
  depends_on:
    admin-api: {condition: service_healthy}

# -----
# Arayuz ve gozlemlenebilirlik
# -----
ui:
  image: nginx:1.27-alpine
  container_name: hr-ui
  ports: ["3000:80"]
  volumes:
    - ./ui:/usr/share/nginx/html:ro
    - ./ops/nginx.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on: [gateway]
  networks: [hookrelay]

prometheus:

```

```

    image: prom/prometheus:v2.54.1
    container_name: hr-prometheus
    ports: ["9090:9090"]
    volumes:
      - ./ops/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    networks: [hookrelay]

grafana:
  image: grafana/grafana:11.2.0
  container_name: hr-grafana
  ports: ["3001:3000"]
  environment:
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: admin
    GF_USERS_ALLOW_SIGN_UP: "false"
    GF_AUTH_ANONYMOUS_ENABLED: "true"
    GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
  volumes:
    - ./ops/grafana/provisioning:/etc/grafana/provisioning:ro
    - ./ops/grafana/dashboards:/var/lib/grafana/dashboards:ro
  depends_on: [prometheus]
  networks: [hookrelay]

volumes:
  pgdata:
  redisdata:
  kafkadata:

networks:
  hookrelay:
    driver: bridge

```

11.7.3 `ops/sql/init.sql`

```

-- HER SERVIS AYRI VERITABANI.
--
-- Mikroservisin en sik atlanan ve en onemli kurali: iki servis ayni
-- tabloya yazamaz. Yazarsa, semayi degistirmek icin iki takimin
-- anlasmasi gerekir ve elinizde "dagitik monolit" olur – mikroservisin
-- butun maliyeti, hicbir faydasi.
--
-- Burada uc AYRI VERITABANI var ama TEK Postgres sunucusunda. Bu bir
-- taviz ve bilincli: yerel gelistirmede uc konteyner calistirmek
-- 600 MB fazladan RAM demek. Onemli olan izolasyonun SEMA duzeyinde
-- gercek olması – hicbir servis digerinin tablosunu goremiyor.
-- Uretimde ayri sunuculara tasimak, sadece bir baglanti dizgisi degisikligi.

CREATE DATABASE hookrelay_control;
CREATE DATABASE hookrelay_ingest;
CREATE DATABASE hookrelay_delivery;

GRANT ALL PRIVILEGES ON DATABASE hookrelay_control TO hookrelay;
GRANT ALL PRIVILEGES ON DATABASE hookrelay_ingest TO hookrelay;
GRANT ALL PRIVILEGES ON DATABASE hookrelay_delivery TO hookrelay;

```

11.7.4 `ops/nginx.conf`

```

server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # Arayuz ve API AYNI KOKENDEN servis ediliyor.
    #
    # Tarayıcı açısından her şey http://localhost:3000 — bu yüzden
    # CORS diye bir problem YOK. Preflight istegi yok, Access-Control
    # başlıkları yok, "neden çalışmıyor" yok. Ayri portlardan servis
    # edip CORS'la ugrasmaktansa tek kökende toplamak her zaman daha
    # basit.
    location /api/ {
        proxy_pass http://gateway:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    location /v1/ {
        proxy_pass http://gateway:8080;
        proxy_set_header Host $host;
        proxy_read_timeout 60s;
    }

    location /sink/ {
        proxy_pass http://gateway:8080;
        proxy_set_header Host $host;
        proxy_read_timeout 130s; # /sink/slow 120 saniyeye kadar bekletebilir
    }

    location / {
        try_files $uri $uri/ /index.html;
        add_header Cache-Control "no-store";
    }
}

```

11.7.5 `ops/prometheus/prometheus.yml`

`relabel_configs` ile `instance` etiketi `admin-api:8081` yerine `admin-api` oluyor. Okunaklılık dışında pratik faydası: port değişirse geçmiş veriyle sürekliliği kopmaz.

```

global:
    scrape_interval: 5s
    evaluation_interval: 15s

scrape_configs:
    - job_name: hookrelay
      metrics_path: /actuator/prometheus
      static_configs:
        - targets:
            - admin-api:8081
            - ingest-api:8082
            - dispatcher:8083

```

```

- retry-scheduler:8084
- chaos-target:8085
- gateway:8080
relabel_configs:
  # instance etiketini "admin-api:8081" yerine "admin-api" yap.
  # Grafana'da okunaklı olması dışında pratik bir faydası da var:
  # port değişirse geçmiş veriyle sürekliliği kopmaz.
  - source_labels: [__address__]
    regex: '([^:]+):.*'
    target_label: instance
    replacement: '$1'

```

11.7.6 `.dockerignore`

```

**/target
.git
.idea
*.iml
docs
ops/grafana/data
ui/node_modules

```

11.7.7 `.gitignore`

```

target/
.idea/
*.iml
.DS_Store
ops/grafana/data/
*.log

```

Kontrol noktası

```

docker compose up -d --build
docker compose ps

```

On üç konteyner **Up** olmalı, kritik olanlar **(healthy)**.

Bölüm 12 — Arayüz

Tek bir `index.html`. Derleme adımı yok, `node_modules` yok, framework yok.

Sebebi: bu bir frontend projesi değil. Arayüzün tek görevi, arkadaki mekanizmayı **görünür kılmak**. React kurmak projeye 200 MB bağımlılık ve bir derleme adımı eklerdi; karşılığında hiçbir şey kazandırmazdı.

Panelin gösterdikleri:

Bölüm	Ne gösterir	Neden önemli
Üst şerit	Başarılı / bekleyen / tükenmiş sayıları	Sistemin nabızı
Hızlı demo	Tek tıkla 1 uygulama + 4 endpoint	Girişi sıfırlar
Endpoint'ler	Sağlık, başarı oranı, devre durumu	Devre kesici canlı görülür
Teslimat günlüğü	Her teslimatın durumu, denemeleri	"Webhook gelmedi" cevabı
Zamanlayıcı	Duraklatılmış katmanlar, geri sayım	pause/seek gözle görülür
Chaos hedefi	Gelen istekler, dönen kodlar	Karşı taraftan doğrulama

Zamanlayıcı kartı özellikle değerli. Dokuzuncu bölümdeki `pause/seek` mekanizması soyut bir anlatım olmaktan çıkıyor: `t3` kutusunun sarıya dönüp "18 sn" yazması, o partition'ın gerçekten duraklatıldığını gösteriyor.

```
setInterval(() => {
  loadStats(); loadDeliveries(); loadScheduler(); loadChaos();
}, 2000);
```

İki saniyede bir yoklama. WebSocket daha zarif olurdu ama bir yönetim paneli için gereksiz karmaşıklık — ve bu rehberin konusu o değil.

12.1 Panelin tamamı

Tek dosya, derleme adımı yok. Uzun ama karmaşık değil: durum bir nesnede (`state`), her bölüm için bir `load*` fonksiyonu, iki `setInterval` ile yenileme.

Okurken şu üç fonksiyona bakın; gerisi görüntüleme:

- `quickSetup()` — tek tıkla uygulama + dört endpoint kurar
- `loadEndpoints()` — sağlık ve devre durumunu birleştirip renklendirir
- `loadScheduler()` — duraklatılmış katmanları sarıya boyar

12.1.1 `ui/index.html`

```
<!doctype html>
<html lang="tr">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>HookRelay – Kontrol Paneli</title>
```

```

<style>
:root{
  --bg:#0b0f14; --panel:#121822; --panel2:#0f151d; --line:#1f2836;
  --text:#e6edf6; --muted:#8b9bb0; --accent:#39d3a0; --accent2:#4aa8ff;
  --warn:#f5a524; --danger:#f2555a; --mono:ui-monospace,"SF Mono",Menlo,Consolas,monospace
}
*{box-sizing:border-box}
body{margin:0;background:var(--bg);color:var(--text);
  font:14px/1.5 -apple-system,BlinkMacSystemFont,"Segoe UI",Roboto,sans-serif}
a{color:var(--accent2);text-decoration:none}

header{display:flex;align-items:center;gap:20px;padding:14px 22px;
  border-bottom:1px solid var(--line);background:var(--panel2);
  position:sticky;top:0;z-index:10;flex-wrap:wrap}
.logo{font-weight:700;font-size:17px;letter-spacing:-.3px}
.logo span{color:var(--accent)}
.stats{display:flex;gap:22px;margin-left:auto;flex-wrap:wrap}
.stat{text-align:right}
.stat b{display:block;font-size:19px;font-variant-numeric:tabular-nums}
.stat small{color:var(--muted);font-size:11px;text-transform:uppercase;letter-spacing:.6px}

main{display:grid;grid-template-columns:400px 1fr;gap:16px;padding:16px;
  align-items:start}
@media(max-width:1100px){main{grid-template-columns:1fr}}

.card{background:var(--panel);border:1px solid var(--line);border-radius:10px;
  margin-bottom:16px;overflow:hidden}
.card h2{margin:0;padding:12px 16px;font-size:13px;text-transform:uppercase;
  letter-spacing:.7px;color:var(--muted);border-bottom:1px solid var(--line);
  display:flex;align-items:center;gap:8px}
.card h2 .right{margin-left:auto;text-transform:none;letter-spacing:0}
.card .body{padding:14px 16px}

button{background:var(--accent);color:#062018;border:0;border-radius:7px;
  padding:8px 14px;font-weight:600;cursor:pointer;font-size:13px}
button:hover{filter:brightness(1.1)}
button.ghost{background:transparent;color:var(--text);border:1px solid var(--line)}
button.ghost:hover{border-color:var(--accent);color:var(--accent)}
button.small{padding:4px 9px;font-size:12px}
button.disabled{opacity:.5;cursor:not-allowed}

input,select{background:var(--panel2);border:1px solid var(--line);color:var(--text);
  border-radius:7px;padding:8px 10px;font-size:13px;width:100%;font-family:inherit}
input:focus,select:focus{outline:0;border-color:var(--accent2)}
label{display:block;font-size:11px;color:var(--muted);margin:10px 0 4px;
  text-transform:uppercase;letter-spacing:.5px}
.row{display:flex;gap:8px}
.row>*{flex:1}

table{width:100%;border-collapse:collapse;font-size:13px}
th{text-align:left;font-size:11px;color:var(--muted);text-transform:uppercase;
  letter-spacing:.5px;padding:8px 12px;border-bottom:1px solid var(--line);
  position:sticky;top:0;background:var(--panel)}
td{padding:8px 12px;border-bottom:1px solid var(--panel2)}
tr:hover td{background:var(--panel2)}

.pill{display:inline-block;padding:2px 8px;border-radius:20px;font-size:11px;
  font-weight:600;letter-spacing:.3px}

```

```

.pill.ok{background:rgba(57,211,160,.16);color:var(--accent)}
.pill.pending{background:rgba(74,168,255,.16);color:var(--accent2)}
.pill.bad{background:rgba(242,85,90,.16);color:var(--danger)}
.pill.warn{background:rgba(245,165,36,.16);color:var(--warn)}
.pill.mute{background:rgba(139,155,176,.14);color:var(--muted)}

.dot{width:8px;height:8px;border-radius:50%;display:inline-block;margin-right:6px}
.dot.ok{background:var(--accent)} .dot.bad{background:var(--danger)}
.dot.warn{background:var(--warn)} .dot.mute{background:var(--muted)}

code,.mono{font-family:var(--mono);font-size:12px}
.muted{color:var(--muted)}
.ep{padding:10px 0;border-bottom:1px solid var(--panel2)}
.ep:last-child{border-bottom:0}
.ep .url{font-family:var(--mono);font-size:11.5px;word-break:break-all;color:var(--muted)}
.ep .top{display:flex;align-items:center;gap:8px;margin-bottom:3px}
.ep .top b{font-size:13px}

.feed{max-height:520px;overflow-y:auto}
.scroll{max-height:300px;overflow-y:auto}
.empty{padding:26px;text-align:center;color:var(--muted);font-size:13px}

.key{background:#1a2430;border:1px dashed var(--accent);border-radius:7px;
padding:9px 11px;font-family:var(--mono);font-size:11.5px;
word-break:break-all;margin-top:8px}
.toast{position:fixed;right:18px;bottom:18px;background:var(--panel);
border:1px solid var(--accent);border-radius:9px;padding:12px 16px;
max-width:380px;box-shadow:0 12px 30px rgba(0,0,0,.5);z-index:100;font-size:13px}
.toast.err{border-color:var(--danger)}

details.detail{background:var(--panel2);border-radius:7px;margin-top:8px;padding:8px 11px}
details.detail summary{cursor:pointer;font-size:12px;color:var(--muted)}
pre{margin:8px 0 0;font-family:var(--mono);font-size:11px;white-space:pre-wrap;
word-break:break-all;color:#c8d6e8;max-height:220px;overflow:auto}
.tiers{display:flex;gap:6px;flex-wrap:wrap}
.tier{background:var(--panel2);border:1px solid var(--line);border-radius:6px;
padding:6px 10px;font-family:var(--mono);font-size:11px}
.tier.paused{border-color:var(--warn);color:var(--warn)}
</style>
</head>
<body>

<header>
  <div class="logo">Hook<span>Relay</span></div>
  <div class="muted" style="font-size:12px">Güvenilir webhook teslimat servisi</div>
  <div class="stats" id="stats"></div>
</header>

<main>
  <!-- SOL SUTUN -->
  <div>
    <div class="card">
      <h2>Hızlı demo</h2>
      <div class="body">
        <p class="muted" style="margin:0 0 10px;font-size:12.5px">
          Tek tıkla bir uygulama ve dört endpoint kurar:
          sağlam, %50 hatalı, tamamen olu ve 410 denen.
        </p>
        <div class="row">
          <button onclick="quickSetup()" id="btnSetup">Demoyu kur</button>
          <button class="ghost" onclick="sendBurst()" id="btnBurst">50 olay gönder</button>
        </div>
      </div>
    </div>
  </div>

```



```

    </div>
    <div id="keyBox"></div>
  </div>
</div>

<div class="card">
  <h2>Uygulamalar <span class="right"><button class="small ghost" onclick="newApp()">+ Yeni</button></span></h2>
  <div class="body" id="apps"><div class="empty">Yukleniyor...</div></div>
</div>

<div class="card">
  <h2>Endpoint'ler <span class="right"><button class="small ghost" onclick="newEndpoint()">+ Yeni</button></span></h2>
  <div class="body scroll" id="endpoints"><div class="empty">Önce bir uygulama seçin</div></div>
</div>

<div class="card">
  <h2>Yeniden deneme zamanlayicisi</h2>
  <div class="body" id="scheduler"><div class="empty">...</div></div>
</div>

<!-- SAG SUTUN -->
<div>
  <div class="card">
    <h2>
      Teslimat gunlugu
      <span class="right">
        <select id="filterStatus" onchange="loadDeliveries()" style="width:auto;display:inline-block">
          <option value="">hepsi</option>
          <option value="PENDING">bekleyen</option>
          <option value="SUCCEEDED">basarili</option>
          <option value="EXHAUSTED">tukenmis</option>
          <option value="DISCARDED">dusurulmus</option>
        </select>
        <button class="small ghost" onclick="replayExhausted()">Tukenmisleri tekrar gonder
        </button>
      </span>
    </h2>
    <div class="feed">
      <table>
        <thead><tr>
          <th>Durum</th><th>Olay</th><th>Endpoint</th><th>Deneme</th>
          <th>Son</th><th>Guncelleme</th><th></th>
        </tr></thead>
        <tbody id="deliveries"><tr><td colspan="7" class="empty">Yukleniyor...</td></tr></tbody>
      </table>
    </div>
  </div>

  <div class="card">
    <h2>Chaos hedefi – gelen istekler <span class="right">
      <button class="small ghost" onclick="clearChaos()">Temizle</button></span></h2>
    <div class="body scroll" id="chaos"><div class="empty">Henüz istek gelmedi</div></div>
  </div>
</div>
</main>

```

```

<script>
const api = (p, o) => fetch(p, o).then(async r => {
  const text = await r.text();
  const body = text ? JSON.parse(text) : null;
  if (!r.ok) throw new Error(body?.message || body?.error || r.status);
  return body;
});

let state = {
  appId: localStorage.getItem('hr_appId') || null,
  apiKey: localStorage.getItem('hr_apiKey') || null,
  endpoints: [], health: {},
};

function toast(msg, isErr) {
  const el = document.createElement('div');
  el.className = 'toast' + (isErr ? ' err' : '');
  el.textContent = msg;
  document.body.appendChild(el);
  setTimeout(() => el.remove(), 4200);
}

const esc = s => String(s ?? '').replace(/&lt;"/g, c =>
  ({'&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;'}[c]));
const shortId = id => id ? String(id).slice(0, 8) : '-';
const ago = ts => {
  if (!ts) return '-';
  const d = Math.floor((Date.now() - new Date(ts)) / 1000);
  if (d < 60) return d + ' sn';
  if (d < 3600) return Math.floor(d / 60) + ' dk';
  return Math.floor(d / 3600) + ' sa';
};

// ----- demo
async function quickSetup() {
  const btn = document.getElementById('btnSetup');
  btn.disabled = true;
  try {
    const name = 'demo-' + Math.random().toString(36).slice(2, 7);
    const created = await api('/api/admin/applications', {
      method: 'POST', headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({name}),
    });
    state.appId = created.application.id;
    state.apiKey = created.apiKey;
    localStorage.setItem('hr_appId', state.appId);
    localStorage.setItem('hr_apiKey', state.apiKey);

    const targets = [
      ['http://chaos-target:8085/sink/ok', 'Saglam – her zaman 200', 0],
      ['http://chaos-target:8085/sink/flaky?rate=50', '%50 hatali – yeniden denemeyi gorun', 0],
      ['http://chaos-target:8085/sink/dead', 'Olu – devre kesiciyi acar', 0],
      ['http://chaos-target:8085/sink/gone', '410 Gone – kalici hata, direkt DLQ', 0],
    ];
    for (const [url, description, rl] of targets) {
      await api(`/api/admin/applications/${state.appId}/endpoints`, {
        method: 'POST', headers: {'Content-Type': 'application/json'},

```

```

        body: JSON.stringify({url, description, eventTypes: ['*'], rateLimitPerSecond: rl}),
    });
}
document.getElementById('keyBox').innerHTML =
    `<div class="key"><b>API anahtari</b><br>${esc(state.apiKey)}</div>`;
toast('Demo kuruldu: 1 uygulama, 4 endpoint. Simdi "50 olay gonder"e basin.');
```

```

    refreshAll();
} catch (e) { toast('Hata: ' + e.message, true); }
finally { btn.disabled = false; }
}

async function sendBurst() {
    if (!state.apiKey) return toast('Once demoyu kurun (API anahtari lazim)', true);
    const btn = document.getElementById('btnBurst');
    btn.disabled = true;
    const types = ['order.created', 'order.paid', 'user.updated', 'invoice.finalized'];
    let sent = 0;
    try {
        await Promise.all(Array.from({length: 50}, (_, i) =>
            api('/v1/events', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': 'Bearer ' + state.apiKey,
                    'Idempotency-Key': 'ui-' + Date.now() + '-' + i,
                },
                body: JSON.stringify({
                    eventType: types[i % types.length],
                    payload: {no: i, tutar: (Math.random()*1000).toFixed(2), birim: 'TRY'},
                }),
            }).then(() => sent++).catch(() => {}))
        ));
        toast(`${sent} olay kabul edildi. 4 endpoint'e ${sent*4} teslimat uretildi.`);
        setTimeout(refreshAll, 600);
    } finally { btn.disabled = false; }
}

// ----- uygulamalar
async function newApp() {
    const name = prompt('Uygulama adi:');
    if (!name) return;
    try {
        const c = await api('/api/admin/applications', {
            method: 'POST', headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({name}),
        });
        state.appId = c.application.id; state.apiKey = c.apiKey;
        localStorage.setItem('hr_appId', state.appId);
        localStorage.setItem('hr_apiKey', state.apiKey);
        document.getElementById('keyBox').innerHTML =
            `<div class="key"><b>API anahtari (bir daha gosterilmez)</b><br>${esc(c.apiKey)}</div>`;
        refreshAll();
    } catch (e) { toast('Hata: ' + e.message, true); }
}

async function loadApps() {
    const apps = await api('/api/admin/applications');
    const box = document.getElementById('apps');
    if (!apps.length) { box.innerHTML = '<div class="empty">Henuz uygulama yok</div>'; return;
    }
}

```

```

if (!state.appId || !apps.find(a => a.id === state.appId)) {
  state.appId = apps[0].id;
  localStorage.setItem('hr_appId', state.appId);
}
box.innerHTML = apps.map(a => `
  <div class="ep">
    <div class="top">
      <span class="dot ${a.enabled ? 'ok' : 'mute'}"></span>
      <b>${esc(a.name)}</b>
      ${a.id === state.appId ? '<span class="pill ok">secili</span>' : ''}
      <span class="right" style="margin-left:auto">
        ${a.id === state.appId ? '' :
          '<button class="small ghost" onclick="selectApp('${a.id}')">Sec</button>`}
      </span>
    </div>
    <div class="url">${esc(a.apiKeyPreview)} · v${a.version}</div>
  </div>`).join('');
}

function selectApp(id) {
  state.appId = id;
  localStorage.setItem('hr_appId', id);
  refreshAll();
}

// ----- endpointler
async function newEndpoint() {
  if (!state.appId) return toast('Once bir uygulama secin', true);
  const url = prompt('Endpoint URL:', 'http://chaos-target:8085/sink/flaky?rate=30');
  if (!url) return;
  try {
    await api(`/api/admin/applications/${state.appId}/endpoints`, {
      method: 'POST', headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({url, eventTypes: ['*'], description: 'elle eklendi'}),
    });
    toast('Endpoint eklendi. Konfigurasyon compacted topic uzerinden dispatcher\'a gidiyor.'
    );
    loadEndpoints();
  } catch (e) { toast('Hata: ' + e.message, true); }
}

async function deleteEndpoint(id) {
  if (!confirm('Endpoint silinsin mi?')) return;
  await api('/api/admin/endpoints/' + id, {method: 'DELETE'});
  toast('Silindi. Compacted topic\'e deleted=true basildi.');
  loadEndpoints();
}

async function toggleEndpoint(id, enabled) {
  await api('/api/admin/endpoints/' + id, {
    method: 'PATCH', headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({enabled: !enabled}),
  });
  loadEndpoints();
}

async function loadEndpoints() {
  const box = document.getElementById('endpoints');
  if (!state.appId) { box.innerHTML = '<div class="empty">Once bir uygulama secin</div>'; re
    turn; }

```

```

const [eps, health] = await Promise.all([
  api(`/api/admin/applications/${state.appId}/endpoints`),
  api(`/api/admin/applications/${state.appId}/health`).catch(() => []),
]);
state.endpoints = eps;
state.health = Object.fromEntries(health.map(h => [h.endpointId, h]));

if (!eps.length) { box.innerHTML = '<div class="empty">Endpoint yok</div>'; return; }

box.innerHTML = eps.map(e => {
  const h = state.health[e.endpointId] || state.health[e.id] || {};
  const cb = h.circuitState || 'CLOSED';
  const cbPill = cb === 'OPEN' ? '<span class="pill bad">devre ACIK</span>'
    : cb === 'HALF_OPEN' ? '<span class="pill warn">yari acik</span>' : '';
  const rate = h.succeeded + h.failed ? Math.round(h.successRate * 100) : null;
  const dot = !e.enabled ? 'mute' : cb === 'OPEN' ? 'bad'
    : rate !== null && rate < 80 ? 'warn' : 'ok';
  return `
    <div class="ep">
      <div class="top">
        <span class="dot ${dot}"></span>
        <b>${esc(e.description || 'endpoint')}</b>
        ${cbPill}
        ${e.enabled ? '' : '<span class="pill mute">kapali</span>'}
        <span style="margin-left:auto;display:flex;gap:4px">
          <button class="small ghost" onclick="toggleEndpoint('${e.id}',${e.enabled})">
            ${e.enabled ? 'Kapat' : 'Ac'}</button>
          <button class="small ghost" onclick="deleteEndpoint('${e.id}')">Sil</button>
        </span>
      </div>
      <div class="url">${esc(e.url)}</div>
      <div class="muted" style="font-size:11.5px;margin-top:4px">
        ${rate !== null ? `basari %${rate} . ` : ''}
        ${h.succeeded || 0}✓ ${h.failed || 0}x
        ${h.shortCircuited ? ` . ${h.shortCircuited} atlandi` : ''}
        ${h.consecutiveFailures ? ` . ust uste ${h.consecutiveFailures} hata` : ''}
        ${h.lastLatencyMs !== null ? ` . son ${h.lastLatencyMs} ms` : ''}
        . en fazla ${e.maxAttempts} deneme
      </div>
      ${h.lastError ? `<div class="muted mono" style="font-size:11px;margin-top:3px;color:
        var(--danger)">${esc(h.lastError)}</div>` : ''}
    </div>`;
}).join('');
}

// ----- teslimatlar
async function loadDeliveries() {
  const status = document.getElementById('filterStatus').value;
  const q = new URLSearchParams({size: '30'});
  if (status) q.set('status', status);
  else if (state.appId) q.set('applicationId', state.appId);

  const page = await api('/api/deliveries?' + q);
  const rows = page.content || [];
  const tbody = document.getElementById('deliveries');

  if (!rows.length) {
    tbody.innerHTML = '<tr><td colspan="7" class="empty">Teslimat yok</td></tr>';
    return;
  }
  const cls = s => s === 'SUCCEEDED' ? 'ok' : s === 'PENDING' ? 'pending'

```

```

        : s === 'EXHAUSTED' ? 'bad' : 'mute';
const label = {SUCCEEDED:'basarili', PENDING:'bekliyor',
               EXHAUSTED:'tukendi', DISCARDED:'dusuruldu'};

tbody.innerHTML = rows.map(d => `
  <tr>
    <td><span class="pill ${cls(d.status)}">${label[d.status]} || d.status</span></td>
    <td class="mono">${esc(d.eventType)}</td>
    <td class="mono muted">${shortId(d.endpointId)}</td>
    <td>${d.attempts}</td>
    <td class="mono">${d.lastStatus ?? '<span class="muted"><--</span>'}</td>
    <td class="muted">${ago(d.updatedAt)}</td>
    <td><button class="small ghost" onclick="showDelivery('${d.id}')">Detay</button></td>
  </tr>` ).join('');
}

async function showDelivery(id) {
  const d = await api('/api/deliveries/' + id);
  const rows = d.attempts.map(a => `
    <tr>
      <td>#${a.attempt}</td>
      <td class="mono">${a.httpStatus ?? '-'}</td>
      <td>${a.latencyMs} ms</td>
      <td class="muted">${new Date(a.occurredAt).toLocaleTimeString('tr-TR')}</td>
      <td class="mono muted" style="font-size:11px">${esc(a.error || a.responseSnippet || ''
      )}</td>
    </tr>` ).join('');

  const w = window.open('', '_blank', 'width=900,height=650');
  w.document.write(`
    <html><head><meta charset="utf-8"><title>Teslimat ${shortId(id)}</title>
    <style>body{background:#0b0f14;color:#e6edf6;font:13px system-ui;padding:20px}
    table{width:100%;border-collapse:collapse}td,th{padding:7px;border-bottom:1px solid #1f2
      836;text-align:left}
    pre{background:#121822;padding:12px;border-radius:8px;overflow:auto;font-size:12px}
    h3{color:#39d3a0}</style></head><body>
    <h3>Teslimat ${esc(id)}</h3>
    <p>${esc(d.delivery.eventType)} · ${esc(d.delivery.status)} · ${d.delivery.attempts} den
      eme</p>
    <h3>Denemeler</h3>
    <table><tr><th>#</th><th>HTTP</th><th>Sure</th><th>Zaman</th><th>Detay</th></tr>${rows}<
      /table>
    <h3>Govde</h3><pre>${esc(d.payload || '')}</pre>
    </body></html>`);
}

async function replayExhausted() {
  if (!confirm('Butun tukenmis teslimatlar yeniden kuyruğa alinsin mi?')) return;
  try {
    const r = await api('/api/deliveries/replay-exhausted', {method: 'POST'});
    toast(`${r.requeued} teslimat yeniden kuyruğa alindi`);
    setTimeout(refreshAll, 800);
  } catch (e) { toast('Hata: ' + e.message, true); }
}

// ----- zamanlayici + chaos
async function loadScheduler() {
  const box = document.getElementById('scheduler');
  try {
    const s = await api('/api/scheduler/state');
    const paused = s.pausedPartitions || {};

```

```

const tiers = (s.tiers || []).map((t, i) => {
  const short = 't' + (i + 1);
  const hit = Object.entries(paused).find(([k]) => k.includes('.') + short + '.');
  return `<div class="tier ${hit ? 'paused' : ''}">${short}${
    hit ? ` · ${Math.ceil(hit[1] / 1000)} sn` : ''}</div>`;
}).join('');
box.innerHTML = `<div class="tiers">${tiers}</div>
<div class="muted" style="font-size:11.5px;margin-top:9px">
  Sari = partition duraklatilmiş, geri sayımda.
  ${Object.keys(paused).length} duraklatılmış partition.</div>`;
} catch { box.innerHTML = '<div class="empty">Zamanlayıcıya ulaşılamadı</div>'; }
}

async function loadChaos() {
  const box = document.getElementById('chaos');
  try {
    const list = await api('/sink/received?limit=25');
    if (!list.length) { box.innerHTML = '<div class="empty">Henüz istek gelmedi</div>'; return; }
    box.innerHTML = `<table><tr><th>Davranis</th><th>Olay</th><th>Deneme</th>
<th>Cevap</th><th>Zaman</th></tr>` + list.map(r => `
<tr><td class="mono">${esc(r.behavior)}</td>
<td class="mono muted">${esc(r.eventType || '-')}</td>
<td>#${r.attempt}</td>
<td><span class="pill ${r.respondedWith < 300 ? 'ok' : 'bad'}">${r.respondedWith}<
  /span></td>
<td class="muted">${ago(r.at)}</td></tr>`).join('') + '</table>';
  } catch { box.innerHTML = '<div class="empty">chaos-target kapalı</div>'; }
}

async function clearChaos() {
  await api('/sink/received', {method: 'DELETE'});
  loadChaos();
}

async function loadStats() {
  try {
    const s = await api('/api/deliveries/stats');
    document.getElementById('stats').innerHTML = `
<div class="stat"><b style="color:var(--accent)">${s.SUCCEEDED || 0}</b><small>başarılı</small></div>
<div class="stat"><b style="color:var(--accent2)">${s.PENDING || 0}</b><small>bekleyen</small></div>
<div class="stat"><b style="color:var(--danger)">${s.EXHAUSTED || 0}</b><small>tukenmiş</small></div>
<div class="stat"><b class="muted">${s.DISCARDED || 0}</b><small>dusurulmuş</small></div>`;
  } catch { }
}

function refreshAll() {
  loadStats(); loadApps(); loadEndpoints(); loadDeliveries();
  loadScheduler(); loadChaos();
}

if (state.apiKey) {
  document.getElementById('keyBox').innerHTML =
    `<div class="key"><b>Kayıtlı API anahtarı</b><br>${esc(state.apiKey)}</div>`;
}
refreshAll();
setInterval(() => { loadStats(); loadDeliveries(); loadScheduler(); loadChaos(); }, 2000);
setInterval(loadEndpoints, 4000);

```

```
</script>  
</body>  
</html>
```


Bölüm 13 — Gözlemlenebilirlik

13.1 Metrikler

Her serviste Micrometer + Prometheus:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

```
management:
  endpoints.web.exposure.include: health,info,prometheus,metrics
  metrics.tags.service: dispatcher
```

Uygulama metrikleri:

```
this.succeeded = Counter.builder("hookrelay.dispatch.succeeded").register(meters);
this.timer = Timer.builder("hookrelay.dispatch.http")
    .publishPercentiles(0.5, 0.95, 0.99)
    .register(meters);
```

`publishPercentiles` neden: ortalama gecikme yalan söyler. 99 istek 10 ms, 1 istek 10 saniye sürdüğünde ortalama 110 ms çıkar ve "iyi" görünür. p99 ise 10 saniyeyi gösterir — asıl bilmeniz gereken sayı odur.

13.2 Hangi metrikler önemli

Metrik	Ne söyler	Alarm eşiği
<code>dispatch.succeeded / failed</code>	Başarı oranı	%95 altı
<code>dispatch.http p99</code>	Müşteri sunucuları yavaşlıyor mu	5 sn üstü
<code>dispatch.short_circuited</code>	Kaç endpoint devre dışı	Artıyorsa bak
<code>outbox.failed</code>	Kafka'ya yazamıyoruz	Sıfırdan büyükse acil
<code>scheduler.paused_partitions</code>	Kaç katman beklemede	Bilgi amaçlı
<code>kafka_consumer_records_lag</code>	Tüketici geride kalıyor mu	Sürekli artıyorsa acil

Son satır en kritiği. Lag sürekli artıyorsa üretim tüketimden hızlı demektir ve sistem er ya da geç patlar. Tek çözüm ölçeklemek — ama partition sayısı tavanı geçemezsiniz.

13.3 Grafana

Panolar `ops/grafana/dashboards/hookrelay.json` içinde ve **provisioning ile otomatik yükleniyor**. Elle içe aktarma yok.

```
grafana:
  volumes:
    - ./ops/grafana/provisioning:/etc/grafana/provisioning:ro
    - ./ops/grafana/dashboards:/var/lib/grafana/dashboards:ro
```

Panoyu dosyada tutmak, versiyonlanabilir olması demek. Grafana arayüzünde elle kurulan bir pano, konteyner silinince kaybolur ve kimse nasıl kurulduğunu hatırlamaz.

13.4 Yapılandırma dosyaları

13.4.1 `ops/grafana/provisioning/datasources/prometheus.yml`

`uid: prometheus` elle verilmiş. Verilmeseydi Grafana rastgele bir uid üretir, pano JSON'undaki referans tutmaz ve paneller boş görünürdü.

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    uid: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: false
```

13.4.2 `ops/grafana/provisioning/dashboards/dashboards.yml`

Panolar dosyadan yükleniyor. Grafana arayüzünde elle kurulan bir pano konteyner silinince kaybolur ve kimse nasıl kurulduğunu hatırlamaz.

Pano JSON'u (`ops/grafana/dashboards/hookrelay.json`) 11 panel içeriyor ve elle yazılmadı — küçük bir Python betiğiyle üretildi. JSON'u elle düzenlemek yerine üretici düzenlemek, panel eklemeyi üç satıra indiriyor.

```
apiVersion: 1
providers:
  - name: hookrelay
    folder: HookRelay
    type: file
    disableDeletion: false
    updateIntervalSeconds: 10
    options:
      path: /var/lib/grafana/dashboards
```

Bölüm 14 — Testler

14.1 Saf birim testleri

En değerli mantık, altyapı gerektirmeden test edilebilir olmalı. İmzalama ve yeniden deneme politikası tam olarak böyle:

```
@Test
@DisplayName("Retry-After katman gecikmesinden uzunsa ona uyulur")
void retry_after_uzunsa_kazanir() {
    var r = policy.afterFailure(1, 6, Duration.ofMinutes(3));

    assertThat(r.get().tier()
        .as("topic değişmez, sadece not-before ileri alınır").isEqualTo(1);
    assertThat(r.get().delay()).isEqualTo(Duration.ofMinutes(3));
}
```

`@DisplayName` ile Türkçe açıklama, `.as(...)` ile iddianın **gerekçesi**. Test patladığında "beklenen 1, gelen 2" yerine neden 1 beklendiğini de okursunuz.

Bu üç sınıf 47 test içeriyor ve saniyeler içinde çalışıyor:

Sınıf	Test	Ne doğrular
WebhookSignerTest	8	İmza, replay, zamanlama saldırısı
TieredRetryPolicyTest	10	Katman ilerleyişi, Retry-After , yaşam süresi
FailureClassifierTest	23	HTTP kodu sınıflandırması, iki ayar altında

14.2 Testcontainers ile entegrasyon

```
@SpringBootTest
@Testcontainers
class IngestFlowIT {

    @Container static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");
    @Container static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.7.1"));
    @Container static GenericContainer<?> redis =
        new GenericContainer<>("redis:7-alpine").withExposedPorts(6379);

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        r.add("spring.data.redis.host", redis::getHost);
        r.add("spring.data.redis.port", () -> redis.getMappedPort(6379));
    }
}
```

Neden H2 değil gerçek Postgres: **FOR UPDATE SKIP LOCKED**, kısmi indeks, **jsonb** — hiçbir H2'de

aynı davranmaz. H2 ile geçen bir test, üretimde patlayan bir kod anlamına gelebilir.

Tuzak: sızıntılı test kurulumu

Bu testin ilk hali şöyleydi ve **bozukt**:

```
private static final UUID APP_ID = UUID.randomUUID(); // statik, paylaşılan

@BeforeEach
void setUp() {
    endpoints.put(new EndpointConfig(UUID.randomUUID(), APP_ID, ...)); // her seferinde YE
    NI
    endpoints.put(new EndpointConfig(UUID.randomUUID(), APP_ID, ...));
}
```

@Container static alanlar sınıf boyunca yaşar — Redis konteyneri her test için yeniden kurulmaz. `hr:app:{id}:eps` kümesi de hiç temizlenmediği için endpoint sayısı her testte ikiye artıyordu:

Test	Kümedeki endpoint	Beklenen deliveryCount	Gerçekleşen
1	2	2	2 ✓
2	4	1	3 ✗
3	6	2	6 ✗

Çözüm — sabit kimlik, böylece `put()` üzerine yazar:

```
private static final UUID EP_ALL = UUID.fromString("11111111-1111-1111-1111-111111111111");
private static final UUID EP_PAID = UUID.fromString("22222222-2222-2222-2222-222222222222");
```

Ve testin kendi varsayımını doğrulaması:

```
assertThat(endpoints.listByApplication(APP_ID))
    .as("her test tam iki endpoint ile başlamalı")
    .hasSize(2);
```

O satır olmasa sızıntı sessizce geri gelebilirdi.

Kural. Konteyner static ise durumu da statictir. Testler arasında paylaşılan her dış sistemi (Redis, Kafka, dosya sistemi) ya @BeforeEach içinde temizleyin ya da sabit anahtarlarla üzerine yazın. "Her test kendi verisini oluşturuyor" varsayımı, paylaşılan konteynerlerle birlikte sessizce yanlış olur.

14.3 Sürekli entegrasyon

```
jobs:
  birim-testleri: # mvn test - saniyeler, Docker gerekmez
  entegrasyon-testleri: # mvn verify - Testcontainers, dakikalar
  imajlar: # docker build x6 - Dockerfile bozulmasin
```

Neden üç ayrı iş, tek `mvn verify` değil. İlk hali tek işti ve kırmızı yandığında hangi katmanın düştüğü belli değildi — log açmak gerekiyordu. Loglar da özel repolarda kimlik istiyor. Üçe bölünce cevap iş listesinde görünüyor: `birim-testleri` yeşil + `entegrasyon-testleri` kırmızı = sorun altyapı testinde.

Bu ayrımın bedeli birim testlerinin iki kez koşması (birkaç saniye). Karşılığında her kırmızıda kazanılan dakikalar.

Testcontainers imajları önden çekilir:

```
- run: |
  docker pull postgres:16-alpine
  docker pull redis:7-alpine
  docker pull confluentinc/cp-kafka:7.7.1
```

Testcontainers'ın başlatma zaman aşımına indirme süresi de dahil. Kafka imajı büyük; soğuk runner'da bu adım olmadan test "container did not start" diye düşebiliyor.

Bu projede CI'nın ilk işi, **yerelde hiç koşturmadık** bir testi koşturmak oldu — ve yukarıdaki sızıntıyı ilk o yakaladı. CI'yı "yeşil rozet" için değil, sizin çalıştıramadığınız ortamı sağladığı için kurun.

14.4 Yük testi

```
k6 run -e API_KEY=hr_xxx ops/k6/load.js # 100 olay/sn, 2 dk
k6 run -e API_KEY=hr_xxx -e SCENARIO=spike ops/k6/load.js # 0 → 1000/sn
k6 run -e API_KEY=hr_xxx -e SCENARIO=burst ops/k6/load.js # idempotency yağmuru
```

Üç senaryo, üç farklı soru:

- **steady** — "sürekli yükte sistem stabil mi?"
- **spike** — "ani tepede ne oluyor?" Kuyruk birikir, `dispatcher` arkadan yetişir. Ölçmek istediğiniz: giriş 202 dönmeye devam ediyor mu.
- **burst** — "idempotency gerçekten çalışıyor mu?" 50 sanal kullanıcı aynı 10 anahtarı döndürüp duruyor. Beklenen: 10 mesaj oluşur, geri kalanı `duplicate: true` döner.

```
thresholds: {
  'http_req_duration{expected_response:true}': ['p(95)<150'],
  'olay_hata': ['rate<0.01'],
}
```

Eşik koymak önemli: eşiksiz yük testi bir grafik üretir, eşikli yük testi bir **karar** üretir.

Ölçülen değerler ve bir uyarı

Bu projede ölçülenler (4 çekirdek / 8 GB, 13 konteyner aynı makinede):

Yüksüz tek istek	19 ms doğrudan, 25–60 ms gateway üzerinden
Steady, 200 sanal kullanıcı	~79 olay/sn, medyan 1.6 s, p95 4.9 s, %0 hata
Burst	3610 istek → 10 benzersiz olay, 3600 mükerrer, %0 hata

p95 eşiği (150 ms) aşıyor. Bu bir **başarısızlık değil, bilgi**: yüksüz gecikme 19 ms ve hiçbir istek hata vermiyor — yani sistem kuyruklanıyor, kaybetmiyor. Aşan şey makinenin kapasitesi.

Eşiği ölçülen değere çekme dürtüsüne direnin. Bir eşik, *kabul edilebilir olanı* söylemeli; *şu an olanı* değil. Ölçüme göre ayarlanan eşik hiçbir zaman patlamaz ve hiçbir şey öğretmez.

Buna karşılık `olay_hata < %1` eşiği **asla** aşılmamalı — yavaşlamak kabul edilebilir, olay kaybetmek değil. İki eşiğin anlamı farklı ve bu fark bilinçli.

Ölçüm iki gerçek darboğaz buldu

Bu testler yazılmasaydı fark edilmeyecek iki hata:

- 1. Outbox seri gönderiyordu.** Her kayıt için `send().get()` — 500 kayıtlık paket, 1 ms gidiş-dönüş ile 500 ms. Poller tek iş parçacıklı olduğu için bu doğrudan sistemin tavanıydı. Düzeltme: önce hepsini gönder, sonra teyitleri toplu. Sıra garantisi ve kayıpsızlık korunuyor.
- 1. Fan-out'ta N+1 Redis çağrısı.** `listByApplication` küme üyelerini alıp her biri için ayrı `GET` yapıyordu — 6 endpoint = 7 gidiş-dönüş, kabul isteğinin sıcak yolunda. `MGET` ile 2'ye indi. Veritabanında herkesin bildiği N+1 tuzağı, önbellekte gözden kaçıyor.

Hiçbiri derleyicinin veya birim testinin yakalayabileceği hatalar değildi.

14.5 Testlerin tamamı

14.5.1 `libs/common/src/test/java/dev/hookrelay/common/crypto/WebhookSigner`

Sekiz test. En değerlisi `zaman_damgasi_kurcalanirsa_redededilir` — replay saldırısının kapalı olduğunu kanıtlıyor.

İlk yazılışı **yanlıştı**: imzayı `Instant.now()` ile üretip `t=` değerini yine `now` ile değiştiriyordu. Aynı saniye içinde kaldıkları için sahte başlık orijinaliyle birebir aynı çıkıyor ve test geçmesi gerekirken geçiyordu — ama hiçbir şey kanıtlamıyordu. Doğrusu bir saat önceki bir imzayı tazelemeye çalışmak.

```
package dev.hookrelay.common.crypto;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.time.Instant;

import static org.assertj.core.api.Assertions.assertThat;

class WebhookSignerTest {

    private final WebhookSigner signer = new WebhookSigner();
    private static final String SECRET = "whsec_test_0123456789abcdef";
    private static final String PAYLOAD = "{\"eventType\":\"order.paid\",\"amount\":149.90}";

    @Test
    @DisplayName("Uretilen imza kendi dogrulamasindan gecer")
    void imza_dogrulanir() {
        String header = signer.sign(SECRET, PAYLOAD, Instant.now());
        assertThat(signer.verify(SECRET, PAYLOAD, header, 300)).isTrue();
    }
}
```

```

@Test
@DisplayName("Imza bicimi t=<ts>,v1=<hex>")
void imza_bicimi() {
    String header = signer.sign(SECRET, PAYLOAD, Instant.ofEpochSecond(1_700_000_000L));
    assertThat(header).startsWith("t=1700000000,v1=");
    assertThat(header.split("v1=")[1]).hasSize(64); // SHA-256 = 32 bayt = 64 hex
}

@Test
@DisplayName("Govde degisirse imza tutmaz")
void govde_kurcalanirsa_reddedilir() {
    String header = signer.sign(SECRET, PAYLOAD, Instant.now());
    String tampered = PAYLOAD.replace("149.90", "1.00");
    assertThat(signer.verify(SECRET, tampered, header, 300)).isFalse();
}

@Test
@DisplayName("Yanlis sir ile dogrulama basarisiz")
void yanlis_sir_reddedilir() {
    String header = signer.sign(SECRET, PAYLOAD, Instant.now());
    assertThat(signer.verify("whsec_baska_sir", PAYLOAD, header, 300)).isFalse();
}

@Test
@DisplayName("Eski imza tolerans disinda reddedilir - replay saldirisi kapali")
void eski_imza_reddedilir() {
    Instant tenMinutesAgo = Instant.now().minusSeconds(600);
    String header = signer.sign(SECRET, PAYLOAD, tenMinutesAgo);

    assertThat(signer.verify(SECRET, PAYLOAD, header, 300)).isFalse(); // 5 dk toleran
    assertThat(signer.verify(SECRET, PAYLOAD, header, 900)).isTrue(); // 15 dk tolerans
}

@Test
@DisplayName("Zaman damgasi kurcalanirsa imza tutmaz")
void zaman_damgasi_kurcalanirsa_reddedilir() {
    // Gercek replay senaryosu: saldirgan bir saat once yakaladigi
    // gecerli istegi tekrar gondermek istiyor. Eskidigi icin
    // tolerans kontrolune takilacak; o yuzden t= degerini
    // guncelliyor. Ama v1 ESKI t'ye gore hesaplanmistir - tutmaz.
    String header = signer.sign(SECRET, PAYLOAD, Instant.now().minusSeconds(3600));
    String forged = header.replaceFirst("t=\\d+", "t=" + Instant.now().getEpochSecond());
    ;
    assertThat(signer.verify(SECRET, PAYLOAD, forged, 300))
        .as("t imzanin icinde oldugu icin tek basina degistirilemez")
        .isFalse();
}

@Test
@DisplayName("Bozuk basliklar cokmeden reddedilir")
void bozuk_baslik_reddedilir() {
    assertThat(signer.verify(SECRET, PAYLOAD, null, 300)).isFalse();
    assertThat(signer.verify(SECRET, PAYLOAD, "", 300)).isFalse();
    assertThat(signer.verify(SECRET, PAYLOAD, "sacmalik", 300)).isFalse();
    assertThat(signer.verify(SECRET, PAYLOAD, "t=abc,v1=xyz", 300)).isFalse();
    assertThat(signer.verify(SECRET, PAYLOAD, "v1=deadbeef", 300)).isFalse();
    assertThat(signer.verify(SECRET, PAYLOAD, "t=1700000000,v1=ZZZZ", 300)).isFalse();
}

```

```

@Test
@DisplayName("API anahtari hash'i belirlenimci ve anahtari sızdırmıyor")
void api_anahtari_hash() {
    String key = ApiKeys.generate();
    assertThat(key).startsWith("hr_").hasSize(51);           // hr_ + 48 hex
    assertThat(ApiKeys.hash(key)).isEqualTo(ApiKeys.hash(key)).hasSize(64);
    assertThat(ApiKeys.hash(key)).doesNotContain(key.substring(3));
    assertThat(ApiKeys.preview(key)).hasSize(13).startsWith("hr_").endsWith("...");
}
}

```

14.5.2 `services/dispatcher/src/test/java/dev/hookrelay/dispatcher/delivery/Tiere

`.as(...)` ile her iddianın **gerekçesi** yazılı. Test patladığında "beklenen 1, gelen 2" yerine neden 1 beklendiğini de okursunuz.

```

package dev.hookrelay.dispatcher.delivery;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;

class TieredRetryPolicyTest {

    private static final List<Duration> DELAYS = List.of(
        Duration.ofSeconds(10), Duration.ofMinutes(1), Duration.ofMinutes(5),
        Duration.ofMinutes(30), Duration.ofHours(2));

    private final RetryPolicy policy =
        new TieredRetryPolicy(new RetryProperties(DELAYS, Duration.ofHours(24)));

    @Test
    @DisplayName("1. deneme basarisiz olunca tl'e (10 sn) gider")
    void ilk_hata_tl() {
        Optional<RetryPolicy.Reschedule> r = policy.afterFailure(1, 6, null);

        assertThat(r).isPresent();
        assertThat(r.get().tier()).isEqualTo(1);
        assertThat(r.get().delay()).isEqualTo(Duration.ofSeconds(10));
    }

    @Test
    @DisplayName("Katmanlar sirayla ilerler: 1→t1, 2→t2, ... 5→t5")
    void katmanlar_sirayla() {
        for (int attempt = 1; attempt <= 5; attempt++) {
            var r = policy.afterFailure(attempt, 6, null);
            assertThat(r).as("deneme %d", attempt).isPresent();
            assertThat(r.get().tier()).isEqualTo(attempt);
            assertThat(r.get().delay()).isEqualTo(DELAYS.get(attempt - 1));
        }
    }
}

```



```

@Test
@DisplayName("Katmanlar bitince bos doner - cagiran DLQ'ya yollar")
void katmanlar_bitince_bos() {
    assertThat(policy.afterFailure(6, 6, null)).isEmpty();
    assertThat(policy.afterFailure(9, 20, null))
        .as("maxAttempts büyük olsa bile katman sayisi tavandir")
        .isEmpty();
}

@Test
@DisplayName("maxAttempts katman sayisinden kucukse erken durur")
void max_attempts_tavani() {
    assertThat(policy.afterFailure(2, 3, null)).isPresent();
    assertThat(policy.afterFailure(3, 3, null)).isEmpty();
}

@Test
@DisplayName("Retry-After katman gecikmesinden uzunsa ona uyulur")
void retry_after_uzunsa_kazanir() {
    var r = policy.afterFailure(1, 6, Duration.ofMinutes(3));

    assertThat(r).isPresent();
    assertThat(r.get().tier()).as("topic degismez, sadece not-before ileri alinir").isEqualTo(1);
    assertThat(r.get().delay()).isEqualTo(Duration.ofMinutes(3));
    assertThat(r.get().notBefore()).isAfter(Instant.now().plus(Duration.ofMinutes(2)));
}

@Test
@DisplayName("Retry-After katman gecikmesinden kisaysa yok sayilir")
void retry_after_kisaysa_yok_sayilir() {
    var r = policy.afterFailure(3, 6, Duration.ofSeconds(2));

    assertThat(r).isPresent();
    assertThat(r.get().delay())
        .as("kendi geri cekilmemizden erken donmeyiz")
        .isEqualTo(Duration.ofMinutes(5));
}

@Test
@DisplayName("Deneme yakmadan erteleme, istenen sureyi karsilayan ilk katmani secer")
void deneme_yakmadan_erteleme() {
    assertThat(policy.withoutConsumingAttempt(Duration.ofSeconds(3)).tier())
        .as("3 sn → 10 sn'lik t1 yeterli").isEqualTo(1);

    assertThat(policy.withoutConsumingAttempt(Duration.ofSeconds(45)).tier())
        .as("45 sn → t1 (10 sn) yetmez, t2 (60 sn)").isEqualTo(2);

    assertThat(policy.withoutConsumingAttempt(Duration.ofMinutes(20)).tier())
        .as("20 dk → t4 (30 dk)").isEqualTo(4);
}

@Test
@DisplayName("En buyuk katmandan uzun bekleme istenirse son katmana konur ama erken islenmez")
void cok_uzun_bekleme() {
    var r = policy.withoutConsumingAttempt(Duration.ofHours(5));

    assertThat(r.tier()).isEqualTo(5);
    assertThat(r.notBefore())
        .as("not-before basligi topic gecikmesinden bagimsiz - erken islenmez")

```

```

        .isAfter(Instant.now().plus(Duration.ofHours(4)));
    }

    @Test
    @DisplayName("Yasam suresi dolan teslimat expired doner")
    void yasam_suresi() {
        assertThat(policy.expired(Instant.now()).isFalse());
        assertThat(policy.expired(Instant.now().minus(Duration.ofHours(23))).isFalse());
        assertThat(policy.expired(Instant.now().minus(Duration.ofHours(25))).isTrue());
        assertThat(policy.expired(null).isFalse());
    }

    @Test
    @DisplayName("Bos ayarla kurulursa makul varsayilanlar devreye girer")
    void varsayilanlar() {
        RetryPolicy p = new TieredRetryPolicy(new RetryProperties(null, null));
        assertThat(p.afterFailure(1, 6, null).isPresent());
        assertThat(p.expired(Instant.now().minus(Duration.ofHours(25))).isTrue());
    }
}

```

14.5.3 `services/dispatcher/src/test/java/dev/hookrelay/dispatcher/delivery/FailureClassifierTest`

İç içe sınıflarla iki ayar altında test ediliyor. `gone_her_zaman_kalici` testi, 410'un ayardan muaf olduğunu kilitliyor.

```

package dev.hookrelay.dispatcher.delivery;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.assertj.core.api.Assertions.assertThat;

class FailureClassifierTest {

    private final FailureClassifier strict = new FailureClassifier(false); // varsayilan
    private final FailureClassifier lenient = new FailureClassifier(true); // 4xx'i de den
        e

    private SendResult status(Integer code) {
        return new SendResult(code, 12, "ok", null, null);
    }

    @ParameterizedTest(name = "HTTP {0} → basarili")
    @ValueSource(ints = {200, 201, 202, 204, 299})
    void ikiyuzler_basarili(int code) {
        assertThat(status(code).success()).isTrue();
        assertThat(strict.permanent(status(code))).isFalse();
    }

    @ParameterizedTest(name = "HTTP {0} → gecici, yeniden denenir")
    @ValueSource(ints = {408, 429, 500, 502, 503, 504})
    void gecici_hatalar(int code) {
        assertThat(status(code).success()).isFalse();
        assertThat(strict.permanent(status(code)))
            .as("408 ve 429 'simdi olmaz' demek, 'asla' degil")

```

```

        .isFalse();
    }

    @Test
    @DisplayName("Ag hatasi (kod yok) gecici sayilir")
    void ag_hatasi_gecici() {
        SendResult r = new SendResult(null, 5000, null, "Connection refused", null);
        assertThat(r.success()).isFalse();
        assertThat(strict.permanent(r))
            .as("baglanti kurulamadi - sunucu yarin ayakta olabilir")
            .isFalse();
    }

    @Nested
    @DisplayName("Varsayilan ayar (retry-on-4xx = false)")
    class Varsayilan {

        @ParameterizedTest(name = "HTTP {0} → KALICI, tek denemede DLQ")
        @ValueSource(ints = {400, 401, 403, 404, 422})
        void dortyuzler_kalici(int code) {
            assertThat(strict.permanent(status(code)))
                .as("ayni istegi tekrar gondermek ayni cevabi alır")
                .isTrue();
        }
    }

    @Nested
    @DisplayName("Ayar acikken (retry-on-4xx = true)")
    class AyarAcik {

        @ParameterizedTest(name = "HTTP {0} → artik gecici, yeniden denenir")
        @ValueSource(ints = {400, 401, 403, 404, 422})
        void dortyuzler_yeniden_denendir(int code) {
            assertThat(lenient.permanent(status(code)))
                .as("token yenilerken gecici 403 donen musteriler icin")
                .isFalse();
        }

        @Test
        @DisplayName("410 Gone AYARDAN ETKILENMEZ - her zaman kalici")
        void gone_her_zaman_kalici() {
            assertThat(strict.permanent(status(410))).isTrue();
            assertThat(lenient.permanent(status(410)))
                .as("'bu kaynak kalici olarak yok' demenin baska yolu yok; ısrar etmek kabalık")
                .isTrue();
        }
    }
}

```

14.5.4 `services/dispatcher/src/test/java/dev/hookrelay/dispatcher/delivery/Deliver

`butun_durumlar_siniflandirilmis` testi enum'a yeni bir değer eklendiğinde kırılıyor. Amaç tam olarak bu: yeni bir **Outcome** ekleyen kişi, onu sınıflandırmaya **zorlanmalı**. Aksi hâlde sağlık projeksiyonu sessizce yanlış sayar.

```

package dev.hookrelay.dispatcher.delivery;

import dev.hookrelay.contracts.DeliveryResult.Outcome;

```

```

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.EnumSource;

import static org.assertj.core.api.Assertions.assertThat;

/**
 * Sağlık projeksiyonunun doğruluğu tamamen bu ayrıma bağlı.
 * Yeni bir Outcome eklendiğinde bu testler onu sınıflandırmaya zorlar.
 */
class DeliveryOutcomeTest {

    @ParameterizedTest(name = "{0} → gerçek HTTP denemesi")
    @EnumSource(value = Outcome.class, names = {"SUCCEEDED", "RETRYING", "EXHAUSTED"})
    void gerçek_denemeler(Outcome o) {
        assertThat(o.isRealAttempt()).isTrue();
    }

    @ParameterizedTest(name = "{0} → istek ATILMADI")
    @EnumSource(value = Outcome.class, names = {"SHORT_CIRCUITED", "DISCARDED"})
    void istek_atilmayanlar(Outcome o) {
        assertThat(o.isRealAttempt())
            .as("bunlar sağlık istatistikinde 'hata' sayılmamalı")
            .isFalse();
    }

    @Test
    @DisplayName("Bes durum var; yenisi eklenirse bu test kırılır ve sınıflandırmaya zorlar")
    void bütün_durumlar_sınıflandırılmış() {
        assertThat(Outcome.values()).hasSize(5);
        long gerçek = java.util.Arrays.stream(Outcome.values())
            .filter(Outcome::isRealAttempt).count();
        assertThat(gerçek).isEqualTo(3);
    }
}

```

14.5.5 `services/ingest-api/src/test/java/dev/hookrelay/ingestapi/IngestFlowIT.java`

Uçtan uca giriş akışı. `@DynamicPropertySource` ile konteynerlerin rastgele portları Spring'e aktarılıyor.

Sabit endpoint kimliklerinin sebebi 14.2'de.

```

package dev.hookrelay.ingestapi;

import com.fasterxml.jackson.databind.ObjectMapper;
import dev.hookrelay.common.crypto.ApiKeys;
import dev.hookrelay.common.redis.AppConfigCache;
import dev.hookrelay.common.redis.EndpointConfigCache;
import dev.hookrelay.contracts.ApplicationConfig;
import dev.hookrelay.contracts.EndpointConfig;
import dev.hookrelay.ingestapi.repo.MessageRepository;
import dev.hookrelay.outbox.OutboxRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;

```

```

import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.springframework.test.web.servlet.MockMvc;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.containers.KafkaContainer;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;

import java.util.Set;
import java.util.UUID;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

/**
 * Uctan uca giris akisi - gercek Postgres, gercek Redis, gercek Kafka.
 *
 * NEDEN H2 DEGIL: bu testin dogruladigi seylerin cogu H2'de calismaz veya
 * farkli calisir - kismi UNIQUE indeks, FOR UPDATE SKIP LOCKED, jsonb.
 * H2 ile gecen bir test, uretimde patlayan bir kod anlamina gelebilir.
 *
 * Docker gerekir. Yoksa Testcontainers testi atlar degil, HATA verir -
 * bu da bilincli: "test gecti" ile "test calismadi" ayni sey degil.
 */
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
class IngestFlowIT {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine")
            .withDatabaseName("hookrelay_ingest");

    @Container
    static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.7.1"));

    @Container
    static GenericContainer<?> redis =
        new GenericContainer<>(DockerImageName.parse("redis:7-alpine"))
            .withExposedPorts(6379);

    @DynamicPropertySource
    static void properties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        registry.add("spring.data.redis.host", redis::getHost);
        registry.add("spring.data.redis.port", () -> redis.getMappedPort(6379));
        // Replikatorler kapali: bu testte konfigurasyonu Redis'e ELLE koyuyoruz,
        // boylece admin-api'ye ihtiyac duymadan giris akisini test edebiliyoruz.
        registry.add("hookrelay.endpoint-config.replicate", () -> "false");
        registry.add("hookrelay.app-config.replicate", () -> "false");
    }
}

```

```

registry.add("hookrelay.outbox.enabled", () -> "false");
}

@Autowired MockMvc mockMvc;
@Autowired ObjectMapper mapper;
@Autowired AppConfigCache apps;
@Autowired EndpointConfigCache endpoints;
@Autowired MessageRepository messages;
@Autowired OutboxRepository outbox;

private static final UUID APP_ID = UUID.randomUUID();

// Endpoint kimlikleri SABIT.
//
// İlk yazilisinda her @BeforeEach icinde UUID.randomUUID() cagriliyordu.
// Redis konteyneri sinif boyunca yasadigi ve hr:app:{id}:eps kumesi hic
// temizlenmedigi icin endpoint sayisi her testte ikiser artiyordu:
// 1. test 2 endpoint gorurken 3. test 6 goruyordu ve deliveryCount
// beklentileri tutmuyordu. Testin kendisi sizintiliydi.
//
// Sabit kimlikle put() ayni anahtarın üzerine yaziyor, kume hep 2 kalir.
private static final UUID EP_ALL = UUID.fromString("11111111-1111-1111-1111-1111111111111");
private static final UUID EP_PAID = UUID.fromString("22222222-2222-2222-2222-2222222222222");

private String apiKey;

@BeforeEach
void setUp() {
    messages.deleteAll();
    outbox.deleteAll();

    apiKey = ApiKeys.generate();
    apps.put(new ApplicationConfig(APP_ID, "test-app", ApiKeys.hash(apiKey),
        true, false, 1));

    // Iki endpoint: biri her seye abone, digeri sadece order.paid'e
    endpoints.put(new EndpointConfig(EP_ALL, APP_ID,
        "http://ornek/hepsi", "whsec_1", Set.of("*"),
        true, false, 0, 6, 10_000, 1));
    endpoints.put(new EndpointConfig(EP_PAID, APP_ID,
        "http://ornek/odeme", "whsec_2", Set.of("order.paid"),
        true, false, 0, 6, 10_000, 1));

    // Testin kendi varsayimini dogrula: tam iki endpoint görünmeli.
    // Bu satir olmasaydi yukaridaki sizinti sessizce geri gelebilirdi.
    assertThat(endpoints.listByApplication(APP_ID))
        .as("her test tam iki endpoint ile baslamali")
        .hasSize(2);
}

@Test
@DisplayName("0lay kabul edilir ve abone endpoint sayisi kadar teslimat uretilir")
void fan_out_dogru_calisir() throws Exception {
    mockMvc.perform(post("/v1/events")
        .header("Authorization", "Bearer " + apiKey)
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            { "eventType": "order.paid", "payload": { "orderId": 4711 } } """)
        .andExpect(status().isAccepted()));
}

```

```

        .andExpect(jsonPath("$.deliveryCount").value(2))
        .andExpect(jsonPath("$.duplicate").value(false));

    assertThat(messages.count()).isEqualTo(1);
    assertThat(outbox.count())
        .as("her endpoint için bir outbox kaydı")
        .isEqualTo(2);
}

@Test
@DisplayName("Abone olmayan olay tipi teslimat üretmez")
void abone_olmayan_tip_atlanir() throws Exception {
    mockMvc.perform(post("/v1/events")
        .header("Authorization", "Bearer " + apiKey)
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"eventType":"user.deleted","payload":{}}"""))
        .andExpect(status().isAccepted())
        .andExpect(jsonPath("$.deliveryCount").value(1));    // sadece "*"
}

@Test
@DisplayName("Aynı Idempotency-Key ikinci kez mesaj oluşturmaz")
void idempotency_calisir() throws Exception {
    String body = """
        {"eventType":"order.paid","payload":{"orderId":1}}""";

    mockMvc.perform(post("/v1/events")
        .header("Authorization", "Bearer " + apiKey)
        .header("Idempotency-Key", "siparis-1")
        .contentType(MediaType.APPLICATION_JSON).content(body))
        .andExpect(status().isAccepted())
        .andExpect(jsonPath("$.duplicate").value(false));

    for (int i = 0; i < 3; i++) {
        mockMvc.perform(post("/v1/events")
            .header("Authorization", "Bearer " + apiKey)
            .header("Idempotency-Key", "siparis-1")
            .contentType(MediaType.APPLICATION_JSON).content(body))
            .andExpect(status().isAccepted())
            .andExpect(jsonPath("$.duplicate").value(true));
    }

    assertThat(messages.count())
        .as("dört istek, tek mesaj")
        .isEqualTo(1);
    assertThat(outbox.count())
        .as("teslimat da bir kez üretilir")
        .isEqualTo(2);
}

@Test
@DisplayName("Gecersiz API anahtarı 401 döner")
void gecersiz_anahar_reddedilir() throws Exception {
    mockMvc.perform(post("/v1/events")
        .header("Authorization", "Bearer hr_sahte")
        .contentType(MediaType.APPLICATION_JSON)
        .content("""
            {"eventType":"x","payload":{}}"""))
        .andExpect(status().isUnauthorized())
        .andExpect(jsonPath("$.code").value("UNAUTHORIZED"));
}

```

```

    }

    @Test
    @DisplayName("Authorization basligi olmadan 401")
    void baslik_yoksa_reddedilir() throws Exception {
        mockMvc.perform(post("/v1/events")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"eventType":"x","payload":{}}"""))
            .andExpect(status().isUnauthorized());
    }
}

```

14.5.6 `ops/k6/load.js`

Üç senaryo tek dosyada, `SCENARIO` ortam değişkeniyle seçiliyor.

`burst` senaryosunda `idemKey` kasten tekrar ediyor: 50 sanal kullanıcı aynı 10 anahtarı döndürüp duruyor. Beklenen sonuç 10 mesaj, geri kalanı mükerrer.

```

// =====
// HookRelay yuk testi
//
// k6 run -e API_KEY=hr_xxx ops/k6/load.js
// k6 run -e API_KEY=hr_xxx -e SCENARIO=spike ops/k6/load.js
//
// Kurulum gerekmiyorsa Docker ile:
// docker run --rm -i --network hookrelay_hookrelay \
// -e API_KEY=hr_xxx -e BASE_URL=http://gateway:8080 \
// grafana/k6 run - < ops/k6/load.js
// =====

import http from 'k6/http';
import { check } from 'k6';
import { Counter, Rate } from 'k6/metrics';

const BASE_URL = __ENV.BASE_URL || 'http://localhost:8080';
const API_KEY = __ENV.API_KEY;
const SCENARIO = __ENV.SCENARIO || 'steady';

const accepted = new Counter('olay_kabul');
const duplicates = new Counter('olay_mukerrer');
const failRate = new Rate('olay_hata');

// Uc senaryo, uc farkli soru:
//
// steady - "surekli yukte sistem stabil mi?" ~100 olay/sn, 2 dakika
// spike - "ani tepede ne oluyor?" 0 → 1000/sn, 30 saniye
// burst - "idempotency gercekten calisiyor mu?" ayni anahtarla yagmur
const scenarios = {
  steady: {
    executor: 'constant-arrival-rate',
    rate: 100, timeUnit: '1s', duration: '2m',
    preAllocatedVUs: 50, maxVUs: 200,
  },
  spike: {
    executor: 'ramping-arrival-rate',
    startRate: 10, timeUnit: '1s',
    preAllocatedVUs: 100, maxVUs: 800,
  },
};

```



```

    stages: [
      { target: 50, duration: '10s' },
      { target: 1000, duration: '20s' }, // TEPE
      { target: 1000, duration: '30s' },
      { target: 10, duration: '20s' },
    ],
  },
  burst: {
    executor: 'constant-vus',
    vus: 50, duration: '30s',
  },
};

// -----
// ESIKLER – VE OLCULEN GERCEK DEGERLER
// -----
// Yuksuz tek istek (olculdu):
//   dogrudan ingest-api : ~19 ms
//   gateway uzerinden   : ~25-60 ms
//
// 4 cekirdek / 8 GB gelistirme makinesinde, 13 konteynerin hepsi ayni
// makinede calisirken steady senaryosu (200 sanal kullaniciya kadar):
//   kabul   ~79 olay/sn   (hedef 100)
//   medyan  ~1.6 s       p95 ~4.9 s      hata %0
//
// Bu sayilar UYGULAMANIN degil MAKINENIN limiti: yuksuz gecikme 19 ms,
// hicbir istek hata vermiyor, sadece kuyruklaniyor. Ayni yigin ayri
// makinelerde calistiginda p95 150 ms'nin altinda kalir.
//
// Esigi dusurmuyoruz: bir esik, "kabul edilebilir" olani soylemeli,
// "su an olan"i degil. Gelistirme makinesinde asilmasi normaldir ve
// asildiginda size makinenin doyduğunu soyler – ki bu da faydali bir bilgi.
// -----
export const options = {
  scenarios: { [SCENARIO]: scenarios[SCENARIO] },
  thresholds: {
    'http_req_duration{expected_response:true}': ['p(95)<150'],
    // Asil kirmizi çizgi bu: HICBIR olay kaybolmamali.
    // Yavaslamak kabul edilebilir, kaybetmek degil.
    'olay_hata': ['rate<0.01'],
  },
};

export function setup() {
  if (!API_KEY) {
    throw new Error('API_KEY gerekli. Once bir uygulama olusturup anahtari alin:\n' +
      " curl -s -XPOST localhost:8080/api/admin/applications " +
      "-H 'Content-Type: application/json' -d '{\"name\":\"yuk-testi\"}');
  }
  return {};
}

export default function () {
  const eventTypes = ['order.created', 'order.paid', 'user.updated', 'invoice.finalized'];
  const eventType = eventTypes[Math.floor(Math.random() * eventTypes.length)];

  // burst senaryosunda anahtar KASITLI olarak tekrar ediyor:
  // 50 sanal kullanıcı aynı 10 anahtarı dondurup duruyor.
  // Beklenen sonuc: 10 mesaj olusur, geri kalan hepsi "mukerrer" doner.
  const idemKey = SCENARIO === 'burst'
    ? `tekrarli-${__ITER % 10}`

```

```
    : `${__VU}-${__ITER}-${Date.now()}`;

const payload = JSON.stringify({
  eventType,
  payload: {
    id: `${__VU}-${__ITER}`,
    amount: Math.round(Math.random() * 100000) / 100,
    currency: 'TRY',
    at: new Date().toISOString(),
  },
});

const res = http.post(`${BASE_URL}/v1/events`, payload, {
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${API_KEY}`,
    'Idempotency-Key': idemKey,
  },
});

const ok = check(res, {
  '202 kabul edildi': (r) => r.status === 202,
});

failRate.add(!ok);
if (res.status === 202) {
  const body = res.json();
  if (body && body.duplicate) duplicates.add(1);
  else accepted.add(1);
}
}
```

Bölüm 15 — Kırma Senaryoları

Öğrenmenin asıl gerçekleştiği bölüm. Her senaryodan önce panel (`localhost:3000`) açık olsun.

15.1 Kafka'yı durdurun

```
docker compose stop kafka
curl -XPOST localhost:8080/v1/events -H "Authorization: Bearer $API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{"eventType":"test","payload":{}}'
```

Beklenen: `202 Accepted` gelir. Olay kabulü **çalışmaya devam eder**.

Çünkü `ingest-api` Kafka'ya yazmıyor — outbox tablosuna yazıyor. Kafka'nın ayakta olup olmaması onu ilgilendirmiyor.

```
docker compose logs ingest-api --tail=20 # outbox hataları görürsünüz
docker compose start kafka             # ve birikmiş her şey akar
```

Beşinci bölümde outbox'ı kurarken anlatılan şeyin canlı hali. Outbox olmasaydı `202` yerine `500` alırdınız ve olay kaybolurdu.

15.2 Redis'i durdurun

```
docker compose stop redis
```

Beklenen: teslimatlar düşürülür. `dispatcher` endpoint konfigürasyonunu okuyamıyor.

Bu, Redis'in bu projede "sadece cache" olmadığının kanıtı. Cache olsaydı Redis çöktüğünde sistem yavaşlar ama çalışırdı. Burada Redis bir **replika** — kaybolunca veri kayboluyor.

```
docker compose start redis
curl -XPOST localhost:8080/api/admin/endpoints/republish
```

`republish` ucu kaynak veritabanından replikayı yeniden kurar. Türetilmiş verinin yeniden üretilebilir olması, sağlıklı bir mimarinin işaretidir.

15.3 dispatcher'ı öldürün

```
docker compose kill -s SIGKILL dispatcher && docker compose start dispatcher
```

Beklenen: commit edilmemiş offset'lerdeki mesajlar tekrar işlenir.

Panelde deneme sayılarına bakın. Bazı teslimatlarda aynı deneme numarasının iki kez işlendiğini göreceksiniz — ama `delivery_attempt` tablosundaki `UNIQUE` kısıt sayesinde ikinci kayıt açılmıyor.

"En az bir kez" teslimatın canlı kanıtı. Sekizinci bölümdeki tartışmanın somut hali.

15.4 Ölçekleyin

```
docker compose up -d --scale dispatcher=3 --no-recreate dispatcher
```

Kafka UI'da (`localhost:8090`) `dispatcher` tüketici grubuna bakın: 12 partition üç örnek arasında paylaştırılıyor (4 + 4 + 4).

Şimdi 13 örnek deneyin. Bir örnek **boş oturur** — partition sayısı paralellik tavanıdır.

15.5 Yavaş müşteri

```
curl -XPOST "localhost:8080/api/admin/applications/$APP_ID/endpoints" \
  -H 'Content-Type: application/json' \
  -d '{"url":"http://chaos-target:8085/sink/slow?ms=25000","eventTypes":["*"]}'
```

Sonra 100 olay gönderin. **Beklenen:** yavaş endpoint'e giden teslimatlar birikir ama **diğer endpoint'ler etkilenmez**.

Bulkhead olmasaydı 12 tüketici iş parçasığının hepsi o endpoint'te takılırdı.

`hookrelay.bulkhead.max-concurrent-per-endpoint` değerini 100 yapıp tekrar deneyin — farkı göreceksiniz.

15.6 Devre kesiciyi açın

`/sink/dead` endpoint'ine 10 olay gönderin. Panelde beşinci hatadan sonra **"devre AÇIK"** rozeti belirir.

Grafana'da `hookrelay.dispatch.short_circuited` sayacının artmaya başladığını görürsünüz. 20 saniye sonra `HALF_OPEN`'a geçer, tek bir yoklama gider, o da patlar, tekrar `OPEN`.

`/sink/dead`'i `/sink/ok`'e çevirin (`PATCH /api/admin/endpoints/{id}`). Bir sonraki yoklama geçer, devre kapanır ve biriken her şey akar.

Bölüm 16 — GitHub'a Hazırlık

Kod bitti. Şimdi projeyi bakanın anlayacağı hâle getirmek gerekiyor — ve bu, kodun kendisi kadar önemli.

16.1 README ilk otuz saniyede kazanılır

Sırasıyla şunlar olmalı:

- 1. Tek cümlelik tanım.** "Güvenilir webhook teslimat servisi."
- 2. Çalıştırma komutu.** İki satır, kopyala-yapıştır.
- 3. Hangi problemi çözüyor.** Naif kodun neden yetmediği — tercihen bir tablo.
- 4. Mimari şeması.** ASCII yeterli; resim dosyası bakım yükü.
- 5. En ilginç teknik detay.** Bizde bu, Kafka'da geciktirmeli mesaj.
- 6. Dürüst sınırlar.**

Altıncı madde çoğu projede yok ve tam da bu yüzden değerli. "Exactly-once yok, çünkü dağıtık sistemde yok" cümlesi, projeyi yazanın ne yaptığını bildiğini gösterir. Bunu yazmayan projeler, ya düşünmemiş ya saklıyor gibi görünür.

16.2 Kod yorumları

Bu projedeki yorumlar **ne yaptığını** değil **neden öyle yaptığını** anlatıyor:

```
// Kötü: sayacı artır
counter.increment();

// İyi: Deneme sayısı ARTMIYOR: aynı attempt ile geri konuyor.
// Müşterinin hatası olmayan bir engelleme, onun deneme hakkını yakmamalı.
int attempt = block.consumesAttempt() ? task.attempt() + 1 : task.attempt();
```

Bir yorum, kodu okuyarak öğrenilemeyecek bir şey söylemiyorsa gereksizdir. Kodu okuyarak öğrenilemeyecek tek şey ise **niyet**.

16.3 Commit geçmişi

Faz faz ilerleyin, her fazda çalışan bir sistem bırakın:

```
feat: contracts — topic kataloğu ve olay şemaları
feat: common — HMAC imzalama ve Redis Lua ilkelleri
feat: outbox — transactional outbox deseni
feat: admin-api — kontrol düzlemi ve compacted config replikasyonu
feat: ingest-api — idempotent olay kabulü ve fan-out
feat: dispatcher — teslimat hattı, devre kesici, bulkhead
feat: retry-scheduler — pause/seek ile geciktirmeli mesaj
feat: ops — compose, prometheus, grafana, k6
docs: README ve mimari kararlar
```

Bir tek "initial commit"le yüklenmiş proje, hazır şablondan indirilmiş gibi görünür.

16.4 Neyi eklemeyin

Ekleme	Neden
Eureka, Config Server	Compose DNS'i zaten var — süs
GraphQL	REST yeterli, gösteriş olur
Kubernetes manifest'leri	Çalıştırılmayacak, bakılmayacak
Rozet (badge) yığını	Beş rozet güven verir, on beş şüphe
"TODO: ileride eklenecek"	Yapılmamış işin ilanı

Bir projeyi güçlü yapan şey ne kadar teknoloji içerdiği değil, **her parçasının neden orada olduğunun açıklanabilmesidir**. Mülakatta "burada X neden var?" sorusuna "modern görünsün diye" cevabı, o teknolojiyi hiç koymamış olmaktan kötüdür.

16.5 Mülakatta ne sorulur

Bu projeyle şu sorulara cevabınız olur:

- "Kafka'da bir mesajı nasıl geciktirirsiniz?" → Dokuzuncu bölüm, [pause/seek](#).
- "Veritabanı ve Kafka'ya atomik yazma nasıl yapılır?" → Beşinci bölüm, outbox.
- "Exactly-once mümkün mü?" → Hayır; en-az-bir-kez + idempotent alıcı.
- "Yavaş bir bağımlılık sisteminizi nasıl etkiler?" → Bulkhead, gürültülü komşu.
- "Devre kesiciyi neden Redis'te yazdınız?" → Paylaşılan durum, çok örnek.
- "Partition sayısını nasıl seçtiniz?" → Paralellik tavanı, artırmanın sıra etkisi.
- "İki servis aynı veriye nasıl bakar?" → Compacted topic ile replikasyon.

Hepsinin cevabı kodda, gerekçesiyle birlikte yazılı.

Bölüm 17 — Gözden Geçirme: Kodun İkinci Okunuşu

Kod çalışıyor. Testler geçiyor. Sistem ayakta. **Şimdi baştan okuyun.**

Bu bölüm, bu projede yapılan gerçek bir gözden geçirme turunun çıktısı. Sekiz bulgu çıktı; biri ciddi bir işlem hatasıydı, biri metrikleri yalancı yapıyordu, geri kalanı okunabilirlik borcuydu. Hiçbirini derleyici veya birim testi yakalayamazdı.

Amaç bulguları ezberletmek değil, **arama yöntemini** göstermek.

17.1 Önce mekanik tarama

İnsan gözünün harcanmayacağı şeyler önce, komutla:

```
# Kullanılmayan import
for f in $(find . -name '*.java'); do
  for imp in $(grep -oP '^import \K[\w.]+(?:=;)' "$f"); do
    cls=${imp##*.}
    grep -v '^import ' "$f" | grep -qw "$cls" || echo "$f → $imp"
  done
done

grep -rn "TODO\|FIXME\|XXX\|HACK" --include=*.java .
grep -rn "printStackTrace\System.out" --include=*.java .
```

Bu projede: sıfır kullanılmayan import, sıfır TODO. Temiz çıkması iyi haber değil — sadece **asıl bakılması gereken yere** geçmenizi sağlıyor.

17.2 Sonra "az geçen alan" taraması

Bir alan dosyada yalnızca iki kez geçiyorsa (tanım + bir kullanım) veya bir kez geçiyorsa (yalnız tanım), incelemeye değer:

```
grep -oP 'private (static )?(final )?[\w<>,\[\] ]+ \K\w+(?= *[:=])' "$f"
```

Çıkan sonuç: **IngestService.log hiç kullanılmıyordu**. Küçük ama anlamlı — birisi log yazmayı planlamış, sonra yazmamış. Bugün ölü kod, yarın "burada log var sanıyordum" diye kaybedilen bir saat.

17.3 Asıl iş: her `catch` bloğunu sorgulayın

Gözden geçirmenin en verimli tek sorusu şudur:

Bu istisnayı yakaladıktan sonra program gerçekten sağlam bir durumda mı?

Bu projede bir yerde cevap **hayırdı**:

```
try { attempts.save(new DeliveryAttempt(...)); }
catch (DataIntegrityViolationException e) { log.debug("zaten var"); }
deliveries.save(delivery); // ← transaction çoktan ölmüş
```

Hibernate kısıt ihlalinin sonra transaction'ı **rollback-only** işaretler. Kod çalışıyor *görünür*, commit anında patlar. Üstelik tam da "mükerrer teslimat" senaryosunda — yani Kafka'da **normal** olan durumda.

Bunu hiçbir birim testi yakalamaz, çünkü mock'lanmış bir repository istisna atmaz. Entegrasyon testi bile yakalamaz — o senaryoyu **kasten** kurmadıysanız.

Çözüm sekizinci bölümde: **ON CONFLICT DO NOTHING**.

Kural olarak: beklediğiniz bir çatışmayı istisna ile karşılamayın, **önleyin**.

17.4 Metriklerin anlamını sorgulayın

Bir metriğin var olması, doğru olduğu anlamına gelmiyor. İki soru sorun:

1. Bu sayı tam olarak neyi sayıyor?
2. Bu sayıya bakan biri hangi kararı verir, ve o karar doğru olur mu?

İki bulgu çıktı.

Birincisi: sağlık projeksiyonu "SUCCEEDED değilse hatadır" diyordu. Silinmiş bir endpoint'in 25.657 düşürülmüş teslimatı, o endpoint'in "25.657 kez hata verdiği" gibi görünüyordu — oysa ona **tek bir istek bile atılmamıştı**. Devresi açık bir endpoint de hiç istek almadığı hâlde saniyede yüzlerce "hata" biriktiriyordu.

Bu sayıya bakan biri "endpoint bozuk" der. Yanlış karar.

Çözüm: `Outcome.isRealAttempt()` ayrımı ve ayrı bir `short_circuited` sütunu.

İkincisi: `ingest.accepted` sayacı, iki mükerrer yolundan **birinde** artıyordu (veritabanı çatışması) ama diğerinde artmıyordu (Redis isabeti). Yani "kabul edilen olay" metriği, mükerrer trafiğin dağılımına göre değişen bir sayıydı. İki yol iki farklı şey ölçüyordu.

Yanlış bir metrik, olmayan bir metriktir: olmayana bakmazsınız, yanlış olana güvenirsiniz.

17.5 "Neden bir eksik?" dedirten satırları arayın

```
delivery.retry(task.attempt() - 1, null, reason, nextAttemptAt);
```

Çalışıyordu. Niyet doğrudu: "bu engelleme bir deneme hakkı yakmadı." Ama okuyan kişi - 1'i görünce durup düşünmek zorunda kalıyordu.

Niyeti kodun kendisi söylemeli:

```
delivery.blocked(reason, nextAttemptAt); // attempts'e dokunmaz
```

Aynı davranış, sıfır soru işareti. Bu tür düzeltmeler "kozmetik" görünür ama gözden geçirmenin en yüksek getirili işidir: her - 1, gelecekte birinin yanlış anlayıp "düzelteceği" bir hatadır.

17.6 Çalışmayan anotasyonları arayın


```
@Transactional
public Delivery ensure(DeliveryTask task) { ... } // yalnız içeriden çağrılıyor
```

Bu anotasyon **hiçbir şey yapmıyor** — Spring'de transaction proxy ile çalışır, aynı sınıftan yapılan çağrı proxy'ye uğramaz. Burada zararsızdı (çağırının transaction'ı zaten vardı) ama yanıltıcıydı.

Çalışmayan bir anotasyon, olmayan bir anotasyondan kötüdür: okuyan kişi "burası kendi transaction'ını açıyor" sanır ve yanlış bir zihin modeli kurar. Sonra o modele dayanarak bir değişiklik yapar.

Düzeltilme: metodu **private** yapın, anotasyonu kaldırın, **neden** kaldırdığınızı yazın.

17.7 Sayfalama ile filtrelemenin sırasını kontrol edin

```
deliveries.findByStatus(EXHAUSTED, PageRequest.of(0, 500))
    .getContent().stream()
    .filter(d -> d.getEndpointId().equals(endpointId)) // ← sonra
    .toList();
```

"En fazla 500 gönder" diyorsunuz. Veritabanı 500 kayıt dönüyor. Java o 500'ün içinden hedef endpoint'e ait 12 tanesini süzüyor. Kullanıcı 500 gönderildiğini sanıyor, 12 gönderilmiş oluyor.

Sessizce eksik iş — hata vermiyor, log basmıyor, sadece yanlış.

Filtreleme her zaman sayfalamadan **önce**, yani veritabanında olmalı.

17.8 Bir düzeltmeyi nasıl kanıtlarsınız

Düzelttiğinizi *sanmak* yetmez. **ON CONFLICT** düzeltmesi için kurulan deterministik kanıt:

Problem: SIGKILL ile mükerrer teslimat üretmeye çalıştık — olmadı. **ack-mode: manual_immediate** ile offset her kayıttan sonra commit edildiği için pencere birkaç mikrosaniye. Rastlantıya bağlı bir test, test değildir.

Çözüm: Aynı çatışmayı **kasten** kurun. Yeniden gönderim (**replay**) tam da bunu yapıyor — **attempt** sayacını 1'e sıfırlıyor, oysa **delivery_attempt** tablosunda zaten bir **attempt = 1** satırı var.

```
# 1. Başarılı bir teslimat seç (attempt=1 kaydı mevcut)
DID=$(curl -s ".../api/deliveries?endpointId=$EP&size=1" | jq -r '.content[0].id')

# 2. Yeniden gönder → attempt 1'e sıfırlanır → INSERT çatışır
curl -XPOST ".../api/deliveries/$DID/replay"
```

Beklenen üç gözlem — üçü birden gerçekleşmeli:

Gözlem	Ne kanıtlar
chaos-target aynı deliveryId 'yi iki kez gördü	İstek gerçekten tekrar gitti
delivery_attempt 'te hâlâ tek satır	ON CONFLICT çatışmayı yuttu
Durum tekrar SUCCEEDED oldu	Transaction çatışmadan sağ çıktı

Üçüncüsü asıl kanıt. Eski kodda transaction **rollback-only** işaretlenirdi ve durum güncellemesi

kaybolurdu — teslimat **PENDING**'de sonsuza kadar takılı kalırdı.

```
-- Genel tutarlılık kontrolü
SELECT count(*) FROM (
  SELECT delivery_id, attempt FROM delivery_attempt
  GROUP BY 1,2 HAVING count(*) > 1
) x; -- → 0 olmalı

SELECT count(*) FROM delivery_attempt a
LEFT JOIN delivery d ON d.id = a.delivery_id
WHERE d.id IS NULL; -- → 0 olmalı (yetim kayıt yok)
```

Ve logda:

```
docker logs hr-dispatcher | grep -c "UnexpectedRollbackException" # → 0
```

Kural. Bir hatayı düzelttiğinizde, düzeltmeden **önce** o hatayı deterministik olarak üretebildiğinizden emin olun. Üretemiyorsanız, düzelttiğinizi de kanıtlayamazsınız.

17.9 Gözden geçirme kontrol listesi

Kendi projenizde uygulayabileceğiniz sıra:

#	Soru	Nasıl aranır
1	Ölü kod var mı?	Kullanılmayan import/alan taraması
2	Her catch sonrası program sağlam mı?	Elle, tek tek
3	Metrikler tam olarak neyi sayıyor?	Her sayacın iki yolunu izle
4	"Neden böyle?" dedirten satır var mı?	Okurken durduğunuz her yer
5	Hangi anotasyonlar çalışmıyor?	Self-invocation, eksik proxy
6	Filtre sayfalamadan önce mi?	Her PageRequest
7	Doküman kodu anlatıyor mu?	Her iddiayı kodda doğrula
8	Kullanılmayan parametre var mı?	Derleyici uyarıları

Yedinci madde bu projede iki kez iş çıkardı: **KARARLAR.md** bir yapılandırma ayarından bahsediyordu ama ayar kodda yoktu. Doküman **yalan söylüyordu** ve altı ay sonra birisi o ayarı arayacaktı.

Çözüm dokümanı zayıflatmak değil, ayarı yazmaktı — çünkü gerekçe doğrudu, eksik olan uygulamaydı.

Kontrol noktası

```
mvn -q clean test
```

Bu projede gözden geçirme sonrası: **47 test**, hepsi geçiyor. Turdan önce 41'di — yeni bulguların her biri kendi testini de getirdi.

Bir bulgunun testi yoksa, o bulgu geri gelir.

Ek A — Komut Referansı

Çalıştırma

```
./scripts/baslat.sh          # derle, ayağa kaldır, hazır olana kadar bekle
./scripts/demo.sh           # uçtan uca demo
./scripts/durdur.sh         # durdur (veri korunur)
./scripts/durdur.sh --temizle # durdur ve veri hacimlerini sil
```

Kırma senaryoları

```
./scripts/kir.sh broker
./scripts/kir.sh redis
./scripts/kir.sh cokuk-tuketici
./scripts/kir.sh olcekle 3
```

Adresler

Kontrol paneli	http://localhost:3000
Gateway	http://localhost:8080
Kafka UI	http://localhost:8090
Grafana	http://localhost:3001 (admin/admin)
Prometheus	http://localhost:9090
Postgres	localhost:5433 (hookrelay/hookrelay)
Redis	localhost:6380
Kafka (dışarıdan)	localhost:29092

API

```
# Uygulama oluştur
curl -XPOST localhost:8080/api/admin/applications \
  -H 'Content-Type: application/json' -d '{"name":"magazam"}'

# Endpoint ekle
curl -XPOST localhost:8080/api/admin/applications/$APP_ID/endpoints \
  -H 'Content-Type: application/json' \
  -d '{"url":"https://ornek.com/webhook","eventTypes":["order.*"],"rateLimitPerSecond":10}'

# Olay gönder
curl -XPOST localhost:8080/v1/events \
  -H "Authorization: Bearer $API_KEY" \
  -H 'Idempotency-Key: siparis-4711' \
```

```
-H 'Content-Type: application/json' \
-d '{"eventType":"order.paid","payload":{"orderId":4711}}'

# Teslimat günlüğü
curl 'localhost:8080/api/deliveries?status=EXHAUSTED'
curl localhost:8080/api/deliveries/$DELIVERY_ID
curl -XPOST localhost:8080/api/deliveries/replay-exhausted

# Zamanlayıcı durumu
curl localhost:8080/api/scheduler/state
```

Kafka

```
docker exec hr-kafka /opt/kafka/bin/kafka-topics.sh \
--bootstrap-server localhost:9092 --list

docker exec hr-kafka /opt/kafka/bin/kafka-consumer-groups.sh \
--bootstrap-server localhost:9092 --describe --group dispatcher

docker exec hr-kafka /opt/kafka/bin/kafka-console-consumer.sh \
--bootstrap-server localhost:9092 \
--topic hookrelay.endpoint-config.v1 --from-beginning \
--property print.key=true
```

Son komut compacted topic'i gösterir. Bir endpoint'i beş kez güncelleyip birkaç dakika bekleyin, sonra tekrar çalıştırın: yalnızca son sürüm kalmış olacak.

Betiklerin tamamı

baslat.sh `scripts/baslat.sh`

`docker compose up` yeterli değil: konteyner "başladı" der ama uygulama henüz hazır değildir. `wait_for` her servisin sağlık ucunu yokluyor.

```
#!/usr/bin/env bash
# HookRelay'i ayaga kaldırır ve hazır olana kadar bekler.
set -euo pipefail
cd "$(dirname "$0")/.."

echo "► İmajlar derleniyor ve konteynerler baslatılıyor..."
docker compose up -d --build

wait_for () {
    local name=$1 url=$2 tries=${3:-60}
    printf "  %-18s" "$name"
    for i in $(seq 1 "$tries"); do
        if curl -fsS "$url" >/dev/null 2>&1; then echo "hazır"; return 0; fi
        sleep 2
    done
    echo "ZAMAN ASIMI"
    echo "    docker compose logs $name --tail=50"
    return 1
}

echo
```

```
echo "► Saglik kontrolleri:"
wait_for admin-api      http://localhost:8081/actuator/health
wait_for ingest-api     http://localhost:8082/actuator/health
wait_for dispatcher     http://localhost:8083/actuator/health
wait_for retry-scheduler http://localhost:8084/actuator/health
wait_for chaos-target   http://localhost:8085/actuator/health
wait_for gateway        http://localhost:8080/actuator/health
wait_for ui             http://localhost:3000
```

```
cat <<'BANNER'
```

```
✓ HookRelay ayakta
```

```
Kontrol paneli  http://localhost:3000
Kafka UI        http://localhost:8090
Grafana         http://localhost:3001  (admin / admin)
Prometheus      http://localhost:9090
```

```
Demoyu calistirmak icin: ./scripts/demo.sh
BANNER
```

demo.sh `scripts/demo.sh`

```
#!/usr/bin/env bash
# -----
# Uctan uca demo.
#
# Bir uygulama ve dort endpoint kurar, 40 olay gonderir, ne oldugunu anlatir.
# Amac: yeniden deneme, devre kesici, kalici hata ve DLQ davranislarinin
# hepsini tek komutta gozle gorulebilir kilmak.
# -----
set -euo pipefail
BASE=${BASE:-http://localhost:8080}
CHAOS_INTERNAL=${CHAOS_INTERNAL:-http://chaos-target:8085}

need () { command -v "$1" >/dev/null || { echo "$1 gerekli"; exit 1; }; }
need curl; need jq

echo "► 1/5 Uygulama olusturuluyor"
APP=$(curl -fsS -XPOST "$BASE/api/admin/applications" \
  -H 'Content-Type: application/json' \
  -d '{"name":"demo-$(date +%s)}"')
APP_ID=$(jq -r '.application.id' <<<"$APP")
API_KEY=$(jq -r '.apiKey' <<<"$APP")
echo "  uygulama: $APP_ID"
echo "  anahtar : $API_KEY"

add_endpoint () {
  curl -fsS -XPOST "$BASE/api/admin/applications/$APP_ID/endpoints" \
    -H 'Content-Type: application/json' \
    -d '{"url":"'${1}'","description":"'${2}'","eventTypes":["*"]}' \
    | jq -r '.id'
}

echo
echo "► 2/5 Endpoint'ler ekleniyor"
EP_OK=$(add_endpoint "$CHAOS_INTERNAL/sink/ok" "saglam")
EP_FLAKY=$(add_endpoint "$CHAOS_INTERNAL/sink/flaky?rate=60" "%60 hatali")
```

```

EP_DEAD=$(add_endpoint "$CHAOS_INTERNAL/sink/dead" "olu sunucu")
EP_GONE=$(add_endpoint "$CHAOS_INTERNAL/sink/gone" "410 Gone")
echo " saglam $EP_OK"
echo " hatali $EP_FLAKY"
echo " olu $EP_DEAD"
echo " 410 Gone $EP_GONE"

echo
echo "► 3/5 Konfigurasyonun compacted topic üzerinden dispatcher'a ulasmasi bekleniyor"
sleep 3

echo
echo "► 4/5 40 olay gönderiliyor (4 endpoint x 40 = 160 teslimat)"
for i in $(seq 1 40); do
    curl -fsS -XPOST "$BASE/v1/events" \
        -H 'Content-Type: application/json' \
        -H "Authorization: Bearer $API_KEY" \
        -H "Idempotency-Key: demo-$i" \
        -d '{"eventType":"order.paid","payload":{"no":$i,"tutar":$((RANDOM % 5000)).00}}' \
        >/dev/null &
done
wait
echo " gönderildi"

echo
echo "► 5/5 Ayni Idempotency-Key ile 5 kez daha (mukerrer testi)"
for i in 1 2 3 4 5; do
    R=$(curl -fsS -XPOST "$BASE/v1/events" \
        -H 'Content-Type: application/json' \
        -H "Authorization: Bearer $API_KEY" \
        -H "Idempotency-Key: demo-1" \
        -d '{"eventType":"order.paid","payload":{"no":1}}' | jq -r '.duplicate')
    echo " deneme $i → duplicate=$R"
done

cat <<BANNER

Simdi http://localhost:3000 adresini acin ve izleyin:

• saglam endpoint → hepsi ilk denemede basarili
• %60 hatali → t1, t2, t3 katmanlarinda ustel azalma
• olu sunucu → 5 hatadan sonra DEVRE ACIK, istek atilmiyor
• 410 Gone → tek denemede DLQ, yeniden denenmiyor

Zamanlayici kartinda duraklatilmis katmanlari sari gorursunuz.

Grafana: http://localhost:3001/d/hookrelay-main
Kafka UI: http://localhost:8090 (retry topic'lerine bakin)

Yuk testi:
k6 run -e API_KEY=$API_KEY ops/k6/load.js

BANNER

```

kir.sh `scripts/kir.sh`

```
#!/usr/bin/env bash
# -----
# KIRMA SENARYOLARI – ogrenmenin asil gerceklestigi yer.
#
# Kullanım: ./scripts/kir.sh <senaryo>
# -----
set -euo pipefail
cd "$(dirname "$0")/.."

case "${1:-}" in
    broker)
        echo "► Kafka durduruluyor. Simdi olay gondermeyi deneyin."
        echo "  Beklenen: ingest 202 dondurmeye DEVAM EDER (outbox'a yaziyor),"
        echo "           teslimat durur, Kafka gelince birikmis hepsi akar."
        docker compose stop kafka
        echo
        echo "  Geri acmak icin: docker compose start kafka"
        ;;
    redis)
        echo "► Redis durduruluyor."
        echo "  Beklenen: dispatcher endpoint konfigurasyonunu okuyamaz,"
        echo "           teslimatlar dusurulur. Redis'in bu projede sadece"
        echo "           'cache' olmadigini gosterir."
        docker compose stop redis
        echo "  Geri: docker compose start redis && curl -XPOST localhost:8080/api/admin/endpoin"
        echo "        ts/republish"
        ;;
    cokuk-tuketici)
        echo "► Dispatcher olduruluyor (SIGKILL) – offset commit edilmeden."
        echo "  Beklenen: yeniden basladiginda commit edilmemis mesajlari"
        echo "           TEKRAR isler. 'En az bir kez' teslimatin canli kaniti."
        docker compose kill -s SIGKILL dispatcher
        sleep 2
        docker compose start dispatcher
        ;;
    olckle)
        N=${2:-3}
        echo "► Dispatcher $N ornege cikariliyor."
        echo "  Kafka UI'da (localhost:8090) tuketici grubuna bakin:"
        echo "  12 partition $N ornek arasinda paylastirilacak."
        docker compose up -d --scale dispatcher=$N --no-recreate dispatcher
        ;;
    *)
        cat <<'HELP'
Senaryolar:

    broker          Kafka'yi durdurur   → outbox'in ne ise yaradigini gosterir
    redis           Redis'i durdurur  → replikanin kritikligini gosterir
    cokuk-tuketici  Dispatcher'i oldurur → "en az bir kez" teslimati gosterir
    olckle [N]      N ornege cikarir   → partition paylasimini gosterir

Her senaryodan once http://localhost:3000 acik olsun.
HELP
;;
esac
```

durdur.sh `scripts/durdur.sh`


```
#!/usr/bin/env bash
# Durdurur. --temizle verilirse veri hacimlerini de siler.
set -euo pipefail
cd "$(dirname "$0")/.."

if [[ "${1:-}" == "--temizle" ]]; then
    echo "► Konteynerler ve VERI HACIMLERI siliniyor"
    docker compose down -v
else
    echo "► Konteynerler durduruluyor (veri korunuyor)"
    echo " Veriyi de silmek icin: $0 --temizle"
    docker compose down
fi
```

git-hazirla.sh `scripts/git-hazirla.sh`

Depoyu faz faz commit'lerle kurar. Tek "initial commit" ile yüklenmiş bir proje, hazır şablondan indirilmiş gibi görünür.

```
#!/usr/bin/env bash
# -----
# Depoyu faz faz commit'lerle hazirlar.
#
# Neden: tek "initial commit" ile yuklenmis proje, hazir sablondan
# indirilmis gibi gorunur. Faz faz gecmis, projenin nasil buyudugunu
# ve her fazin calisir durumda birakildigini gosterir.
#
# Kullanim: ./scripts/git-hazirla.sh
# -----
set -euo pipefail
cd "$(dirname "$0")/.."

[[ -d .git ]] && { echo "Zaten bir git deposu var. Devam edilmiyor."; exit 1; }

git init -q -b main
git add .gitignore .editorconfig LICENSE pom.xml
git commit -q -m "chore: cok modullu Maven iskeleti"

git add libs/contracts
git commit -q -m "feat: contracts - topic katalogu ve olay semalari"

git add libs/common
git commit -q -m "feat: common - HMAC imzalama ve Redis Lua ilkelleri"

Hiz siniri, dagitik devre kesici ve idempotency deposu Lua ile yazildi:
oku-hesapla-yaz ucusu atomik olmak zorunda."

git add libs/outbox
git commit -q -m "feat: outbox - transactional outbox deseni"

Veritabani commit'i ile Kafka yayini tek transaction'a giremez.
Kayit ayni transaction'da bir tabloya yazilir, ayri bir poller basar."

git add services/admin-api
git commit -q -m "feat: admin-api - kontrol duzlemi ve compacted config replikasyonu"

git add services/ingest-api
git commit -q -m "feat: ingest-api - idempotent olay kabulü ve fan-out"
```

```
git add services/dispatcher
git commit -q -m "feat: dispatcher - teslimat hattı, devre kesici, bulkhead

Chain of Responsibility ile on kontroller, endpoint bazlı bulkhead,
Strategy ile katmanlı yeniden deneme politikası. Hata sınıflandırması
ayrı bir bean'de (FailureClassifier): sınıflandırma bir politikadır,
veri değil."

git add services/retry-scheduler
git commit -q -m "feat: retry-scheduler - pause/seek ile Kafka'da geciktirmeli mesaj"

git add services/chaos-target services/gateway
git commit -q -m "feat: chaos-target ve gateway"

git add Dockerfile .dockerignore docker-compose.yml ops scripts ui
git commit -q -m "feat: ops - compose, nginx, prometheus, grafana, k6, kontrol paneli"

git add README.md README.en.md CONTRIBUTING.md .github docs
git commit -q -m "docs: README, mimari kararlar (ADR) ve sıfırdan inşa rehberi"

git add .github CONTRIBUTING.md LICENSE .editorconfig 2>/dev/null || true
git diff --cached --quiet || git commit -q -m "ops: CI, lisans ve kod kuralları"

git add -A
git diff --cached --quiet || git commit -q -m "chore: kalan dosyalar"

echo "✓ $(git rev-list --count HEAD) commit oluşturuldu"
git --no-pager log --oneline
```

Ek B — Sık Karşılaşılan Hatalar

`Topic ... not present in metadata after 60000 ms`

Topic yok ve otomatik oluşturma kapalı. Sebep genellikle `admin-api`'nin henüz ayağa kalkmamış olması — topic'leri o oluşturuyor.

```
docker compose logs admin-api --tail=50
docker exec hr-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Konteynerler `kafka:9092`'ye bağlanamıyor

`KAFKA_ADVERTISED_LISTENERS` yanlış. Konteynerler `kafka:9092`, makineniz `localhost:29092` kullanılmalı. İkisini karıştırmak Docker'da Kafka'nın bir numaralı hatası.

`No existing transaction found for transaction marked with propagation 'mandatory'`

`OutboxRecorder.record(...)` transaction dışından çağrıldı. Muhtemelen `@Transactional` metodu aynı sınıf içinden çağırdınız (self-invocation). Yedinci bölüm, 7.5.

Teslimatlar `DISCARDED` oluyor

`dispatcher` endpoint konfigürasyonunu bulamıyor. Redis boş olabilir:

```
docker exec hr-redis redis-cli KEYS 'hr:ep:*'
curl -XPOST localhost:8080/api/admin/endpoints/republish
```

`CommitFailedException: ... group has already rebalanced`

İşlem `max.poll.interval.ms`'ten uzun sürdü. `max.poll.records` değerini düşürün veya işlemi hızlandırın. `dispatcher`'da 50 kayıt / 5 dakika ayarlı; yavaş endpoint'ler varsa 20'ye düşürün.

Flyway `checksum mismatch`

Uygulanmış bir migration dosyasını değiştirdiniz. Asla yapmayın — yeni bir `V{n+1}__...sql` ekleyin. Yerel geliştirmede:

```
./scripts/durdur.sh --temizle && ./scripts/baslat.sh
```

Grafana panoları boş

Prometheus servisleri toplayamıyor olabilir. `http://localhost:9090/targets` adresinde hepsi `UP` olmalı. Değilse ilgili servisin `/actuator/prometheus` ucu açık mı bakın.

Port çakışması

Compose'da host portları bilinçli olarak kaydırıldı: Postgres **5433**, Redis **6380**, Kafka **29092**. Makinenizde zaten çalışan Postgres/Redis varsa çakışmasın diye. Yine de çakışıyorsa `docker-compose.yml` içinde `ports` değerlerini değiştirin.

Kapanış

Elinizde şu an çalışan bir sistem var. Ama asıl kazanılan şey kod değil; şu soruların cevabı:

- Neden iki sistem tek transaction'a giremez ve bu nasıl aşılır?
- Kafka neden bir kuyruk değil ve bunun sonuçları neler?
- Bir topic'in partition sayısı hangi kararları kilitler?
- Devre kesici neden paylaşılan durum ister?
- Yavaş bir bağımlılık bütün sistemi nasıl düşürür, nasıl engellenir?
- "Exactly-once" neden yok ve yerine ne konur?

Bu soruların hepsinin cevabı, bu projede **çalışan kod** olarak duruyor.

Buradan sonra

Sistemi büyütmek isterseniz, zorluk sırasına göre:

- 1. Endpoint bazlı olay tipi filtresi** — `order.*` gibi kalıp eşleme. Kolay, faydalı.
- 2. Webhook alıcı SDK'sı** — imza doğrulayan küçük bir kütüphane (Java + Node). Projeyi "kullanılabilir" yapan şey budur.
- 3. DLQ'dan otomatik kurtarma politikası** — endpoint sağlığa dönünce otomatik yeniden gönderim.
- 4. OpenTelemetry ile dağıtık izleme** — bir teslimatın dört servisteki yolunu Jaeger'da izlemek.
- 5. Kafka Streams ile anomali tespiti** — "bu endpoint son 5 dakikada %90 hata veriyor" penceresi.
- 6. Çok kiracılı hız sınırı** — uygulama bazlı global kota.

İlk ikisi bir hafta sonu. Üçüncüsü, DLQ'nun neden otomatik tüketicisi olmaması gerektiğini düşünmenizi gerektirir — ve o düşünce, kodun kendisinden değerlidir.